

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2017

Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc

Tháng 4 năm 2017

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017

IOI China candidate team papers / National training team 2017 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2017. Khác với một số tập trước đó, lớp văn bản của tập này trích xuất khá tốt bằng pdftotext. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, và chín bài đầu tiên: bài của Mao Xiao về công thức truy hồi của dãy số, Yang Jiaqi về matching trên đồ thị tổng quát bằng đại số tuyến tính, Yuan Yutao về bài toán tính tổng đa thức, Zhong Zhixian về bài toán tập độc lập trong thi tin học, Chen Junkun về *Đồ thị con kỳ diệu*, Sun Yaofeng về bài toán bao đóng bắc cầu động, Wang Leping về *A + B Problem*, Xu Mingkuan về thuật toán chia khối kích thước phi chuẩn, và Weng Wentao về cây palindrome cùng các ứng dụng.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2017*. Trang bìa ghi thời điểm phát hành là tháng 4 năm 2017. Tập PDF gồm 204 trang, được tạo bằng LaTeX với gói hyperref.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Một số nghiên cứu về công thức truy hồi của dãy số	Mao Xiao
12	Matching trên đồ thị tổng quát dựa trên đại số tuyến tính	Yang Jiaqi
27	Tính tổng đa thức	Yuan Yutao
32	Bàn về bài toán tập độc lập trong thi tin học	Zhong Zhixian
50	Báo cáo ra đề và mở rộng của <i>Đồ thị con kỳ diệu</i>	Chen Junkun
75	Tìm hiểu bài toán bao đóng bắc cầu động	Sun Yaofeng
90	Báo cáo ra đề <i>A + B Problem</i>	Wang Leping
99	Bước đầu tìm hiểu thuật toán chia khối kích thước phi chuẩn	Xu Mingkuan
122	Cây palindrome và ứng dụng	Weng Wentao
139	Báo cáo ra đề <i>Cây đen trắng</i>	Yan Shuyi
144	Báo cáo ra đề <i>Đa giác đều</i>	Yang Jingqin
155	Bàn về lời giải tuyến tính cho quy hoạch động có đơn điệu quyết định	Feng Zhe
165	Báo cáo ra đề <i>Cây đoạn bị thao túng</i>	Shen Rui
179	Bước đầu về logic máy tính và nghệ thuật: mô hình cường độ nốt khi chơi piano dựa trên logic	Zhao Shengyu
190	Báo cáo ra đề <i>Tái cấu trúc bộ gen</i>	Hong Huadun

3 Một số nghiên cứu về công thức truy hồi của dãy số

Mao Xiao, Trường Trung học Yali

Tóm tắt

Bài viết này là luận văn đội tuyển quốc gia tin học Trung Quốc năm 2017 của tác giả. Trong bài, tác giả trước hết giới thiệu đa thức truy hồi, sau đó giới thiệu thuật toán Berlekamp–Massey, rồi trình bày triển vọng ứng dụng của chúng trong thi tin học, gồm đếm công thức truy hồi và tìm đa thức đặc trưng của một số ma trận thưa.

Dẫn nhập

Đa thức truy hồi là một thuật ngữ do tác giả đặt ra. Trong thi tin học, công thức truy hồi thường được cho tường minh, còn nhiệm vụ của thí sinh thường là: biết một số hạng đầu của dãy và một công thức truy hồi nào đó, hãy tính một số hạng của dãy. Tuy nhiên, bài viết này nghiên cứu những công thức truy hồi ẩn. Vì vậy tác giả đưa ra khái niệm hoàn toàn mới là đa thức truy hồi. Đây là một khái niệm rất mạnh: nó không chỉ có các ứng dụng như đếm công thức truy hồi, mà còn giúp ta học và hiểu thuật toán Berlekamp–Massey.

Berlekamp–Massey là một thuật toán xuất sắc nhưng bị lạnh nhạt. Trong thi tin học, Berlekamp–Massey không mấy phổ biến; ngay cả những người biết nó cũng thường chỉ xem nó như một đường tắt. Ngay cả trên phạm vi thế giới, tôi cũng chưa nghe nói thuật toán này từng xuất hiện như thuật toán chuẩn của một bài nào. Tuy vậy, nó có khá nhiều ứng dụng.

Do đó, bài viết này sẽ trình bày hai nội dung ấy và giới thiệu một số ứng dụng của chúng.

3.1 Đa thức truy hồi

Trong mục này, tác giả sẽ đưa ra nhiều định nghĩa mới, đồng thời giới thiệu nhiều kết luận hữu ích.

3.1.1 Bậc nhỏ nhất

Định nghĩa 1.1. Với dãy $\{s_0, s_1, \dots, s_{l-1}\}$, ta đặt đa thức tương ứng của dãy là

$$S(x) = \sum_{k=0}^{l-1} s_k x^k.$$

Định nghĩa 1.2. Với một đa thức khác không A , **bậc** của nó được định nghĩa là bậc của hạng tử cao nhất, ký hiệu $\deg(A)$. Riêng đa thức không có bậc là vô cùng nhỏ.

Định nghĩa 1.3. Trên một trường nào đó, với dãy $\{r_0, r_1, \dots, r_{m-1}\}$ và dãy $\{a_0, a_1, \dots, a_{n-1}\}$, nếu r không toàn bằng không và với mọi $0 \leq p \leq n - m$ ta có

$$\sum_{k=0}^{m-1} a_{p+k} \times r_{m-k-1} = 0, \quad (1.1)$$

thì gọi r là một **công thức truy hồi**¹ của dãy a .

Nếu $r_0 = 1$, ta gọi r là **công thức truy tiến**. Nếu r là công thức truy tiến có độ dài ngắn nhất trong mọi công thức truy tiến của dãy a , ta gọi nó là **công thức truy tiến ngắn nhất** của dãy a .

¹Một số nơi gọi khái niệm này là công thức truy tiến. Tác giả không tìm được khác biệt giữa hai thuật ngữ, nên trong bài chọn thuật ngữ ít dùng hơn để định nghĩa riêng, còn thuật ngữ kia giữ nghĩa thường gặp.

Định nghĩa 1.4. Với dãy a và đa thức khác không $R(x)$, luôn tồn tại duy nhất công thức truy hồi ngắn nhất r sao cho đa thức tương ứng của nó là $R(x)$. Ta gọi r là **công thức truy hồi ngắn nhất tương ứng** của $R(x)$. Độ dài của r trừ một được gọi là **bậc nhỏ nhất** của đa thức $R(x)$, ký hiệu d_R hoặc $d_{R(x)}$. Đa thức $R(x)$ được gọi là **đa thức truy hồi bậc** d_R của a .

Chứng minh. Cho một đa thức khác không $R(x)$ và một dãy a độ dài n . Dãy gồm các hệ số từ hạng tử bậc 0 đến hạng tử bậc $\max(\deg(R), n)$ của nó chắc chắn là một công thức truy hồi của a , và đa thức tương ứng là $R(x)$, nên r tồn tại.

Nếu bậc nhỏ nhất của đa thức khác không $R(x)$ là d , thì r nhất định là dãy gồm các hệ số từ bậc 0 đến bậc d của nó, nên r là duy nhất. ■

3.1.2 Các phép toán trên đa thức truy hồi

Sau khi có đa thức truy hồi, ta có thể nhận được một số kết luận hữu ích. Từ các kết luận này, ta có thể làm nhiều việc, và chúng cũng giúp ích cho thuật toán Berlekamp–Massey.

Bổ đề 1.1. Đa thức $R(x)$ có bậc nhỏ nhất d đối với dãy a khi và chỉ khi d không nhỏ hơn bậc của $R(x)$, và các hệ số từ bậc d đến bậc $n - 1$ của $R(x) \times A(x)$ đều bằng không.

Chứng minh. Ta nhận thấy công thức (1.1) có dạng tích chập; từ đó dễ suy ra bổ đề. ■

Định lý 1.1. Giả sử $F(x), G(x)$ là hai đa thức có bậc nhỏ nhất không vượt quá d đối với dãy a . Khi đó $F(x) + G(x)$ cũng có bậc nhỏ nhất không vượt quá d đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ và $G(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc d đến bậc $n - 1$ của $(F(x) + G(x)) \times A(x)$ đều bằng không; đồng thời bậc của $F(x) + G(x)$ hiển nhiên không vượt quá d . Do đó $F(x) + G(x)$ có bậc nhỏ nhất không vượt quá d đối với dãy a . ■

Bổ đề 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d đối với dãy a . Khi đó với $k \in \mathbb{N}$ và $k \geq 0$, $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a .

Chứng minh. Theo Bổ đề 1.1, các hệ số từ bậc d đến bậc $n - 1$ của $F(x) \times A(x)$ đều bằng không. Vì vậy các hệ số từ bậc $d + k$ đến bậc $n + k - 1$ của $x^k F(x) \times A(x)$ đều bằng không; đồng thời bậc của $x^k F(x) \times A(x)$ hiển nhiên không vượt quá $d + k$. Do đó $x^k F(x)$ có bậc nhỏ nhất không vượt quá $d + k$ đối với dãy a . ■

Định lý 1.2. Giả sử $F(x)$ có bậc nhỏ nhất d_F đối với dãy a . Khi đó $F(x) \times G(x)$ có bậc nhỏ nhất không vượt quá $d_F + \deg(G)$ đối với dãy a .

Chứng minh. Suy ra từ Định lý 1.1 và Bổ đề 1.2. ■

3.1.3 Cơ sở

Định nghĩa 1.5. Với dãy a và dãy đa thức $\{\text{Base}_0(x), \text{Base}_1(x), \text{Base}_2(x), \dots\}$, nếu với mọi k , $\text{Base}_k(x)$ là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức có hạng tử thấp nhất là x^k , thì gọi dãy đa thức Base là **một cơ sở** của dãy a .

Ta nhận thấy một cơ sở của một dãy thật ra chính là cơ sở của không gian tuyến tính do nó sinh ra. Ta cũng có thể nhận thấy tọa độ của một đa thức theo một cơ sở liên hệ chặt chẽ với bậc nhỏ nhất của nó.

Định lý 1.3. Nếu Base là cơ sở của dãy a , thì mọi đa thức có bậc nhỏ nhất d đều có thể được biểu diễn tuyến tính một cách duy nhất bởi một số $\text{Base}_k(x)$ có bậc nhỏ nhất không vượt quá d ; hơn nữa trong biểu diễn đó nhất định tồn tại ít nhất một $\text{Base}_k(x)$ có bậc nhỏ nhất đúng bằng d .

Chứng minh. Giả sử $R(x)$ là một đa thức có bậc nhỏ nhất d . Ta xét cách phân rã nó.

Nếu $R(x)$ bằng không thì không cần phân rã. Ngược lại, giả sử hạng tử thấp nhất của nó là x^k , hệ số là C_r , và hệ số tương ứng của $\text{Base}_k(x)$ là C_{Base} . Khi đó hạng tử thấp nhất của $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ ít nhất là x^{k+1} . Vì $\text{Base}_k(x)$ là một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức có hạng tử thấp nhất x^k , bậc nhỏ nhất của nó hiển nhiên không vượt quá d . Theo Định lý 1.1, $R(x) - C_{\text{Base}}^{-1} C_r \text{Base}_k(x)$ cũng có bậc nhỏ nhất không vượt quá d . Ta phân rã biểu thức này theo cách đệ quy. Cuối cùng ta thu được một biểu diễn tuyến tính thỏa điều kiện.

Vì các $\text{Base}_k(x)$ là một cơ sở của không gian tuyến tính mà chúng sinh ra, cách biểu diễn hiển nhiên là duy nhất.

Nếu trong biểu diễn, mọi $\text{Base}_k(x)$ đều có bậc nhỏ nhất nhỏ hơn d , thì hoặc đa thức này là đa thức không, hoặc theo Bổ đề 1.1 bậc nhỏ nhất của nó sẽ nhỏ hơn d , mâu thuẫn. ■

3.2 Thuật toán Berlekamp–Massey

Mục này giới thiệu thuật toán Berlekamp–Massey². Tuy các tài liệu như Wikipedia [1] đều đã giới thiệu quy trình của thuật toán Berlekamp–Massey, tác giả không tìm được bất kỳ bản quy trình thuật toán tiếng Trung nào. Đồng thời, tác giả cũng muốn dùng ngôn ngữ của bài viết này để trình bày lại quy trình của thuật toán.

3.2.1 Giới thiệu thuật toán

Thuật toán Berlekamp–Massey có thể tìm một cơ sở Base cho dãy a độ dài n trên một trường bất kỳ bằng $\mathcal{O}(n^2)$ phép toán và không cần thêm không gian đáng kể.

3.2.2 Quy trình thuật toán

Theo định nghĩa, mọi đa thức có hạng tử thấp nhất x^k , với $k \geq n$, đều có bậc nhỏ nhất đúng bằng bậc của nó. Vì vậy với $\text{Base}_k(x)$, ta chỉ cần chọn bất kỳ Cx^k nào với $C \neq 0$. Không mất tính tổng quát, giả sử mọi đa thức tìm được đều có hệ số của hạng tử thấp nhất bằng 1, khi đó ở đây ta đặt $C = 1$.

Giả sử ta đã tìm được một đa thức có bậc nhỏ nhất nhỏ nhất trong các đa thức từ $\text{Base}_{k+1}(x)$ đến $\text{Base}_n(x)$. Xét cách tìm $\text{Base}_k(x)$.

Ta xét $x \text{Base}_k(x)$. Theo Định lý 1.3, ta chỉ cần làm cho bậc nhỏ nhất lớn nhất trong biểu diễn tuyến tính của nó càng nhỏ càng tốt.

²Wikipedia tiếng Trung phiên âm thuật toán này là Berlekamp–Massey.

Trước hết, vì hạng tử thấp nhất của $x \text{Base}_k(x)$ là x^{k+1} , trong biểu diễn nhất định có hạng $\text{Base}_{k+1}(x)$. Xét các hạng còn lại: rõ ràng bậc nhỏ nhất lớn nhất trong số chúng phải càng nhỏ càng tốt.

Đặt $\text{res}(x) = x \text{Base}_k(x) - \text{Base}_{k+1}(x)$. Ta tính $\text{pro}(x) = \text{res}(x) \times A(x)$. Theo Bổ đề 1.1, để biết các hệ số từ bậc $k+1$ đến bậc $n-1$ của $\text{pro}(x)$ đều bằng không.

Xét hạng tử bậc n . Nếu nó bằng không, ta đã biết $x \text{Base}_k(x)$; ngược lại giả sử nó là u . Xét $\text{Base}_l(x)$ với $l > k+1$. Nếu hệ số bậc n của $\text{Base}_l(x) \times A(x)$ khác không, giả sử hệ số bậc n là v . Để biết các hệ số từ bậc $k+1$ đến bậc n của $\text{res}(x) - uv^{-1} \text{Base}_l(x)$ đều bằng không. Vì bậc nhỏ nhất lớn nhất phải càng nhỏ càng tốt, nếu tồn tại đa thức thỏa điều kiện thì $\text{Base}_l(x)$ nhất định là đa thức có bậc nhỏ nhất nhỏ nhất trong mọi đa thức thỏa điều kiện. Lấy $\text{Base}_l(x)$ như vậy, ta có thể tìm được $x \text{Base}_k(x)$. Nếu không tồn tại $\text{Base}_l(x)$ như vậy, thì chỉ có thể là bậc nhỏ nhất của $x \text{Base}_k(x)$ đã vượt quá n . Khi đó ta chỉ cần tùy ý chọn một đa thức có hạng tử thấp nhất là hạng bậc x^{k+1} và hệ số bằng 1.

Khi đã biết $x \text{Base}_k(x)$, hiển nhiên ta cũng biết $\text{Base}_k(x)$.

Dùng phương pháp này, mỗi $P_k(x)$ đều có thể được tìm trong không quá $\mathcal{O}(n)$ phép toán, vì vậy tổng số phép toán là $\mathcal{O}(n)$, còn số Base cần tính chỉ là $\mathcal{O}(n)$. Do đó tổng số phép toán là $\mathcal{O}(n^2)$.

Rõ ràng ngoài việc lưu $A(x)$ và Base, thuật toán này không cần không gian phụ đáng kể.

3.2.3 Mã giả

Theo mô tả trên, thuật toán có thể được cài đặt bằng phương pháp sau.

Thuật toán 1 Berlekamp–Massey Algorithm I

Require: $a, n = |a|$

Ensure: Base

```

1:  $\text{Base}_k = x^k$  ( $k \geq n$ )
2:  $l \leftarrow n + 1$ 
3:  $v \leftarrow 1$ 
4: for  $k \leftarrow n - 1$  to 1 do
5:    $u \leftarrow [x^n] \text{Base}_{k+1}(x) \times A(x)$ 
6:    $\text{Base}_k(x) \leftarrow (\text{Base}_{k+1}(x) - uv^{-1} \text{Base}_l(x)) \div x$ 
7:   if  $(u \neq 0) \wedge (d_{\text{Base}_{k+1}} < d_{\text{Base}_l})$  then
8:      $l \leftarrow k + 1$ 
9:      $v \leftarrow u$ 
10:  end if
11: end for
```

Theo mô tả thuật toán, giá trị ban đầu của l và v không quan trọng; chỉ cần giá trị ban đầu thỏa $l \geq n + 1$ là có thể bảo đảm đáp án đúng. Tuy nhiên, $l = n + 1, v = 1$ là giá trị khởi tạo tham khảo trên Wikipedia tiếng Anh, nên tác giả xem nó như một phương án đã thành thông lệ.

Tuy nhiên, vì phép toán đa thức khá phiền phức, khi cài đặt thật sự ta thường không dùng đa thức một cách tường minh.

Ký hiệu r_k là công thức truy hồi ngắn nhất tương ứng với $\text{Base}_{n-k+1} \div x^{n-k+1}$. Khi đó ta dễ thấy r_k là công thức truy tiến ngắn nhất của tiền tố độ dài k của dãy a . Vì vậy quá trình trên dễ dàng được đổi thành quá trình tính r .

Ta xem dãy như một mảng có độ dài thay đổi, chỉ số bắt đầu từ 0. Thuật toán mới như sau.

Thuật toán 2 Berlekamp–Massey Algorithm II

Require: $a, n = |a|$

Ensure: r_k ($0 \leq k \leq n$)

```
1:  $r_0 = \{1\}$ 
2:  $\text{last} \leftarrow \{1\}$ 
3:  $v \leftarrow 1$ 
4:  $s \leftarrow 1$ 
5: for  $k \leftarrow 1$  to  $n$  do
6:    $r_k \leftarrow r_{k-1}$ 
7:    $u \leftarrow \sum_{i=0}^{|r_k|-1} a_{k-i-1} \times r_i$ 
8:   if ( $u \neq 0$ ) then
9:     if  $|r_k| < |\text{last}| + s$  then
10:       $r_k.\text{append}(\{0\} \times (|\text{last}| + s - |r_k|))$ 
11:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
12:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
13:      end for
14:       $\text{last} \leftarrow r_{k+1}$ 
15:       $v \leftarrow u$ 
16:       $s \leftarrow 0$ 
17:    else
18:      for  $i \leftarrow 0$  to  $|\text{last}| - 1$  do
19:         $r_k[i + s] \leftarrow r_k[i + s] - uv^{-1} \text{last}[i]$ 
20:      end for
21:    end if
22:  end if
23:   $s \leftarrow s + 1$ 
24: end for
```

Ngoài ra, nếu ta chỉ cần tìm công thức truy tiến ngắn nhất của cả dãy, thì ở mọi thời điểm ta thật ra chỉ cần lưu một r_k và một last. Nếu không tính kích thước từng phần tử, độ phức tạp không gian có thể giảm xuống $\mathcal{O}(n)$.

3.3 Đếm công thức truy hồi

3.3.1 Đa thức truy hồi nhỏ nhất và đa thức truy hồi thứ nhì không tầm thường

Định nghĩa 3.1. Trong mọi $\text{Base}_k(x)$ đã tìm được, một đa thức có bậc nhỏ nhất nhỏ nhất được gọi là **đa thức truy hồi nhỏ nhất**, ký hiệu $M(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; bậc nhỏ nhất của nó được ký hiệu d_M .

Trong mọi $\text{Base}_k(x)$ đã tìm được, nếu tồn tại một đa thức không thể biểu diễn dưới dạng $x^k M(x)$, với $k \in \mathbb{N}, k \geq 0$, thì đa thức có bậc nhỏ nhất nhỏ nhất trong số đó được gọi là **đa thức truy hồi thứ nhì không tầm thường**, ký hiệu $S(x)$. Nếu có nhiều đa thức như vậy thì chọn tùy ý; nếu không tồn tại thì đặt $S(x) = \text{Base}_{n+1}(x)$. Bậc nhỏ nhất của nó được ký hiệu d_S .

3.3.2 Biểu diễn tuyến tính

Định nghĩa 3.2. Với các đa thức $A(x), B(x), C(x)$, nếu tồn tại duy nhất đa thức $F_B(x)$ có bậc không vượt quá $d_A - d_B$ và đa thức $F_C(x)$ có bậc không vượt quá $d_A - d_C$ sao cho

$$A(x) = F_B(x)B(x) + F_C(x)C(x),$$

thì gọi $A(x)$ là **có thể biểu diễn tuyến tính** bởi $B(x), C(x)$.

Bổ đề 3.1. Với các đa thức $A(x), B(x), C(x), D(x), E(x)$, nếu $A(x)$ có thể biểu diễn tuyến tính bởi $B(x), C(x)$, và $B(x), C(x)$ đều có thể biểu diễn tuyến tính bởi $D(x), E(x)$, thì $A(x)$ cũng có thể biểu diễn tuyến tính bởi $D(x), E(x)$.

Chứng minh. Đặt

$$\begin{aligned} A(x) &= F_B(x)B(x) + F_C(x)C(x), \\ B(x) &= F_{BD}(x)D(x) + F_{BE}(x)E(x), \\ C(x) &= F_{CD}(x)D(x) + F_{CE}(x)E(x). \end{aligned}$$

Dễ biết

$$\begin{aligned} A(x) &= (F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x))D(x) \\ &\quad + (F_D(x)F_{BE}(x) + F_C(x)F_{CE}(x))E(x). \end{aligned}$$

Vì

$$\deg(F_B(x)) \leq d_A - d_B, \quad \deg(F_{BD}(x)) \leq d_B - d_D,$$

nên

$$\deg(F_B(x)F_{BD}(x)) = \deg(F_B(x)) + \deg(F_{BD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Do đó

$$\deg(F_B(x)F_{BD}(x) + F_C(x)F_{CD}(x)) \leq d_A - d_D.$$

Tương tự,

$$\deg(F_B(x)F_{BE}(x) + F_C(x)F_{CE}(x)) \leq d_A - d_E.$$

Quá trình này bảo đảm tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.1. Sau khi tính xong $\text{Base}_k(x)$, nó là đa thức nhỏ nhất hoặc là đa thức thứ nhì không tầm thường.

Tạm thời ta chưa chứng minh định lý này; trước hết giới thiệu một bổ đề mới, rồi chứng minh cả hai cùng lúc.

Bổ đề 3.2. Mọi $\text{Base}_k(x)$ đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Xét quy trình của thuật toán Berlekamp–Massey.

Khi $k = n$, hiển nhiên Định lý 3.1 và Bổ đề 3.2 đều đúng.

Giả sử sau khi tính xong $\text{Base}_{k+1}(x)$, cả hai đều đúng.

Xét quá trình tính $x \text{Base}_k(x)$. Cuối cùng nó nhất định là $\text{Base}_{k+1}(x)$ cộng với một hằng số nhân với một $\text{Base}_l(x)$ nào đó. Theo mô tả thuật toán, ta dễ nhận thấy $\text{Base}_{k+1}(x)$ và hạng còn lại đều là một trong $M(x)$ và $S(x)$.

Vì vậy bậc nhỏ nhất của $x \text{Base}_k(x)$ bằng $\max(d_M, d_S)$, còn bậc nhỏ nhất của $\text{Base}_k(x)$ phải trừ thêm một. Do đó nó nhất định là đa thức nhỏ nhất hoặc đa thức thứ nhì tầm thường. Như vậy ta đã chứng minh Định lý 3.1.

Hiển nhiên với chính nó, Bổ đề 3.2 đúng. Tuy nhiên vì $M(x), S(x)$ đã thay đổi, ta còn phải xét $\text{Base}_l(x)$ với $l > k$.

Xét sự thay đổi của $M(x)$ và $S(x)$. $M(x)$ hoặc không đổi, hoặc trở thành $S(x)$ mới.

Xét $S(x)$. Theo quy trình thuật toán, ta có

$$x \text{Base}_k(x) = C_M M(x) + C_S S(x).$$

Do đó

$$S(x) = x \text{Base}_k(x) - C_M M(x).$$

Vì $d_S > d_{\text{Base}_k}$ và $d_S \geq d_M$, $S(x)$ trước khi thay đổi có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi.

Như vậy, $M(x)$ và $S(x)$ trước khi thay đổi đều có thể được biểu diễn tuyến tính bởi $M(x)$ và $S(x)$ sau khi thay đổi. Theo Bổ đề 3.1, $\text{Base}_l(x)$ với $l > k$ vẫn có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Đồng thời, với trường hợp $k > n$, vì $\text{Base}_k(x) = x^{k-n} \text{Base}_n(x)$, bổ đề hiển nhiên đúng.

Tương tự, quá trình này giữ tính duy nhất, nên bổ đề được chứng minh. ■

Định lý 3.2. Mọi đa thức đều có thể được biểu diễn tuyến tính bởi $S(x)$ và $M(x)$.

Chứng minh. Dễ suy ra từ Định lý 1.3 và Bổ đề 3.2. ■

3.3.3 Định lý đếm công thức truy hồi

Định lý 3.3 (Định lý đếm đa thức truy hồi). Giả sử kích thước của trường là q . Khi đó số đa thức truy hồi bậc k của một dãy là

$$\text{num}_k = q^{\max(k-d_M-1,0)+\max(k-d_S-1,0)} (q^{[k \geq d_M]+[k \geq d_S]} - 1).$$

Trong công thức này, $[x]$ bằng 1 khi x đúng, và bằng 0 nếu không.

Chứng minh. Dễ suy ra từ Định lý 3.2. ■

Định lý 3.4 (Định lý đếm công thức truy hồi). Số công thức truy hồi của một dãy là

$$\sum_{k=1}^n (n - k + 1) \text{num}_k.$$

Chứng minh. Một đa thức truy hồi bậc x tương ứng đúng một công thức truy hồi có độ dài $x, x+1, \dots, n$, tổng cộng $n - x + 1$ công thức, nên có công thức trên. ■

3.4 Đa thức đặc trưng của ma trận thưa đặc biệt

Thuật toán Berlekamp–Massey không chỉ có tác dụng rất mạnh trong lĩnh vực công thức truy hồi; nó cũng có thể giúp ích cho các bài toán liên quan đến ma trận.

Vì vậy, chương này sẽ giới thiệu cách dùng thuật toán đó để tính đa thức đặc trưng của ma trận thưa đặc biệt.

3.4.1 Điều kiện quan trọng

Khi mỗi trị riêng chỉ tương ứng với một khối Jordan³, đa thức đặc trưng chính là đa thức tối tiểu. Đây là điều kiện quan trọng để thuật toán này hoạt động.

³Nói cách khác, chiều cao của mỗi vector đều bằng bội số của nó.

3.4.2 Quá trình thực hiện

Theo định lý Cayley–Hamilton, đa thức đặc trưng của một ma trận $n \times n$ A là đa thức triệt tiêu của nó; tức là nếu đa thức đặc trưng là p thì $p(A) = 0$.

Nhớ lại định nghĩa công thức truy hồi, ta thấy nếu chọn ngẫu nhiên một vector v , trước hết hiển nhiên $p(Av) = 0$. Nếu kích thước của trường số đủ lớn, thì với xác suất rất cao, dãy vector $\{v, Av, A^2v, \dots\}$ tồn tại một công thức truy tiến ngắn nhất duy nhất r có độ dài n . Khi đó $R(x)$ chính là đa thức đặc trưng.

Nếu A là ma trận thưa và số phần tử là e , thì từ $A^k v$ có thể tính $A^{k+1}v$ trong $\mathcal{O}(e)$ phép toán.

Khi xét dùng thuật toán Berlekamp–Massey, ta có hai khó khăn.

Khó khăn thứ nhất là xử lý nhanh dãy vector. Ta có thể thiết kế một hàm băm, trong đó hàm băm là một biến đổi tuyến tính, biến mọi vector thành một số; như vậy ta có thể xử lý dãy vector.

Khó khăn thứ hai là xử lý một dãy vô hạn. Tuy dãy này dài vô hạn, ta có thể nhận thấy trong trường hợp ngẫu nhiên, độ dài công thức truy tiến ngắn nhất của tiền tố độ dài $2n$ xấp xỉ n . Vì vậy ta có thể lấy một tiền tố đủ dài để tính. Một cách tốt hơn là xét Thuật toán 2: thật ra ta có thể liên tục tăng tiền tố cho đến khi độ dài công thức truy tiến ngắn nhất đạt n .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính đa thức đặc trưng của ma trận thưa đặc biệt bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.3 Định thức của ma trận thưa

Xét cách tính định thức $\det(A)$. Hiển nhiên nếu ma trận này thỏa điều kiện quan trọng ở Mục 4.1, ta có thể tìm đa thức đặc trưng, rồi lấy hạng tử tự do nhân với $(-1)^n$.

Nếu ma trận này không thỏa điều kiện đó, ta có thể lấy ngẫu nhiên một ma trận đường chéo B ; số phép toán là $\mathcal{O}(ne)$. Nếu trường số đủ lớn, thì AB có xác suất rất cao thỏa điều kiện này. Vì $\det(AB) = \det(A) \det(B)$, ta chỉ cần tính $\det(AB)$, rồi chia cho $\det(B)$. Vì B là ma trận đường chéo, $\det(B)$ chính là tích các phần tử trên đường chéo của B .

Như vậy, ta thu được một thuật toán Monte Carlo có thể tính định thức của ma trận thưa bằng $\mathcal{O}(n(n + e))$ phép toán và không gian tuyến tính theo kích thước ma trận.

3.4.4 Đếm cây khung của đồ thị thưa

Hiển nhiên, có thể dùng định lý ma trận-cây Kirchhoff để tính số cây khung. Nếu đồ thị đủ thưa, thì định thức của ma trận Laplace hiển nhiên có thể được tính bằng phương pháp ở Mục 4.3.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(n(n + m))$, độ phức tạp không gian là $\mathcal{O}(n + m)$, rất tốt.

3.5 Tổng kết

Toàn bài đều xoay quanh công thức truy hồi. Phần thứ nhất trình bày đa thức truy hồi, phần thứ hai nghiên cứu thuật toán Berlekamp–Massey, còn phần thứ ba và thứ tư chủ yếu giới thiệu một số ứng dụng của chúng.

Tôi hy vọng bài viết này có thể đóng vai trò gợi mở. Đây vẫn chỉ là một nghiên cứu chưa chín muồi về công thức truy hồi, và các ứng dụng được giới thiệu cũng khá cơ bản. Khi khoa học kỹ thuật không ngừng tiến bộ, nhất định còn nhiều ứng dụng thú vị hơn đang chờ chúng ta khám phá. Vì vậy, mong rằng bài viết này có thể gợi ý cho độc giả và dẫn ra một số ứng dụng hấp dẫn hơn.

Lời cảm ơn

1. Cảm ơn bạn Du Yuhao. Lần đầu tiên tôi nghe nói về thuật toán này là từ bạn ấy, và sự xuất hiện của một phần nội dung trong bài viết này không thể tách rời sự giúp đỡ của bạn ấy.
2. Cảm ơn các bạn Du Yuhao, Yang Jiaqi và Weng Wentao đã sửa lỗi.
3. Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Tài liệu tham khảo

[1] Wikipedia.

https://en.wikipedia.org/wiki/Berlekamp%E2%80%93Massey_algorithm

4 Matching trên đồ thị tổng quát dựa trên đại số tuyến tính

Yang Jiaqi, Trường Trung học Tưởng niệm Trung Sơn, thành phố Trung Sơn

Tóm tắt

Bài toán matching trên đồ thị là một bài toán kinh điển của lý thuyết đồ thị, có vị trí quan trọng cả trong học thuật lẫn trong thi thuật toán. Bài viết này chủ yếu giới thiệu một thuật toán matching trên đồ thị tổng quát dựa trên đại số tuyến tính, đồng thời đưa ra một số ứng dụng đơn giản của nó. Thuật toán này rất khác với các phương pháp tổ hợp truyền thống để giải bài toán này; hy vọng bản thân thuật toán và những tư tưởng dùng trong quá trình thu được thuật toán có thể gợi mở cho độc giả.

Dẫn nhập

Bài toán matching trên đồ thị có thể được chia theo tính chất của đồ thị thành hai loại: matching trên đồ thị hai phía và matching trên đồ thị tổng quát. Trong đó, matching hai phía đã xuất hiện nhiều năm trong giới thi thuật toán và là một điểm kiến thức thường gặp. Vì thuật toán của nó tương đối đơn giản, nó đã được phổ cập rất tốt: hiện nay phần lớn thí sinh đều đã nắm vững và có thể vận dụng thành thạo thuật toán matching hai phía.

Ngược lại, matching trên đồ thị tổng quát là một điểm kiến thức mới nổi. Vài năm gần đây, nó đã xuất hiện trong bài tập đội tuyển, kỳ thi mùa đông và các kỳ thi giao lưu đội tuyển. Nhưng các kỳ thi ấy đều ở cấp trên NOI; trong các kỳ thi dưới NOI, đề về matching tổng quát còn hiếm gặp, và hiện nay số thí sinh nắm được thuật toán matching tổng quát cũng tương đối ít. Tác giả cho rằng một nguyên nhân quan trọng khiến thuật toán này chưa phổ biến là thuật toán cây hoa, cách thường dùng hiện nay để giải loại bài toán này, khá phức tạp và có ngưỡng học tập cao. Vì vậy bài viết này sẽ giới thiệu một thuật toán matching dễ nhớ, dễ cài đặt, với hy vọng thúc đẩy việc phổ cập thuật toán matching trên đồ thị tổng quát.

Như tiêu đề bài viết đã nói, thuật toán được giới thiệu ở đây là một phương pháp đại số. Dù trong thi thuật toán, phương pháp tổ hợp thường gặp hơn, phương pháp đại số cũng là một công cụ mạnh để giải các bài toán đồ thị. Ví dụ nổi tiếng nhất có lẽ là định lý Matrix-Tree để đếm cây khung. Phương pháp đại số đem lại một góc nhìn hoàn toàn mới khi giải quyết vấn đề; hy vọng bài viết này có thể giúp độc giả cảm nhận sức hấp dẫn riêng của phương pháp đại số.

Bài viết trước hết sẽ bắt đầu từ bài toán matching hoàn hảo, giải bài toán quyết định tương ứng, sau đó thu được một thuật toán xây dựng phương án matching hoàn hảo và từng bước tối ưu nó bằng một số kiến thức đại số tuyến tính. Tiếp theo, bài viết chuyển bài toán matching cực đại về matching hoàn hảo, rồi cuối cùng trình bày một số ứng dụng và mở rộng của thuật toán này.

4.1 Kiến thức chuẩn bị

4.1.1 Lý thuyết đồ thị

Nếu không nói gì thêm, các đồ thị trong bài đều là đồ thị vô hướng.

Ta dùng $G = (V, E)$ để biểu diễn một đồ thị, trong đó V là tập đỉnh và E là tập cạnh. Ta thường dùng n để biểu diễn kích thước $|V|$ của tập đỉnh, dùng m để biểu diễn kích thước $|E|$ của tập cạnh, và thường xem $V = \{v_1, v_2, \dots, v_n\}$.

Khi không gây nhập nhằng, ta dùng uv để biểu diễn cạnh (u, v) .

Một **matching** M của đồ thị $G = (V, E)$ là một tập con của tập cạnh E , thỏa mãn hai cạnh bất kỳ trong M không có đỉnh chung. Nếu đỉnh v là đầu mút của một cạnh nào đó trong M , ta nói v nằm trong matching M . Giá trị lớn nhất của kích thước mọi matching của đồ thị G được ký hiệu là $\nu(G)$; nếu $|M| = \nu(G)$, ta gọi M là một **matching cực đại** của G . Đặc biệt, nếu mọi đỉnh của G đều nằm trong matching M , ta gọi M là một **matching hoàn hảo** của G . Rõ ràng G chỉ có thể có matching hoàn hảo khi $|V|$ là số chẵn.

Với một đồ thị $G = (V, E)$ và một tập đỉnh $U \subseteq V$, ta ký hiệu đồ thị thu được bằng cách xóa mọi đỉnh trong U khỏi G là $G - U$, tức là

$$G - U = (V \setminus U, E \cap \{uv \mid u, v \in V \setminus U\});$$

đồ thị con cảm sinh của G theo U được ký hiệu là $G[U]$, tức là

$$G[U] = G - (V \setminus U).$$

4.1.2 Đại số tuyến tính

Với một ma trận A có m hàng và n cột, ta đặt tập chỉ số hàng là $\{1, \dots, m\}$, tập chỉ số cột là $\{1, \dots, n\}$. Phần tử ở hàng i , cột j của ma trận A được ký hiệu là $A_{i,j}$.

Với một ma trận $m \times n$ A và các tập bất kỳ $I \subseteq \{1, \dots, m\}, J \subseteq \{1, \dots, n\}$, ta ký hiệu ma trận con thu được bằng cách giữ lại mọi hàng trong I và mọi cột trong J là $A_{I,J}$. Ta viết tắt $A_{I,\{1,\dots,n\}}$ thành $A_{I,\cdot}$, và $A_{\{1,\dots,m\},J}$ thành $A_{\cdot,J}$.

Tương tự, ta ký hiệu $A^{I,J}$ là ma trận con thu được bằng cách xóa khỏi A mọi hàng trong I và mọi cột trong J .

Với một ma trận vuông $n \times n$ A , định thức của nó được ký hiệu là $\det A$, và

$$\det A = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn } \pi} \prod_{k=1}^n A_{k,\pi(k)}.$$

Trong đó Σ_n biểu diễn tập mọi hoán vị của $\{1, \dots, n\}$, còn $\text{sgn } \pi$ biểu diễn dấu của hoán vị π . Nếu số cặp nghịch thế của dãy $\{\pi(1), \dots, \pi(n)\}$ là x , thì $\text{sgn } \pi = (-1)^x$.

Cho m vector n chiều $\{v_1, \dots, v_m\}$. Nếu tồn tại $\lambda_1, \lambda_2, \dots, \lambda_m$ sao cho

$$v = \sum_{i=1}^m \lambda_i v_i,$$

thì gọi v là một tổ hợp tuyến tính của $\{v_1, \dots, v_m\}$. Nếu không vector nào trong m vector này có thể được biểu diễn thành tổ hợp tuyến tính của các vector còn lại, ta nói m vector này độc lập tuyến tính; ngược lại, chúng phụ thuộc tuyến tính.

Với một ma trận A , nếu xem mỗi hàng của nó là một vector, thì số lượng cực đại các vector độc lập tuyến tính có thể chọn ra được gọi là **hạng hàng**; nếu xem mỗi cột là một vector, số lượng cực đại các

vector cột độc lập tuyến tính được gọi là **hạng cột**. Hạng hàng và hạng cột của một ma trận luôn bằng nhau, nên ta gọi chung là **hạng** của ma trận A , ký hiệu $\text{rank } A$.

Cuối cùng, ta nêu một kết luận mà không chứng minh.

Định lý 1.1. Bốn bài toán sau có cùng độ phức tạp:

1. tính định thức của ma trận;
2. nhân hai ma trận;
3. tìm ma trận nghịch đảo;
4. khử Gauss trên ma trận, tức phân rã LU.

Ta đặt độ phức tạp của cả bốn bài toán này là $\mathcal{O}(n^\omega)^4$.

4.2 Sự tồn tại của matching hoàn hảo

4.2.1 Ma trận Tutte

Định nghĩa 2.1 (Ma trận Tutte). Với một đồ thị vô hướng $G = (V, E)$, định nghĩa ma trận Tutte của G là một ma trận $n \times n$ $\tilde{A}(G)$, trong đó

$$\tilde{A}(G)_{i,j} = \begin{cases} x_{i,j}, & \text{nếu } v_i v_j \in E \text{ và } i < j, \\ -x_{j,i}, & \text{nếu } v_i v_j \in E \text{ và } i > j, \\ 0, & \text{trong các trường hợp khác.} \end{cases}$$

Ở đây, $x_{i,j}$ là một biến duy nhất gắn với cạnh $v_i v_j$.

Ta xét ví dụ sau.



Hình 1: Đồ thị G và ma trận Tutte của nó

Nửa trái của Hình 1 cho một đồ thị G , còn nửa phải cho ma trận Tutte $\tilde{A}(G)$ tương ứng với G . Có thể thấy mỗi cạnh của G đều tương ứng với một biến, nên $\tilde{A}(G)$ dùng tổng cộng $|E|$ biến.

4.2.2 Định lý Tutte

Ma trận Tutte có liên hệ mật thiết với sự tồn tại của matching hoàn hảo trong đồ thị. Thật vậy, ta có định lý sau.

Định lý 2.1 (Tutte). Đồ thị G có matching hoàn hảo khi và chỉ khi $\det \tilde{A}(G) \neq 0$.

Để chứng minh định lý này, trước hết ta cần dùng khái niệm “phủ chu trình chẵn” để đặc trưng một đồ thị có matching hoàn hảo.

⁴Trong học thuật, người ta thường cho rằng $2 \leq \omega < 2.373$.

Định nghĩa 2.2. Một **phủ chu trình** của đồ thị $G = (V, E)$ là việc dùng một số chu trình để phủ mọi đỉnh của G , sao cho mỗi đỉnh của G nằm đúng trong một chu trình.

Nói một cách hình thức, mỗi phủ chu trình của G tương ứng với một hoán vị π từ 1 đến n , thỏa mãn $v_i v_{\pi(i)} \in E$.

Nếu mọi chu trình trong phủ chu trình này đều có độ dài chẵn, ta gọi nó là một **phủ chu trình chẵn**; ngược lại, ta gọi nó là một **phủ chu trình lẻ**.

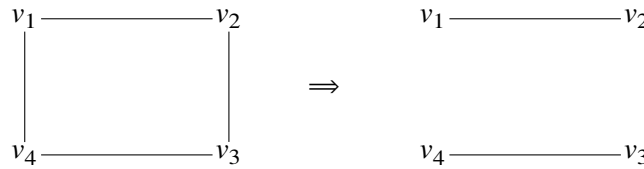
Chú ý rằng Định nghĩa 2.2 không yêu cầu các cạnh $v_i v_{\pi(i)}$ đôi một khác nhau, nghĩa là trong một phủ chu trình có thể xuất hiện cạnh lặp.

Bổ đề 2.2. Đồ thị $G = (V, E)$ có matching hoàn hảo khi và chỉ khi G có một phủ chu trình chẵn.

Chứng minh. Nếu G có một matching hoàn hảo, ta trực tiếp dùng chu trình độ dài hai do mỗi cặp đỉnh trong matching tạo thành để phủ G .

Nếu G có một phủ chu trình chẵn, thì trong mỗi chu trình chẵn, lấy cách một cạnh một cạnh, ta sẽ thu được một matching hoàn hảo của G . ■

Hình 2 dưới đây minh họa cách thu được một matching hoàn hảo từ một phủ chu trình chẵn của G .



Hình 2: Phủ chu trình chẵn và matching hoàn hảo

Có Bổ đề 2.2, ta có thể chứng minh định lý Tutte, tức Định lý 2.1.

Chứng minh định lý Tutte. Ta có

$$\det \tilde{A}(G) = \sum_{\pi \in \Sigma_n} (-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \tilde{A}(G)_{k, \pi(k)}. \quad (2.1)$$

Các hạng tử khác 0 ở vế phải của đẳng thức có dạng

$$(-1)^{\text{sgn}(\pi)} \prod_{k=1}^n \pm x_{k, \pi(k)},$$

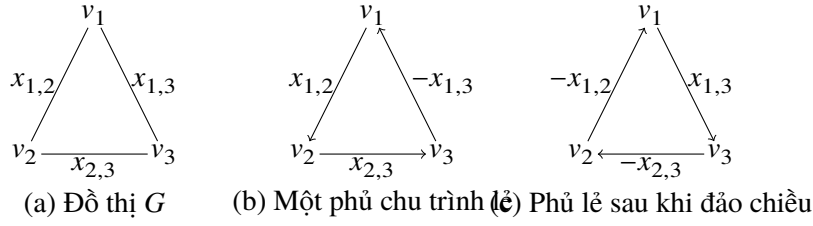
trong đó $v_1 v_{\pi(1)}, \dots, v_n v_{\pi(n)}$ là các cạnh của G . Những cạnh này tạo thành một phủ chu trình của G , tương ứng với phân rã thành chu trình của π .

Tiếp theo, nếu ta đảo chiều một chu trình bất kỳ có độ dài lớn hơn 2 trong π , ta sẽ thu được cùng một phủ chu trình. Bây giờ xét dấu của nó thay đổi ra sao. Gọi π' là hoán vị thu được sau khi đảo chiều một chu trình trong π . Việc đảo chiều một chu trình không làm đổi dấu của hoán vị π , tức $\text{sgn}(\pi) = \text{sgn}(\pi')$, nhưng làm đổi dấu mọi biến $x_{i, \pi(i)}$ trên chu trình đó.

Vì vậy, nếu ta đảo chiều một chu trình chẵn trong π , dấu của hạng tử thu được giống với dấu của hạng tử tương ứng với π ; nếu không, dấu ngược lại.

Do đó mọi phủ chu trình chứa chu trình lẻ đều bị triệt tiêu từng cặp một bởi một hạng tử dương và một hạng tử âm. Vì $\det \tilde{A}(G) \neq 0$, điều này có nghĩa là đồ thị G tồn tại một phủ chu trình chẵn, nên G có matching hoàn hảo. ■

Để hiểu trực quan hơn chứng minh này, ta lấy một chu trình ba đỉnh làm ví dụ để giải thích vì sao phủ chu trình lẻ sẽ không xuất hiện trong $\det \tilde{A}(G)$.



Hình 3: Chu trình ba đỉnh G

Ta lấy một phủ chu trình lẻ của G để phân tích. Như Hình 3(b), giả sử phủ chu trình ta chọn tương ứng với hoán vị $\pi = (2\ 3\ 1)$. Khi đó đóng góp của hạng tử này vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi)x_{1,2}x_{2,3}(-x_{1,3})$. Bây giờ đảo chiều π , ta thu được Hình 3(c), tương ứng với hoán vị $\pi' = \pi^{-1} = (3\ 1\ 2)$, và đóng góp của nó vào $\det \tilde{A}(G)$ là $\text{sgn}(\pi')(-x_{1,2})(-x_{2,3})x_{1,3}$.

Rõ ràng $\text{sgn}(\pi) = \text{sgn}(\pi')$; đồng thời có thể thấy đóng góp của π' chính là đóng góp của π sau khi nhân biến tương ứng với mỗi cạnh trên chu trình với -1 . Vì vậy hai hạng tử này trái dấu nhau, và tổng đóng góp của chúng là 0.

4.2.3 Ngẫu nhiên hóa

Định lý Tutte, tức Định lý 2.1, cho ta một thuật toán quyết định rất tốt để kiểm tra đồ thị G có matching hoàn hảo hay không: chỉ cần xét $\det \tilde{A}(G)$ có bằng không hay không. Nhưng cách làm này có một vấn đề đáng kể: mỗi hạng tử của ma trận Tutte đều là một biến, và tổng số biến lên tới $|E|$, nên việc tính trực tiếp định thức rất khó, thậm chí cần thời gian mũ.

Tuy nhiên, ta không nhất thiết phải tính ra $\det \tilde{A}(G)$; việc ta cần làm chỉ là phán đoán $\det \tilde{A}(G)$ có đồng nhất bằng 0 hay không.

Quyết định một đa thức có đồng nhất bằng 0 hay không là một bài toán kinh điển. Giả sử hiện tại ta có một đa thức n biến $P(x_1, x_2, \dots, x_n)$, và muốn quyết định nó có đồng nhất bằng 0 hay không. Ta nên làm gì?

Trước hết có thể quan sát rằng nếu $P(x_1, x_2, \dots, x_n) = 0$, thì bất kể ta thay giá trị nào vào đa thức P , kết quả thu được chắc chắn đều là 0. Vậy liệu ta có thể chọn n số ngẫu nhiên $r_1, r_2, \dots, r_n$, rồi dựa vào việc $P(r_1, r_2, \dots, r_n)$ có bằng 0 hay không để suy ra $P(x_1, x_2, \dots, x_n)$ có đồng nhất bằng 0 hay không? Bổ đề Schwartz–Zippel dưới đây cho ta biết cách làm trông có vẻ không đáng tin này thực ra có xác suất cao cho kết quả đúng.

Bổ đề 2.3 (Schwartz, Zippel). Với một đa thức n biến bậc d $P(x_1, x_2, \dots, x_n)$ trên trường F , không đồng nhất bằng 0, giả sử $r_1, r_2, \dots, r_n$ là n số ngẫu nhiên trong F được chọn độc lập, khi đó

$$\Pr[P(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|F|}.$$

Chứng minh. Ta thử chứng minh định lý này bằng quy nạp.

Xét trường hợp một biến: khi đó P có nhiều nhất d nghiệm, nên xác suất ta vừa đúng chọn vào một nghiệm không vượt quá $d/|F|$.

Bây giờ giả sử định lý đúng cho trường hợp $n - 1$ biến. Ta xét cách phân loại mọi hạng tử của P theo bậc của x_1 :

$$P(x_1, x_2, \dots, x_n) = \sum_{i=0}^d x_1^i P_i(x_2, \dots, x_n).$$

Vì $P \neq 0$, tồn tại một i sao cho $P_i \neq 0$. Xét i lớn nhất thỏa điều kiện đó, khi ấy $\deg P_i \leq d - i$.

Theo giả thiết quy nạp,

$$\Pr[P_i(r_2, \dots, r_n) = 0] \leq \frac{d-i}{|F|}.$$

Nếu $P_i(r_2, \dots, r_n) \neq 0$, thì $P(x_1, r_2, \dots, r_n)$ là một đa thức một biến bậc i , nên

$$\Pr[P(r_1, r_2, \dots, r_n) = 0 \mid P_i(r_2, \dots, r_n) \neq 0] \leq \frac{i}{|F|}.$$

Đặt sự kiện $P(r_1, r_2, \dots, r_n) = 0$ là A , và sự kiện $P_i(r_2, \dots, r_n) = 0$ là B . Khi đó

$$\begin{aligned} \Pr[A] &= \Pr[A \cap B] + \Pr[A \cap B^c] \\ &= \Pr[B] \Pr[A \mid B] + \Pr[B^c] \Pr[A \mid B^c] \\ &\leq \Pr[B] + \Pr[A \mid B^c] \\ &\leq \frac{d-i}{|F|} + \frac{i}{|F|} = \frac{d}{|F|}. \end{aligned}$$

■

Ta biết rằng với một đồ thị G cho trước, bậc của đa thức $\det \tilde{A}(G)$ là cấp $\mathcal{O}(n)$. Vì vậy ta có thể chọn một số nguyên tố p cấp n^2 , rồi trên trường modulo p , tức F_p , gán ngẫu nhiên một giá trị cho biến $x_{i,j}$ tương ứng với mỗi cạnh $v_i v_j$. Sau đó ta kiểm tra⁵ $\det \tilde{A}(G)$ có bằng 0 hay không để phán đoán G có matching hoàn hảo hay không. Theo bổ đề Schwartz–Zippel, xác suất phán đoán sai không vượt quá $\frac{n}{p} = \mathcal{O}(1/n)$; xác suất này chấp nhận được đối với chúng ta. Hơn nữa, ta có thể giảm xác suất sai bằng cách điều chỉnh kích thước của p , hoặc ngẫu nhiên nhiều lần.

Từ đó, ta có thuật toán quyết định sau.

Thuật toán 1 Lovasz’s testing algorithm

```

1: if  $\det \tilde{A}(G) \neq 0$  then
2:   print “YES”
3: else
4:   print “NO”
5: end if
```

Định lý 2.4. Độ phức tạp thời gian của Thuật toán 1 là $\mathcal{O}(n^\omega)$.

4.3 Xây dựng phương án matching hoàn hảo

4.3.1 Một thuật toán vét cạn

Theo Thuật toán 1, hiện tại ta đã có thể quyết định đồ thị G có matching hoàn hảo hay không. Từ đó ta lập tức thu được một thuật toán vét cạn để xây dựng matching: duyệt từng cạnh, xóa hai đầu mút của nó, rồi kiểm tra đồ thị còn lại có matching hoàn hảo hay không.

⁵Từ đây trở đi, khi cần tính định thức, ma trận nghịch đảo, hoặc khử ma trận $\tilde{A}(G)$, ta đều hiểu là sau khi đã gán ngẫu nhiên các biến thì thực hiện phép toán tương ứng trên F_p .

Thuật toán 2 A naive matching algorithm

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:     if  $v_i v_j \in E(G)$  and  $\det \tilde{A}(G - \{v_i, v_j\}) \neq 0$  then
5:        $G \leftarrow G - \{v_i, v_j\}$ 
6:        $M \leftarrow M \cup \{v_i v_j\}$ 
7:     end if
8:   end for
9: end for
10: return  $M = 0$ 

```

Thuật toán 2 cần thực hiện $\mathcal{O}(n^2)$ vòng lặp; trong mỗi vòng lặp cần tính định thức của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.1. Độ phức tạp thời gian của Thuật toán 2 là $\mathcal{O}(n^{\omega+2})$.

4.3.2 Một số kiến thức đại số tuyến tính

Nút thắt của Thuật toán 2 nằm ở việc cần phán đoán mỗi cạnh có thể nằm trong matching hoàn hảo hay không. Nếu ta có thể tăng tốc quá trình phán đoán này, ta sẽ giảm được độ phức tạp của thuật toán.

Để đạt mục tiêu đó, trước hết ta cần biết một số kiến thức đại số tuyến tính.

Định nghĩa 3.1. Phần phụ của ma trận A theo hàng i và cột j , ký hiệu $M_{i,j}$, là $\det A^{(i),\{j\}}$.

Định nghĩa 3.2. Đại số phụ của ma trận A theo hàng i và cột j , ký hiệu $C_{i,j}$, là $(-1)^{i+j} M_{i,j}$.

Ma trận các đại số phụ của A là một ma trận $n \times n$ C , thỏa mãn phần tử ở hàng i , cột j của nó là đại số phụ của A theo hàng i và cột j .

Định nghĩa 3.3. Ma trận phụ hợp $\text{adj } A$ của ma trận A là chuyển vị của ma trận các đại số phụ của A , tức $\text{adj } A = C^T$.

Định lý 3.2 (Khai triển Laplace). Giả sử A là một ma trận $n \times n$. Khi đó với một hàng bất kỳ $i \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{j=1}^n A_{i,j} C_{i,j} = \sum_{j=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Tương tự, với một cột bất kỳ $j \in \{1, \dots, n\}$, ta có

$$\det A = \sum_{i=1}^n A_{i,j} (\text{adj } A)_{j,i}.$$

Định lý 3.3. Nếu ma trận A khả nghịch, thì

$$A^{-1} = \text{adj } A / \det A.$$

Chứng minh. Theo Định nghĩa 3.2 và Định nghĩa 3.3, hệ số ở hàng i , cột i của $A \times \text{adj } A$ là $\sum_{j=1}^n A_{i,j} C_{i,j}$. Theo Định lý 3.2, hệ số này bằng $\det A$.

Nếu $i \neq j$, thì hệ số ở hàng i , cột j của $A \times \text{adj } A$ là $\sum_{k=1}^n A_{i,k} C_{j,k}$. Thực ra, điều này tương đương với việc thay các phần tử hàng j của A bằng các phần tử hàng i , rồi lấy định thức. Vì có hai hàng giống nhau, định thức bằng 0.

Vì vậy ta có

$$A \times \text{adj } A = \text{adj } A \times A = \det A \times I_n,$$

tức là

$$A^{-1} = \text{adj } A / \det A.$$

■

4.3.3 Tính chất của ma trận phản đối xứng

Trước hết ta định nghĩa khái niệm ma trận phản đối xứng.

Định nghĩa 3.4. Với một ma trận $n \times n$ A , nếu $A^T = -A$, tức $A_{i,j} = -A_{j,i}$, thì ta gọi A là một ma trận phản đối xứng.

Ma trận Tutte trong Định nghĩa 2.1 chính là một ma trận phản đối xứng. Có thể thấy các thuật toán phía trước của ta đều xoay quanh việc tính định thức của ma trận Tutte; đây chính là lý do ta nghiên cứu tính chất của ma trận phản đối xứng.

Bổ đề 3.4. Với một ma trận phản đối xứng $n \times n$ A , nếu n lẻ thì $\det A = 0$.

Chứng minh. Từ kiến thức về định thức, ta biết $\det A = \det A^T$. Lại vì $A^T = -A$, nên $\det A = \det(-A) = (-1)^n \det A$.

Khi n lẻ, $(-1)^n = -1$, tức $\det A = -\det A$, do đó $\det A = 0$.

■

Định lý 3.5. Với một ma trận phản đối xứng $n \times n$ A và một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$, nếu $A_{i_1, \cdot}, A_{i_2, \cdot}, \dots, A_{i_k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , thì $\det A_{I,I} \neq 0$.

Chứng minh. Không mất tính tổng quát, giả sử $I = \{1, 2, \dots, k\}$.

Vì $A_{1, \cdot}, A_{2, \cdot}, \dots, A_{k, \cdot}$ là một nhóm độc lập tuyến tính cực đại theo hàng của A , từ tính phản đối xứng của A , ta có $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$ là một nhóm độc lập tuyến tính cực đại theo cột của A .

Do đó với mọi $k < i \leq n$, $A_{\cdot, i}$ đều có thể được biểu diễn tuyến tính bởi $A_{\cdot, 1}, A_{\cdot, 2}, \dots, A_{\cdot, k}$. Vì vậy

$$\text{rank } A_{I,I} = \text{rank } A_{I, \cdot} = k,$$

tức $\det A_{I,I} \neq 0$.

■

Hệ quả 3.6. Với một ma trận phản đối xứng A , tồn tại một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ sao cho

$$\text{rank } A = \text{rank } A_{I,I} = k.$$

Định lý 3.7. Hạng của một ma trận phản đối xứng là số chẵn.

Chứng minh. Theo Hệ quả 3.6, một ma trận phản đối xứng tồn tại một định thức con chính đầy hạng có hạng bằng hạng của chính nó. Vì định thức con chính của ma trận phản đối xứng cũng là phản đối xứng, kết hợp với Bổ đề 3.4 ta biết hạng của định thức con chính này là số chẵn. Do đó hạng của một ma trận phản đối xứng là số chẵn. ■

Bổ đề 3.8. Với một ma trận phản đối xứng khả nghịch $n \times n$ A , và hai chỉ số bất kỳ $i, j \in \{1, \dots, n\}, i \neq j$, ta có

$$\det A^{(i),\{j\}} \neq 0 \iff \det A^{\{i,j\},\{i,j\}} \neq 0.$$

Chứng minh. Nếu $\det A^{(i),\{j\}} \neq 0$, thì $\text{rank} A^{(i),\{j\}} = n - 1$. Vì $A^{\{i,j\},\{i,j\}}$ thu được từ $A^{(i),\{j\}}$ bằng cách xóa thêm một hàng và một cột, nên $\text{rank} A^{\{i,j\},\{i,j\}} \geq n - 3$. Lại vì $A^{\{i,j\},\{i,j\}}$ là ma trận phản đối xứng, theo Định lý 3.7, hạng của nó là số chẵn, nên $\text{rank} A^{\{i,j\},\{i,j\}}$ nhất định bằng $n - 2$. Do đó $\det A^{\{i,j\},\{i,j\}} \neq 0$.

Ngược lại, nếu $\det A^{\{i,j\},\{i,j\}} \neq 0$, thì $\text{rank} A^{\{i,j\},\{i,j\}} = n - 2$; đồng thời $\text{rank} A^{(i),\{j\}} = n - 2$. Theo Định lý 3.7,

$$\text{rank} A^{(i),\{i,j\}} = \text{rank} A^{(i),\{i\}} = n - 2.$$

Điều này cho thấy cột thứ j của $A^{(i),\emptyset}$ là tổ hợp tuyến tính của các cột khác. Vì vậy $\text{rank} A^{(i),\{j\}} = \text{rank} A^{(i),\emptyset} = n - 1$, suy ra $\det A^{(i),\{j\}} \neq 0$. ■

4.3.4 Thuật toán Rabin–Vazirani

Bây giờ ta thử thu được một thuật toán xây dựng phương án matching tốt hơn Thuật toán 2.

Định lý 3.9 (Rabin, Vazirani). Giả sử $G = (V, E)$ là một đồ thị có matching hoàn hảo, và $\tilde{A} = \tilde{A}(G)$ là ma trận Tutte tương ứng của G . Khi đó

$$(\tilde{A}^{-1})_{i,j} \neq 0 \iff G - \{v_i, v_j\} \text{ có matching hoàn hảo.}$$

Chứng minh. Định lý này có thể được suy ra từ Định lý 3.3, Bổ đề 3.8 và định lý Tutte, tức Định lý 2.1. ■

Dựa trên Định lý 3.9, ta thu được thuật toán sau.

Thuật toán 3 Matching algorithm of Rabin and Vazirani

```

1:  $M \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   if  $v_i$  is not yet matched then
4:     compute  $\tilde{A}(G)^{-1}$ 
5:     find  $j$ , such that  $v_i v_j \in E(G)$  and  $(\tilde{A}(G)^{-1})_{j,i} \neq 0$ 
6:      $G \leftarrow G - \{v_i, v_j\}$ 
7:      $M \leftarrow M \cup \{v_i v_j\}$ 
8:   end if
9: end for
10: return  $M = 0$ 
```

Thuật toán 3 cần thực hiện tổng cộng $\mathcal{O}(n)$ vòng lặp; trong mỗi vòng lặp cần tìm nghịch đảo của một ma trận vuông cấp $\mathcal{O}(n)$. Do đó:

Định lý 3.10. Độ phức tạp của Thuật toán 3 là $\mathcal{O}(n^{\omega+1})$.

4.3.5 Phương pháp xây dựng dựa trên khử Gauss

Nút thắt của Thuật toán 3 nằm ở việc tính nghịch đảo của ma trận Tutte $\tilde{A}(G)$. Thực ra, trong mỗi vòng lặp ta chỉ xóa hai hàng và hai cột của $\tilde{A}(G)$, nhưng lại cần tính lại ma trận nghịch đảo từ đầu. Nếu mỗi lần không cần tính lại nghịch đảo từ đầu, mà có thể dùng một phương pháp hiệu quả để duy trì ma trận nghịch đảo, ta sẽ giảm được độ phức tạp của thuật toán này.

Thuật toán của ta dựa trên định lý sau.

Định lý 3.11. Nếu

$$A = \begin{pmatrix} a_{1,1} & v^T \\ u & B \end{pmatrix}, \quad A^{-1} = \begin{pmatrix} \hat{a}_{1,1} & \hat{v}^T \\ \hat{u} & \hat{B} \end{pmatrix},$$

trong đó $\hat{a}_{1,1} \neq 0$, thì

$$B^{-1} = \hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}.$$

Chứng minh. Vì $AA^{-1} = I$, ta có

$$\begin{pmatrix} a_{1,1}\hat{a}_{1,1} + v^T\hat{u} & a_{1,1}\hat{v}^T + v^T\hat{B} \\ u\hat{a}_{1,1} + B\hat{u} & u\hat{v}^T + B\hat{B} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & I_{n-1} \end{pmatrix}.$$

Từ đó,

$$\begin{aligned} B(\hat{B} - \hat{u}\hat{v}^T/\hat{a}_{1,1}) &= I_{n-1} - u\hat{v}^T - B\hat{u}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{a}_{1,1}\hat{v}^T/\hat{a}_{1,1} \\ &= I_{n-1} - u\hat{v}^T + u\hat{v}^T \\ &= I_{n-1}. \end{aligned}$$

■

Chú ý rằng trong định lý trên, thao tác sửa $\hat{B}$ đúng là một thao tác khử trong khử Gauss. Trong định lý, ta chứng minh với ví dụ khử hàng thứ nhất và cột thứ nhất.

Là một ứng dụng trực tiếp của Định lý 3.11, ta thu được thuật toán sau.

Thuật toán 4 Simple matching algorithm

- 1: $M \leftarrow \emptyset$
- 2: compute $\tilde{A}(G)^{-1}$
- 3: **for** $i \leftarrow 1$ **to** n **do**
- 4: **if** the i -th row is not yet eliminated **then**
- 5: find j such that $v_i v_j \in E(G)$ and $(\tilde{A}(G)^{-1})_{j,i} \neq 0$
- 6: $G \leftarrow G - \{v_i, v_j\}$
- 7: $M \leftarrow M \cup \{v_i v_j\}$
- 8: update $\tilde{A}(G)^{-1}$ by eliminating the i -th row and the j -th column and then the j -th row and the i -th column
- 9: **end if**
- 10: **end for**
- 11: **return** $M = 0$

Thuật toán 4 cần lặp $\mathcal{O}(n)$ lần; trong mỗi vòng lặp cần thực hiện hai thao tác khử. Độ phức tạp thời gian của một thao tác khử là $\mathcal{O}(n^2)$. Vì vậy:

Định lý 3.12. Độ phức tạp thời gian của Thuật toán 4 là $\mathcal{O}(n^3)$.

4.4 Matching cực đại

4.4.1 Kích thước của matching cực đại

Đến đây, mọi thuật toán của ta đều dùng để giải matching hoàn hảo, nhưng trong ứng dụng thực tế, bài toán ta cần giải thường là matching cực đại.

Tương tự ý tưởng giải bài toán matching hoàn hảo phía trước, trước hết ta bắt đầu từ bài toán đơn giản nhất: với một đồ thị cho trước $G = (V, E)$, làm thế nào tìm được kích thước matching cực đại $\nu(G)$ của G ? Với bài toán này, ta có định lý sau.

Định lý 4.1. Kích thước matching cực đại $\nu(G)$ của đồ thị G là

$$\frac{1}{2} \text{rank } \tilde{A}(G).$$

Chứng minh. Đặt $k = \text{rank } \tilde{A}(G)$. Theo Hệ quả 3.6, ta có thể chọn một tập chỉ số hàng $I = \{i_1, i_2, \dots, i_k\}$ của $\tilde{A}(G)$ sao cho

$$\text{rank } \tilde{A}(G) = \text{rank } \tilde{A}(G)_{I,I} = k.$$

Đặt $U = \{v_{i_1}, v_{i_2}, \dots, v_{i_k}\}$. Khi đó $\tilde{A}(G)_{I,I} = \tilde{A}(G[U])$. Vì $\text{rank } \tilde{A}(G[U]) = |U|$, đồ thị $G[U]$ tồn tại matching hoàn hảo; điều này cho thấy $\nu(G) \geq k/2$.

Nếu $\nu(G) > k/2$, ta xét lấy ra mọi đỉnh nằm trong một matching cực đại. Khi đó nhất định tồn tại một tập chỉ số hàng I sao cho $|I| > k$ và $\det \tilde{A}(G)_{I,I} \neq 0$. Điều này có nghĩa là hạng của một ma trận con của $\tilde{A}(G)$ lớn hơn hạng của chính nó, hiển nhiên là không thể.

Tổng hợp lại, $\nu(G) = k/2$. ■

4.4.2 Chuyển hóa thứ nhất

Tiếp theo ta thử chuyển bài toán matching cực đại về bài toán matching hoàn hảo quen thuộc. Nếu với một đồ thị G , ta đã tìm được kích thước matching cực đại $\nu(G)$ của nó, thì ta có thể thêm $n - 2\nu(G)$ đỉnh mới vào G , rồi nối cạnh từ mỗi đỉnh mới đến từng đỉnh trong n đỉnh ban đầu. Đồ thị mới thu được khi đó là một đồ thị có matching hoàn hảo, và matching hoàn hảo của đồ thị mới tương ứng với matching cực đại của đồ thị ban đầu.

Như vậy ta đã giải được bài toán matching cực đại. Chuyển hóa này có tính tổng quát rất tốt, nhưng có một nhược điểm: trong trường hợp xấu nhất, ta có thể phải thêm n đỉnh mới, làm kích thước dữ liệu, tức số đỉnh, tăng gấp đôi, nên hằng số của thuật toán sẽ tăng đáng kể.

4.4.3 Chuyển hóa thứ hai

Ta xét một cách làm khác, cũng chuyển bài toán matching cực đại về bài toán matching hoàn hảo, nhưng lần này thực hiện bằng cách xóa đỉnh. Với một đồ thị $G = (V, E)$ và một matching cực đại M của nó, xét tập đỉnh U gồm mọi đỉnh nằm trong matching M . Rõ ràng $G[U]$ có matching hoàn hảo, và matching hoàn hảo của $G[U]$ chính là matching cực đại của G . Ý tưởng của ta là tìm một tập đỉnh U như vậy, rồi áp dụng thuật toán giải matching hoàn hảo phía trước cho $G[U]$.

Thực ra, quá trình chứng minh Định lý 4.1 đã chỉ đường cho ta. Từ Bổ đề 3.5 và Hệ quả 3.6, ta chỉ cần tìm một nhóm độc lập tuyến tính cực đại theo hàng của $\tilde{A}(G)$. Điều này có thể được tìm bằng một lượt khử Gauss. Vì vậy, ta có thể chuyển bài toán matching cực đại về matching hoàn hảo trong độ phức tạp $\mathcal{O}(n^\omega)$, và nó không làm tăng hằng số của thuật toán.

4.5 Ứng dụng

Thuật toán được trình bày trong bài là thuật toán tổng quát để giải bài toán matching cực đại trên đồ thị tổng quát, nên tất nhiên nó có thể giải mọi bài cần dùng matching tổng quát.

Ngoài ra, nó còn có một số ứng dụng khá đặc sắc. Dưới đây ta xét hai ví dụ.

4.5.1 Vertex Geography⁶

Mô tả bài toán. Alice và Bob chơi một trò chơi trên một đồ thị vô hướng $G = (V, E)$. Hai người luân phiên thao tác, Alice đi trước.

Ban đầu có một quân cờ đặt trên một đỉnh nào đó. Sau đó, trong mỗi lượt, người chơi có thể di chuyển quân cờ này sang một đỉnh kề và chưa từng đi tới trước đó, tính cả điểm xuất phát. Người không thể thao tác sẽ thua. Hỏi những đỉnh nào là đỉnh thắng cho người đi trước?

Giới thiệu thuật toán. Trước hết ta có một kết luận: một đỉnh v_i là trạng thái thắng khi và chỉ khi nó nhất định nằm trong matching cực đại của đồ thị G .⁷

Vì vậy bài toán chuyển thành phán đoán một đỉnh nào đó có nhất định nằm trong matching cực đại của G hay không. Ta làm theo Mục 4.2: thêm $k = n - 2\nu(G)$ đỉnh mới và nối các cạnh tương ứng. Gọi đồ thị mới thu được là G' , các đỉnh mới có chỉ số lần lượt là $n+1, n+2, \dots, n+k$. Sau đó tùy ý chọn một đỉnh mới v_j ($n+1 \leq j \leq n+k$). Khi đó với mỗi đỉnh v_i của đồ thị ban đầu ($1 \leq i \leq n$), nếu $v_i v_j$ có thể xuất hiện trong một matching hoàn hảo của G' , thì v_i không nhất định nằm trong matching cực đại của G .

Theo Định lý 3.9, ta chỉ cần kiểm tra $(\tilde{A}(G')^{-1})_{i,j}$ có bằng không hay không.

Có thể thấy thuật toán matching được giới thiệu trong bài có ưu thế trong các bài toán kiểu “một đỉnh có nhất định nằm trong matching cực đại hay không” và “một cạnh có nhất định nằm trong matching cực đại hay không”. Lý do là thuật toán này tránh được việc phân tích đường tăng, mà chỉ cần kiểm tra một vị trí nào đó của ma trận nghịch đảo có bằng không hay không.

4.5.2 Matching trọng số cực đại

Mô tả bài toán. Matching trọng số cực đại là bài toán trong đó mỗi cạnh $v_i v_j$ của đồ thị G mang một trọng số $w_{i,j}$, và mục tiêu hiện tại của ta là tìm một matching M có tổng trọng số cạnh lớn nhất. Matching cực đại tương đương với matching trọng số cực đại khi mọi trọng số cạnh đều bằng 1.

Giới thiệu thuật toán. Ta có thể nối cạnh trọng số 0 giữa mọi cặp đỉnh; khi n lẻ, thêm một đỉnh mới và nối nó với mọi đỉnh. Vì vậy mỗi matching có trọng số của đồ thị ban đầu sẽ tương ứng với ít nhất một matching hoàn hảo có trọng số của đồ thị mới; và một matching hoàn hảo có trọng số của đồ thị mới tương ứng với một matching có trọng số của đồ thị ban đầu.

Bây giờ ta chuyển thành bài toán tìm matching hoàn hảo có trọng số của đồ thị mới G' . Ta thêm một biến y . Đặt

$$\tilde{A}'(G')_{i,j} = \tilde{A}(G')_{i,j} \times y^{w_{i,j}}.$$

Như vậy, điều ta cần tìm tương đương với bậc cao nhất của y trong $\det \tilde{A}'(G')$; giá trị này chia cho 2 chính là matching trọng số cực đại.

⁶Bài toán kinh điển này có tên tiếng Anh là Undirected Vertex Geography (UVG).

⁷Chứng minh kết luận này không phải trọng tâm của bài viết, nên được lược bỏ ở đây; độc giả quan tâm có thể tham khảo các tài liệu liên quan.

Trước hết, ta gán một giá trị ngẫu nhiên cho mọi $x_{i,j}$. Sau đó giả sử tổng trọng số của mọi cạnh không vượt quá W . Ta có thể lần lượt thay mọi giá trị từ 0 đến W vào y , tính một lần định thức, rồi dùng $W + 1$ kết quả này để nội suy một đa thức theo y . Khi đó bậc cao nhất của đa thức này là thứ ta cần tìm.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(Wn^\omega + W^2)$; khi cả W và n đều nhỏ, nó có giá trị nhất định.

4.6 Tổng kết

Đến đây, ta đã có thể tìm kích thước matching cực đại trên đồ thị tổng quát trong độ phức tạp $\mathcal{O}(n^\omega)$, và xây dựng một phương án matching trong độ phức tạp $\mathcal{O}(n^3)$.

Thuật toán được giới thiệu trong bài vẫn còn tiềm năng rất lớn đang chờ chúng ta khám phá, và các ứng dụng được giới thiệu trong bài còn khá cơ bản. Vì vậy tôi hy vọng bài viết này có thể đóng vai trò gợi mở, mong rằng có độc giả sẽ mở rộng thuật toán trong bài, phát triển nó hơn nữa và thu được những ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
- Cảm ơn thầy Song Xinbo và thầy Zhuo Mingcong đã quan tâm và chỉ dẫn trong nhiều năm.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã chỉ dẫn.
- Cảm ơn bạn Zhou Zixin đã chia sẻ thuật toán này với tôi.
- Cảm ơn các bạn Gao Wenyuan và Wu Hongxun đã đọc duyệt bài viết này.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

- [1] Mucha M, Sankowski P. Maximum matchings via Gaussian elimination[C]//Foundations of Computer Science, 2004. Proceedings. 45th Annual IEEE Symposium on. IEEE, 2004: 248–255.
- [2] Ivan I, Virza M, Yuen H. Algebraic Algorithms for Matching[J]. 2011.
- [3] Cheung H Y. Algebraic algorithms in combinatorial optimization[D]. The Chinese University of Hong Kong, 2011.
- [4] Huang C C, Kavitha T. Efficient algorithms for maximum weight matchings in general graphs with small edge weights[C]//Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, 2012: 1400–1412.
- [5] Chen Yinbo. Bàn sơ về thuật toán matching trên đồ thị và ứng dụng của nó[C]//Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015. 2015.

5 Tính tổng đa thức: báo cáo ra đề

Yuan Yutao, Trường Trung học Yali, thành phố Trường Sa

5.1 Nội dung bài toán

Cho n, m, a, b, c . Định nghĩa $f_0(x) = 1$; với $k > 0$,

$$f_k(x) = \sum_{i=0}^x (ai^2 + bi + c)f_{k-1}(i).$$

Với mọi số nguyên i thỏa mãn $0 \leq i < n$, hãy tính giá trị $f_i(m)$ modulo 1004535809.

5.1.1 Giới hạn dữ liệu

Mã dữ liệu	n	m	a	b	c
0	2000	≤ 2000	0	0	1
1	2000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
2	300	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
3	1000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
4	2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
5	3000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
6	50000	$\leq 10^9$	0	0	$\leq 10^9$
7	250000	$\leq 10^9$	0	0	$\leq 10^9$
8	50000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
9	250000	≤ 2000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
10	50000	≤ 50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
11	250000	≤ 50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
12	50000	$\leq 10^9$	0	1	0
13	50000	$\leq 10^9$	0	$\leq 10^9$	$\leq 10^9$
14	250000	$\leq 10^9$	0	1	0
15	250000	$\leq 10^9$	0	$\leq 10^9$	$\leq 10^9$
16	50000	$\leq 10^9$	1	0	0
17	50000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$
18	250000	$\leq 10^9$	1	0	0
19	250000	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$	$\leq 10^9$

5.2 Giới thiệu thuật toán

5.2.1 Thuật toán một

Quy hoạch động trực tiếp theo định nghĩa.

Độ phức tạp thời gian là $\mathcal{O}(nm)$. Điểm kỳ vọng: 10–15 điểm.

5.2.2 Thuật toán hai

Tổng tiền tố của một đa thức $f(x)$ vẫn là một đa thức, và có thể được tìm trong $\mathcal{O}(n^2)$ hoặc $\mathcal{O}(n \log n)$ thời gian. Ta lần lượt tìm mọi đa thức $f_i(x)$.

Độ phức tạp thời gian là $\mathcal{O}(n^3)$ hoặc $\mathcal{O}(n^2 \log n)$. Dùng đồng thời với thuật toán một, điểm kỳ vọng là 15–30 điểm.

5.2.3 Thuật toán ba

Bậc của đa thức $f_k(x)$ không vượt quá $3k$, nên chỉ cần truy hồi để tìm $f_k(x)$ với $0 \leq x \leq 3k$, rồi nội suy để tìm đáp án.

Độ phức tạp thời gian là $\mathcal{O}(n^2)$. Dùng đồng thời với thuật toán một, điểm kỳ vọng là 30–35 điểm.

5.2.4 Thuật toán bốn

Khi $a = b = 0$, ta có

$$f_k(x) = \binom{x+k}{k} c^k.$$

Có thể tính bằng truy hồi.

Độ phức tạp thời gian là $\mathcal{O}(n)$. Dùng đồng thời với thuật toán ba, điểm kỳ vọng là 40–45 điểm.

5.2.5 Thuật toán năm

Đặt

$$g_k(x) = \sum_{i=0}^{\infty} f_i(k).$$

Từ định nghĩa, khi $k > 0$ ta có

$$g_k(x) = (ak^2 + bk + c)xg_k(x) + g_{k-1}(x),$$

tức là

$$g_k(x) = \frac{g_{k-1}(x)}{1 - (ak^2 + bk + c)x}.$$

Đồng thời

$$g_0(x) = \frac{1}{1 - cx}.$$

Do đó

$$g_k(x) = \prod_{i=0}^k \frac{1}{1 - (ai^2 + bi + c)x}.$$

Tính mẫu số bằng vét cạn hoặc chia để trị, với độ phức tạp thời gian $\mathcal{O}(m^2)$ hoặc $\mathcal{O}(m \log^2 m)$. Sau đó tìm nghịch đảo đa thức để thu được đáp án.

Độ phức tạp thời gian là $\mathcal{O}(m^2 + n \log n)$ hoặc $\mathcal{O}(m \log^2 m + n \log n)$. Điểm kỳ vọng: 45–60 điểm.

5.2.6 Thuật toán sáu

Đặt

$$E(x) = \prod_{i=0}^k (1 - (ai^2 + bi + c)x) = \sum_{i=0}^{k+1} (-1)^{k-i} e_{k-i} x^k.$$

Đặt

$$p_i = \sum_{j=0}^k (aj^2 + bj + c)^i.$$

Từ đồng nhất thức Newton, ta có

$$e_i = \frac{1}{i} \sum_{j=1}^i (-1)^{j-1} e_{i-j} p_j.$$

Đặt

$$P(x) = \sum_{i=0}^{\infty} (-1)^i p_{i+1}.$$

Khi đó thu được

$$E'(x) = E(x)P(x).$$

Có thể dùng FFT chia để trị để tìm $E(x)$: mỗi lần dùng nửa trước của các hệ số chập với $P(x)$ để cập nhật nửa sau của các hệ số.

Tiếp theo xét cách tìm p_i . Ta chia làm hai trường hợp.

Khi $a = 0, b \neq 0$, đặt $u = \frac{c}{b}$. Khi đó

$$p_i = b^i \sum_{j=0}^k (j+u)^i = b^i \left(\sum_{j=0}^{k+u} j^i - \sum_{j=0}^{u-1} j^i \right).$$

Có thể dùng các số Bernoulli để tính.

Khi $a \neq 0$, đặt $u = \frac{b}{2a}, v = \frac{c}{a}$. Khi đó

$$\begin{aligned} p_i &= a^i \sum_{j=0}^k (j^2 + 2uj + v)^i \\ &= a^i \sum_{j=0}^k ((j+u)^2 + (v-u^2))^i \\ &= a^i \sum_{t=0}^i \binom{i}{t} (v-u^2)^{i-t} \sum_{j=0}^k (j+u)^{2t}. \end{aligned}$$

Có thể tìm $\sum_{j=0}^k (j+u)^{2t}$ tương tự trường hợp trước, rồi cộng để thu được đáp án.

Độ phức tạp thời gian là $\mathcal{O}(n \log^2 n)$. Điểm kỳ vọng: 80 điểm.

5.2.7 Thuật toán bảy

Từ $E'(x) = E(x)P(x)$, $E(0) = 1$, ta giải được

$$E(x) = \exp \left(\int_0^x P(t) dt \right).$$

Vì vậy có thể tìm $E(x)$ bằng một lần lấy exp đa thức.

Độ phức tạp thời gian là $\mathcal{O}(n \log n)$. Điểm kỳ vọng: 90–100 điểm.

5.3 Ý tưởng ra đề

Tính tổng tiền tố của một dãy đa thức là một bài toán rất kinh điển. Nếu liên tục tính tổng tiền tố nhiều lần trên một dãy đa thức, ta có thể dùng tổ hợp để tìm công thức tổng quát. Vì vậy, một mở rộng tự nhiên là thực hiện những phép khác giữa hai lần lấy tổng tiền tố; trong bài này, phép đó chính là nhân với một dãy đa thức khác. Vì tôi không tìm được thuật toán hiệu quả để tìm công thức tổng quát, nên đã thay đổi nội dung truy vấn và thu được bài toán này.

5.4 Tổng kết

Bài này chủ yếu kiểm tra mức độ hiểu biết của thí sinh về hàm sinh, khả năng vận dụng các thuật toán liên quan đến đa thức, cũng như các kiến thức toán học như đồng nhất thức Newton và vi tích phân cơ bản.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp cơ hội học tập và trao đổi.

Cảm ơn thầy Wang Xingming và thầy Zhu Quanmin đã dạy dỗ tôi.

Cảm ơn các bạn học đã giúp đỡ tôi.

Tài liệu tham khảo

- [1] Jin Ce. Phép toán hàm sinh và bài toán đếm tổ hợp[C]//Luận văn đội tuyển quốc gia năm 2015. 2015.
- [2] Peng Yuxiang. “Inverse Element of Polynomial”. <http://picks.logdown.com/posts/189620-inverse-element-of-polynomial>
- [3] Peng Yuxiang. “Newton’s Method of Polynomial”. <http://picks.logdown.com/posts/209226-newtons-method-of-polynomial>
- [4] Liu Rujia. *Nhập môn kinh điển về thi thuật toán*. Tsinghua University Press.

6 Bàn về bài toán tập độc lập trong thi tin học

Zhong Zhixian, Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tóm tắt

Tìm thuật toán tốt hơn cho các bài toán NP-hard là một nội dung quan trọng của tin học. Trong đó, các bài toán liên quan đến tập độc lập khá thường gặp và cũng nhiều lần xuất hiện trong thi tin học. Bài viết này bắt đầu từ các tính chất của tập độc lập, rồi giới thiệu vài lớp thuật toán tập độc lập dựa trên nhiều tư tưởng khác nhau. Với đồ thị tổng quát, trên cơ sở thuật toán tìm kiếm, bài viết đề xuất thuật toán tối ưu bằng quy hoạch động, đồng thời phân tích và kiểm thử hiệu suất của vài thuật toán trên dữ liệu ngẫu nhiên. Với đồ thị đặc biệt, bài viết giới thiệu hai tư tưởng thuật toán cho đồ thị đặc biệt, đề xuất tư tưởng giải bài toán tập độc lập trên “đồ thị k -xương rồng” và các ứng dụng mở rộng của nó.

6.1 Lời nói đầu

Không ít bài toán liên quan đến tập độc lập, như tập độc lập lớn nhất, tập độc lập có trọng số lớn nhất, đếm tập độc lập, đều là các bài toán NP-hard kinh điển trong lý thuyết đồ thị và xuất hiện rộng rãi trong thi tin học. Tuy nhiên hiệu suất của các thuật toán giải bài toán tập độc lập thường không cao. Vì vậy, tôi đã nghiên cứu sâu hơn lớp bài toán này, với hy vọng dùng được phương pháp hiệu quả hơn để giải chúng.

Nghiên cứu trong bài viết này chia làm hai phần. Phần thứ nhất giới thiệu hai thuật toán giải bài toán tập độc lập trên đồ thị tổng quát: thuật toán tập độc lập dựa trên tìm kiếm tập độc lập cực đại và thuật toán tập độc lập dựa trên quy hoạch động. Phần thứ hai giới thiệu thuật toán tập độc lập cho hai loại đồ thị đặc biệt, lần lượt dựa trên tư tưởng matching đồ thị và tư tưởng chia giai đoạn trên đồ thị.

6.2 Định nghĩa và quy ước

Bài toán tập độc lập có nhiều dạng. Để tiện mô tả, dưới đây đưa ra một số định nghĩa.

Định nghĩa 2.1. Với đồ thị vô hướng $G = (V, E)$ và hai đỉnh $u, v \in V$, nếu $(u, v) \in E$ thì gọi u, v là **kề nhau** (adjacent). **Lân cận** (neighborhood) của đỉnh $v \in V$ được định nghĩa là tập các đỉnh trong V kề với v , ký hiệu $N(v)$. Ngoài ra, $N_G(v)$ biểu thị lân cận của v trong đồ thị G .

Định nghĩa 2.2. **Bậc** (degree) của đỉnh v , ký hiệu $\deg(v)$, được định nghĩa là kích thước của $N(v)$, tức $\deg(v) = |N(v)|$. Ngoài ra, $\deg_G(v)$ biểu thị bậc của v trong đồ thị G .

Định nghĩa 2.3. Một **tập độc lập** (independent set) của đồ thị vô hướng $G = (V, E)$ được định nghĩa là một tập con của V sao cho mọi cặp đỉnh trong tập con đó đều không kề nhau. Nói hình thức, I là một tập độc lập của G khi và chỉ khi $I \subseteq V$ và với mọi $u, v \in I$, $(u, v) \notin E$.

Định nghĩa 2.4. Một **tập độc lập lớn nhất** (maximum independent set) của đồ thị vô hướng $G = (V, E)$ là một tập độc lập I trong G có số đỉnh $|I|$ lớn nhất.

Định nghĩa 2.5. **Số độc lập** (independence number) của đồ thị vô hướng $G = (V, E)$ được định nghĩa là số đỉnh $|I|$ của một tập độc lập lớn nhất I của G , ký hiệu $\alpha(G)$.

Định nghĩa 2.6. **Đồ thị con cảm sinh** (induced subgraph)⁸ của đồ thị vô hướng $G = (V, E)$ trên $S \subseteq V$ được định nghĩa là đồ thị có tập đỉnh là S và tập cạnh gồm các cạnh có cả hai đầu mút nằm trong S , ký hiệu $G[S]$.

Mọi bài toán trong bài này đều lấy bài toán tập độc lập lớn nhất làm ví dụ: với đồ thị vô hướng $G = (V, E)$ cho trước, hãy tìm một tập độc lập lớn nhất I của G .

Để tiện, quy ước n là bậc của đồ thị G (tức số đỉnh), m là số cạnh của G , tức $n = |V|$, $m = |E|$.

6.3 Bài toán tập độc lập trên đồ thị tổng quát

Hiện nay, đa số bài toán liên quan đến tập độc lập trên đồ thị tổng quát không có thuật toán thời gian đa thức, mà chỉ có thể dùng các thuật toán mũ với độ phức tạp tương đối tốt.

Thực ra đã có không ít thuật toán tìm tập độc lập lớn nhất của đồ thị với độ phức tạp lý thuyết rất tốt, có thể nhanh chóng tính tập độc lập lớn nhất của đồ thị vô hướng hàng trăm đỉnh⁹. Nhưng các thuật toán ấy thường quá phức tạp khi cài đặt, khó áp dụng trong thi tin học. Tác giả chọn nghiên cứu một số thuật toán tương đối hiệu quả và dễ cài đặt hơn.

6.3.1 Thuật toán tập độc lập dựa trên tìm kiếm tập độc lập cực đại

Thuật toán tìm kiếm ngây thơ. Thuật toán tìm kiếm ngây thơ nhất rất đơn giản: dùng tìm kiếm theo chiều sâu để liệt kê các tập con $I \subseteq V$, tức theo một thứ tự nào đó liệt kê mỗi đỉnh $v \in V$ có thuộc I hay không; hề tồn tại $(u, v) \in E$ sao cho $u, v \in I$ thì quay lui. Trong tất cả các tập độc lập I được liệt kê, in ra một tập có $|I|$ lớn nhất. Độ phức tạp của thuật toán là $\mathcal{O}(2^n)$, quá kém hiệu quả.

Riêng với bài toán tập độc lập lớn nhất, có thể thêm vài phép cắt nhánh. Ví dụ:

1. Nếu $\deg(v) = 0$, không có cạnh nào liên thuộc với v , nên luôn có thể cho $v \in I$.
2. Nếu $\deg(v) = 1$, xét đỉnh u duy nhất liên thuộc với v . Nếu $u \notin I$, ta luôn có thể cho $v \in I$; ngược lại, xóa u khỏi I và thêm v vào, kích thước của I không đổi. Vì vậy luôn có thể cho $v \in I$.
3. Khi tìm kiếm, ghi lại giá trị lớn nhất a của kích thước tập độc lập hiện đã tìm được. Ký hiệu P là tập các đỉnh trong $V - I$ không kề với đỉnh nào của I . Khi $|I| + |P| \leq a$, có thể cắt nhánh theo tính tối ưu.

Tuy nhiên sau khi thêm các phép cắt nhánh này, độ phức tạp vẫn còn rất cao và khó chấp nhận.

Tập độc lập cực đại và thuật toán Bron–Kerbosch. Thuật toán tìm kiếm ngây thơ quá kém hiệu quả. Có cách nào tốt để tối ưu không? Ta đưa ra khái niệm tập độc lập cực đại:

⁸Đồ thị con cảm sinh có hai loại: cảm sinh bởi đỉnh và cảm sinh bởi cạnh. Trong bài này đều chỉ loại thứ nhất.

⁹Năm 2001, Robson đề xuất một thuật toán tìm tập độc lập lớn nhất có độ phức tạp xấp xỉ $\mathcal{O}(1.1888^n)$, xem Tài liệu tham khảo [2].

Định nghĩa 3.1. Một **tập độc lập cực đại** (maximal independent set) của đồ thị vô hướng $G = (V, E)$ là một tập độc lập I của G sao cho với mọi đỉnh $v \in V - I$, tập $I + \{v\}$ không phải là tập độc lập.

Thông thường, số tập độc lập cực đại của một đồ thị ít hơn số tập độc lập rất nhiều. Ví dụ, đường đi gồm 50 đỉnh có 32 951 280 099 tập độc lập, nhưng chỉ có 1 221 537 tập độc lập cực đại. Ngoài ra, không ít bài toán tối ưu tổ hợp liên quan đến tập độc lập có thể chỉ xét tập độc lập cực đại; bài toán tập độc lập lớn nhất là một ví dụ như vậy.

Định lý 3.1. Mọi tập độc lập lớn nhất đều là tập độc lập cực đại.

Chứng minh. Giả sử I là một tập độc lập lớn nhất. Với $v \in V - I$ bất kỳ, nếu $I + \{v\}$ là tập độc lập, thì vì $|I + \{v\}| = |I| + 1 > |I|$, $I + \{v\}$ là một tập độc lập lớn hơn I . Nói cách khác, I không phải là tập độc lập lớn nhất, mâu thuẫn với giả thiết. Vì vậy I nhất định là tập độc lập cực đại. ■

Do đó, nếu tìm được mọi tập độc lập cực đại của G , ta sẽ tìm được tập độc lập lớn nhất của G .

Có nhiều thuật toán tìm tập độc lập cực đại; trong đó thuật toán Bron–Kerbosch là một thuật toán cài đặt tương đối gọn. Dưới đây giới thiệu thuật toán Bron–Kerbosch để tìm tập độc lập cực đại.

Thuật toán Bron–Kerbosch có thể tìm mọi tập độc lập cực đại của đồ thị vô hướng bất kỳ G . Tư tưởng cơ bản của nó vẫn là tối ưu tìm kiếm. Mã giả như sau¹⁰:

Thuật toán 1 BronKerbosch(R, P, X)

```

1: if  $P = X = \emptyset$  then
2:   print  $R$ 
3: end if
4: Chọn đỉnh  $u \in P \cup X$  sao cho  $|P \cap (\{u\} \cup N(u))|$  nhỏ nhất
5: for all  $v \in P \cap (\{u\} \cup N(u))$  do
6:   BronKerbosch( $R \cup \{v\}, P - (\{v\} \cup N(v)), X - (\{v\} \cup N(v))$ )
7:    $P \leftarrow P - \{v\}$ 
8:    $X \leftarrow X \cup \{v\}$ 
9: end for
```

Gọi hàm BronKerbosch(R, P, X) sẽ in ra mọi tập độc lập cực đại chứa tất cả các đỉnh trong R , chứa tùy ý nhiều đỉnh trong P , và không chứa đỉnh nào trong X . Gọi BronKerbosch($\emptyset, V, \emptyset$) sẽ thu được mọi tập độc lập cực đại của G .

Khi cài đặt, tập hợp có thể được lưu bằng nén bit: dùng một mảng A gồm $\lceil n/64 \rceil$ số nguyên 64 bit để lưu một tập A' kích thước n , trong đó $x \in A'$ khi và chỉ khi

$$A[\lfloor x/64 \rfloor] \text{ AND } 2^{x \bmod 64} > 0$$

(chỉ số mảng A bắt đầu từ 0). Vì trong bài toán tập độc lập, bậc n của đồ thị sẽ không quá lớn, kích thước mảng nén bit có thể xem xấp xỉ như một hằng số.

Điểm mấu chốt nhất của thuật toán trên là pivoting. Trong quá trình thuật toán có một bước chọn đỉnh $u \in P \cup X$ sao cho $|P \cap (\{u\} \cup N(u))|$ nhỏ nhất; u được gọi là đỉnh pivot. Sau đó liệt kê đỉnh đầu tiên v trong $\{u\} \cup N(u)$ thuộc tập độc lập R . Đó là quá trình pivoting.

Vì sao phải làm như vậy?

Nếu trực tiếp tìm kiếm tập độc lập cực đại, hiệu suất sẽ rất thấp vì thuật toán sẽ duyệt qua rất nhiều tập không cực đại. Chẳng hạn khi G là đồ thị không n đỉnh¹¹, hiển nhiên V là tập độc lập cực đại duy nhất của G . Nhưng khi tìm kiếm ngây thơ liệt kê một đỉnh nào đó không thuộc tập độc lập cực đại, dù

¹⁰Thuật toán Bron–Kerbosch có nhiều cách cài đặt; bài này giới thiệu một trong số đó.

¹¹Đồ thị không được định nghĩa là đồ thị không có cạnh, tức $G = (V, E)$ là đồ thị không khi và chỉ khi $E = \emptyset$.

không thể tìm ra tập độc lập cực đại, thuật toán vẫn sẽ tiếp tục tìm kiếm, lãng phí rất nhiều thời gian. Tính đúng đắn của pivoting dựa trên định lý sau:

Định lý 3.2. Với đồ thị vô hướng $G = (V, E)$ và $v \in V$, mọi tập độc lập cực đại I của G đều thỏa $I \cap (\{v\} \cup N(v)) \neq \emptyset$.

Chứng minh. Giả sử tồn tại tập độc lập cực đại I thỏa $I \cap (\{v\} \cup N(v)) = \emptyset$. Khi đó với mọi $u \in I$, $u \notin \{v\} \cup N(v)$, tức $u \neq v$ và $(u, v) \notin E$.

Vì vậy $I \cup \{v\}$ cũng là một tập độc lập, và $I \subset I \cup \{v\}$. Điều này nói rằng I không cực đại, mâu thuẫn.

■

Như vậy ta đã chứng minh tính đúng đắn của định lý. Dù cách này vẫn sẽ duyệt qua một số tập không cực đại, các tập như thế rõ ràng đã ít hơn rất nhiều.

Số lượng tập độc lập cực đại. Trước đó ta chỉ biết trực giác rằng số tập độc lập cực đại tương đối ít. Ở đây ta đưa ra cận trên cho số lần gọi đệ quy của thuật toán Bron–Kerbosch:

Định lý 3.3. Số lần gọi đệ quy của thuật toán Bron–Kerbosch là $\mathcal{O}(3^{n/3})$.

Chứng minh của định lý này khá phức tạp, bài viết lược bỏ¹².

Từ đó có thể thu được hệ quả:

Định lý 3.4. Số tập độc lập cực đại của đồ thị vô hướng n đỉnh là $\mathcal{O}(3^{n/3})$.

Cận trên này rất dễ đạt tới: chỉ cần dựng $\lfloor n/3 \rfloor$ tam giác độc lập lẫn nhau. Nhưng khi đồ thị được sinh ngẫu nhiên, cận trên này còn rất lỏng. Để minh họa điểm đó, tác giả đã nghiên cứu số tập độc lập cực đại của đồ thị ngẫu nhiên.

Sinh ngẫu nhiên một đồ thị $G = (V, E)$ trong các đồ thị vô hướng đơn n đỉnh có m cạnh. Ký hiệu x là số tập độc lập cực đại của G , tức

$$x = \sum_{S \subseteq V} [S \text{ là tập độc lập cực đại}].$$

Xét cách tính kỳ vọng $\mathbb{E}(x)$. Điều kiện để S là tập độc lập cực đại của G là:

- S là tập độc lập, tức với mọi $u, v \in S$, $(u, v) \notin E$;
- với mọi $v \in V - S$, $S \cup \{v\}$ không phải là tập độc lập, tức mỗi đỉnh trong $V - S$ kề với ít nhất một đỉnh trong S .

Dùng nguyên lý bao hàm–loại trừ, liệt kê k đỉnh trong $V - S$ không kề với đỉnh nào của S . Ký hiệu $i = |S|$. Khi đó $n - i - k$ đỉnh còn lại có thể nối cạnh với các đỉnh trong S , và hai đỉnh bất kỳ trong $V - S$ (tổng cộng $\frac{1}{2}(n - i)(n - i - 1)$ cặp đỉnh) cũng có thể nối cạnh. Vậy số đồ thị G thỏa S là tập độc lập cực đại là

$$\sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m}.$$

¹²Bạn đọc có hứng thú có thể đọc Tài liệu tham khảo [3].

Do số đồ thị đơn n đỉnh có m cạnh là $\binom{n(n-1)/2}{m}$, ta có

$$\begin{aligned}\mathbb{E}(x) &= \sum_{S \subseteq V} P([S \text{ là tập độc lập cực đại}]) \\ &= \sum_{i=0}^n \sum_{\substack{S \subseteq V \\ |S|=i}} \binom{n(n-1)/2}{m}^{-1} \sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m} \\ &= \binom{n(n-1)/2}{m}^{-1} \sum_{i=0}^n \binom{n}{i} \sum_{k=0}^{n-i} (-1)^k \binom{n-i}{k} \binom{(n-i-k)i + \frac{(n-i)(n-i-1)}{2}}{m}.\end{aligned}$$

Dưới đây là một số kết quả tính toán, đã làm tròn:

$\mathbb{E}(x)$	$m = n$	$m = \lfloor \sqrt{3}n \rfloor$	$m = 2n$	$m = 3n$	$m = n^2/4$
$n = 20$	84	101	99	81	49
$n = 30$	706	933	909	691	157
$n = 40$	5.95×10^3	8.67×10^3	8.40×10^3	5.88×10^3	403
$n = 50$	5.02×10^4	8.07×10^4	7.76×10^4	5.01×10^4	891
$n = 60$	4.23×10^5	7.51×10^5	7.18×10^5	4.27×10^5	1779
$n = 70$	3.57×10^6	6.99×10^6	6.64×10^6	3.63×10^6	3291
$n = 80$	3.01×10^7	6.51×10^7	6.14×10^7	3.10×10^7	5730
$n = 90$	2.54×10^8	6.06×10^8	5.68×10^8	2.64×10^8	9506
$n = 100$	2.15×10^9	5.64×10^9	5.25×10^9	2.25×10^9	15154

Có thể thấy trong trường hợp ngẫu nhiên, số tập độc lập cực đại nhỏ hơn $3^{n/3}$ rất nhiều. Ngoài ra, khi m gần $\sqrt{3}n$, kỳ vọng $\mathbb{E}(x)$ của số tập độc lập cực đại x là lớn nhất; còn với đồ thị quá dày, số tập độc lập lại khá ít.

Theo kết luận này còn có thể suy ra xác suất $x \geq k \cdot \mathbb{E}(x)$ không vượt quá $1/k$, nên trong đa số trường hợp, số tập độc lập cực đại của đồ thị ngẫu nhiên sẽ không lớn hơn kỳ vọng quá nhiều¹³.

Cần chú ý rằng độ phức tạp của thuật toán Bron–Kerbosch không liên quan trực tiếp đến số tập độc lập cực đại, nên dùng kỳ vọng số tập độc lập cực đại để phân tích thời gian chạy kỳ vọng của thuật toán Bron–Kerbosch là không chính xác. Thực tế có những thuật toán tìm kiếm tập độc lập cực đại có độ phức tạp liên quan trực tiếp đến số tập độc lập cực đại, nhưng chúng vượt ra ngoài phạm vi bài này; bạn đọc có hứng thú có thể tự tìm hiểu.

Ứng dụng. Sau khi giới thiệu tính chất và thuật toán của tập độc lập cực đại, ta hãy xem nó có những ứng dụng nào.

Ví dụ 1 (Bài toán tô màu 3 cho đồ thị). Cho đồ thị vô hướng đơn $G = (V, E)$ bậc n . Dùng ba màu tô các đỉnh trong V sao cho hai đầu mút u, v của mỗi cạnh $(u, v) \in E$ có màu khác nhau. $n \leq 40$.

Bài toán tô màu 3 cho đồ thị cũng là một bài toán NP-hard nổi tiếng. Thuật toán ngây thơ mỗi lần liệt kê màu của một đỉnh kề với đỉnh đã xác định màu, cần liệt kê $\mathcal{O}(2^n)$ trường hợp, không thể qua dữ liệu $n = 40$. Làm thế nào để giải hiệu quả hơn?

Trước hết đưa ra một định lý:

Định lý 3.5. Đồ thị vô hướng $G = (V, E)$ tô được bằng 3 màu khi và chỉ khi G tồn tại một tập độc lập cực đại I sao cho đồ thị $G - I$ là đồ thị hai phía¹⁴.

¹³Vì tác giả chưa nghiên cứu phương sai của x , không thể thu được mô tả chính xác hơn về phân bố của x .

¹⁴Đồ thị vô hướng $G = (V, E)$ là đồ thị hai phía (bipartite graph) nếu có thể chia V thành hai tập S và $V - S$ sao cho hai đầu mút của mỗi cạnh không nằm trong cùng một tập, tức với mọi $(u, v) \in E$, $u \in S, v \in V - S$ hoặc $u \in V - S, v \in S$.

Chứng minh. (Tính đủ) Giả sử I là một tập độc lập cực đại của G và $G - I$ là đồ thị hai phía. Theo tính chất của đồ thị hai phía, tồn tại tập đỉnh $X \subseteq V - I$. Ký hiệu $Y = V - I - X$, sao cho với mọi $u, v \in X$ hoặc $u, v \in Y$, ta có $(u, v) \notin E$.

Vì I là tập độc lập của G , nên với mọi $u, v \in I$, ta có $(u, v) \notin E$.

Do đó tô các đỉnh trong I bằng màu 1, các đỉnh trong X bằng màu 2, các đỉnh trong Y bằng màu 3 là một phương án hợp lệ.

(Tính cần) Giả sử $G = (V, E)$ tô được bằng 3 màu. Ký hiệu X, Y, Z lần lượt là tập các đỉnh được tô màu 1, 2, 3.

Theo định nghĩa, với mọi $(u, v) \in E$, hai đỉnh u, v không thuộc cùng một tập trong ba tập này, nên X, Y, Z đều là tập độc lập.

Nếu X không phải là tập độc lập cực đại, thì tồn tại $v \in V - X$ sao cho $X + \{v\}$ là tập độc lập. Thêm v vào tập X ; đồng thời, nếu $v \in Y$ thì xóa v khỏi Y , ngược lại $v \in Z$ và xóa v khỏi Z . Lặp quá trình này cho đến khi X là tập độc lập cực đại.

Hiển nhiên lúc này Y, Z vẫn là các tập độc lập của G , tức với $u, v \in Y$ hoặc $u, v \in Z$, ta có $(u, v) \notin E$. Vì vậy $G[Y \cup Z]$ là đồ thị hai phía. Bài toán được chứng minh. ■

Kiểm tra một đồ thị có phải đồ thị hai phía hay không và tô màu nó có thể thực hiện trong $\mathcal{O}(m)$ thời gian. Vì vậy chỉ cần liệt kê mọi tập độc lập cực đại I của đồ thị G , rồi kiểm tra đồ thị $G - I$ có phải đồ thị hai phía hay không:

1. Nếu với mọi tập độc lập cực đại I , đồ thị $G - I$ đều không phải đồ thị hai phía, thì đồ thị G không tô được bằng 3 màu.
2. Nếu tồn tại một tập độc lập cực đại I sao cho đồ thị $G - I$ là đồ thị hai phía, thì đồ thị G tô được bằng 3 màu: tô các đỉnh trong I bằng màu 1, còn $G - I$ được tô hai phía bằng màu 2 và 3.

Trong bài này, vì G là đồ thị đơn, $m = \mathcal{O}(n^2)$, nên độ phức tạp thời gian của thuật toán là $\mathcal{O}(3^{n/3}n^2)$.

Ví dụ 2 (Mùa thể thao của Tiểu Q, test 10). Cho một hệ m phương trình đồng dư tuyến tính n ẩn

$$\begin{cases} a_{0,0}x_0 + a_{0,1}x_1 + \cdots + a_{0,n-1}x_{n-1} \equiv c_0 \pmod{b_0}, \\ a_{1,0}x_0 + a_{1,1}x_1 + \cdots + a_{1,n-1}x_{n-1} \equiv c_1 \pmod{b_1}, \\ \dots \\ a_{m-1,0}x_0 + a_{m-1,1}x_1 + \cdots + a_{m-1,n-1}x_{n-1} \equiv c_{m-1} \pmod{b_{m-1}}. \end{cases}$$

Hãy tìm một nghiệm $(x_1, x_2, \dots, x_n)$ thỏa mãn càng nhiều phương trình càng tốt. Đây là bài nộp đáp án.¹⁵

Trong ví dụ này chỉ thảo luận test 10. Ở test này, bằng cách lập mô hình đồ thị, xem mỗi phương trình là một đỉnh và nối cạnh giữa hai phương trình xung đột nhau, ta có thể chuyển thành bài toán tập độc lập lớn nhất trên một đồ thị vô hướng có số đỉnh $n = 90$, số cạnh $m = 223$. Vì quá trình chuyển đổi cụ thể vượt khỏi phạm vi bài viết, nên lược bỏ.

Dùng thuật toán Bron–Kerbosch để tìm mọi tập độc lập cực đại, rồi in ra tập lớn nhất trong số đó. Cách làm này hiệu quả ra sao?

Tác giả so sánh thuật toán tìm kiếm ngây thơ Simple-Search và thuật toán tìm kiếm dựa trên tập độc lập cực đại Maximal-Search. Hai thuật toán chỉ dùng phép cắt nhánh cơ bản nhất: nếu ngay cả khi thêm tất cả các đỉnh còn lại vào I , kích thước tập thu được cũng không vượt quá tập độc lập lớn nhất hiện đã tìm được, thì cắt nhánh. Vì mục đích kiểm thử chỉ là xem Maximal-Search có hiệu suất chạy tốt hơn

¹⁵Nguồn bài: bài 3 của WC2013.

Simple-Search hay không, ở đây không thêm các phép cắt nhánh khác phụ thuộc vào tính chất của bài toán.

Với Simple-Search, chương trình của tác giả chạy vài giờ vẫn chỉ tìm được tập độc lập kích thước 33, và chương trình chưa kết thúc. Còn với Maximal-Search, chương trình của tác giả chỉ mất chưa đến 1 phút để tìm được một tập độc lập kích thước 34, và chỉ mất 3 phút để chứng minh số độc lập của đồ thị thật sự bằng 34. Có thể thấy dùng tập độc lập cực đại để tìm kiếm quả thật có thể nâng cao hiệu suất chạy rất nhiều.

Bằng cách thêm nhiều tối ưu cắt nhánh hơn, có thể rút ngắn thêm thời gian chạy của thuật toán; bạn đọc có hứng thú có thể thử.

6.3.2 Thuật toán tập độc lập dựa trên quy hoạch động

Quy hoạch động là một phương pháp xử lý bài toán hiệu quả và linh hoạt. Trong bài toán tập độc lập, quy hoạch động không chỉ giải được các bài toán tối ưu như tập độc lập lớn nhất, tập độc lập trọng số lớn nhất, mà còn giải được bài toán đếm như đếm tập độc lập. Dưới đây vẫn lấy bài toán tập độc lập lớn nhất làm ví dụ, nhưng để thể hiện tính phổ dụng của quy hoạch động, phần thảo luận tiếp theo sẽ không thêm các phép cắt nhánh chỉ dành cho bài toán tối ưu, như cắt nhánh theo tính tối ưu.

Thuật toán. Nếu muốn dùng quy hoạch động để giải bài toán tập độc lập, ta cần chuyển bài toán thành các bài toán con có quy mô nhỏ hơn. Với tập độc lập, ta có hai định lý sau:

Định lý 3.6. Với đồ thị vô hướng $G = (V, E)$ và $V' \subseteq V$, với mọi $I \subseteq V'$, I là tập độc lập của G khi và chỉ khi I là tập độc lập của $G[V']$.

Chứng minh. (Tính đủ) Khi I là tập độc lập của $G[V']$, với mọi $(u, v) \in E$, nếu $u, v \in V'$ thì hiển nhiên u, v không đồng thời thuộc I ; nếu một trong u, v không thuộc V' , giả sử $u \notin V'$, thì $u \notin I$. Vì vậy I là tập độc lập của G .

(Tính cần) Khi I là tập độc lập của G , vì mỗi cạnh của $G[V']$ đều thuộc G , mỗi cạnh của $G[V']$ có ít nhất một đầu mút không thuộc I . Do đó I là tập độc lập của $G[V']$. ■

Định lý 3.7. Với đồ thị vô hướng $G = (V, E)$ và $v \in V$, nếu $I \subseteq V$ và $v \in I$, thì I là tập độc lập của G khi và chỉ khi $I - \{v\}$ là tập độc lập của $G[V - \{v\} - N(v)]$.

Chứng minh. Ký hiệu $T = V - \{v\} - N(v)$.

(Tính đủ) Khi I là tập độc lập của G , $N(v) \cap I = \emptyset$, nên $I - \{v\} \subseteq T$. Vì mọi cặp đỉnh trong I không kề nhau trong G , $I - \{v\} \subseteq I$, và $G[T] \subseteq G$, nên $I - \{v\}$ là tập độc lập của $G[T]$.

(Tính cần) Khi $I - \{v\}$ là tập độc lập của $G[T]$, vì $I - \{v\} \subseteq V - \{v\} - N(v)$, nên $N(v) \cap I = \emptyset$. Mặt khác, các cạnh mà G có thêm so với $G[T]$ đều kề với v hoặc với các đỉnh trong $N(v)$, nên I là tập độc lập của G . ■

Theo hai định lý trên, ta có thể dùng quy hoạch động nén trạng thái để tìm số độc lập $\alpha(G)$ của đồ thị vô hướng bất kỳ $G = (V, E)$.

Với tập đỉnh $S \subseteq V$, định nghĩa $f(S)$ là số độc lập của đồ thị con cảm sinh bởi S trên G , tức $f(S) = \alpha(G[S])$; hiển nhiên $f(\emptyset) = 0$.

Xét trường hợp $S \neq \emptyset$. Lấy tùy ý $v \in S$, xét một tập $I \subseteq S$. Nếu $v \notin I$, thì I là tập độc lập của $G[S]$ khi và chỉ khi I là tập độc lập của $G[S - \{v\}]$. Nếu $v \in I$, thì I là tập độc lập của $G[S]$ khi và chỉ khi

$I - \{v\}$ là tập độc lập của $G[S - \{v\} - N(v)]$. Từ đó có:

$$f(S) = \begin{cases} 0, & S = \emptyset, \\ \max\{f(S - \{v\}), f(S - \{v\} - N(v)) + 1\}, & \forall v \in S, S \neq \emptyset. \end{cases} \quad (3.1)$$

Khi cài đặt, đánh số các đỉnh trong đồ thị là $0, 1, \dots, n-1$; đỉnh v có thể chọn là đỉnh có số hiệu lớn nhất trong S . Tương tự, có thể dùng kỹ thuật nén bit để lưu tập S . Ngoài ra, có thể dùng tìm kiếm có nhớ để tính f , trạng thái được lưu bằng Hash Table.

Thuật toán này không chỉ tìm được số độc lập, mà còn tìm được một tập độc lập lớn nhất. Xem mã giả sau, trong đó hàm f là hàm tìm kiếm có nhớ được định nghĩa theo công thức (3.1):

Thuật toán 2 Subset-Dynamic-Programming

```

1:   $S = V$ 
2:   $I = \emptyset$ 
3:  while  $S \neq \emptyset$  do
4:    Cho  $v$  là đỉnh có số hiệu lớn nhất trong  $S$ 
5:    if  $f(S - \{v\}) > f(S - \{v\} - N(v)) + 1$  then
6:       $S \leftarrow S - \{v\}$ 
7:    else
8:       $S \leftarrow S - \{v\} - N(v)$ 
9:       $I \leftarrow I + \{v\}$ 
10:   end if
11: end while
12: return  $I = 0$ 

```

Nếu cài đặt trực tiếp, độ phức tạp là $\mathcal{O}(2^{n/2})$, giả sử mỗi thao tác với Hash Table tốn $\mathcal{O}(1)$ thời gian. Lý do là trong $n/2$ tầng đệ quy đầu, mỗi tầng nhiều nhất có 2 nhánh; sau khi đệ quy quá $n/2$ tầng, S chỉ còn chứa các đỉnh trong nửa đầu theo số hiệu, nên tổng độ phức tạp là $\mathcal{O}(2^{n/2})$.

Với n bất kỳ, đều có thể dựng đồ thị G khiến thuật toán này có độ phức tạp $\Theta(2^{n/2})$: nối đỉnh 0 với $\lfloor n/2 \rfloor$, đỉnh 1 với $\lfloor n/2 \rfloor + 1, \dots$, đỉnh $\lfloor n/2 \rfloor - 1$ với $2\lfloor n/2 \rfloor - 1$. Khi đó $\lfloor n/2 \rfloor$ tầng đệ quy đầu đều có 2 nhánh, và mọi nhánh đều khác nhau.

Ví dụ 3 (Đếm clique). Cho đồ thị vô hướng đơn $G = (V, E)$. Hỏi G có bao nhiêu clique. Một clique được định nghĩa là một tập đỉnh $S \subseteq V$ sao cho hai đỉnh bất kỳ trong S đều có cạnh nối. $n \leq 50$.

Ký hiệu $\overline{G}$ là đồ thị bù của G . Không khó thấy S là clique của G khi và chỉ khi S là tập độc lập của $\overline{G}$.

Điều này là vì nếu S là clique của G , các đỉnh trong S đôi một kề nhau trong G , nên đôi một không kề nhau trong $\overline{G}$, tức S là tập độc lập của $\overline{G}$. Ngược lại, nếu S không phải clique của G , tức tồn tại hai đỉnh u, v không kề nhau trong G , thì u, v kề nhau trong $\overline{G}$. Nói cách khác, S không phải tập độc lập của $\overline{G}$.

Vì vậy bài toán chuyển thành bài toán đếm tập độc lập: tìm số tập độc lập của $\overline{G}$. Hiển nhiên dạng bài toán đếm này không thể dùng chiến lược tối ưu tìm kiếm, nhưng dùng thuật toán Subset-Dynamic-Programming nêu trên thì có thể giải trong thời gian $\mathcal{O}(2^{n/2})$.

Tối ưu hiệu suất. Qua kiểm thử thực tế, hiệu suất của thuật toán Subset-Dynamic-Programming không bằng thuật toán tìm kiếm đã tối ưu. Vì sao? Xét bài toán tập độc lập lớn nhất: độ phức tạp tệ nhất của thuật toán này là $\mathcal{O}(2^{n/2})$, nhưng khi cắt nhánh tìm kiếm, đỉnh bậc 1 có thể được xử lý trực tiếp, nên chỉ cần $\mathcal{O}(n)$ thời gian. Có thể tương tự cách tối ưu thuật toán tìm kiếm để dùng một số phép “cắt nhánh” tối ưu DP nói trên không?

Câu trả lời là có. Tuy nhiên tác giả không định lặp lại những tối ưu trước đó: đã dùng thuật toán dựa trên DP thì nên nghiên cứu các tối ưu phổ dụng hơn. Ví dụ, khi tìm tập độc lập trọng số lớn nhất, xử lý

trực tiếp đỉnh bậc 1 là sai. Sau khi nghiên cứu, tác giả thấy các tối ưu sau có thể nâng cao hiệu suất DP rất nhiều:

1. Trong công thức chuyển trạng thái, đỉnh v không lấy đỉnh có số hiệu lớn nhất trong S , mà lấy đỉnh bậc lớn nhất trong $G[S]$, chú ý là đồ thị con cảm sinh chứ không phải đồ thị gốc.
2. Khi đồ thị $G[S]$ không liên thông, ký hiệu tập đỉnh của các thành phần liên thông lần lượt là $S_1, S_2, \dots, S_k$. Vì mỗi thành phần liên thông độc lập với nhau, có thể chuyển thành các bài toán con nhỏ hơn:

$$f(S) = \sum_{i=1}^k f(S_i).$$

3. Khi đồ thị $G[S]$ không chứa chu trình, có thể đổi sang quy hoạch động trên cây để giải.

Thuật toán DP sau tối ưu được ký hiệu là Optimized-Subset-Dynamic-Programming. Tác giả không thể phân tích độ phức tạp của thuật toán này trong trường hợp xấu nhất, nhưng qua kiểm thử trên đồ thị ngẫu nhiên, Optimized-Subset-Dynamic-Programming nhanh hơn Subset-Dynamic-Programming trước đó rất nhiều. Trong đó tối ưu 1 và 2 có hiệu quả rõ rệt, nhất là với đồ thị khá thưa. Lý do là khi liên tục xóa các đỉnh bậc lớn trong đồ thị thưa, đồ thị con cảm sinh $G[S]$ rất dễ trở nên không liên thông.

Ví dụ 4 (Mùa thể thao của Tiểu Q, test 10). Đề bài xem Ví dụ 2.

Phần trên đã nhắc đến thời gian cần thiết để dùng Maximal-Search tìm nghiệm tối ưu của bài toán này. Bây giờ ta thử dùng thuật toán tập độc lập dựa trên quy hoạch động Subset-Dynamic-Programming. Đáng tiếc là kiểm thử cho thấy hiệu suất chạy của thuật toán này không cao, và do số trạng thái quá nhiều, mức tiêu thụ bộ nhớ cũng không thể chấp nhận.

Ta thử tiếp Optimized-Subset-Dynamic-Programming. Điều đáng ngạc nhiên là hiệu suất chạy của thuật toán này rất cao: qua kiểm thử của tác giả, thuật toán chỉ mất 1 giây để thu được nghiệm tối ưu, tức một tập độc lập kích thước 34. Hơn nữa trong quá trình chạy, thuật toán không dùng bất kỳ tối ưu nào phụ thuộc vào tính chất riêng của bài toán, như cắt nhánh theo tính tối ưu. Có thể thấy trên đồ thị thưa, Optimized-Subset-Dynamic-Programming quả thật là một thuật toán xuất sắc.

Liên hệ với thuật toán tìm kiếm. Thực ra, nếu bỏ phần ghi nhớ khỏi thuật toán này, tức không dùng Hash Table để lưu giá trị f , ta thu được một thuật toán tìm kiếm có tối ưu. Thuật toán tìm kiếm này, ký hiệu Optimized-Search, vẫn nhanh hơn thuật toán tìm kiếm ngây thơ Simple-Search, nhưng chậm hơn Optimized-Subset-Dynamic-Programming.

Optimized-Subset-Dynamic-Programming kết hợp các tư tưởng tối ưu của tìm kiếm và quy hoạch động; nó không chỉ có tính phổ dụng khá mạnh mà độ khó cài đặt cũng nhỏ.

Tuy nhiên Optimized-Subset-Dynamic-Programming có một nhược điểm: độ phức tạp không gian khá lớn. Còn thuật toán tìm kiếm Optimized-Search có không gian cấp đa thức và có thể chạy trong thời gian dài hơn. Vì vậy có thể chỉ ghi nhớ giá trị $f(S)$ của các S nhỏ, phần còn lại dùng phương pháp tìm kiếm; như thế có thể giải bài toán với yêu cầu không gian thấp hơn.

Kiểm thử và so sánh. Sau khi nghiên cứu thuật toán quy hoạch động và các tối ưu nói trên, tác giả đã cài đặt thuật toán này (cùng phiên bản đã tối ưu) và các thuật toán dựa trên tìm kiếm trước đó, rồi dùng đồ thị ngẫu nhiên để kiểm thử hiệu suất chạy kỳ vọng của chúng. Trong hàng đầu của bảng sau, n, m biểu thị đồ thị ngẫu nhiên n đỉnh m cạnh; cột cuối 90, 223 biểu thị đồ thị tương ứng với test 10 của bài *Mùa thể thao của Tiểu Q* trong WC2013. Thời gian trong bảng biểu thị ước lượng thời gian chạy kỳ vọng của thuật toán trên đồ thị tương ứng; dấu “-” nghĩa là thời gian chạy quá dài, chưa đo được.

Thuật toán	40, 60	50, 85	60, 120	90, 223
Simple-Search	2s	-	-	-
Maximal-Search	< 0.01s	< 0.1s	1s	-
Subset-Dynamic-Programming	< 0.01s	< 0.1s	1s	-
Optimized-Subset-Dynamic-Programming-1	< 0.01s	< 0.01s	< 0.01s	1s
Optimized-Subset-Dynamic-Programming-2	< 0.01s	< 0.01s	< 0.01s	1s

Maximal-Search và Subset-Dynamic-Programming lần lượt dùng các tối ưu “cắt nhánh tìm kiếm” và “ghi nhớ”, hiệu quả của chúng khá gần nhau. Optimized-Subset-Dynamic-Programming kết hợp ưu điểm của cả hai, nên hiệu suất nghiêm ngặt cao hơn hai thuật toán này. Điều đáng nói là khác biệt giữa Optimized-Subset-Dynamic-Programming-1 và Optimized-Subset-Dynamic-Programming-2 nằm ở việc thuật toán sau thêm tối ưu 3, tức đổi sang DP trên cây. Dù tối ưu 3 trông rất hiệu quả vì biến bài toán cấp mũ thành bài toán thời gian tuyến tính, qua kiểm thử, hiệu suất chạy của hai thuật toán gần như không khác biệt.

6.4 Bài toán tập độc lập trên đồ thị đặc biệt

Phần trên giới thiệu tư tưởng cơ bản để giải tập độc lập trên đồ thị tổng quát: tối ưu tìm kiếm và quy hoạch động. Các phương pháp này đều có độ phức tạp cấp mũ. Trong mục này, ta sẽ bàn tiếp bài toán tập độc lập trên đồ thị đặc biệt: khi bản thân đồ thị có một tính chất đặc biệt nào đó, liệu có thể giải cùng bài toán bằng độ phức tạp đa thức hay không?

6.4.1 Thuật toán tập độc lập lớn nhất dựa trên tư tưởng matching đồ thị

Tập độc lập lớn nhất của đồ thị hai phía. Tập độc lập lớn nhất của đồ thị hai phía là một bài toán kinh điển. Ta có định lý sau:

Định lý 4.1. Với đồ thị hai phía n đỉnh G , $\alpha(G) = n - \nu(G)$, trong đó $\alpha(G)$ và $\nu(G)$ lần lượt là số độc lập và số matching của đồ thị G .

Chứng minh của định lý này có thể tìm thấy trong nhiều tài liệu, nên ở đây lược bỏ.

Dùng thuật toán Hungary hoặc luồng mạng để tìm một số matching $\nu(G)$ của đồ thị hai phía $G = (X, Y, E)$, ta thu được số độc lập $\alpha(G)$ của G . Ngoài ra, sau khi dựng mạng luồng, tìm lát cắt nhỏ nhất có thể thu được một tập độc lập lớn nhất I .

Thuật toán này chỉ giải được bài toán tập độc lập dạng tối ưu, không giải được các bài toán phức tạp hơn như đếm hoặc có thêm ràng buộc khác.

Tập độc lập lớn nhất của đồ thị không móng. Tập độc lập lớn nhất của đồ thị hai phía cho ta một ý tưởng: khi tìm matching lớn nhất của đồ thị, ta có thể liên tục tăng kích thước matching bằng cách tìm đường tăng. Vậy khi tìm tập độc lập lớn nhất của các đồ thị khác, có thể dùng cách tăng tương tự không? Đáng tiếc là trên đồ thị bất kỳ, đồ thị con cảm sinh bởi hiệu đối xứng¹⁶ của hai tập độc lập không nhất thiết là một số đường đi hoặc chu trình, nên không thể dùng phương pháp tìm đường tăng để tìm tập độc lập lớn nhất.

Tuy nhiên, trên một loại đồ thị đặc biệt, thuật toán này là khả thi.

Định nghĩa 4.1. Đồ thị không móng (claw-free graph) được định nghĩa là đồ thị vô hướng mà mọi đồ thị con cảm sinh đều không phải $K_{1,3}$. Trong đó $K_{1,3}$ được gọi là **móng** (claw), tức đồ thị hai phía đầy đủ có một phần chứa 1 đỉnh và phần kia chứa 3 đỉnh.

¹⁶Hiệu đối xứng của hai tập A, B là $A \triangle B = \{x \mid [x \in A] \neq [x \in B]\}$.

Tập độc lập lớn nhất của đồ thị không móng có thể được giải bằng thuật toán tương tự matching trên đồ thị tổng quát. Thuật toán này dựa trên định lý sau:

Định lý 4.2. Giả sử I_1, I_2 là hai tập độc lập của đồ thị không móng G . Khi đó mỗi thành phần liên thông của $G[I_1 \Delta I_2]$ đều là một đường đi đơn hoặc một chu trình đơn.

Chứng minh. Giả sử $v \in I_1$, khi đó $N(v) \cap I_1 = \emptyset$. Trong $G[I_1 \Delta I_2]$, bậc của v là $|N(v) \cap I_2|$.

Giả sử tồn tại ba đỉnh khác nhau $v_1, v_2, v_3 \in N(v) \cap I_2$. Vì I_2 là tập độc lập, nên v_1, v_2, v_3 đôi một không kề nhau. Do đó $G[\{v, v_1, v_2, v_3\}]$ là một móng, mâu thuẫn.

Vì vậy $|N(v) \cap I_2| \leq 2$, tức mọi đỉnh của $G[I_1 \Delta I_2]$ đều có bậc không vượt quá 2. Định lý được chứng minh. ■

Chú ý rằng nếu $|I_1| < |I_2|$, thì $G[I_1 \Delta I_2]$ nhất định tồn tại một thành phần liên thông C sao cho số đỉnh thuộc I_2 trong thành phần này nhiều hơn số đỉnh thuộc I_1 . Vì các đỉnh thuộc I_1 và I_2 trong C xuất hiện xen kẽ nhau, nếu C là chu trình đơn thì số đỉnh thuộc I_1 và I_2 trong C bằng nhau; còn nếu C là đường đi đơn thì số đỉnh thuộc I_1 và I_2 trong C chênh nhau 1. Vì vậy nhất định tồn tại một đường đi có số đỉnh thuộc I_2 nhiều hơn số đỉnh thuộc I_1 đúng 1. Ta gọi C là đường tăng (augment path) của I_1 .

Như vậy, có thể tương tự thuật toán matching lớn nhất trên đồ thị tổng quát để dùng thuật toán đường tăng tìm tập độc lập lớn nhất của đồ thị không móng $G = (V, E)$. Ban đầu đặt $I = \emptyset$. Mỗi lần xuất phát từ một đỉnh để tìm một đường tăng, rồi đảo trạng thái các đỉnh trên đường tăng: đỉnh trước đó không thuộc tập độc lập thì thêm vào tập độc lập, đỉnh trước đó thuộc tập độc lập thì xóa khỏi tập độc lập. Cài đặt thuật toán này tương tự thuật toán hoa của Edmonds; chi tiết cài đặt và chứng minh tính đúng đắn xem Tài liệu tham khảo [4], ở đây không trình bày thêm.

6.4.2 Thuật toán tập độc lập lớn nhất dựa trên tư tưởng chia giai đoạn trên đồ thị

6.4.3 Quy hoạch động trên đồ thị phân tầng

Với đồ thị $G = (V, E)$, chia tập đỉnh V thành k tập rời nhau $V_1, V_2, \dots, V_k$ sao cho với mọi $u \in V_i, v \in V_j$, nếu $|i - j| > 1$ thì $(u, v) \notin E$. Khi đó gọi dãy tập $\langle V_1, V_2, \dots, V_k \rangle$ là một phân tầng của G . Nếu mỗi V_i có không nhiều đỉnh, ta có thể quyết định theo thứ tự $V_1, V_2, \dots, V_k$, và ở mỗi giai đoạn chỉ cần nén trạng thái cách chọn của một tầng, hiệu quả cao hơn nhiều so với DP nén trạng thái trên toàn bộ đồ thị tổng quát.

Ký hiệu $f(i, S)$ là tập độc lập lớn nhất trong $G[V_1 \cup V_2 \cup \dots \cup V_i]$ có chứa S làm tập con. Công thức chuyển trạng thái như sau:

$$f(i, S) = \begin{cases} -\infty, & S \text{ không độc lập,} \\ 0, & i = 0, \\ \max\{f(i-1, S') \mid S' \subseteq V_{i-1}, S \cup S' \text{ độc lập}\} + |S'|, & \text{trường hợp còn lại.} \end{cases}$$

Số độc lập của G là giá trị lớn nhất của $f(k, I)$ trên mọi tập độc lập I của $G[V_k]$. Quá trình này cũng tìm được một tập độc lập lớn nhất của G , phương pháp tương tự như trước, nên ở đây không trình bày thêm.

Độ phức tạp thời gian của thuật toán này là $\mathcal{O}\left(\sum_{i=1}^{k-1} 2^{|V_i|+|V_{i+1}|}\right)$. Tuy nhiên trong đa số trường hợp, số tập độc lập trong mỗi V_i không nhiều, nên hiệu suất thời gian thực tế cao hơn cận trên lý thuyết rất nhiều.

6.4.4 Quy hoạch động trên “đồ thị k -xương rồng”

Ví dụ 5. Cho đồ thị vô hướng đơn $G = (V, E)$, bảo đảm mỗi cạnh thuộc đúng một chu trình đơn. Hãy tìm số độc lập của G . $|V| \leq 50\,000$, $|E| \leq 60\,000$.¹⁷

Nếu G không liên thông, chỉ cần tìm số độc lập của từng thành phần liên thông của G rồi cộng lại. Dưới đây giả sử G là đồ thị liên thông. Đồ thị đơn liên thông mà mỗi cạnh thuộc nhiều nhất một chu trình đơn được gọi là “xương rồng”. Nó có tính chất đặc biệt gì?

Ta có thể thử mạnh dạn: chọn tùy ý $r \in V$, lấy r làm gốc và tìm kiếm theo chiều sâu (depth-first search, DFS) trên đồ thị G , thu được một cây tìm kiếm theo chiều sâu (cây DFS) $T = (V, E_T)$. Hiển nhiên T là một cây khung của G . Định nghĩa cạnh cây là cạnh thuộc E_T , cạnh không cây là cạnh không thuộc E_T , tức thuộc $E - E_T$.

Cây DFS có một tính chất rất quan trọng:

Định lý 4.3. Với mọi cạnh không cây $e = (u, v)$, trong T , hoặc u là tổ tiên của v , hoặc v là tổ tiên của u .

Chứng minh. Giả sử $e = (u, v)$ là cạnh không cây và u được thăm trước v . Khi thăm u , nếu v không nằm trong cây con gốc u , thì khi liệt kê cạnh ra e của u , v chưa được thăm, nên bước tiếp theo sẽ đi theo cạnh e để thăm v . Từ đó $e \in E_T$, mâu thuẫn với giả thiết. Vậy u là tổ tiên của v . Tương tự, nếu v được thăm trước u , thì v là tổ tiên của u . ■

Với cạnh không cây $e = (u, v)$, nếu cạnh cây e' nằm trên đường đi đơn từ u đến v trong cây T , thì gọi cạnh cây e' bị cạnh không cây e **phủ** (cover). Với xương rồng, ta có:

Định lý 4.4. Mỗi cạnh cây bị nhiều nhất một cạnh không cây phủ.

Chứng minh. Giả sử một cạnh cây e bị hai cạnh không cây (u_1, v_1) , (u_2, v_2) phủ. Khi đó (u_1, v_1) cùng đường đi đơn từ v_1 đến u_1 trong T tạo thành một chu trình đơn C_1 ; (u_2, v_2) cùng đường đi đơn từ v_2 đến u_2 trong T tạo thành một chu trình đơn C_2 . Cạnh e đồng thời thuộc C_1 và C_2 , mâu thuẫn với định nghĩa xương rồng. Vì vậy e bị nhiều nhất một cạnh không cây phủ. ■

Tính chất này cho phép ta làm quy hoạch động trên cây T . Ý tưởng ban đầu là: ký hiệu $f(i, 0)$ là kích thước tập độc lập lớn nhất trong cây con gốc đỉnh $i \in V$ của T , $f(i, 1)$ là kích thước tập độc lập lớn nhất trong cây con i khi đỉnh cha của i thuộc tập độc lập. Tuy nhiên cách này không bảo đảm hai đầu mút của cạnh không cây không đồng thời thuộc tập độc lập.

Xét thêm một chiều trạng thái. Ký hiệu $f(i, j, k)$ là kích thước tập độc lập lớn nhất trong cây con gốc i , trong trường hợp đỉnh cha của i thuộc ($j = 1$) hoặc không thuộc ($j = 0$) tập độc lập, và đỉnh đầu trên u_i của cạnh không cây $e'_i = (u_i, v_i)$ phủ cạnh e_i nối i với cha của nó thuộc ($k = 1$) hoặc không thuộc ($k = 0$) tập độc lập. Khi e_i không thuộc chu trình, $k = 0$. Khi chuyển, liệt kê i có thuộc tập độc lập hay không; chú ý khi i là đỉnh đầu dưới v_i của e'_i và $k = 1$, không thể chọn i . Công thức chuyển trạng thái như sau, trong đó Chi_i biểu thị tập con của đỉnh i :

- Khi $j = 1$ hoặc $k = 1 \wedge i = v_i$:

$$f(i, j, k) = \sum_{c \in \text{Chi}_i} f(c, 0, [e'_c = e'_i]k).$$

- Ngược lại:

$$f(i, j, k) = \max \left\{ \sum_{c \in \text{Chi}_i} f(c, 0, [e'_c = e'_i]k), 1 + \sum_{c \in \text{Chi}_i} f(c, 1, [e'_c = e'_i]k + [u_c = i]) \right\}.$$

¹⁷Nguồn bài: LYDSY 4316.

Sau khi tiền xử lý mọi u_i, v_i trong $\mathcal{O}(m)$ thời gian, ta có thể dùng quy hoạch động $\mathcal{O}(n)$ nói trên để tìm tập độc lập lớn nhất. Độ phức tạp thời gian là $\mathcal{O}(n + m)$.

Sau khi nghiên cứu thuật toán trên, tác giả suy nghĩ liệu thuật toán này có thể mở rộng hay không. Qua phân tích, tác giả nhận thấy sau khi sửa thuật toán một cách đơn giản, nó không chỉ xử lý được xương rồng mà còn xử lý được đồ thị trong đó mỗi cạnh thuộc không nhiều chu trình đơn.

Ta gọi các đồ thị như vậy là “đồ thị k -xương rồng”¹⁸, tức đồ thị mà mỗi cạnh thuộc không nhiều nhất k chu trình đơn, trong đó k tương đối nhỏ.

Khi giải bài toán tập độc lập trên đồ thị k -xương rồng $G = (V, E)$, ta cũng chọn một đỉnh làm gốc và DFS trên đồ thị để thu được cây DFS T . Tiếp theo với mỗi đỉnh i , ký hiệu C_i là tập các cạnh không cây phủ cạnh e_i nối i với cha của nó. Khi đó có một tính chất:

Định lý 4.5. Trong đồ thị k -xương rồng, với mọi $i \in V$, $|C_i| \leq k$.

Chứng minh. Với mỗi $(u, v) \in C_i$, cạnh (u, v) và đường đi từ u đến v trong T đều tương ứng với một chu trình đơn chứa e_i . Vì vậy nhất định tồn tại $|C_i|$ chu trình đơn chứa e_i .

Do số chu trình đơn chứa e_i không vượt quá k , nên $|C_i| \leq k$. ■

Tiếp theo, ta lợi dụng tính chất này để thiết kế quy hoạch động. Trong trạng thái, cần ghi lại đỉnh đầu trên của mọi cạnh trong C_i có thuộc tập độc lập hay không; cách chuyển tương tự. Cụ thể, ký hiệu $f(i, j, S)$ là kích thước tập độc lập lớn nhất trong cây con gốc i của T , trong trường hợp đỉnh cha của i thuộc ($j = 1$) hoặc không thuộc ($j = 0$) tập độc lập, và trong tập U_i gồm các đỉnh đầu trên của các cạnh trong C_i , những đỉnh thuộc tập độc lập tạo thành tập S . Công thức chuyển trạng thái như sau:

- Khi $j = 1$ hoặc $S \cup \{i\}$ không phải là tập độc lập:

$$f(i, j, S) = \sum_{c \in \text{Chi}_i} f(c, 0, S \cap U_i).$$

- Ngược lại:

$$f(i, j, S) = \max \left\{ \sum_{c \in \text{Chi}_i} f(c, 0, S \cap U_i), 1 + \sum_{c \in \text{Chi}_i} f(c, 1, (S \cup \{i\}) \cap U_i) \right\}.$$

Tương tự thuật toán trước, khi xem k là hằng số, độ phức tạp của thuật toán này là $\mathcal{O}(n)$.

Thực ra điều kiện “đồ thị k -xương rồng” là quá rộng. Chỉ cần số cạnh không cây phủ mỗi cạnh cây đều không vượt quá k , thậm chí chỉ cần $\sum_{v \in V} 2^{|C_v|}$ không lớn, thuật toán này đều có hiệu suất rất cao.

Thuật toán này có thể mở rộng sang các bài toán phức tạp hơn, như đếm tập độc lập.

Ví dụ 6 (Bài toán đếm tập con, test 7 và 8). Cho đồ thị vô hướng $G = (V, E)$, $|V| = n$, $|E| = m$. Hỏi có bao nhiêu tập con $V' \subseteq V$ thỏa $|V'| = k$ và với mọi $(u, v) \in E$, $u \notin V' \vee v \notin V'$. Vì đáp án có thể rất lớn, chỉ cần in đáp án lấy modulo p . Đây là bài nộp đáp án.¹⁹

Trong ví dụ này chỉ thảo luận test 7 và 8. Hai test này thỏa G là đồ thị liên thông, quy mô dữ liệu như bảng sau:

Mã test	$n =$	$m =$	$k =$	$p =$
7	4998	5002	666	1000000009
8	11986	12011	1098	1000000007

¹⁸Tên gọi bắt nguồn từ LYDSY 4863.

¹⁹Nguồn bài: tự sáng tác, UOJ #259.

Bài toán yêu cầu số tập độc lập kích thước k trong G . Vì G là đồ thị liên thông và $m - n$ rất nhỏ, có thể thấy G nhận được bằng cách thêm $m - n + 1$ cạnh vào một cây.

Nếu sau khi dựng cây DFS của G , ta vét cạn một đầu mút của mỗi cạnh không cây có được chọn hay không, rồi dùng quy hoạch động trên cây để thống kê số tập độc lập, thì có thể qua test 7 khá nhanh. Nhưng test 8 cần chạy quá lâu và cần tối ưu. Theo tư tưởng đã nêu trong mục này, số chu trình đơn mà mỗi cạnh thuộc về không nhiều, nên có thể dùng thuật toán trên để giải.

Ký hiệu:

- C_i là tập các u_e thỏa $e \in E'$, u_e là tổ tiên của i , không tính i , và v_e nằm trong cây con gốc i .
- $f(i, j, S, s)$ là số tập độc lập kích thước s trong j cây con đầu tiên của i , với điều kiện tập các đỉnh trong C_i thuộc tập độc lập là S .
- $g(i, j, S, s)$ là số tập độc lập kích thước s trong i và j cây con đầu tiên của i , với điều kiện tập các đỉnh trong C_i thuộc tập độc lập là S .
- $F(i, S, s) = f(i, t_i, S, s)$, $G(i, S, s) = g(i, t_i, S, s)$, trong đó t_i là số con của i .

Với $f(i, j, S, s)$, giả sử trong i và $j - 1$ cây con đầu tiên của i có tổng cộng s_1 đỉnh, trong cây con thứ j có s_2 đỉnh, và con thứ j là c_j . Khi đó

$$f(i, j, S, s) = \sum_{x=\max\{s-s_1, 0\}}^{\min\{s, s_2\}} f(i, j-1, S, s-x) G(c_j, S \cap C_{c_j}, x), \quad 0 \leq s \leq s_1 + s_2.$$

Với $g(i, j, S, s)$, nếu tồn tại cạnh $e \in E'$ sao cho $u_e \in S$ và $v_e = i$, thì i không thể thuộc tập độc lập, nên

$$g(i, j, S, s) = f(i, j, S, s).$$

Ngược lại, tồn tại tập độc lập chứa i ; số tập độc lập như vậy được ký hiệu $g'(i, j, S, s)$. Khi đó

$$g'(i, j, S, s) = \sum_{x=\max\{s-1, 0\}}^{\min\{s-1, s_2\}} g'(i, j-1, S, s-x) F(c_j, (S \cup \{i\}) \cap C_{c_j}, x), \quad 0 \leq s \leq s_1 + s_2.$$

Do đó

$$g(i, j, S, s) = g'(i, j, S, s) + f(i, j, S, s), \quad 0 \leq s \leq s_1 + s_2.$$

Biên là $f(i, 0, S, 0) = g'(i, 0, S, 1) = 1$; mọi trạng thái chưa định nghĩa có giá trị 0. Đáp án chính là $G(r, \emptyset, k)$.

Chỉ cần tính các trạng thái thỏa $s \leq k$. Độ phức tạp là $\mathcal{O}(k \sum_{v \in V} 2^{|C_v|})$, trong đó $|C_v|$ thường không quá 12. Toàn bộ tính toán được thực hiện theo modulo p ; bộ nhớ có thể cấp phát động.

Cần chú ý rằng chọn các đỉnh $r \in V$ khác nhau làm gốc, cũng như dùng các thứ tự khác nhau để DFS, sẽ cho hiệu suất chạy khác nhau. Có thể chọn một gốc r để DFS sao cho

$$\sum_{v \in V} 2^{|C_v|}$$

nhỏ nhất, rồi mới thực hiện thuật toán trên để giảm độ phức tạp thời gian. Qua thực nghiệm, số trạng thái của test 7 xấp xỉ 5×10^5 , có thể qua test này trong 1 giây; số trạng thái của test 8 xấp xỉ 10^8 , có thể qua test này trong 2 phút.

6.5 Tổng kết

Các phương pháp tối ưu thuật toán cho bài toán NP-hard nhiều không kể xiết. Bài viết này chỉ nhắc đến vài thuật toán tối ưu cho bài toán tập độc lập. Các phương pháp này giải những bài toán tương tự nhau, nhưng tư tưởng lại khác nhau: với bài toán tối ưu thông thường, như tập độc lập lớn nhất, có thể dùng thuật toán tìm kiếm có cắt nhánh theo tính tối ưu để giảm lượng liệt kê; với bài toán đếm hoặc có ràng buộc bổ sung, như đếm tập độc lập, có thể dùng quy hoạch động nén trạng thái và nâng cao hiệu suất chạy bằng cách tối ưu số trạng thái; với đồ thị có thể “tăng” và đồ thị có tính giai đoạn rõ rệt, lại có thể dùng thuật toán độ phức tạp đa thức để hoàn thành hiệu quả.

Đồng thời, bài viết phân tích và so sánh thời gian chạy của vài thuật toán trong trường hợp ngẫu nhiên, giúp mọi người có nhận thức sâu hơn về hiệu suất giải bài toán tập độc lập.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn cô Chen Ying của Trường Trung học số 1 Phúc Châu đã quan tâm và chỉ dẫn.

Cảm ơn các anh chị khóa trên của Trường Trung học số 1 Phúc Châu đã đem lại nhiều gợi mở và chỉ dẫn.

Cảm ơn tổ huấn luyện OI đã giúp đỡ tôi.

Cảm ơn bạn Chen Junkun đã cung cấp cho tôi không ít kỹ thuật tối ưu thuật toán tập độc lập, giúp tôi rất nhiều trong nghiên cứu DP trên cây DFS.

Cảm ơn mọi người đã cung cấp các tài liệu mà bài viết này từng tham khảo.

Cảm ơn quý vị đã dành thời gian quý báu trong lúc bận rộn để đọc bài.

Tài liệu tham khảo

- [1] Wikipedia, Bron–Kerbosch algorithm.
- [2] Robson, J. M. (2001), Finding a maximum independent set in time $\mathcal{O}(2^{n/4})$.
- [3] Tomita, Etsuji; Tanaka, Akira; Takahashi, Haruhisa (2006), The worst-case time complexity for generating all maximal cliques and computational experiments.
- [4] Minty, George J. (1980), “On maximal independent sets of vertices in claw-free graphs”, *Journal of Combinatorial Theory. Series B*, 28 (3): 284–304, doi:10.1016/0095-8956(80)90074-X, MR 579076.

7 Báo cáo ra đề và mở rộng của *Đồ thị con kỳ diệu*

Chen Junkun, Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tóm tắt

Đồ thị con kỳ diệu là một bài truyền thống do tôi ra trong đợt kiểm tra chéo đội tuyển tập huấn năm 2017. Bằng cách làm yếu các điều kiện khác nhau của bài toán, ta có thể chuyển nó thành các bài toán đồ thị thỏa những điều kiện đặc thù khác nhau, rồi giải bằng nhiều thuật toán. Bài viết này tập trung nghiên cứu một lớp bài toán DP có cập nhật trên đồ thị dưới một điều kiện đặc biệt, đề xuất “cây tròn–vuông” và thuật toán dựa trên tư tưởng phân trị theo chuỗi, đồng thời mở rộng ứng dụng của hai thuật toán đó. Tôi hy vọng thông qua bài *Đồ thị con kỳ diệu*, có thể chia sẻ với mọi người một phương pháp xử lý chung cho một lớp thành phần song liên thông đỉnh và phương pháp giải các bài toán DP trên cây có cập nhật.

7.1 Đề bài

7.1.1 Mô tả tóm tắt

Cho một đồ thị vô hướng đơn $G = (V, E)$. Đặt $n = |V|$, $m = |E|$; mỗi đỉnh có một trọng số, lần lượt là $v_1, v_2, \dots, v_n$. Đồ thị G thỏa một điều kiện đặc biệt: với mọi tập V' gồm 7 đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V'$, $x \in V - \{p, q\}$, và sau khi xóa đỉnh x khỏi đồ thị thì p và q không liên thông.

Bây giờ, cho hằng số k . Cần tìm một đồ thị con liên thông $G' = (V', E')$ của G sao cho với mọi $p \in V'$, bậc của p trong G' không vượt quá k . Hãy cực đại hóa tổng trọng số các đỉnh trong đồ thị con, tức $\sum_{p \in V'} v_p$ (chỉ cần in giá trị lớn nhất), đồng thời tính số đồ thị con khác nhau đạt được giá trị lớn nhất đó, lấy modulo 64123. Hai đồ thị con $G_1 = (V_1, E_1)$ và $G_2 = (V_2, E_2)$ được coi là khác nhau khi và chỉ khi $V_1 \neq V_2$ hoặc $E_1 \neq E_2$.

Bài còn cần hỗ trợ q lần cập nhật; mỗi lần sửa trọng số v_p của một đỉnh p . Hãy trả lời hai câu hỏi trên trước mọi cập nhật và sau mỗi lần cập nhật.

7.1.2 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, k , lần lượt là số đỉnh, số cạnh của G và hằng số giới hạn bậc k .

Dòng thứ hai chứa n số nguyên $v_1, v_2, \dots, v_n$, là trọng số ban đầu của các đỉnh.

Trong m dòng tiếp theo, dòng thứ i chứa hai số nguyên $x_i, y_i \in [1, n]$, biểu thị cạnh thứ i nối hai đỉnh x_i và y_i .

Dòng tiếp theo chứa một số nguyên q , là số lần sửa trọng số đỉnh.

Trong q dòng tiếp theo, dòng thứ i chứa hai số nguyên p_i, w_i , biểu thị trong lần sửa thứ i , đặt $v_{p_i} \leftarrow w_i$.

7.1.3 Dữ liệu ra

Ban đầu và sau mỗi lần sửa, in một dòng đáp án, tổng cộng $q + 1$ dòng. Mỗi dòng chứa hai số, lần lượt là “tổng trọng số của đồ thị con có tổng trọng số lớn nhất” và “số đồ thị con hợp lệ khác nhau có tổng trọng số đạt giá trị lớn nhất, lấy phần dư khi chia cho 64123”.

7.1.4 Ví dụ

Dữ liệu vào

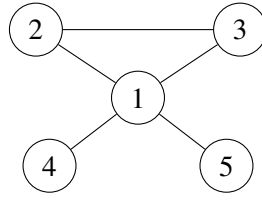
```
5 5 2
1 2 1 2 2
2 3
2 1
1 4
1 3
1 5
1
2 1
```

Dữ liệu ra

```
6 4
5 5
```

7.1.5 Giải thích ví dụ

Trong ví dụ, đồ thị G như sau:



Ban đầu có 4 phương án có tổng trọng số đỉnh đạt giá trị lớn nhất 6:

Số	V'	E'
1	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 2), (1, 4)\}$
2	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 3), (1, 4)\}$
3	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 2), (1, 5)\}$
4	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 3), (1, 5)\}$

Sau khi sửa trọng số đỉnh 2 thành 1, có 5 phương án có tổng trọng số đỉnh đạt giá trị lớn nhất 5:

Số	V'	E'
1	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 2), (1, 4)\}$
2	$\{1, 2, 3, 4\}$	$\{(2, 3), (1, 3), (1, 4)\}$
3	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 2), (1, 5)\}$
4	$\{1, 2, 3, 5\}$	$\{(2, 3), (1, 3), (1, 5)\}$
5	$\{1, 4, 5\}$	$\{(1, 4), (1, 5)\}$

7.1.6 Giới hạn và quy ước

Với mọi dữ liệu, bảo đảm $n \geq 2$, $m, q \geq 0$, $2 \leq k \leq 3$, $1 \leq v_i, w_i \leq 5000$. Thông tin từng subtasks như sau (chấm gộp):

Subtask	Điểm	Test	n	m	k	q	Tính chất
1	7	1	≤ 10	≤ 20	$= 3$	≤ 100	Không
2	18	2,3	≤ 10000	$= n - 1$	$= 2$	≤ 20000	1
3	7	4,5	≤ 50000	$= n - 1$	$= 2$	≤ 50000	1
4	15	6,7,8	≤ 100000	$= n - 1$	$= 2$	≤ 200000	1
5	12	9,10	≤ 100	≤ 300	$= 2$	$= 0$	2,3
6	9	11,12	≤ 1000	≤ 3000	$= 3$	$= 0$	3
7	5	13	≤ 30000	≤ 100000	$= 3$	$= 0$	Không
8	14	14,15	≤ 100000	≤ 300000	$= 3$	$= 0$	Không
9	3	16	≤ 30000	≤ 55000	$= 3$	≤ 10000	Không
10	10	17–20	≤ 30000	≤ 100000	$= 3$	≤ 10000	Không

Tính chất 1: G là một cây vô căn có n đỉnh.

Tính chất 2: mọi v_i, w_i đều bằng 1.

Tính chất 3: với mọi tập V' gồm 5 đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V'$, $x \in V - \{p, q\}$, và sau khi xóa x khỏi đồ thị thì p và q không liên thông.

Với mỗi test, nếu trong đáp án của thí sinh, số thứ nhất của mọi dòng đều đúng nhưng một số dòng có số thứ hai sai, vẫn được 60% số điểm của test đó (làm tròn đến số nguyên).

7.1.7 Hạn chế

Giới hạn thời gian: 4 giây trên TUOJ.

Giới hạn bộ nhớ: 1 GB.

7.2 Phân tích sơ bộ

7.2.1 Ký hiệu và quy ước

Dùng ký hiệu $[x]$: nếu mệnh đề x đúng thì $[x] = 1$, ngược lại $[x] = 0$.

Dùng ký hiệu $\mathcal{O}(A) + \mathcal{O}(B) + \mathcal{O}(C)$ để mô tả thuật toán có độ phức tạp tiền xử lý $\mathcal{O}(A)$, độ phức tạp mỗi lần sửa $\mathcal{O}(B)$, và độ phức tạp bộ nhớ $\mathcal{O}(C)$. Khi đó tổng thời gian là $\mathcal{O}(A + qB)$.

Dùng ký hiệu đại diện $*$. Nếu một biến xuất hiện có chứa $*$, nó biểu thị tập mọi biến thỏa điều kiện. Ví dụ, nếu với mỗi $p \in V$ định nghĩa $f(p)$, thì $f(*)$ là tập $\{f(p) \mid p \in V\}$; nếu với mỗi $p \in V, 1 \leq d \leq k$ định nghĩa $f(p, d)$, thì $f(a, *)$ là tập $\{f(a, b) \mid 1 \leq b \leq k\}$.

Đồ thị gốc có thể không liên thông. Nếu vậy, ta có thể tính nghiệm tối ưu cho từng thành phần liên thông. Vì thế các phân tích sau mặc định đang xét một đồ thị liên thông.

Bài này cần tính số phương án tối ưu. Có thể dùng thuật toán chung sau để đếm: định nghĩa thông tin (v, c) , trong đó v là giá trị tối ưu, c là số phương án sau khi lấy modulo; định nghĩa phép cộng

$$(v_1, c_1) + (v_2, c_2) = (\max(v_1, v_2), c_1[v_1 \geq v_2] + c_2[v_2 \geq v_1] \bmod 64123),$$

và phép nhân

$$(v_1, c_1) \times (v_2, c_2) = (v_1 + v_2, c_1 c_2 \bmod 64123).$$

Dùng thông tin này là có thể đồng thời duy trì nghiệm tối ưu và số phương án; phần sau sẽ không nhắc lại.

7.2.2 Thuật toán một: vét cạn

Trong subtask 1, dữ liệu rất nhỏ, $m \leq 20$, nên có thể xét mọi đồ thị con.

Chú ý mục tiêu tối ưu và các lần sửa chỉ liên quan đến V' , còn E' chỉ ảnh hưởng đến số phương án. Vì vậy có thể tiền xử lý, với mỗi V' , số cách chọn E' .

Khi tiền xử lý, liệt kê tập cạnh E' , kiểm tra đồ thị con sinh ra có thỏa giới hạn bậc và liên thông hay không; nếu có thì cập nhật số phương án cho tập đỉnh của đồ thị con đó. Cần xử lý riêng trường hợp một đỉnh.

Sau mỗi lần sửa, chỉ cần tính lại tổng trọng số của mỗi V' , rồi cộng số phương án của các V' đạt tối ưu.

Độ phức tạp là $\mathcal{O}(m2^m) + \mathcal{O}(2^n) + \mathcal{O}(2^n + m)$. Thuật toán qua được subtask 1, kỳ vọng 7 điểm. Thuật toán này chưa dùng bất kỳ tính chất nào của đề, nên độ phức tạp rất tệ. Trước hết, ta xét một lớp dữ liệu đặc biệt: dữ liệu là cây.

7.3 Dữ liệu trên cây với $k = 2$

Subtask 2 đến 4, tổng 40 điểm, thỏa G là một cây và $k = 2$. Không khó thấy các test này yêu cầu tìm đường đi có tổng trọng số lớn nhất trên cây, có hỗ trợ sửa trọng số một đỉnh. Sau đây gọi “chuỗi dài nhất” là chuỗi có tổng trọng số lớn nhất.

7.3.1 Thuật toán hai: vét cạn trên cây

Trong subtask 2, dữ liệu cho phép thuật toán $\mathcal{O}(n)$ cho mỗi lần sửa.

Bài toán này có cấu trúc con tối ưu, nên có thể dùng quy hoạch động. Chọn tùy ý một đỉnh gốc r để biến cây thành cây có gốc; ký hiệu $\text{Ch}(i)$ là tập con của đỉnh i . Đặt $f(i)$ là khoảng cách từ i đến đỉnh xa

nhất trong cây con của i ²⁰, và $g(i)$ là tổng trọng số của chuỗi dài nhất nằm hoàn toàn trong cây con của i . Khi đó

$$f(i) = v_i + \max_{p \in \text{Ch}(i)} f(p),$$

$$g(i) = \max \left(f(i), \max_{p \in \text{Ch}(i)} g(p), v_i + \max_{\substack{p, q \in \text{Ch}(i) \\ p \neq q}} (f(p) + f(q)) \right).$$

Chỉ cần ghi thêm số phương án trong $f(i)$ và $g(i)$ là tính được số phương án tối ưu. Đáp án là $f(r)$ và số phương án tương ứng.

Mỗi lần sửa, chỉ cần cập nhật các giá trị DP trên đường từ p đến gốc r , nên thời gian mỗi lần sửa là $\mathcal{O}(n)$.

Độ phức tạp là $\mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(n)$. Thuật toán qua được subtask 2, có thể qua subtask 3, kỳ vọng 18 đến 25 điểm.

7.3.2 Thuật toán ba: phân trị theo đỉnh

Subtask 3 và 4 có số lần sửa và kích thước cây lớn hơn, nên cần tối ưu độ phức tạp mỗi lần sửa.

Đây là một bài toán thống kê đường đi, do đó có thể dùng phân trị theo đỉnh; nội dung và cách cài đặt cụ thể có thể xem [1]. Trong quá trình phân trị, chỉ cần xét các đường đi qua trọng tâm hiện tại g và số phương án của chúng. Gọi tập nhánh của trọng tâm g là Br (chính là tập cạnh đi ra từ g trong thành phần liên thông hiện tại). Đặt $f(i)$ là khoảng cách từ g đến đỉnh xa nhất trong cây con i , với thành phần hiện tại được gốc ở g . Khi đó chuỗi dài nhất qua trọng tâm g là

$$\max \left(f(g), \max_{\substack{p, q \in \text{Br} \\ p \neq q}} (f(p) + f(q)) \right) - v_g.$$

Mỗi lần sửa đỉnh i , xét mọi thành phần trong các tầng phân trị chứa i . Số thành phần như vậy chỉ là $\mathcal{O}(\log n)$, nên có thể lần lượt sửa vết cạn. Trong mỗi thành phần, phần bị sửa là khoảng cách từ mọi đỉnh trong cây con của i , với gốc hiện tại g , đến g ; mọi giá trị đó cùng cộng thêm hiệu giữa v_i mới và v_i cũ.

Dễ nghĩ đến việc lấy thứ tự DFS của thành phần khi gốc là g , rồi dùng cây đoạn duy trì giá trị nhỏ nhất của khoảng cách đến g trên một đoạn; khi đó mỗi lần sửa là sửa một đoạn.

Sau khi sửa độ sâu, cần duy trì giá trị đường đi qua trọng tâm. Giá trị $f(g)$ chỉ cần truy vấn gốc cây đoạn. Để duy trì

$$\max_{\substack{p, q \in \text{Br} \\ p \neq q}} f(p) + f(q),$$

có thể dùng heap để duy trì $f(p)$ của từng nhánh; khi đó chỉ cần lấy giá trị lớn nhất và lớn nhì. Mỗi $f(p)$ của một nhánh cũng có thể truy vấn đoạn trên cây đoạn.

Độ phức tạp là $\mathcal{O}(n \log n) + \mathcal{O}(\log^2 n) + \mathcal{O}(n \log n)$. Thuật toán qua được subtask 2, 3, 4, kỳ vọng 40 điểm.

7.4 Dữ liệu trên cây với $k = 3$

Xét một lớp dữ liệu đặc biệt: dữ liệu trên cây với $k = 3$. Lớp này không xuất hiện riêng trong subtasks, nhưng là hướng rất gần với lời giải chính. Ta sẽ suy nghĩ về lớp bài toán này và thử mở rộng lời giải của nó sang đồ thị tổng quát.

²⁰Ở đây khoảng cách được định nghĩa là tổng trọng số đỉnh trên đường đi đơn duy nhất giữa hai đỉnh.

7.4.1 Thuật toán bốn: DP trên cây

Vẫn có thể thiết kế một thuật toán quy hoạch động. Ghi $f(i, d)$ là tổng trọng số lớn nhất của một đồ thị con liên thông mà mọi đỉnh đều nằm trong cây con của i , có chứa i , và bậc của i đúng bằng d . Đặt $g(i) = \max f(i, *)$. Khi đó chuyển $f(i, *)$ bằng cách với mỗi $p \in \text{Ch}(i)$, thực hiện một DP ba lô với trọng lượng 1, giá trị $g(p)$, rồi cuối cùng cộng thêm a_i .

Độ phức tạp của thuật toán là $\mathcal{O}(nk^2) + \mathcal{O}(nk^2) + \mathcal{O}(nk)$; điểm kỳ vọng và các test qua được giống thuật toán hai.

7.4.2 Hướng tối ưu?

Trong thuật toán ba, ta dùng phân trị theo đỉnh, chia cây thành $\mathcal{O}(\log n)$ cấu trúc phân trị. Khi sửa, chỉ cần lần lượt sửa các cấu trúc ấy rồi dùng cấu trúc dữ liệu (cây đoạn) để duy trì mỗi cấu trúc phân trị, nhờ đó hỗ trợ sửa hiệu quả.

Tuy nhiên với bài $k = 3$, đối tượng cần tìm không còn là đường đi có tổng trọng số lớn nhất mà là một thành phần liên thông. Trong phân trị theo đỉnh, không chỉ khó xử lý “thành phần liên thông hợp lệ đi qua trọng tâm”, mà còn khó hỗ trợ sửa hiệu quả.

Dưới đây là một bài ví dụ gợi mở cho việc suy nghĩ về bài gốc.

Ví dụ 1. Tập độc lập trọng số lớn nhất trên cây, phiên bản có cập nhật. Cho một cây vô căn có n đỉnh, mỗi đỉnh có trọng số a_i . Có q thao tác; mỗi thao tác sửa a_i của một đỉnh. Ban đầu và sau mỗi lần sửa, hãy in tổng trọng số của tập độc lập có tổng trọng số lớn nhất trên cây. Dữ liệu: $n, q \leq 1000000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 3 giây, bộ nhớ 512 MB.²¹

Nếu không có cập nhật, có thể thiết kế DP như sau. Chọn tùy ý một đỉnh làm gốc. Đặt $f(i)$ là tổng trọng số lớn nhất của tập độc lập trọng số lớn nhất nằm trong cây con của i , trong đó i thuộc tập độc lập; đặt $g(i)$ là giá trị tương tự khi i không thuộc tập độc lập. Khi đó

$$g(i) = \sum_{p \in \text{Ch}(i)} \max(f(p), g(p)), \quad f(i) = a_i + \sum_{p \in \text{Ch}(i)} g(p).$$

Nhưng DP này không cho phép tính nhanh nghiệm tối ưu khi dữ liệu vào thay đổi.

Xét tiếp trường hợp đặc biệt: cây là một chuỗi. Bài toán trên chuỗi dễ duy trì bằng cây đoạn. Với mỗi nút cây đoạn, ghi thông tin $\text{ans}(a, b)$, trong đó $a, b \in \{0, 1\}$. Nếu đoạn mà nút đó duy trì là $[l, r]$, thì $\text{ans}(a, b)$ là tổng trọng số lớn nhất của tập độc lập có mọi đỉnh nằm trong $[l, r]$, thỏa $a = [l \text{ thuộc tập độc lập}]$, $b = [r \text{ thuộc tập độc lập}]$. Phép gộp thông tin đoạn là

$$\text{ans}(a, b) = \max_{c+d < 2} (\text{ans}_l(a, c) + \text{ans}_r(d, b)).$$

Trong cây đoạn, số nút chứa một vị trí chỉ là $\mathcal{O}(\log n)$, nên sau khi sửa cũng chỉ cần tính lại $\mathcal{O}(\log n)$ nút đó. Vì vậy thuật toán này hỗ trợ tính nghiệm tối ưu sau cập nhật trong $\mathcal{O}(\log n)$ thời gian.

Nhưng khi mở rộng lên cây, ta gặp không ít rắc rối. Thuật toán cây đoạn trên chuỗi thực chất ghi lại thông tin của mọi đỉnh trong thành phần liên thông hiện tại (trên chuỗi là một đoạn) nối với bên ngoài (các thành phần liên thông của cấu trúc phân trị khác), cụ thể là chúng có thuộc tập độc lập hay không. Trên chuỗi, một thành phần liên thông có nhiều nhất 2 điểm nối ra ngoài, nên ghi được. Trên cây, nếu phân trị theo đỉnh, một thành phần liên thông có thể nối với các thành phần liên thông khác qua $\mathcal{O}(\log n)$ đỉnh, rất phức tạp để duy trì, càng khó hỗ trợ cập nhật hiệu quả. Vì vậy phân trị động theo đỉnh không phù hợp.

²¹Nguồn: bài toán kinh điển.

Điểm mấu chốt khiến cây đoạn và phân trị theo đỉnh hiệu quả là tư tưởng phân trị: mỗi lần sửa chỉ ảnh hưởng đến $\mathcal{O}(\log n)$ cấu trúc phân trị, nhờ đó bảo đảm độ phức tạp. Ở đây, ta có thể xét phân trị dựa trên chuỗi (gọi tắt là phân trị theo chuỗi).

Phân trị theo chuỗi thực hiện sau khi phân rã nặng–nhẹ: lấy ra một chuỗi nặng, xóa chuỗi đó khỏi cây, rồi đệ quy xử lý từng cây con còn lại (gốc của những cây con này là các con nhẹ của một đỉnh trên chuỗi nặng). Sau khi đệ quy xong, xét đóng góp của chuỗi nặng vào đó.²²

Dùng phân trị theo chuỗi, xét tối ưu trực tiếp DP ở trên. Dễ thấy đỉnh đầu của mỗi chuỗi nặng hoặc là gốc cây, hoặc có cha thuộc một chuỗi nặng khác. Vì vậy các chuỗi nặng cũng tạo thành quan hệ cây có gốc: nếu chuỗi nặng A là cha của chuỗi nặng B , thì cha của đỉnh đầu chuỗi B nằm trong A , còn đỉnh đầu của B là một con nhẹ của một đỉnh trong A . Gọi cây này là cây chuỗi nặng. Ta thay đổi thứ tự DP theo thứ tự của cây chuỗi nặng: trước đây xử lý mọi đỉnh trong cây con, còn bây giờ xử lý mọi chuỗi nặng trong cây con của cây chuỗi nặng hiện tại.

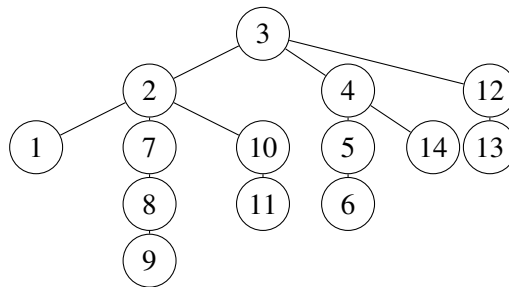
Với mỗi chuỗi nặng, sau khi đã đệ quy xử lý mọi chuỗi nặng trong các cây con theo cạnh nhẹ của từng đỉnh và đã xét ảnh hưởng của các giá trị DP đó lên đáp án, bài toán trên chuỗi nặng gần như trở thành bài tập độc lập động trên dãy, nên có thể dùng cây đoạn trên dãy để duy trì. Ta dùng lại thiết kế trạng thái $\text{ans}(a, b)$ của dãy, có sửa đổi nhỏ. Giả sử các đỉnh trên chuỗi nặng theo thứ tự khoảng cách đến gốc tăng dần là $p_1, p_2, \dots, p_c$, và nút cây đoạn hiện tại duy trì đoạn $[l, r]$. Ghi $\text{ans}(a, b)$ là tổng trọng số lớn nhất của tập độc lập có mọi đỉnh nằm trong các đỉnh $p_l, p_{l+1}, \dots, p_r$ và trong các cây con của con nhẹ của chúng, thỏa $a = [p_l \text{ thuộc tập độc lập}]$, $b = [p_r \text{ thuộc tập độc lập}]$. Phép gộp thông tin giống trên dãy.

Còn một vấn đề: làm sao tính ảnh hưởng của mọi cây con theo cạnh nhẹ của một đỉnh, tức làm sao tính $\text{ans}(a, b)$ cho đoạn dài 1 trên chuỗi. Chỉ có $\text{ans}(0, 0)$ và $\text{ans}(1, 1)$ có ý nghĩa. Gọi đỉnh đơn đó là i , và $\text{Ch}_0(i)$ là tập con nhẹ của i . Từ DP ở trên, dễ có

$$\text{ans}(0, 0) = \sum_{p \in \text{Ch}_0(i)} \max(f(p), g(p)), \quad \text{ans}(1, 1) = a_i + \sum_{p \in \text{Ch}_0(i)} g(p).$$

Chỉ cần thống kê hai tổng này.

Quan sát hình sau:



Đừng quên rằng con nhẹ của một đỉnh chính là p_1 của chuỗi nặng chứa con nhẹ ấy. Vì vậy có thể mượn thông tin của chuỗi nặng chứa con nhẹ để tính giá trị DP. Với con nhẹ p ,

$$f(p) = \max(\text{ans}(1, 0), \text{ans}(1, 1)), \quad g(p) = \max(\text{ans}(0, 0), \text{ans}(0, 1)).$$

Do đó chỉ cần mở hai biến cho mỗi đỉnh để duy trì hai tổng nói trên của mọi con nhẹ.

Mỗi lần sửa, trước hết sửa chuỗi nặng chứa đỉnh bị sửa, rồi nhảy tới chuỗi nặng chứa cha của đỉnh đầu chuỗi hiện tại, thực hiện cập nhật một điểm; cứ thế nhảy đến gốc. Khi nhảy giữa các chuỗi nặng, không được tính lại hai tổng bằng $\mathcal{O}(|\text{Ch}_0(p_i)|)$, mà phải trực tiếp cộng trừ vào hai biến duy trì tổng.

Như vậy bài toán được giải với độ phức tạp $\mathcal{O}(n) + \mathcal{O}(\log^2 n) + \mathcal{O}(n)$.

²²Cài đặt cụ thể và chứng minh chi tiết về phân trị theo chuỗi có thể xem [1].

Nhìn lại thuật toán này, nó đổi thứ tự DP thành thứ tự của cây chuỗi nặng, từ đó chuyển bài toán thành bài toán trên chuỗi rồi dùng thuật toán trên dây để giải hiệu quả. Vì thuật toán không trực tiếp khai thác tính chất riêng của bài toán trên cây, mà chia cây thành nhiều chuỗi và lần lượt giải bài toán trên dây; trong khi bài toán tương tự trên dây thường có nhiều tính chất hơn và dễ duy trì hơn bài toán trên cây, thuật toán có khả năng mở rộng rất mạnh. Bây giờ xét mở rộng thuật toán này sang dữ liệu trên cây với $k = 3$.

7.4.3 Thuật toán năm: phân trị theo chuỗi

Xét dùng phân trị theo chuỗi để giải bài trên cây với $k = 3$ có cập nhật. Vẫn tối ưu trực tiếp DP của thuật toán bốn. Trước hết, sau khi xử lý xong mọi chuỗi nặng trong cây con, với mỗi chuỗi nặng, điều cần xét là mọi cây con liên thông có đỉnh cao nhất nằm trên chuỗi nặng này. Giao của cây con ấy với chuỗi nặng là một đoạn trên chuỗi. Vì vậy, sau khi đã xét đóng góp của các cây con theo cạnh nhẹ, bài toán trên mỗi chuỗi nặng giống một bài tổng đoạn con lớn nhất có giới hạn bậc đỉnh, và cũng có thể duy trì bằng cây đoạn.

Gọi các đỉnh trên chuỗi nặng theo thứ tự khoảng cách đến gốc tăng dần là $p_1, p_2, \dots, p_c$. Với mỗi nút cây đoạn, giả sử đoạn là $[l, r]$, ghi các thông tin sau, với tiền đề chung là “mọi đỉnh đều nằm trong cây con của p_l ”:

- $\text{cro}(a, b)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_l và p_r , trong đó bậc của p_l là a , bậc của p_r là b ;
- $\text{lef}(a)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_l nhưng không chứa p_r , trong đó bậc của p_l là a ;
- $\text{rig}(b)$: tổng trọng số lớn nhất của đồ thị con liên thông chứa p_r nhưng không chứa p_l , trong đó bậc của p_r là b ;
- mid : tổng trọng số lớn nhất của đồ thị con liên thông không chứa cả p_l lẫn p_r .

Cách duy trì thông tin giống bài tổng đoạn con lớn nhất. Khi gộp hai đoạn, liệt kê trạng thái của nút trái và nút phải, kiểm tra bậc của đỉnh giữa có thể gộp hay không (để gộp cần thêm cạnh giữa hai đỉnh đó, nên bậc của cả hai đều phải nhỏ hơn k); nếu gộp được thì cập nhật trạng thái sau khi gộp. Độ phức tạp gộp thông tin là $\mathcal{O}(k^4)$, có thể tối ưu bằng tổng tiền tố xuống $\mathcal{O}(k^2)$.

Vì cần đếm số phương án, thông tin không được dư thừa: phải phân loại nghiêm ngặt để mỗi đồ thị con thuộc đúng một lớp. Một số chi tiết cần chú ý là: $\text{cro}(a, *)$ của nút trái có thể cập nhật $\text{lef}(a)$ của nút hiện tại; $\text{cro}(*, b)$ của nút phải có thể cập nhật $\text{rig}(b)$; $\text{rig}(*)$ của nút trái và $\text{lef}(*)$ của nút phải đều có thể cập nhật mid . Trong trạng thái sau khi gộp, nếu cả hai nút con đều không phải đoạn đơn thì bậc của p_l sau gộp là bậc của p_l trong trạng thái nút trái, bậc của p_r sau gộp là bậc của p_r trong trạng thái nút phải. Nhưng nếu một nút con là đoạn đơn, ví dụ nút trái là đoạn đơn, thì bậc của p_l sau gộp sẽ thay đổi (trong bài này chỉ tăng 1), nên cũng cần duy trì.

Cuối cùng còn phải tính thông tin $\text{cro}(*, *)$, $\text{lef}(*, *)$, $\text{rig}(*, *)$, mid cho từng điểm trên chuỗi. Nhờ phân rẽ nặng–nhẹ, con nhẹ của một đỉnh chính là đầu trái của chuỗi nặng chứa con nhẹ đó, nên lại có thể mượn thông tin cây đoạn của chuỗi chứa con nhẹ. Với một đỉnh đơn, chỉ có mid và $\text{cro}(a, a)$ có ý nghĩa.

Để tiện, với mỗi chuỗi nặng, ghi một “thông tin chuỗi nặng”: $\text{val}(d)$ là tổng trọng số lớn nhất của đồ thị con nằm hoàn toàn trong cây con của đỉnh đầu chuỗi, chứa đỉnh đầu chuỗi và bậc của đỉnh đầu chuỗi là d ; ans là tổng trọng số lớn nhất nằm hoàn toàn trong cây con của đỉnh đầu chuỗi nhưng không chứa đỉnh đầu chuỗi. Có thể lấy $\text{val}(d)$ từ $\text{lef}(d)$ và $\text{cro}(d, *)$ của gốc cây đoạn của chuỗi; lấy ans từ mid và $\text{rig}(*, *)$.

mid của một đỉnh đơn là giá trị lớn nhất trong ans của thông tin chuỗi nặng của các con nhẹ. Còn $\text{cro}(a, a)$ của đỉnh đơn là kết quả DP ba lô trên các mảng $\text{val}(\ast)$ của thông tin chuỗi nặng của từng con nhẹ. Để hỗ trợ sửa, với mỗi đỉnh dùng một cây đoạn hoặc cây cân bằng để duy trì phép gộp ba lô các $\text{val}(\ast)$ của mọi chuỗi nặng con nhẹ và giá trị ans lớn nhất. Như vậy có thể tính thông tin đỉnh đơn trên chuỗi và hỗ trợ sửa.

Mỗi lần sửa, cũng trước hết sửa chuỗi nặng chứa đỉnh bị sửa, rồi nhảy tới chuỗi nặng chứa cha của đỉnh đầu chuỗi hiện tại để cập nhật một điểm; cứ thế đến gốc. Tương tự, khi nhảy giữa các chuỗi nặng, cần sửa trong cây đoạn hoặc cây cân bằng duy trì thông tin chuỗi nặng con nhẹ.

Độ phức tạp là $\mathcal{O}(n) + \mathcal{O}(\log^2 n k^2) + \mathcal{O}(n)$. Các subtasks qua được và điểm kỳ vọng giống thuật toán ba.

Đến đây, ta đã giải thành công mọi dữ liệu trên cây. Tiếp theo xét dữ liệu trên đồ thị tổng quát.

7.5 Dữ liệu đồ thị tổng quát

Trong subtask 5 đến 8, tổng 40 điểm, không có cập nhật; chỉ cần giải một lần.

7.5.1 Tính chất đặc biệt

Chú ý điều kiện đặc biệt của đề: với mọi tập V' gồm $t \in \{5, 7\}$ đỉnh bất kỳ trong V , luôn tồn tại ba đỉnh đôi một khác nhau p, q, x sao cho $p, q \in V', x \in V - \{p, q\}$, và sau khi xóa x khỏi đồ thị thì p và q không liên thông. Xét ý nghĩa của điều kiện này.

Nhắc lại kiến thức về thành phần song liên thông đỉnh.²³ Ta biết hai đỉnh $p, q \in V$ song liên thông đỉnh khi và chỉ khi không tồn tại $x \in V - \{p, q\}$ sao cho sau khi xóa x , p và q không liên thông. Trong một đồ thị vô hướng, có thể dùng thuật toán Tarjan để tìm từng thành phần song liên thông đỉnh. Nếu hai đỉnh p, q cùng nằm trong một thành phần song liên thông đỉnh C , thì p, q song liên thông đỉnh; ngược lại thì không. Hai thành phần song liên thông đỉnh có không quá một đỉnh chung. Nếu hai thành phần A, B có đỉnh chung x , thì với mọi $p \in A - \{x\}, q \in B - \{x\}$, sau khi xóa x , p và q không liên thông. Ta gọi x là điểm khớp (giữa A và B).

Từ các tính chất trên, đoán điều kiện đặc biệt tương đương với mệnh đề: đồ thị không có thành phần song liên thông đỉnh có kích thước ít nhất t , tức mọi thành phần song liên thông đỉnh đều có kích thước nhỏ hơn t . Thử chứng minh:

Chứng minh. (Đủ) Phản chứng. Nếu tồn tại một thành phần song liên thông đỉnh có kích thước không nhỏ hơn t , chỉ cần lấy t đỉnh bất kỳ trong thành phần đó để tạo V' . Khi đó các cặp đỉnh trong V' đều song liên thông đỉnh, tức không tồn tại $x \in V - \{p, q\}$ sao cho sau khi xóa x , p và q không liên thông, mâu thuẫn đề bài.

(Cần) Vẫn phản chứng. Nếu tồn tại t đỉnh tạo thành tập V' sao cho không tồn tại ba đỉnh đôi một khác nhau $p, q \in V', x \in V - \{p, q\}$ làm cho sau khi xóa x , p và q không liên thông, thì theo định nghĩa, mọi $p, q \in V'$ đều song liên thông đỉnh. Vì vậy V' là một thành phần song liên thông đỉnh hoặc là tập con của một thành phần song liên thông đỉnh; do đó nhất định tồn tại một thành phần song liên thông đỉnh có kích thước ít nhất t , mâu thuẫn giả thiết. ■

Điều kiện đặc biệt trong đề có $t = 7$, nên mỗi thành phần song liên thông đỉnh có kích thước không vượt quá 6. Có thể thiết kế thuật toán dựa trên tính chất này. Trong các thuật toán sau, nếu không gây nhầm lẫn, ta gọi tất thành phần song liên thông đỉnh là “khối song liên thông”.

²³Cài đặt cụ thể của thành phần song liên thông đỉnh và chứng minh các định lý liên quan có thể xem [2].

7.5.2 Thuật toán sáu: vét cạn trên các khối song liên thông

Subtask 5 và 6 thỏa tính chất 3, nên mỗi khối song liên thông có kích thước không vượt quá 4.

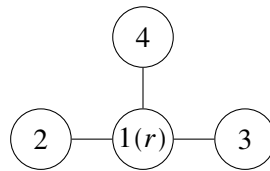
Quá trình Tarjan tìm khối song liên thông như sau. Trong DFS, duy trì một ngăn xếp. Mỗi khi DFS tới đỉnh i , đẩy i vào ngăn xếp, tăng bộ đếm thời gian, rồi đặt $\text{dfn}(i)$ và $\text{low}(i)$ bằng giá trị bộ đếm hiện tại. Với mỗi cạnh (i, p) , nếu đó là cạnh ngược lên tổ tiên, dùng $\text{dfn}(p)$ cập nhật $\text{low}(i)$; ngược lại DFS vào p , sau khi quay về dùng $\text{low}(p)$ cập nhật $\text{low}(i)$. Nếu $\text{low}(p) \geq \text{dfn}(i)$, ta tìm được một khối song liên thông C : pop ngăn xếp đến khi pop p . Nếu tập đỉnh được pop là S , thì tập đỉnh của khối vừa tìm là $V_C = S \cup \{i\}$, và gọi i là gốc của C .

Nếu một đỉnh p nằm trong một khối C và không phải gốc của C , ta nói p thật sự thuộc C . Khi đó, trừ đỉnh đầu tiên gọi Tarjan (gốc của cây DFS), mỗi đỉnh thật sự thuộc đúng một khối. Mỗi khối có đúng một gốc, và gốc của nó thật sự thuộc một khối khác; vì vậy các khối tạo thành một rừng cây có gốc, trong đó cha của mỗi khối là khối chứa gốc của nó (nếu có). Gọi rừng cây có gốc này là cây khối song liên thông. Kết hợp với quá trình Tarjan, có thể thấy Tarjan tương đương với duyệt cây khối theo thứ tự DFS.

Bây giờ xét DP trên cấu trúc cây. Thiết kế trạng thái $f(i, d)$: tổng trọng số lớn nhất của một đồ thị con liên thông có mọi đỉnh nằm trong khối song liên thông gốc i (và mọi khối trong cây con của các khối ấy), đồng thời bậc của i trong đồ thị con là d . Vì Tarjan duyệt cây khối theo DFS, với mọi $p \in S$, toàn bộ khối có gốc p và mọi khối dưới nó đã được xét xong (khi tìm khối có gốc i , đỉnh i chắc chắn còn trên ngăn xếp), nên $f(p, *)$ cũng đã được tính.

Xét xử lý trong khối C để cập nhật đáp án và $f(i, *)$. Vì $|V_C| \leq 4$, số cạnh không vượt quá $\binom{4}{2} = 6$, nên có thể giống thuật toán một: vét cạn tập cạnh để liệt kê một đồ thị con của C , tức liệt kê phần cạnh của đồ thị con cần tìm bên trong khối C .

Tiếp theo, còn phải xét cách nối đồ thị con này với các cây con trong cây khối. Hình sau là một đồ thị con đang được liệt kê (giả sử $k = 3$):



Ví dụ, trong phần cạnh hiện tại của khối, bậc của đỉnh 2 là 1; ta còn có thể thêm vào 2 nhiều nhất 2 bậc từ phía dưới. Do đó cách nối trong cây con chỉ cần thỏa bậc của 2 không vượt quá 2, tức có thể ghép ²⁴ đồ thị con hiện tại với một đồ thị con “mọi đỉnh đều nằm trong khối có gốc 2 (và mọi khối trong cây con dưới nó), đồng thời bậc của đỉnh 2 không vượt quá 2”. Vậy trọng số tốt nhất của đồ thị con hiện tại có thể cộng thêm $\max(f(2, 0), f(2, 1), f(2, 2))$.

Thực hiện phép ghép như vậy cho mỗi đỉnh trong đồ thị con G' đang xét, ta tính được tổng trọng số lớn nhất khi phần nằm trong C là G' và đã xét mọi khối trong cây con của C ; gọi giá trị đó là $g(G')$. Nếu G' chứa gốc i , dùng $g(G')$ cập nhật $f(i, *)$ (đây là một bài toán ba lô); ngược lại dùng $g(G')$ cập nhật đáp án toàn cục ans, vì khi đó đồ thị con hiện tại không còn liên thông với i và các đỉnh phía trên, nên cần cập nhật đáp án tại đây. Cần chú ý: khi tính tổng trọng số lớn nhất của G' , chỉ cộng trọng số các đỉnh không phải gốc trong tập đỉnh của G' ; trọng số của gốc được xét trong khối chứa thật sự đỉnh gốc, tránh tính sai tổng trọng số đồ thị con.²⁵ Sau khi Tarjan chạy xong, ta thu được đáp án. Số phương án có thể được duy trì đồng thời trong quá trình này.

²⁴Ở đây “ghép” hai đồ thị con nghĩa là lấy hợp tập đỉnh và hợp tập cạnh của chúng.

²⁵Nếu cộng trọng số của gốc, thì trong mọi khối có cùng gốc đó trọng số gốc đều bị cộng một lần, làm $f(i, *)$ tính lặp trọng số gốc.

Gọi c là kích thước khối song liên thông lớn nhất trong đồ thị. Độ phức tạp xấu nhất là

$$\mathcal{O}\left(c2^{\binom{c}{2}}n + m\right) + \mathcal{O}\left(c2^{\binom{c}{2}}n + m\right) + \mathcal{O}(n + m).$$

Thuật toán qua được subtask 1, 2, 5, 6; có thể qua subtask 3, kỳ vọng 46 đến 53 điểm.²⁶

7.5.3 Thuật toán bảy: tiền xử lý đồ thị con

Subtask 7 và 8 không còn thỏa tính chất 3; một khối song liên thông có thể có $\binom{6}{2} = 15$ cạnh, và tổng số đỉnh n rất lớn, nên độ phức tạp của thuật toán sáu không qua được.

Nút thắt của thuật toán sáu nằm ở việc liệt kê mọi đồ thị con của mỗi khối. Xét tối ưu việc liệt kê. Trong thuật toán một, ta gộp các trạng thái tương đương và không đổi (tiền xử lý số cách nối cạnh cho mỗi tập đỉnh), từ đó tối ưu độ phức tạp; ở đây cũng có thể làm tương tự. Nhưng bây giờ còn phải xét giới hạn bậc, nên cần ghi trạng thái: một số tứ phân 6 chữ số deg²⁷ nén trạng thái bậc của từng đỉnh (gọi tắt là “nén trạng thái bậc”), trong đó chữ số thứ i , từ thấp đến cao, biểu thị bậc của đỉnh thứ i trong đồ thị. Hai trạng thái có cùng deg có cùng tổng trọng số và cách chuyển như nhau, nên chỉ cần tiền xử lý số cách nối cạnh cho từng trạng thái deg. Có một điểm tiện lợi: với một đồ thị có ít nhất một cạnh, mọi đỉnh trong nó đều có bậc ít nhất 1, nên ghi deg cũng biểu thị được tập đỉnh. Đặt $f(\text{deg})$ là số đồ thị con khác nhau có nén trạng thái bậc là deg. Khi đó chỉ cần tính từng $f(\text{deg})$ là có thể thống kê nghiệm tối ưu và số phương án, không cần liệt kê từng đồ thị con.

Xét cách tính $f(\text{deg})$. Có thể ngay từ đầu tiền xử lý mọi đồ thị có không quá 6 đỉnh thỏa giới hạn bậc (không quá 20000 đồ thị), rồi sắp theo số đỉnh. Khi xử lý mỗi khối, chỉ cần liệt kê mọi đồ thị có số đỉnh không vượt quá kích thước khối hiện tại, dùng phép bit để kiểm tra nhanh tập cạnh của đồ thị đó có phải là tập con của khối hiện tại hay không. Như vậy mỗi khối chỉ cần chưa đến 20000 phép tính. Số trạng thái khác nhau về lý thuyết chỉ là 4^6 , thực tế chưa đến 1600, nên có thể qua dữ liệu subtask 7, 8.

Để tăng tốc chương trình nhằm qua subtask 3, có thể tối ưu thêm. Thuật toán này thực chất là DP trên cây theo giai đoạn là cây con; sau khi sửa đỉnh p , chỉ các giá trị DP trên đường từ p đến gốc có thể thay đổi. Vì vậy mỗi lần sửa cũng chỉ cần sửa các đỉnh trên đường này. Chú ý trong DP ban đầu có thao tác “cập nhật đáp án toàn cục”; có thể không ghi biến toàn cục ans, mà ghi $g(i)$ là tổng trọng số lớn nhất sau khi xét mọi khối có gốc i và mọi khối trong cây con của chúng. Khi đó mỗi lần cũng chỉ cần cập nhật $g(i)$ trên đường từ đỉnh bị sửa đến gốc. Thực tế, sau các tối ưu này, thuật toán có thể qua một phần test của subtask 10.

Gọi c là kích thước khối lớn nhất; $S(s, k)$ là số đồ thị con có không quá s đỉnh và mọi đỉnh có bậc không vượt quá k ; ²⁸ $G(s, k)$ là số nén trạng thái bậc của các đồ thị con có không quá s đỉnh và mọi đỉnh có bậc không vượt quá k .²⁹ Độ phức tạp của thuật toán là

$$\mathcal{O}\left(c2^{\binom{c}{2}} + (S(c, k) + G(c, k)c)n + m\right) + \mathcal{O}(G(c, k)cn) + \mathcal{O}(S(c, k) + G(c, k)n + m).$$

Thuật toán qua được subtask 1, 2, 5, 6, 7, 8; có thể qua subtask 3, kỳ vọng 65 đến 72 điểm.

7.6 Cách chuyển hóa tốt hơn

Hai subtasks cuối có nhiều cập nhật, cần hỗ trợ sửa hiệu quả hơn.

²⁶Với dữ liệu cây, $c = 2$.

²⁷Cơ số là lũy thừa của 2 cho phép dùng phép bit để truy vấn, sửa từng chữ số nhanh; vì vậy dù k có giá trị nào, vẫn dùng cơ số 4.

²⁸Theo trên, $S(6, 3) \leq 20000$.

²⁹Theo trên, $G(6, 3) \leq 1600$.

7.6.1 Cây tròn–vuông: cải tiến cây khối song liên thông

Ở trên ta đã nhắc đến cấu trúc cây khối song liên thông. Nhưng cấu trúc này tương đương với việc gom mọi đỉnh của một khối lại với nhau, gây nhiều bất tiện: trong chuyển trạng thái vừa phải liệt kê đồ thị con, vừa phải thống kê trọng số, lại phải xử lý riêng nhiều chi tiết. Vì vậy cấu trúc cây này khó được duy trì trực tiếp bằng phân trị theo chuỗi.

Bây giờ xét sửa cây khối song liên thông để DP gọn hơn và dễ tối ưu hơn.

Ví dụ 2. Thương nhân. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Có q truy vấn; mỗi truy vấn cho hai đỉnh x, y , yêu cầu tìm một đường đi đơn từ x đến y sao cho trọng số nhỏ nhất trên đường đi là nhỏ nhất. Dữ liệu: $n, m, q \leq 5000000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 2 giây, bộ nhớ 512 MB.³⁰

Nếu $x = y$, đáp án là a_x . Sau đây mặc định $x \neq y$.

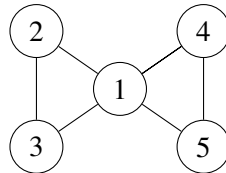
Trước hết, nếu tồn tại một khối song liên thông C chứa đồng thời x, y , thì đáp án là trọng số nhỏ nhất trong khối đó. Chứng minh như sau.

Chứng minh. Trước hết chứng minh với mọi $q \in C$, tồn tại một đường đi đơn từ x đến y đi qua q . Tạo đỉnh phụ p , nối $(p, x), (p, y)$. Vì x, y liên thông, tồn tại một đường đi từ y đến x không qua p . Khi đó từ p đến x có hai đường đi: $p \leftrightarrow x$ và $p \leftrightarrow y \rightarrow x$; do đó p và x song liên thông đỉnh. Tương tự, p và y song liên thông đỉnh, nên x, p, y cùng thuộc một khối. Vì hai khối có nhiều nhất một đỉnh chung, p nhất định thuộc khối $C \cup \{p\}$. Vì vậy với mọi $q \in C$, tồn tại hai đường đi không giao nhau về đỉnh giữa p và q ; trong đó một đường đi qua x , một đường đi qua y . Nhờ đó tồn tại đường đi đơn $x \rightarrow q \rightarrow y$. Vậy mọi đỉnh q trong khối đều có thể nằm trên một đường đi đơn từ x đến y .

Tiếp theo chứng minh với mọi $q \notin C$, không tồn tại đường đi đơn từ x đến y đi qua q . Với điểm q ngoài khối, lấy một khối bất kỳ D chứa nó. Giả sử trên đường đi trong cây khối từ C đến D , dãy khối là $C, P_1, P_2, \dots, P_c, D$. Khi đó từ x đến q phải đi qua điểm chung của C và P_1 ; từ q đến y cũng phải đi qua điểm chung đó. Điểm này bị đi qua hai lần, nên q không nằm trên bất kỳ đường đi đơn nào từ x đến y . ■

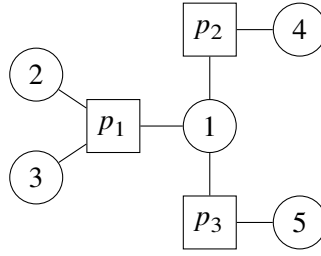
Tiếp tục xét trên cây khối. Có thể đoán tính chất: với mọi truy vấn x, y , đáp án là giá trị nhỏ nhất của các đỉnh trong mọi khối trên đường đi cây khối từ khối chứa x đến khối chứa y . Nhưng mô tả này chưa đủ cụ thể: x, y có thể nằm trong nhiều khối, vậy cặp khối nào mới đúng? Chọn tùy ý thì không đúng; phản ví dụ là đồ thị ghép từ ba tam giác. Một mô tả đúng là chọn cặp gần nhất. Nhưng mô tả ấy vẫn chưa đủ cụ thể và cũng khó cài đặt. Hạn chế của cây khối lộ rõ; ta cần một cách chuyển đồ thị thành cây tốt hơn.

Xét cách chuyển sau: với mỗi khối song liên thông C , tạo một đỉnh phụ p_C , nối p_C với mọi đỉnh trong C , và xóa mọi cạnh gốc trong khối (vì hai khối có nhiều nhất một đỉnh chung, nên không cạnh nào bị xóa lặp). Gọi các đỉnh của đồ thị gốc là đỉnh tròn, các đỉnh phụ mới là đỉnh vuông. Ví dụ đồ thị sau:



Chuyển thành cây tròn–vuông như sau (một cạnh cũng là một khối):

³⁰Nguồn: giản lược từ UOJ 30, đồng thời mở rộng dữ liệu.



Mỗi đỉnh vuông trong hình tương ứng với một khối. Để thấy đồ thị sau chuyển hóa tương đương với việc sửa cây khối như sau: 1. trong cây khối, mỗi khối nối trực tiếp với khối cha; sau chuyển hóa, đỉnh vuông của mỗi khối trước hết nối với đỉnh tròn là gốc của khối đó, rồi từ đỉnh tròn gốc nối với đỉnh vuông của khối cha; 2. các đỉnh không trở thành gốc của khối nào được giữ lại trên cây dưới dạng lá. Từ các sửa đổi này có thể thấy đồ thị sau chuyển hóa vẫn là một cây. Ta gọi cây này là cây tròn–vuông của đồ thị gốc.

Vì mỗi đỉnh phụ nối ra một “cây hoa cúc”, hình dạng cây tròn–vuông không phụ thuộc vào hình dạng cây khối. Nói cách khác, dù bắt đầu thuật toán Tarjan từ đỉnh nào, cây tròn–vuông dựng được đều giống nhau. Hơn nữa, vì cây giữ lại mọi đỉnh của đồ thị gốc, nó xử lý quan hệ giữa các khối thuận tiện hơn: nếu hai khối A, B của đồ thị gốc có điểm chung q , thì trên cây tròn–vuông có các cạnh (p_A, q) và (q, p_B) . Thay vì xem cây tròn–vuông là kết quả sửa cây khối, nên xem nó như một cách hoàn toàn mới để tổ chức thông tin giữa các khối.

Xét giải ví dụ trên cây tròn–vuông. Đặt trọng số của mỗi đỉnh vuông bằng trọng số nhỏ nhất trong khối tương ứng. Ta bất ngờ có: đáp án chính là trọng số nhỏ nhất trên đường đi từ x đến y trong cây tròn–vuông. Chứng minh tính chất này như sau. Mỗi khối nằm trên đường đi tròn–vuông từ x đến y đều có thể được đi vào từ một đỉnh tròn và đi ra từ một đỉnh tròn khác, do đó có thể đi qua bất kỳ đỉnh nào trong khối. Còn muốn đến một khối không nằm trên đường đi đó thì phải đi qua lặp lại một đỉnh tròn nào đó (trên đường giữa hai đỉnh vuông nhất định có ít nhất một đỉnh tròn), nên phải lặp đỉnh.

Bài toán còn lại là hỏi giá trị nhỏ nhất trên đường đi x đến y của một cây. Bài này rất giống bài kinh điển *Vận chuyển hàng hóa* của NOIP2013; dùng thuật toán trong mục 3.2.1 của [4] có thể đạt $\mathcal{O}(m + na(n)) + \mathcal{O}(q \log \log n) + \mathcal{O}(n \log \log n + m)$. Vì thuật toán không cần xử lý tình huống đặc biệt, mã cài đặt khá đẹp.

Ví dụ này thể hiện sức mạnh và sự tiện lợi của cây tròn–vuông. Bây giờ xét áp dụng cây tròn–vuông vào bài gốc.

7.6.2 Thuật toán tám: dùng cây tròn–vuông giải subtask 7 và 8

Ta đã có công cụ mạnh là cây tròn–vuông. Thử dùng nó giải subtask 7, 8 để tìm một thuật toán gọn hơn.

Trước hết dựng cây tròn–vuông của đồ thị gốc. Trong cây tròn–vuông, mọi cạnh của đồ thị gốc đều không được giữ lại, nên mọi cách nối cạnh trong đồ thị con phải được xử lý ở đỉnh vuông tương ứng. Bài này khá giống bài trên cây với $k = 3$, nên có thể mượn thuật toán bốn. Vẫn ghi trạng thái $f(i, d)$: nếu i là đỉnh tròn, nó biểu thị tổng trọng số lớn nhất của đồ thị con có mọi đỉnh nằm trong cây con tròn–vuông của i , có chứa i , và bậc của i trong đồ thị con đúng bằng d ; nếu i là đỉnh vuông, nó biểu thị tổng trọng số lớn nhất của đồ thị con có mọi đỉnh nằm trong cây con tròn–vuông của i , có chứa gốc r_i của khối tương ứng với i , và bậc của r_i trong đồ thị con đúng bằng d .

Xét chuyển trạng thái. DFS trên cây. Nếu i là đỉnh tròn thì mọi con của nó đều là đỉnh vuông, và gốc của các khối tương ứng đều là i . Chỉ cần làm DP ba lô trên các $f(p, *)$ của mọi con $p \in \text{Ch}(i)$, rồi cuối cùng cộng v_i .

Nếu i là đỉnh vuông, cần liệt kê cách nối cạnh trong khối tương ứng (dùng cách của thuật toán bảy để tối ưu liệt kê). Với mỗi đồ thị con được liệt kê, ta cũng xét mỗi đỉnh không phải gốc p trong đồ thị con

còn có thể thêm bao nhiêu bậc từ phía dưới. Nếu có thể thêm d bậc, trọng số đồ thị con hiện tại cộng thêm $\max_{k=0}^d f(p, k)$. Trên cây tròn–vuông, các con của một đỉnh vuông chính là mọi đỉnh không phải gốc trong khối tương ứng, nên các $f(p, *)$ của chúng đều đã tính xong. Cuối cùng, nếu đồ thị con này chứa gốc r_i , gọi bậc của r_i trong đồ thị con là d , thì dùng trọng số đồ thị con cập nhật $f(i, d)$; ngược lại dùng trọng số đó cập nhật đáp án toàn cục ans.

Thuật toán này nhìn qua không khác thuật toán bấy nhiêu. Nhưng nó có một đặc điểm: mọi cách nối cạnh được xử lý tại đỉnh vuông, mọi phép tính trọng số và giới hạn bậc được xử lý tại đỉnh tròn. Việc phân công rõ ràng giữa đỉnh tròn và đỉnh vuông không chỉ khiến mô tả thuật toán rõ hơn, giảm độ phức tạp mã, mà còn rất thuận lợi cho tối ưu thuật toán sau này.

Độ phức tạp, subtasks qua được và điểm kỳ vọng đều giống thuật toán bấy.

7.6.3 Thuật toán chín: lời giải trọn điểm

Bây giờ mọi thứ đã sẵn sàng. Kết hợp phân trị theo chuỗi và cây tròn–vuông, ta đã rất gần lời giải trọn điểm. DP trên cây tròn–vuông gọn hơn DP trên cây khối trước đó, nên dễ dùng phân trị theo chuỗi để tối ưu. Trên mỗi chuỗi nặng, bài toán vẫn là tổng đoạn con lớn nhất có giới hạn bậc đỉnh, nên cách giải tương tự phần trước.

Trong cây đoạn trên chuỗi nặng, vẫn duy trì bốn thông tin $\text{cro}(*, *)$, $\text{lef}(*, *)$, $\text{rig}(*, *)$, mid . Định nghĩa ở đây gần giống trước nhưng có khác biệt nhỏ. Với mỗi đoạn, định nghĩa “điểm thật bên trái” p'_l và “điểm thật bên phải” p'_r : nếu p_l là đỉnh tròn thì $p'_l = p_l$, ngược lại p'_l là cha của p_l trên cây tròn–vuông, tức gốc của khối tương ứng với p_l (khi $l \neq 1$, đỉnh này chính là p_{l-1}); nếu p_r là đỉnh tròn thì $p'_r = p_r$, ngược lại $p'_r = p_{r+1}$ (ở đây p_{r+1} chắc chắn tồn tại vì cuối chuỗi nặng nhất định là lá). Nói cách khác, ở đây ngoài việc ghi thông tin bậc của gốc khối (đỉnh tròn cha) cho đỉnh vuông, trạng thái còn ghi thông tin của đỉnh tròn kế tiếp trong chuỗi nặng. Khi đó $\text{cro}(a, b)$ có nghĩa là “đồ thị con có trọng số lớn nhất khi bậc của p'_l đúng bằng a , bậc của p'_r đúng bằng b ”. Các trạng thái khác cũng sửa tương ứng: bậc được ghi là bậc của p'_l và p'_r .

Mỗi lần gộp thông tin, tại trung điểm $m = \lfloor (l + r) / 2 \rfloor$, ta đang xét bậc của cùng một đỉnh, tức đỉnh tròn trên cạnh (p_m, p_{m+1}) (mỗi cạnh có một đầu tròn và một đầu vuông). Đỉnh này vừa là điểm thật bên phải của nút trái, vừa là điểm thật bên trái của nút phải; do đó điều kiện hợp lệ khi gộp là tổng bậc của đỉnh đó trong hai trạng thái con không vượt quá k . Ở đây vẫn cần chú ý các chi tiết đã nói trong thuật toán năm, chẳng hạn trường hợp một nút con là đoạn đơn.

Còn vấn đề cuối cùng: tính thông tin đỉnh đơn trên mỗi chuỗi nặng. Nếu đỉnh đó là đỉnh tròn, vẫn dùng cách trong thuật toán năm: dùng cây đoạn duy trì thông tin của các chuỗi nặng đi ra, và cuối cùng mọi $\text{cro}(a, a)$ đều cộng thêm v_i . Nếu đỉnh đó là đỉnh vuông, cần liệt kê mọi trạng thái đồ thị con trong khối tương ứng, tức trạng thái dạng (set, deg) như trên. Giả sử đỉnh đơn cây đoạn là i ; trong trạng thái này, với các đỉnh tròn không phải điểm thật trái cũng không phải điểm thật phải, chúng nhất định là con nhẹ, nên dùng thông tin chuỗi nặng của chúng để ghép. Sau khi tính nghiệm tối ưu của trạng thái này, dựa vào việc trạng thái có chứa hai điểm thật hay không và bậc của chúng để quyết định cập nhật loại thông tin nào trong bốn loại thông tin của đỉnh đơn; số phương án phải nhân với số đồ thị con khác nhau của (set, deg) đã tiền xử lý. Một cách cài đặt tiện lợi là trong nén trạng thái, đặt điểm thật trái của đỉnh đơn là đỉnh số 0, điểm thật phải là đỉnh số 1; cách này giảm nhiều trường hợp riêng. Đừng quên ans trong thông tin chuỗi nặng của con nhẹ cũng phải cập nhật mid .

Mỗi lần sửa, trong quá trình nhảy lên theo phân rã cây, cần cập nhật thông tin đỉnh đơn của cha đỉnh đầu mọi chuỗi nặng đi qua. Nếu đỉnh này là đỉnh vuông, vẫn cần 1600×4 phép tính (số 4 là số đỉnh trong khối sau khi bỏ p_{i-1} và p_{i+1}) để liệt kê trạng thái và duy trì thông tin bằng vét cạn. Khi khởi tạo, ta tiền xử lý và lưu từng nén trạng thái đồ thị con cùng số phương án của nó, không cần mỗi lần đều duyệt mọi đồ thị con.

Kế thừa các ký hiệu $c, S(s, k), G(s, k)$ của thuật toán bầy, độ phức tạp là

$$\mathcal{O}\left(c2^{\binom{s}{2}} + (S(c, k) + G(c, k)c)n + m\right) + \mathcal{O}\left((G(c, k)c + k^4 \log n) \log n\right) + \mathcal{O}(S(c, k) + G(c, k)n + m).$$

Thuật toán qua được mọi subtask, kỳ vọng 100 điểm.

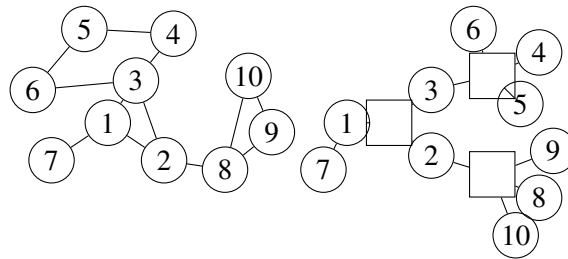
7.7 Mở rộng

Trong bài này, ta dựng cây tròn–vuông của đồ thị gốc, từ đó chuyển bài toán về cây và giải bằng phân trị theo chuỗi. Trên thực tế, cả cây tròn–vuông lẫn phân trị theo chuỗi đều có khả năng mở rộng lớn. Sau đây giới thiệu một số ứng dụng của chúng.

7.7.1 Bài toán trên xương rồng

Một xương rồng được định nghĩa là một đồ thị vô hướng liên thông không có khuyên, trong đó mỗi cạnh thuộc không quá một chu trình đơn.

Dễ thấy mỗi chu trình trong xương rồng là một khối song liên thông, nên tự nhiên cũng có thể dùng cây tròn–vuông để giải. Vì cần xử lý thông tin trên chu trình, ta còn phải duy trì thứ tự các đỉnh tròn kề với từng đỉnh vuông. Đồng thời, để phân biệt chu trình độ dài 2 (cạnh song song) với cạnh cây thông thường, ta có thể không tạo đỉnh vuông cho cạnh cây thông thường, mà chỉ tạo đỉnh vuông cho chu trình. Sau đây là một xương rồng và cây tròn–vuông của nó:



Thực ra, cây tròn–vuông ban đầu được đề xuất chính là để giải các bài toán tính trên xương rồng hiệu quả và tiện lợi hơn. Cách xử lý thành phần song liên thông đỉnh được cải tiến từ cách xử lý xương rồng.

Sau nghiên cứu, gần như mọi bài toán xương rồng tính như DP trên xương rồng,³¹ đường đi ngắn nhất trên xương rồng,³² phân rã chuỗi trên xương rồng,³³ xương rồng ảo,³⁴ và phân trị theo đỉnh trên xương rồng³⁵ đều có thể giải trực tiếp bằng cây tròn–vuông. Hơn nữa, theo nghiên cứu của các bạn khác,³⁶ ngay cả xương rồng động và k -xương rồng động³⁷ cũng có thể được giải bằng tư tưởng tương tự cây tròn–vuông.

Ứng dụng của cây tròn–vuông trên xương rồng không liên quan đến chủ đề chính của bài này và khá dài, nên được lược bỏ. Thuật toán cụ thể có trong blog UOJ của tác giả³⁸ và trong slide trao đổi học viên WC2017 của tôi.³⁹

³¹<http://www.lydsy.com/JudgeOnline/problem.php?id=4316>

³²<http://www.lydsy.com/JudgeOnline/problem.php?id=2125>

³³<http://uoj.ac/problem/158>

³⁴<http://uoj.ac/problem/189>

³⁵<http://uoj.ac/problem/23>

³⁶<http://awd.blog.uoj.ac/blog/2319>

³⁷Đồ thị vô hướng liên thông trong đó mỗi cạnh thuộc nhiều nhất k chu trình đơn. Xương rồng thường nói là 1-xương rồng.

³⁸<http://immortalco.blog.uoj.ac/blog/1955>

³⁹Xem tài liệu tham khảo [3], có thể tải từ blog UOJ của tôi.

7.7.2 DP cây con có hỗ trợ sửa dữ liệu vào

Trong bài này, ta dùng phân trị theo chuỗi để hỗ trợ hiệu quả thao tác sửa một đỉnh và duy trì giá trị DP toàn cục. Thực tế, thuật toán này có thể mở rộng sang một lớp bài tổng quát hơn: sửa dữ liệu vào tại một đỉnh và hỏi giá trị DP của một cây con trong cây có gốc.

Ví dụ 3. Bài toán khổng chế dịch bệnh. Cho một cây có gốc gồm n đỉnh, mỗi đỉnh có trọng số a_i . Với mỗi đỉnh, bạn có thể trả chi phí a_i để chọn toàn bộ lá trong cây con của i . Có m thao tác, mỗi thao tác thuộc một trong hai loại:

1. sửa a_i của một đỉnh;
2. nhập i , hỏi chi phí nhỏ nhất để chọn mọi lá trong cây con của i .

Dữ liệu $n, m \leq 1000000$. Giới hạn thời gian 1 giây, bộ nhớ 256 MB. ⁴⁰

Bài gốc có tính chất đặc biệt: sau cập nhật, a_i không giảm. Lời giải chuẩn của bài gốc xuất phát từ tính chất này và thiết kế một thuật toán trả góp dựa trên sự thay đổi của điểm quyết định. Ở đây không có bảo đảm ấy, nên thuật toán cũ không dùng được.

Nếu mỗi truy vấn đều làm một DP vét cạn riêng, có thể ghi $f(i)$ là chi phí nhỏ nhất để chọn mọi lá trong cây con của i . Nếu i là lá thì $f(i) = a_i$, ngược lại

$$f(i) = \min \left(a_i, \sum_{p \in \text{Ch}(i)} f(p) \right).$$

Thời gian là tuyến tính.

Bây giờ xét dữ liệu có cập nhật. Cây trong bài là cây có gốc, và việc tính giá trị DP có thứ tự nghiêm ngặt; nếu dùng phân trị theo đỉnh thì khó duy trì thứ tự tính DP, càng khó hỗ trợ truy vấn.

Tiếp tục xét phân trị theo chuỗi. Mượn tư tưởng ở trên, tính DP theo thứ tự của cây chuỗi nặng. Vì cây chuỗi nặng cũng là cây có gốc, tính DP theo thứ tự của nó chỉ tương đương với việc chọn thứ tự chuyển trạng thái giữa các con của mỗi đỉnh, không ảnh hưởng tính đúng đắn của chuyển trạng thái. Do đó dùng phân trị theo chuỗi ở đây là hoàn toàn khả thi.

Vấn đề đầu tiên là chuyển trên một chuỗi. Giả sử chuỗi nặng hiện tại có độ dài m ; ghi p_i là đỉnh thứ i từ trên xuống trên chuỗi, c_i là tổng $f(x)$ của mọi con nhẹ x của đỉnh i (để tiện, nếu i là lá thì đặt $c_i = \infty$).

Vì tính DP theo thứ tự cây chuỗi nặng, ta cũng chỉ cần dùng một biến duy trì c_i , giống thuật toán tập độc lập trọng số lớn nhất trên cây. Khi sửa, chỉ cần cộng vào c_i hiệu giá trị $f(x)$ của con nhẹ đó.

Để tính $f(i)$ của từng đỉnh trên chuỗi nặng, có thể DP như sau. Trước hết $f(p_m) = a_{p_m}$; tiếp theo với mọi $1 \leq i < m$,

$$f(p_i) = \min(a_{p_i}, f(p_{i+1}) + c_{p_i}).$$

Phân tích công thức này. Xem một vị trí (a_{p_i}, c_{p_i}) là một phép biến đổi (a, b) thỏa

$$\text{trans}_{(a,b)}(x) = \min(a, x + b).$$

Khi đó $f(p_i)$ là kết quả của các phép biến đổi $(a_{p_m}, c_{p_m}), (a_{p_{m-1}}, c_{p_{m-1}}), \dots, (a_{p_i}, c_{p_i})$ lần lượt tác động lên 0. Hai phép biến đổi như vậy gộp lại vẫn có dạng cũ:

$$\begin{aligned} \text{trans}_{(c,d)}(\text{trans}_{(a,b)}(x)) &= \text{trans}_{(c,d)}(\min(a, b + x)) \\ &= \min(c, d + \min(a, b + x)) \\ &= \min(\min(c, a + d), b + d + x). \end{aligned}$$

⁴⁰Nguồn: cải biên từ BZOJ 4712, đồng thời mở rộng dữ liệu.

Tức là trước hết thực hiện (a, b) , rồi thực hiện (c, d) , tương đương với thực hiện $(\min(c, a + d), b + d)$.

Vì vậy có thể dùng cây đoạn trực tiếp duy trì phép biến đổi hợp của một đoạn. Khi truy vấn, hỏi hậu tố của chuỗi nặng chứa đỉnh đó, độ phức tạp $\mathcal{O}(\log n)$. Khi sửa a_p , trước hết sửa vị trí p trong cây đoạn của chuỗi nặng chứa p , sau đó tính $f_{\text{top}(p)}$ mới, cập nhật $c_{\text{fa}(\text{top}(p))}$ và sửa cây đoạn tương ứng; tiếp tục cập nhật lên đến gốc. Độ phức tạp $\mathcal{O}(\log^2 n)$.

Như vậy bài toán được phân trị theo chuỗi giải với tiền xử lý $\mathcal{O}(n \log n)$, mỗi lần sửa $\mathcal{O}(\log^2 n)$, mỗi truy vấn $\mathcal{O}(\log n)$, và bộ nhớ $\mathcal{O}(n)$.

Bài toán này cũng trực tiếp dùng phân trị theo chuỗi để tối ưu bản thân DP, đồng thời khéo léo khai thác tính chất chuyển trạng thái trên chuỗi, cuối cùng thu được một thuật toán rất hiệu quả và mã cài đặt đẹp. Vì vậy, phân trị theo chuỗi quả thật là một công cụ mạnh cho một lớp bài toán DP trên cây có cập nhật.

7.7.3 DP cây con hỗ trợ Link/Cut và sửa dữ liệu vào

Dễ thấy trong các ví dụ trên, cài đặt cụ thể của phân trị theo chuỗi không quá khác phân rã cây nặng–nhẹ thông thường: đều dùng cây đoạn để duy trì một số thông tin có thể gộp hiệu quả. Vậy các bài này có thể mở rộng sang phân rã chuỗi động, tức LCT, để đồng thời hỗ trợ Link/Cut và sửa dữ liệu vào hay không?

Câu trả lời là có. Lấy ví dụ 1: chỉ cần với mỗi đỉnh dùng hai biến duy trì hai tổng của mọi cạnh nhẹ, phần còn lại giống hệt phân trị theo chuỗi. Không chỉ vậy, vì LCT hỗ trợ đổi gốc, dùng LCT còn có thể tính giá trị DP cho cây con bất kỳ khi lấy một đỉnh tùy ý làm gốc. Độ phức tạp của LCT là $\mathcal{O}(\log n)$ trả góp cho mỗi thao tác, thậm chí tốt hơn thuật toán phân trị theo chuỗi.

Dĩ nhiên, phân trị theo chuỗi dùng LCT cũng có hạn chế. Một là LCT có hằng số lớn, thời gian chạy thực tế chưa chắc tốt hơn thuật toán phân trị theo chuỗi có hằng số rất nhỏ. Hai là nếu thông tin duy trì trên cạnh nhẹ không có phép trừ (ví dụ duy trì giá trị nhỏ nhất của mọi cạnh nhẹ), thì cần dùng Splay để bảo đảm độ phức tạp $\mathcal{O}(\log n)$, tăng độ khó cài đặt. Ba là với một số DP thuộc lớp này, duy trì bằng LCT sẽ tạo ra rất nhiều chi tiết; chẳng hạn nếu ví dụ 3 cần hỗ trợ Link/Cut và truy vấn cây con bất kỳ, thì phải xử lý việc đỉnh lá thay đổi, và để hỗ trợ đảo khi đổi gốc còn phải duy trì hai loại nhãn xuôi và ngược.

7.7.4 Truy vấn đường đi đơn có hỗ trợ sửa

Trên đồ thị tổng quát, cũng có thể có bài toán yêu cầu hỗ trợ sửa và truy vấn thông tin của một đường đi đơn giữa hai đỉnh. Khi đó cây tròn–vuông và phân rã cây nặng–nhẹ cùng phát huy tác dụng.

Ví dụ 4. Tourists. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Có q thao tác:

1. cho hai đỉnh x, y , yêu cầu tìm một đường đi đơn từ x đến y sao cho trọng số nhỏ nhất trên đường đi là nhỏ nhất;
2. nhập p, v , đặt $a_p \leftarrow v$.

Dữ liệu $n, m, q \leq 100000$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 1 giây, bộ nhớ 512 MB.⁴¹

Bài này là phiên bản mạnh hơn của ví dụ 2, yêu cầu hỗ trợ sửa. Nếu giống ví dụ 2, đặt trọng số của mỗi đỉnh vuông bằng trọng số nhỏ nhất trong khối tương ứng, rồi dùng phân rã cây nặng–nhẹ duy trì giá trị nhỏ nhất trên đường đi, thì mỗi lần sửa trọng số một đỉnh sẽ phải duyệt mọi khối chứa đỉnh đó, độ phức tạp không chấp nhận được.

⁴¹Nguồn: UOJ 30.

Đừng quên thuật toán năm của *Đồ thị con kỳ diệu*. Trong thuật toán năm, đỉnh tròn và đỉnh vuông phân công rõ ràng: đỉnh tròn chỉ duy trì trọng số, đỉnh vuông chỉ duy trì cách nối cạnh. Nhờ đó hai loại đỉnh được tách rời; khi sửa đỉnh tròn, không cần duyệt mọi đỉnh vuông kề nó. Tư tưởng ở đây là: không để thông tin có thể thay đổi bị quá nhiều cấu trúc đồng thời duy trì. Trong bài này, cho mỗi đỉnh vuông chỉ duy trì thông tin của các đỉnh tròn con của nó, không duy trì thông tin của đỉnh tròn cha. Khi trọng số một đỉnh thay đổi, chỉ cần cập nhật trọng số đỉnh vuông cha của nó. Thao tác sửa là $\mathcal{O}(\log n)$.

Xét truy vấn. Tách đường đi x đến y thành hai chuỗi $x \rightarrow z \rightarrow y$, trong đó z là LCA của x, y trên cây tròn–vuông. Để thấy mọi đỉnh vuông nằm hoàn toàn trong $x \rightarrow z$ hoặc $z \rightarrow y$ và có cha là z thì đỉnh tròn cha của nó cũng nằm trên đường $x \rightarrow z \rightarrow y$, nên không cần xét riêng. Nhưng nếu z là đỉnh vuông, đỉnh tròn cha của nó chưa được xét; chỉ cần xét vết cận điểm này. Các thao tác còn lại đều là bài kinh điển phân rã cây nặng–nhẹ kết hợp cây đoạn. Độ phức tạp $\mathcal{O}(n + m) + \mathcal{O}(\log^2 n) + \mathcal{O}(n + m)$.

Thực ra bài này cũng có lời giải không dùng cây tròn–vuông, nhưng tối ưu độ phức tạp của lời giải đó không dễ nghĩ ra. Nhìn từ góc độ cây tròn–vuông sẽ dễ nghĩ cách tối ưu hơn, giảm phần nào độ khó tư duy.

Ví dụ 5. Đường đi đơn kỳ diệu. Cho một đồ thị vô hướng liên thông có n đỉnh, m cạnh, mỗi đỉnh có trọng số a_i . Bảo đảm mỗi thành phần song liên thông đỉnh có kích thước không vượt quá $c = 8$. Có q thao tác:

1. nhập x, y , tìm một đường đi đơn từ x đến y sao cho tổng trọng số các đỉnh trên đường đi lớn nhất;
2. nhập p, v , đặt $a_p \leftarrow v$;
3. hỏi đường đi đơn dài nhất của cả đồ thị.

Dữ liệu $n \leq 100000$, $m \leq 500000$, $q \leq 2000000$, $-10^9 \leq a_i \leq 10^9$, bảo đảm số thao tác loại 2 không vượt quá 20000. Giới hạn thời gian 6 giây, bộ nhớ 2 GB.⁴²

Bài này có ràng buộc giống *Đồ thị con kỳ diệu*: giới hạn kích thước khối song liên thông, nên có thể dùng lời giải tương tự.

Nếu chỉ có thao tác 2 và 3, bài toán giống *Đồ thị con kỳ diệu* (vì ở đây không cần đếm, và với cùng tập đỉnh, đáp án của đường đi đơn và chu trình đơn là như nhau, nên hai bài tương đương), dùng thuật toán chín của *Đồ thị con kỳ diệu* là được. Tại mỗi đỉnh vuông, tiền xử lý mọi đường đi đơn; cần ghi tập đỉnh đi qua và hai đầu mút, nên tổng trạng thái là $\mathcal{O}(2^{c-2} \binom{c}{2})$. Cũng có thể ghi bậc đỉnh như thuật toán năm của *Đồ thị con kỳ diệu*; nếu không lưu chu trình, số trạng thái là như nhau. Độ phức tạp thao tác 2 là $\mathcal{O}(2^{c-2} \binom{c}{2} + \log n \log n)$ mỗi lần, thao tác 3 là $\mathcal{O}(1)$.

Bây giờ xét hỗ trợ thao tác 1. Chú ý mọi đỉnh tròn trên đường đi từ x đến y trong cây tròn–vuông đều bắt buộc đi qua, nên đáp án có thể trực tiếp cộng trọng số các đỉnh đó. Còn mỗi đỉnh vuông tương đối độc lập: chỉ cần trong mỗi đỉnh vuông lấy giá trị đường đi lớn nhất rồi cộng lại. Nói cách khác, nếu các đỉnh trên đường đi cây tròn–vuông từ x đến y lần lượt là $p_0, s_1, p_1, s_2, \dots, p_{m-1}, s_m, p_m$, trong đó $p_0 = x, p_m = y$, thì với mỗi $1 \leq i \leq m$, đóng góp của s_i là đường đi đơn trọng số lớn nhất từ p_{i-1} đến p_i (đường đi này chắc chắn nằm hoàn toàn trong khối tương ứng với s_i và độc lập với các khối khác).

Với mỗi chuỗi nặng, giả sử các đỉnh trên chuỗi theo thứ tự là $p_0, s_1, p_1, s_2, \dots, p_{m-1}, s_m, p_m$ (nếu đầu chuỗi nặng là đỉnh vuông thì không có p_0 , không cần duy trì thông tin của s_1). Chỉ cần đặt đóng góp của mỗi p_i là a_{p_i} , đóng góp của mỗi s_i là độ dài đường đi đơn dài nhất từ p_{i-1} đến p_i trừ $a_{p_{i-1}} + a_{p_i}$. Khi đó trên chuỗi nặng chỉ cần hỏi tổng đóng góp trên đoạn để hoàn thành truy vấn. Thực chất thông tin được duy trì ở đây là $\text{cro}(1, 1)$ của đoạn.

Một truy vấn có thể đi qua nhiều chuỗi nặng, nên còn phải xét đường đi được chọn trong khối của

⁴²Nguồn: tự sáng tác.

đỉnh vuông tại vị trí vượt chuỗi. Ở đây hai đầu mút s và t của đường đi đều đã biết. Có thể khi khởi tạo hoặc sửa, tiền xử lý trực tiếp đường đi trọng số lớn nhất cho từng cặp đầu mút; khi đó truy vấn ở đây là $\mathcal{O}(1)$. Vì vậy mỗi thao tác 1 có độ phức tạp $\mathcal{O}(\log^2 n)$.

Như vậy bài toán được giải với tiền xử lý $\mathcal{O}(2^{c-2} \binom{c}{2} n + m)$, thao tác 1 có độ phức tạp $\mathcal{O}(\log^2 n)$, thao tác 2 có độ phức tạp $\mathcal{O}(2^{c-2} \binom{c}{2} + \log n \log n)$, thao tác 3 có độ phức tạp $\mathcal{O}(1)$, và bộ nhớ $\mathcal{O}(2^{c-2} \binom{c}{2} n + m)$.

Bài này mở rộng một trường hợp đặc biệt của *Đồ thị con kỳ diệu*, đồng thời dùng hai tác dụng của phân rã nặng–nhẹ: phân trị và truy vấn trên chuỗi. Nó giải được cả hai dạng vấn đề “duy trì thông tin toàn cục” và “hỗ trợ truy vấn giữa hai đỉnh”, thể hiện sức mạnh của việc phối hợp cây tròn–vuông với phân rã nặng–nhẹ.

7.8 Ý tưởng ra đề và tổng kết

7.8.1 Ý tưởng ra đề

Những năm gần đây, trong một số cuộc thi trực tuyến hoặc hệ thống chấm trực tuyến, đã xuất hiện các bài cần hỗ trợ hỏi giá trị DP toàn cục hoặc của một cấu trúc con sau khi sửa một điểm, như BZOJ 4712, Codeforces 480E, Codeforces 750E. Tôi nghiên cứu sâu lớp bài này và suy ra một số thuật toán hiệu quả dựa trên tư tưởng phân trị.

Trong đó, loại DP cây theo giai đoạn là cây con, cần sửa dữ liệu vào một đỉnh, như BZOJ 4712, là thú vị nhất. Nhưng thuật toán do người ra đề BZOJ 4712 đưa ra là một thuật toán trả góp dựa trên tính chất đặc biệt của phép sửa. Tôi nghiên cứu bài này và cuối cùng thiết kế được một thuật toán dùng phân trị theo chuỗi. Thuật toán ấy không chỉ chạy nhanh hơn mà còn có phạm vi áp dụng rộng hơn, nên tôi mở rộng nó sang phạm vi rộng hơn và cải biên thành ví dụ 3. Lời giải này rất đẹp và khá rộng dụng, nên khiến tôi rất hứng thú; vì vậy tôi muốn mở rộng nó sang ứng dụng tổng quát hơn.

Với thành phần song liên thông đỉnh, một bài toán kinh điển, trước đây tôi đã nghiên cứu ra một cấu trúc có thể xử lý quan hệ giữa các khối hiệu quả và gọn: cây tròn–vuông. Nó có thể chuyển đồ thị thành cây, rồi dùng lời giải của bài toán trên cây. Kết hợp cây tròn–vuông và phân trị theo chuỗi, tôi ra bài toán này.

Để tránh thuật toán vét cạn hoặc thuật toán sai qua được nhờ dữ liệu yếu, tôi thiết kế thông tin cần tính là nghiệm tối ưu và số phương án tối ưu. Thông tin này vừa không có phép trừ, vừa không được đóng góp lặp hoặc bỏ sót, nên dễ dựng dữ liệu mạnh hơn.

Một mục đích khác của việc ra bài là hy vọng có thể chia sẻ với mọi người kết quả nghiên cứu của tôi về việc dùng cây tròn–vuông và phân trị theo chuỗi để giải một lớp bài toán, mong khơi gợi thêm suy nghĩ về bài toán DP động trên cây và bài toán thành phần song liên thông đỉnh.

Đồ thị con kỳ diệu là một bài tương đối khó. Subtask của nó chia thành hai mảng: dữ liệu là cây và dữ liệu $q = 0$. Chúng lần lượt làm yếu một điều kiện của đề, đồng thời gợi mở một phần tư tưởng của lời giải chính: phân trị trên cây và cây tròn–vuông. Thí sinh có thể suy nghĩ từ nhiều góc độ và chọn nhiều thuật toán khác nhau. Để cài đúng lời giải chính của bài, cần tư duy đủ chặt chẽ và khả năng xử lý chi tiết. Ngoài ra lời giải chính có lượng mã và nhiều chi tiết cài đặt, nên đây là một bài khó.

7.8.2 Tổng kết mở rộng

Cây tròn–vuông có thể xử lý các bài toán trên xương rỗng rất hiệu quả, gọn và tổng quát; nó cũng có thể giải một số bài truy vấn đường đi đơn trên đồ thị tổng quát. Phân trị theo chuỗi có thể giải một lớp bài DP theo cây con có hỗ trợ sửa, và còn có thể kết hợp với LCT để hỗ trợ Link/Cut và truy vấn cây con bất kỳ. Dùng phân rã nặng–nhẹ để duy trì cây tròn–vuông có thể giải hiệu quả một lớp bài truy vấn đường đi đơn giữa hai đỉnh có hỗ trợ sửa một đỉnh.

Ngày nay, thành phần song liên thông đỉnh và bài toán DP có cập nhật vẫn có độ khó nhất định đối với nhiều thí sinh. Tôi hy vọng bài này có thể khiến thí sinh suy nghĩ thêm về ứng dụng của phân trị theo chuỗi trong DP trên cây có cập nhật, cũng như về cây tròn–vuông; từ đó có nhiều mở rộng hơn và giải được những bài toán rộng hơn. Nếu bạn nào có thu hoạch nghiên cứu ở hướng này, hoan nghênh đến trao đổi với tôi.

Lời cảm ơn

1. Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.
2. Cảm ơn cha mẹ đã sinh thành và dưỡng dục.
3. Cảm ơn cô Chen Ying của Trường Trung học số 1 Phúc Châu đã quan tâm và chỉ dạy.
4. Cảm ơn các anh chị khóa trên của Trường Trung học số 1 Phúc Châu đã đem lại gợi mở và chỉ dẫn.
5. Cảm ơn các huấn luyện viên đội tuyển quốc gia đã giúp đỡ tôi.
6. Cảm ơn bạn Liu Yifan của Trường Trung học số 1 Phúc Châu đã giúp đỡ tôi rất nhiều trong nghiên cứu cây tròn–vuông và đã đặt tên cho cây tròn–vuông.
7. Cảm ơn bạn Zhong Zhixian đã kiểm đề bài này và thẩm định bản thảo bài viết.
8. Cảm ơn mọi người đã cung cấp các tài liệu mà bài viết này từng tham khảo.
9. Cảm ơn quý vị đã dành thời gian quý báu trong lúc bận rộn để đọc bài.

Tài liệu tham khảo

- [1] Qi Zichao. *Ứng dụng thuật toán phân trị trong các bài toán đường đi trên cây*.
- [2] Tarjan, R. E. (1972), “Depth-first search and linear graph algorithms”, *SIAM Journal on Computing*, 1 (2): 146–160, doi:10.1137/0201010.
- [3] Chen Junkun, Zhong Zhixian. “Making Graphs into Trees”, trao đổi học viên WC2017.
- [4] Ren Zhizhou. *Bàn sơ về tư tưởng heuristic trong thi tin học*.
- [5] Wu Zuofan. *Báo cáo ra đề “Tài xế tàu rời Qin Chuan”*.
- [6] Xu Yinzhan. *Cây đoạn trong một lớp bài toán phân trị*.

8 Tìm hiểu bài toán bao đóng bắc cầu động

Sun Yaofeng, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết này khảo sát bài toán bao đóng bắc cầu động: lần lượt đưa ra các thuật toán tương ứng cho bài toán bao đóng bắc cầu tĩnh, bài toán bao đóng bắc cầu động bộ phận và bài toán bao đóng bắc cầu động hoàn toàn; đồng thời thông qua các ví dụ tự sáng tác để giảng giải ứng dụng của bài toán bao đóng bắc cầu động trong thi tin học.

8.1 Lời nói đầu

“Bao đóng bắc cầu” là một bài toán rất kinh điển trong lý thuyết đồ thị, dùng để tìm quan hệ có thể đi tới giữa mọi cặp đỉnh trong đồ thị có hướng. Hiện nay, trong thi thuật toán, bài toán bao đóng bắc cầu vẫn chủ yếu dùng ở tầng tĩnh, không hỗ trợ thao tác thêm, xóa cạnh; vì vậy không gian ra đề hẹp, tầm vóc bài toán chưa cao. Để thuận theo xu thế các thuật toán kiểu cấu trúc dữ liệu phát triển từ tĩnh sang động, đồng thời mở rộng lựa chọn ra đề trong thi thuật toán, tôi đã học tập và khảo sát hướng bài toán này.

Bài viết sẽ giới thiệu chi tiết một số phương pháp giải bài toán bao đóng bắc cầu động, gồm mở rộng thuật toán đường đi ngắn nhất, thuật toán gộp cây có hướng, cũng như thuật toán ngẫu nhiên dựa trên đại số tuyến tính. Đồng thời, bài viết cũng dùng các ví dụ để giảng giải ứng dụng của bài toán bao đóng bắc cầu động trong thi tin học. Hy vọng bài viết có thể đưa bài toán này vào tầm nhìn của thí sinh và ít nhiều giúp ích cho các bạn tham gia thi tin học.

Mục 2 mô tả bài toán bao đóng bắc cầu động và đưa ra các định nghĩa liên quan. Các mục 3 đến 5 lần lượt giới thiệu phương pháp tương ứng cho bài toán bao đóng bắc cầu tĩnh, bài toán bao đóng bắc cầu động bộ phận chỉ có thao tác thêm cạnh, và bài toán bao đóng bắc cầu động hoàn toàn. Trong mỗi mục đều có ví dụ. Mục 6 tổng kết và quy nạp nội dung của bài viết.

8.2 Dẫn nhập bài toán

8.2.1 Định nghĩa

Định nghĩa đồ thị có hướng $G = (V, E)$, trong đó $V = \{1, 2, \dots, n\}$ là tập đỉnh, còn E là tập cạnh.

Ta gọi đỉnh v là đỉnh có thể tới được từ đỉnh u khi và chỉ khi trong đồ thị G tồn tại một đường đi từ u đến v . Khi đó, gọi u là **tổ tiên** (*ancestor*) của v , và v là **hậu duệ** (*descendant*) của u . Đặc biệt, mỗi đỉnh u vừa là tổ tiên của chính nó, vừa là hậu duệ của chính nó. Đồng thời, gọi tập gồm mọi tổ tiên của đỉnh u là $\text{anc}(u)$, và tập gồm mọi hậu duệ của đỉnh u là $\text{desc}(u)$.

Nếu đồ thị có hướng $G^* = (V, E^*)$ có cùng tập đỉnh với G , và cạnh (u, v) được đặt vào E^* khi và chỉ khi trong G tồn tại đường đi từ u đến v , thì gọi G^* là **bao đóng bắc cầu** (*transitive closure*) của G .

8.2.2 Bài toán bao đóng bắc cầu động

Có thể mô tả hình thức bài toán bao đóng bắc cầu động như sau [7]:

Cho một đồ thị có hướng G , thực hiện một chuỗi thao tác trên nó, gồm ba kiểu:

- $\text{Insert}(x, y)$: thêm vào G một cạnh có hướng từ x đến y ;
- $\text{Delete}(x, y)$: xóa khỏi G cạnh có hướng từ x đến y ;
- $\text{Query}(x, y)$: hỏi trong G có tồn tại đường đi từ x đến y hay không.

Theo số kiểu thao tác được hỗ trợ, ta có các định nghĩa sau:

- bài toán hỗ trợ cả thao tác thêm cạnh lẫn xóa cạnh được gọi là bài toán bao đóng bắc cầu động hoàn toàn;
- bài toán chỉ hỗ trợ một trong hai thao tác thêm, xóa cạnh được gọi là bài toán bao đóng bắc cầu động bộ phận;
- bài toán không có thao tác thêm, xóa cạnh được gọi là bài toán bao đóng bắc cầu tĩnh.

Nếu không nói gì thêm, bài viết dùng n và m để chỉ số đỉnh và số cạnh của G , dùng q để chỉ số thao tác được thực hiện.

8.3 Bài toán bao đóng bắc cầu tĩnh

8.3.1 Thuật toán Floyd–Warshall

Thuật toán Floyd–Warshall [4] thường được dùng để giải bài toán đường đi ngắn nhất giữa mọi cặp đỉnh. Đồng thời, nó cũng có thể dùng để giải bài toán bao đóng bắc cầu tĩnh. Ý tưởng đại khái là dùng phép hoặc logic ($\vee$) và phép và logic ($\wedge$) để thay thế các phép toán số học min và $+$ trong bài toán đường đi ngắn nhất.

Định nghĩa 3.1. Nếu trong đồ thị G tồn tại một đường đi từ đỉnh i đến đỉnh j , và các đỉnh trung gian (không tính i và j) đều lấy từ tập $\{1, 2, \dots, k\}$, thì $t_{i,j}^{(k)}$ bằng 1; ngược lại $t_{i,j}^{(k)}$ bằng 0.

Ta có thể thu được $t_{i,j}^{(k)}$ như sau:

$$t_{i,j}^{(0)} = \begin{cases} 0, & \text{nếu } i \neq j \text{ và } (i,j) \notin E, \\ 1, & \text{nếu } i = j \text{ hoặc } (i,j) \in E. \end{cases}$$

Với $k \geq 1$,

$$t_{i,j}^{(k)} = t_{i,j}^{(k-1)} \vee (t_{i,k}^{(k-1)} \wedge t_{k,j}^{(k-1)}). \quad (3.1)$$

Vì vậy có thể tính được $t_{i,j}^{(n)}$ trong độ phức tạp $\mathcal{O}(n^3)$, từ đó thu được bao đóng bắc cầu.

Định nghĩa 3.2. Tập $T_i^{(k)} = \{j \mid t_{i,j}^{(k)} = 1\}$.

Với $k \geq 1$, theo công thức (3.1), ta có

$$T_i^{(k)} = \begin{cases} T_i^{(k-1)}, & \text{nếu } k \notin T_i^{(k-1)}, \\ T_i^{(k-1)} \cup T_k^{(k-1)}, & \text{nếu } k \in T_i^{(k-1)}. \end{cases}$$

Với các thao tác tập hợp như hợp ($\cup$) và giao ($\cap$), ta có thể tối ưu bằng cách nén bit.

Gọi w là độ dài từ máy, thường $w = 32$. Với một tập S , giả sử S có nhiều nhất n phần tử, ta có thể dùng một dãy 01 độ dài n để biểu diễn mỗi phần tử có tồn tại trong S hay không. Để tăng tốc, cứ mỗi w bit trong dãy 01 lại nén thành một kiểu số nguyên, rồi thực hiện phép toán bit hàng loạt. Như vậy có thể thao tác trên tập S trong độ phức tạp $\mathcal{O}(n/w)$.

Dùng kỹ thuật này, ta có thể tính $T_i^{(n)}$ trong độ phức tạp $\mathcal{O}(n^3/w)$, từ đó thu được bao đóng bắc cầu.

8.3.2 Thứ tự tô-pô

Định nghĩa 3.3. Với đồ thị có hướng không chu trình (DAG) $G = (V, E)$, **thứ tự tô-pô** là một thứ tự tuyến tính của mọi đỉnh trong G , cần thỏa mãn: với mọi cạnh $(u, v) \in E$, vị trí của u trong thứ tự tô-pô phải đứng trước vị trí của v .

Định lý 3.1. Nếu đồ thị G chứa chu trình, thì không tồn tại thứ tự tô-pô.

Định lý 3.2. Có thể tìm thứ tự tô-pô của đồ thị có hướng không chu trình trong độ phức tạp $\mathcal{O}(n + m)$ [4].

Lần lượt duyệt các đỉnh trong thứ tự tô-pô từ sau ra trước. Giả sử hiện tại đang duyệt tới đỉnh u , các cạnh đi ra từ u là $(u, v_1), (u, v_2), \dots, (u, v_k)$, khi đó

$$\text{desc}(u) = \text{desc}(v_1) \cup \text{desc}(v_2) \cup \dots \cup \text{desc}(v_k) \cup \{u\}.$$

Dùng tối ưu nén bit, có thể tính bao đóng bắc cầu cho đồ thị tô-pô trong độ phức tạp $\mathcal{O}(nm/w)$.

8.3.3 Thành phần liên thông mạnh

Định nghĩa 3.4. Thành phần liên thông mạnh (SCC) của đồ thị có hướng $G = (V, E)$ là một tập đỉnh cực đại $C \subseteq V$ sao cho với hai đỉnh bất kỳ $u, v \in C$, u và v đều có thể tới được nhau.

Định lý 3.3. Có thể tìm mọi SCC trong đồ thị có hướng trong độ phức tạp $\mathcal{O}(n + m)$ [4].

Nếu hai đỉnh x và y thuộc cùng một SCC, thì $\text{desc}(x) = \text{desc}(y)$. Vì vậy ta có thể co các đỉnh trong cùng một SCC thành một đỉnh rồi xử lý thống nhất.

Gọi đồ thị có hướng G' là đồ thị thu được sau khi co SCC của G . Khi đó G' là một đồ thị tô-pô. Dùng thuật toán ở mục 3.2, ta có thể tính bao đóng bắc cầu của G' ; xử lý thêm một chút sẽ thu được bao đóng bắc cầu của G . Độ phức tạp thời gian của thuật toán này là $\mathcal{O}(nm/w)$.

8.3.4 Ví dụ 1

Ví dụ 1. Explosion. (2014 ACM/ICPC Asia Regional Beijing Online)

Có n căn phòng, trong mỗi phòng có chìa khóa để mở một số cửa. Ban đầu mọi cửa phòng đều bị khóa, và trong tay bạn không có chìa khóa nào.

Khi không thể dùng chìa khóa trong tay để mở cửa phòng, bạn sẽ chọn ngẫu nhiên một căn phòng chưa mở và phá nó, từ đó lấy được mọi chìa khóa trong phòng đó. Khi có thể dùng chìa khóa trong tay để mở cửa phòng, bạn sẽ lập tức mở nó và nhận chìa khóa, không chọn phá phòng.

Hỏi kỳ vọng số lần phá phòng để mở được toàn bộ các phòng.

Dữ liệu: $n \leq 2000$.

Trước hết chuyển bài toán thành mô hình đồ thị:

- dựng đồ thị có hướng G gồm n đỉnh, phòng thứ i tương ứng với đỉnh i ;
- nếu trong phòng i có chìa khóa mở phòng j , thì nối cạnh có hướng từ i đến j ;
- nếu phá phòng i , thì mọi phòng tương ứng với các đỉnh $j \in \text{desc}(i)$ trong G đều được mở.

Theo tính tuyến tính của kỳ vọng, chỉ cần tính xác suất mỗi phòng bị phá, rồi cộng tất cả lại là đáp án.

Quan sát thấy phòng i có thể bị phá khi và chỉ khi mọi phòng tương ứng với các đỉnh $j \in \text{anc}(i)$ trong G đều chưa bị phá. Nói cách khác, phòng i là phòng đầu tiên được chọn để phá trong $|\text{anc}(i)|$ phòng ấy, nên xác suất nó bị phá là $1/|\text{anc}(i)|$. Vì vậy đáp án cuối cùng là

$$\sum_{i=1}^n \frac{1}{|\text{anc}(i)|}.$$

Dùng thuật toán ở mục 3.1 hoặc 3.3 là có thể tính $\text{anc}(i)$; độ phức tạp thời gian là $\mathcal{O}(n^3/w)$ hoặc $\mathcal{O}(nm/w)$.

8.3.5 Ví dụ 2

Ví dụ 2. Đường đi xor lớn nhất. (Tự sáng tác)

Cho đồ thị có hướng G gồm n đỉnh, m cạnh; trọng số của đỉnh thứ i là val_i .

Định nghĩa cặp đỉnh (u, v) là hợp lệ khi và chỉ khi tồn tại một đường đi từ u đến v .

Hỏi trong mọi cặp hợp lệ (u, v) , giá trị lớn nhất của $\text{val}_u \text{ xor } \text{val}_v$ là bao nhiêu.

Dữ liệu: $0 \leq n, m, \text{val}_i \leq 100000$.

Đặt

$$S(u) = \{\text{val}_v \mid v \in \text{desc}(u)\}.$$

Khi đó bài toán là: với mỗi đỉnh u , tìm giá trị xor lớn nhất giữa val_u và một số trong tập $S(u)$.

Dùng kỹ thuật nén bit, có thể nén tập $S(u)$ thành tổng cộng $|\text{val}_i|/w$ số nguyên. Với mỗi số nguyên, có thể tính trong $\mathcal{O}(1)$ số bit 1 trong biểu diễn nhị phân của nó, tức số phần tử của $S(u)$ mà nó chứa, rồi tính tổng tiền tố cho các giá trị này.

Dùng tư tưởng tham lam, duyệt từ bit cao nhất xuống bit thấp nhất, và thử bit hiện tại bằng 0 hay 1. Với đáp án đang thử, ta chỉ cần phán đoán trong tập $S(u)$ có tồn tại phần tử có trọng số nằm trong một đoạn hay không. Nhờ tổng tiền tố đã tiền xử lý, có thể phán đoán trong $\mathcal{O}(1)$.

Cách tính tập $S(u)$ tương tự bao đóng bắc cầu. Dùng thuật toán ở mục 3.3, có thể giải bài toán trong độ phức tạp

$$\mathcal{O}\left(\frac{(n+m)|\text{val}_i|}{w} + n \log |\text{val}_i|\right).$$

8.4 Bài toán bao đóng bắc cầu động bộ phận chỉ có thao tác thêm cạnh

8.4.1 Thuật toán đường đi ngắn nhất

Dựng đồ thị có hướng có trọng số $G' = (V, E')$:

- với cạnh có sẵn $(u, v) \in E$ trong G , nối trong G' cạnh từ u đến v có trọng số 0;
- nếu thao tác thứ i thêm cạnh (x_i, y_i) , thì nối trong G' cạnh từ x_i đến y_i có trọng số i .

Định nghĩa 4.1. Nếu trong G' tồn tại một đường đi từ đỉnh i đến đỉnh j , và trọng số của mọi cạnh đi qua đều không vượt quá w , thì $h_{i,j}^{(w)} = 1$; ngược lại $h_{i,j}^{(w)} = 0$.

Định nghĩa 4.2.

$$g_{i,j} = \min\{w \mid h_{i,j}^{(w)} = 1\}.$$

Hệ quả 4.1. Sau thao tác thứ $g_{i,j}$, trong đồ thị G sẽ tồn tại một đường đi từ đỉnh i đến đỉnh j .

Nếu có thể tính $g_{i,j}$ cho mọi cặp i, j , ta sẽ thu được bao đóng bắc cầu sau mỗi thao tác.

Quan sát thấy bài toán này tương đương với tìm đường đi ngắn nhất giữa mọi cặp đỉnh. Dùng thuật toán Floyd–Warshall, độ phức tạp thời gian là $\mathcal{O}(n^3)$. Dùng thuật toán Dijkstra kết hợp tối ưu bằng heap Fibonacci [4], độ phức tạp thời gian là $\mathcal{O}(n^2 \log n + n(m+q))$.

Cần chú ý rằng thuật toán này là thuật toán offline.

8.4.2 Mở rộng thuật toán Floyd–Warshall

Bản chất của thuật toán Floyd–Warshall khi tính bao đóng bắc cầu là thêm động các “đỉnh trung chuyển” để cập nhật bao đóng bắc cầu. Thực ra thứ tự thêm các đỉnh trung chuyển vào đồ thị không nhất thiết phải là $1, 2, \dots, n$; nó có thể thêm mọi đỉnh theo một thứ tự bất kỳ. Tính chất đẹp đẽ này khiến nó có thể hỗ trợ bài toán động ở một mức độ nhất định.

Ta dùng ví dụ dưới đây để hiểu rõ hơn ứng dụng của nó.

Ví dụ 3. Tình hình đường sá thời gian thực. (Vòng chung kết 2016 Ji Suan Zhi Dao)

Cho đồ thị vô hướng n đỉnh, định nghĩa $d(x, y, z)$ là đường đi ngắn nhất từ x đến z không đi qua đỉnh y . Hãy tính giá trị

$$\sum_{x=1}^n \sum_{y=1}^n \sum_{z=1}^n d(x, y, z).$$

Dữ liệu: $n \leq 200$.

Xét dùng tư tưởng phân trị.

Gọi thủ tục $\text{Work}(l, r)$ là xử lý đáp án khi $y = l, l+1, \dots, r$; lúc này các đỉnh số $1, 2, \dots, l-1, r+1, \dots, n$ đã được thêm vào đồ thị. Đồng thời duy trì $\text{dist}_{x,z}$ là đường đi ngắn nhất giữa mọi cặp đỉnh.

Khi $l = r$, duyệt $\text{dist}_{x,z}$ trong $\mathcal{O}(n^2)$ là có thể thu được đáp án ứng với $y = l$.

Nếu $l \neq r$, đặt $\text{mid} = \lfloor (l+r)/2 \rfloor$. Thêm các đỉnh $l, l+1, \dots, \text{mid}$ vào đồ thị rồi đệ quy $\text{Work}(\text{mid}+1, r)$; sau đó thêm các đỉnh $\text{mid}+1, \text{mid}+2, \dots, r$ vào đồ thị rồi đệ quy $\text{Work}(l, \text{mid})$. Mỗi khi thêm một đỉnh, dùng thuật toán Floyd–Warshall để cập nhật đường đi ngắn nhất giữa mọi cặp đỉnh.

Độ phức tạp thời gian là $\mathcal{O}(n^3 \log n)$.

“Cạnh trung chuyển”. Bất chước tư tưởng thêm động đỉnh trung chuyển của Floyd–Warshall, với mỗi cạnh được thêm bởi thao tác, ta dùng nó làm “cạnh trung chuyển” để cập nhật bao đóng bắc cầu.

Giả sử thao tác hiện tại thêm cạnh (u, v) . Nếu đã có $v \in \text{desc}(u)$, thì việc thêm cạnh (u, v) sẽ không cập nhật bao đóng bắc cầu, nên bỏ qua thao tác này.

Ngược lại, đặt

$$C = (V - \text{anc}(v)) \cap \text{anc}(u),$$

tức tập gồm mọi đỉnh có thể tới u nhưng không thể tới v . Với mọi đỉnh $x \in C$, việc thêm cạnh (u, v) sẽ làm

$$\text{desc}(x) = \text{desc}(x) \cup \text{desc}(v).$$

Còn với mọi đỉnh $x \notin C$, việc thêm cạnh (u, v) sẽ không cập nhật $\text{desc}(x)$.

Duy trì $\text{anc}(x)$.

Định nghĩa 4.3. Nếu đồ thị có hướng $\tilde{G} = (V, \tilde{E})$ có cùng tập đỉnh với G , và tập cạnh

$$\tilde{E} = \{(u, v) \mid (v, u) \in E\},$$

thì gọi $\tilde{G}$ là **đồ thị đảo** của G .

Dễ thấy tập tổ tiên của đỉnh x trong G bằng tập hậu duệ của đỉnh x trong $\tilde{G}$. Vì vậy trong mỗi thao tác thêm cạnh, chỉ cần thêm cạnh (u, v) vào G , thêm cạnh (v, u) vào $\tilde{G}$, rồi dùng cách duy trì $\text{desc}(x)$ để duy trì $\text{anc}(x)$.

Phân tích độ phức tạp. Mỗi khi thực hiện phép hợp tập trên $\text{desc}(x)$, tập mới $\text{desc}'(x)$ đều có thêm phần tử so với $\text{desc}(x)$. Mặt khác, tổng số quan hệ có thể đi tới khác nhau chỉ là n^2 , nên phép hợp tập ấy nhiều nhất chỉ được thực hiện n^2 lần. Trong trường hợp xấu nhất, mỗi thao tác thêm cạnh có thể thực hiện n phép hợp tập.

Dùng tối ưu nén bit, độ phức tạp thời gian là

$$\mathcal{O}\left(qn + \min\{n^2, qn\} \frac{n}{w}\right).$$

Tối ưu nhỏ. Có thể dùng nén bit để tăng tốc việc duyệt mọi phần tử của tập C .

Theo phần xử lý trước đó, tập C đã được biểu diễn bởi n/w số nguyên. Duyệt n/w số nguyên này; nếu trong biểu diễn nhị phân của một số có bit 1 (tức số đó khác 0), nghĩa là nó chứa ít nhất một phần tử. Lấy ra mọi vị trí có bit 1 là tìm được mọi phần tử của tập C .

Tổng hợp lại, độ phức tạp thời gian là

$$\mathcal{O}\left(q\frac{n}{w} + \min\{n^2, qn\}\frac{n}{w}\right).$$

Tiểu kết. Dễ thấy thuật toán này tốt nhất khi đồ thị dày và số thao tác nhiều.

8.4.3 Thuật toán gộp cây có hướng

Định nghĩa 4.4. Với đồ thị có hướng $G = (V, E)$, **cây có hướng** $T_x = (V, E_x)$ ($E_x \subseteq E$) gốc x là một đồ thị tô-pô thỏa mãn các điều kiện sau:

- gốc x không có cạnh vào, các đỉnh còn lại có nhiều nhất một cạnh vào;
- nếu đỉnh $y \in \text{desc}(x)$ trong G , thì trong T_x cũng tồn tại một đường đi từ gốc x đến y ;
- nếu đỉnh $y \notin \text{desc}(x)$ trong G , thì trong T_x , y không có cạnh vào.

Định nghĩa 4.5. Nếu trong cây có hướng $T_x = (V, E_x)$ có cạnh $(u, v) \in E_x$, thì gọi đỉnh v là con của đỉnh u .

Định nghĩa 4.6. Trong cây có hướng $T_x = (V, E_x)$, tập cây con của đỉnh u là

$$\text{subtree}_x(u) = \{v \mid \text{trong } T_x \text{ tồn tại đường đi từ } u \text{ đến } v\}.$$

Hệ quả 4.2. Trong cây có hướng $T_x = (V, E_x)$, với mọi đỉnh $u \in V$, ta có

$$\text{subtree}_x(u) \subseteq \text{desc}(u).$$

Trong thuật toán này, với mỗi đỉnh $x \in V$, ta xây một cây có hướng T_x để duy trì $\text{desc}(x)$. Với mỗi thao tác thêm cạnh, chỉ cần gộp một số cây có hướng.

Quy trình gộp. Giả sử thao tác hiện tại thêm cạnh (u, v) . Nếu đã có $v \in \text{desc}(u)$, thì việc thêm cạnh (u, v) sẽ không cập nhật bao đóng bắc cầu, nên bỏ qua thao tác này.

Ngược lại, đặt $C = (V - \text{anc}(v)) \cap \text{anc}(u)$. Với mọi đỉnh $x \in C$, gộp cây có hướng T_v vào các con của đỉnh u trong cây có hướng T_x .

Gọi thủ tục $\text{Merge}(x, y, a, b)$ là gộp cây con của đỉnh b trong cây có hướng T_y vào các con của đỉnh a trong cây có hướng T_x . Khi đó với mọi $x \in C$, chỉ cần gọi $\text{Merge}(x, v, u, v)$.

Cụ thể, thủ tục $\text{Merge}(x, y, a, b)$ như sau:

1. thêm cạnh (a, b) vào cây có hướng T_x , tức để b trở thành con của a ;
2. duyệt mọi con w của đỉnh b trong cây có hướng T_y :
 - nếu $w \in \text{desc}(x)$, theo Hệ quả 4.2 ta có
$$\text{subtree}_y(w) \subseteq \text{desc}(w) \subseteq \text{desc}(x),$$
nên không cần tiếp tục đệ quy cây con của đỉnh w trong T_y ;
 - nếu $w \notin \text{desc}(x)$, gọi đệ quy $\text{Merge}(x, y, b, w)$.

Mã giả. Mã giả sau giúp hiểu rõ hơn quá trình gộp cây có hướng:

Thuật toán 1 Gộp cây có hướng

```
0: procedure Merge( $x, y, a, b$ )
1:   insert  $b$  in  $T_x$  as a child of  $a$ .
2:   for each  $w$  child of  $b$  in  $T_y$  do
3:     if  $w$  is not in desc( $x$ ) then
4:       Merge( $x, y, b, w$ )
5:     end if
6:   end for
```

Phân tích độ phức tạp. Mỗi khi gọi thủ tục Merge(x, y, a, b), ta đều thêm đỉnh b vào cây có hướng T_x , đồng thời duyệt mọi con của đỉnh b trong cây có hướng T_y . Một đỉnh không thể được thêm nhiều lần vào cùng một cây có hướng.

Ta có thể phóng đại số con của đỉnh b trong cây có hướng T_y thành số cạnh đi ra từ đỉnh b trong đồ thị G sau thao tác hiện tại. Khi đó trong quá trình gộp, một cây có hướng nhiều nhất sẽ duyệt mọi cạnh trong G một lần. Nói cách khác, mỗi cạnh được thao tác thêm vào sẽ đóng góp $\mathcal{O}(n)$, nên quá trình gộp có độ phức tạp phân bổ $\mathcal{O}(n)$ cho mỗi thao tác.

Đồng thời, mỗi thao tác chỉ cần duyệt $\mathcal{O}(n)$ để tìm mọi đỉnh trong tập C . Vì vậy thuật toán này có độ phức tạp phân bổ $\mathcal{O}(n)$ cho mỗi thao tác.

Tiểu kết. Dễ thấy thuật toán này tốt nhất khi đồ thị thưa và số thao tác ít; nó bổ sung cho thuật toán đã nêu ở mục 4.2. Đồng thời, thuật toán này còn có thể tìm một đường đi từ đỉnh u đến đỉnh v trong độ phức tạp $\mathcal{O}(n)$.

8.5 Bài toán bao đóng bắc cầu động hoàn toàn

8.5.1 Thuật toán offline

Cây đoạn thời gian. Dựng một cây đoạn thời gian [6] kích thước q ; tổng cộng q lá của nó đại diện cho q thao tác. Với mỗi nút i trên cây đoạn thời gian, gọi đoạn nó phủ là $[l_i, r_i]$.

Nếu cạnh (u, v) được thêm vào G ở thao tác thứ l , và bị xóa khỏi G ở thao tác thứ r , thì cạnh (u, v) tồn tại trên đoạn $[l, r - 1]$ của cây đoạn thời gian. Có thể tìm $\mathcal{O}(\log q)$ nút đại diện cho đoạn này trên cây đoạn thời gian, rồi lưu cạnh (u, v) trên các nút ấy.

Vì đồ thị ban đầu G có m cạnh và có q thao tác, nên mọi cạnh có tổng cộng $\mathcal{O}(m + q)$ đoạn thời gian. Các đoạn này sẽ được ánh xạ lên tổng cộng $\mathcal{O}((m + q) \log q)$ nút của cây đoạn thời gian.

Duyệt cây đoạn thời gian; đến một nút thì thêm vào đồ thị mọi cạnh được lưu trên nút đó. Mỗi khi thêm một cạnh, dùng thuật toán đã nêu ở mục 4.2 để duy trì bao đóng bắc cầu.

Độ phức tạp thời gian là

$$\mathcal{O}\left((m + q) \log q \frac{n^2}{w}\right).$$

Cần chú ý rằng thuật toán này là thuật toán offline.

Ví dụ 4. Bài khó của G nhỏ. (Tự sáng tác)

Cho một đồ thị có hướng G gồm n đỉnh, m cạnh; trọng số của đỉnh thứ i là val_i .

Định nghĩa $F(u, v)$ như sau: nếu trong G tồn tại một đường đi từ u đến v , thì $F(u, v) = 1$; ngược lại $F(u, v) = 0$.

Thực hiện q thao tác, gồm hai kiểu:

- cho đỉnh p , rồi cho tot số a_i , thêm vào đồ thị các cạnh từ a_i đến p ;
- cho đỉnh p , rồi cho tot số a_i , xóa khỏi đồ thị các cạnh từ a_i đến p .

Sau mỗi thao tác, cần trả lời giá trị

$$\sum_{u=1}^N \sum_{v=1}^N F(u, v) \times (\text{val}_u \text{ xor } \text{val}_v).$$

Dữ liệu: $tot < n \leq 400, q \leq 800$.

Dùng thuật toán ở mục 5.1.1, có thể giải bài toán trong độ phức tạp

$$\mathcal{O}\left((m + q \times tot) \log q \frac{n^2}{w}\right),$$

nhưng vẫn có thể tiếp tục tối ưu.

Vì mỗi thao tác chỉ thêm, xóa các cạnh đi vào một đỉnh nào đó, ta có thể xét dùng “đỉnh trung chuyển” thay cho “cạnh trung chuyển” để cập nhật bao đóng bắc cầu.

Cụ thể, đặt $\text{pre}(x) = \{y \mid (y, x) \in E\}$, tức tập các đỉnh có cạnh đi vào x . Khi thêm đỉnh x vào đồ thị, duyệt mọi đỉnh $t \in V$; nếu thỏa mãn

$$\text{desc}(t) \cap \text{pre}(x) \neq \emptyset,$$

thì đặt

$$\text{desc}(t) = \text{desc}(t) \cup \text{desc}(x).$$

Như vậy có thể duy trì bao đóng bắc cầu sau khi thêm đỉnh trong độ phức tạp $\mathcal{O}(n^2/w)$.

Vì ban đầu có n đỉnh và có q thao tác, nên mọi đỉnh có tổng cộng $\mathcal{O}(n + q)$ đoạn thời gian. Các đoạn này được ánh xạ lên tổng cộng $\mathcal{O}((n + q) \log q)$ nút của cây đoạn thời gian.

Duyệt cây đoạn thời gian; đến một nút thì thêm vào đồ thị mọi tập $\text{pre}(x)$ được lưu trên nút đó, còn độ phức tạp để thêm một $\text{pre}(x)$ là $\mathcal{O}(n^2/w)$.

Tổng hợp lại, độ phức tạp thời gian là

$$\mathcal{O}\left((n + q) \log q \frac{n^2}{w} + qn^2\right).$$

Mở rộng ví dụ 4. Nếu mỗi thao tác vừa thêm, xóa các cạnh đi vào một đỉnh, vừa thêm, xóa các cạnh đi ra từ một đỉnh, thì làm thế nào?

Có thể vừa ghi $\text{pre}(x)$, vừa ghi thêm tập $\text{suf}(x)$, tức tập các đỉnh có cạnh đi ra từ x . Đồng thời:

- với mọi đỉnh $y \in \text{pre}(x)$, ghi thời điểm thao tác mà cạnh (y, x) được thêm vào;
- với mọi đỉnh $y \notin \text{pre}(x)$, ghi thời điểm thao tác mà cạnh (y, x) bị xóa.

Các đỉnh trong $\text{suf}(x)$ cũng được xử lý tương tự.

Như vậy, khi thêm đỉnh x , dựa vào $\text{pre}(x)$, $\text{suf}(x)$ và thời điểm thao tác tương ứng với chúng, ta có thể phán đoán mỗi cạnh vào, ra của x có tồn tại hay không. Dùng thuật toán tương tự ví dụ 4 là tính được bao đóng bắc cầu.

Độ phức tạp thời gian là

$$\mathcal{O}\left((n + q) \log q \frac{n^2}{w} + qn^2\right).$$

8.5.2 Một thuật toán dựa trên đại số tuyến tính

Chuyển hóa bài toán.

Định nghĩa 5.1. **Phủ chu trình** của đồ thị có hướng $G = (V, E)$ là dùng một số chu trình để phủ mọi đỉnh trong đồ thị, sao cho mỗi đỉnh xuất hiện đúng một lần trên một chu trình.

Giả sử muốn hỏi trong đồ thị G có tồn tại đường đi từ đỉnh i đến đỉnh j hay không. Ta xử lý G như sau để chuyển bài toán thành bài toán phủ chu trình:

1. nối một tự khuyên cho mỗi đỉnh (giả định đồ thị gốc không có tự khuyên);
2. xóa mọi cạnh đi vào đỉnh i , cũng như mọi cạnh đi ra từ đỉnh j , gồm cả các tự khuyên vừa nối;
3. nối một cạnh từ đỉnh j đến đỉnh i .

Gọi đồ thị mới là G' . Khi đó trong G tồn tại đường đi từ đỉnh i đến đỉnh j khi và chỉ khi G' tồn tại phủ chu trình.

Định nghĩa 5.2. Với đồ thị có hướng $G = (V, E)$, định nghĩa ma trận $n \times n$ A là

$$A_{i,j} = \begin{cases} x_{i,j}, & \text{nếu } i = j \text{ hoặc cạnh } (i,j) \in E, \\ 0, & \text{trường hợp khác.} \end{cases}$$

Trong đó $x_{i,j}$ là một biến, nên trong ma trận A có tổng cộng $n + m$ biến.

Định nghĩa 5.3. Định nghĩa ma trận $AE(j, i)$ là ma trận thu được sau khi thực hiện các thao tác sau trên A :

- xóa mọi phần tử ở hàng j và cột i của A về 0;
- đặt $A_{j,i} = x_{j,i}$.

Định lý 5.1. Đặt $Z = AE(j, i)$. Khi đó trong đồ thị G tồn tại đường đi từ đỉnh i đến đỉnh j khi và chỉ khi

$$\sum_{\pi \in \Sigma_n} \prod_{k=1}^n Z_{k, \pi(k)} \neq 0. \quad (5.1)$$

Chứng minh. Ý nghĩa của việc duyệt hoán vị π là duyệt một cách phủ chu trình có thể hợp lệ, trong đó $\pi(k)$ biểu thị đỉnh kế tiếp của đỉnh k . Nếu với mọi đỉnh k đều có $Z_{k, \pi(k)} \neq 0$, thì cách phủ chu trình này là hợp lệ. ■

Định lý 5.2. Công thức (5.1) đúng khi và chỉ khi $\det Z \neq 0$.

Chứng minh. Tổng các tích ở vế trái của (5.1) thực chất là một đa thức n^2 biến bậc n . Theo định nghĩa định thức,

$$\det Z = \sum_{\pi \in \Sigma_n} \text{sign}(\pi) \prod_{k=1}^n Z_{k, \pi(k)}.$$

$\det Z$ cũng là một đa thức n^2 biến bậc n , và so với tổng các tích ở (5.1), nó chỉ thêm hệ số $+1$ hoặc -1 trước mỗi hạng tử. Hệ số ấy không ảnh hưởng tới việc tổng các tích này có đồng nhất bằng 0 hay không, nên Định lý 5.2 được chứng minh. ■

Hệ quả 5.1. Trong đồ thị G tồn tại đường đi từ đỉnh i đến đỉnh j khi và chỉ khi

$$\det(AE(j, i)) \neq 0.$$

Chứng minh. Suy ra từ Định lý 5.1 và Định lý 5.2. ■

Một số kiến thức đại số tuyến tính.

Định nghĩa 5.4. $A_{i,j}$ là ma trận con còn lại sau khi xóa hàng i và cột j khỏi ma trận A .

Định nghĩa 5.5. Phần phụ của ma trận A theo hàng i và cột j , ký hiệu $M_{i,j}$, là $\det A_{i,j}$.

Định nghĩa 5.6. Đại số phụ của ma trận A theo hàng i và cột j , ký hiệu $C_{i,j}$, là $(-1)^{i+j}M_{i,j}$.

Định nghĩa 5.7. Ma trận các phần phụ của A là một ma trận $n \times n$ C , trong đó $C_{i,j} = (-1)^{i+j}M_{i,j}$.

Định nghĩa 5.8. Ma trận phụ hợp của A là chuyển vị của ma trận các phần phụ của A , tức

$$\text{adj } A = C^T.$$

Định lý 5.3. Với ma trận $n \times n$ A , với một hàng bất kỳ $i \in \{1, 2, \dots, n\}$, ta có

$$\det A = \sum_{j=1}^n A_{i,j}(\text{adj } A)_{j,i}.$$

Định lý 5.4. Nếu ma trận A khả nghịch, thì

$$A^{-1} = \frac{\text{adj } A}{\det A}.$$

Xét dùng các kiến thức đại số tuyến tính ở trên để đơn giản hóa bài toán gốc. Với ma trận A của đồ thị G , ta có

$$\det(AE(j, i)) = (\text{adj } A)_{i,j}. \quad (5.2)$$

Lại theo Định lý 5.4,

$$\text{adj } A = \det A \times A^{-1}.$$

Do đó, nếu sau mỗi thao tác ta có thể duy trì động định thức và ma trận nghịch đảo của ma trận A ứng với G , thì sẽ thu được ma trận phụ hợp của A . Theo (5.2), ta thu được $\det(AE(j, i))$; rồi theo Hệ quả 5.1, ta có thể phán đoán trong G có tồn tại đường đi từ đỉnh i đến đỉnh j hay không, từ đó thu được bao đóng bắc cầu của G .

Tính trực tiếp ma trận phụ hợp của A là rất khó. May mắn là bài toán này không cần tính giá trị chính xác của chúng; chỉ cần phán đoán các đa thức ấy có bằng 0 hay không. Vì vậy có thể dùng bổ đề sau để đơn giản hóa bài toán.

Bổ đề 5.1 (Schwartz–Zippel). [1] Với một đa thức n biến bậc d $P(x_1, x_2, \dots, x_n)$ trên trường F , không đồng nhất bằng 0, giả sử $r_1, r_2, \dots, r_n$ là n số ngẫu nhiên trong F được chọn độc lập, khi đó

$$\Pr[P(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|F|}.$$

Vì vậy, chỉ cần gán mỗi biến trong ma trận A thành một số ngẫu nhiên trong trường F . Dựa vào việc $\det(AE(j, i))$ trên trường F có bằng 0 hay không, ta có thể phán đoán trong G có tồn tại đường đi từ đỉnh i đến đỉnh j hay không.

Vì quy mô dữ liệu của loại bài toán này đều khá nhỏ, và ta lấy số nguyên tố lớn $P = 10^9 + 7$ làm trường modulo, xác suất sai có thể bỏ qua.

Duy trì động định thức và ma trận nghịch đảo. Với đồ thị ban đầu G , có thể dùng khử Gauss [2] trong độ phức tạp $\mathcal{O}(n^3)$ để tính định thức và ma trận nghịch đảo của A .

Với mỗi thao tác thêm, xóa cạnh, giả sử thao tác hiện tại sửa một vị trí nào đó ở cột i của ma trận A . Gọi vector cột δ biểu thị thay đổi của cột i trong ma trận A , và gọi A' là ma trận sau thao tác hiện tại. Khi đó

$$A'_{j,i} = A_{j,i} + \delta_j, \quad j \in \{1, 2, \dots, n\}.$$

Gọi vector hàng e_i^T là $(0, 0, \dots, 1, \dots, 0)$, trong đó cột chứa 1 là cột thứ i . Khi ấy δe_i^T là ma trận chỉ có cột thứ i khác 0, và cột đó bằng δ . Do đó

$$A' = A + \delta e_i^T.$$

Nếu ma trận B thỏa $A' = A \times B$, thì

$$B = I + A^{-1} \delta e_i^T = I + (A^{-1} \delta) e_i^T = I + b e_i^T,$$

trong đó vector cột $b = A^{-1} \delta$, có thể tính trong độ phức tạp $\mathcal{O}(n^2)$.

Viết ma trận B ra sẽ thấy nó chỉ khác ma trận đơn vị ở cột thứ i :

$$B_{r,c} = \begin{cases} 1, & r = c, c \neq i, \\ 1 + b_i, & r = c = i, \\ b_r, & c = i, r \neq i, \\ 0, & \text{trường hợp khác.} \end{cases}$$

Quan sát được $\det B = 1 + b_i$. Vì vậy từ công thức

$$\det A' = \det A \times \det B$$

ta thu được định thức của ma trận sau thao tác.

Vì dạng của B rất đơn giản, cũng có thể quan sát được ma trận nghịch đảo B^{-1} :

$$(B^{-1})_{r,c} = \begin{cases} 1, & r = c, c \neq i, \\ \frac{1}{1 + b_i}, & r = c = i, \\ -\frac{b_r}{1 + b_i}, & c = i, r \neq i, \\ 0, & \text{trường hợp khác.} \end{cases}$$

Ma trận nghịch đảo B^{-1} là một ma trận thưa, trong đó chỉ có $\mathcal{O}(n)$ phần tử khác 0. Vì vậy theo công thức

$$A'^{-1} = B^{-1} \times A^{-1}$$

ta có thể dùng phép nhân ma trận thưa để tính A'^{-1} trong độ phức tạp $\mathcal{O}(n^2)$.

Sau khi duy trì động được định thức và ma trận nghịch đảo, xử lý thêm một chút là thu được bao đóng bắc cầu của G . Độ phức tạp thời gian xấu nhất cho một thao tác của thuật toán này là $\mathcal{O}(n^2)$.

Tiểu kết. Quy trình chính của thuật toán này là:

1. thông qua xử lý đồ thị G , chuyển bài toán bao đóng bắc cầu thành bài toán phủ chu trình;
2. định nghĩa ma trận A cho G ; khi $\det(AE(j, i)) \neq 0$, điều đó cho thấy tồn tại đường đi từ đỉnh i đến đỉnh j ;
3. dùng đại số tuyến tính và Bổ đề 5.1 để đơn giản hóa bài toán thành tính ma trận phụ hợp của A trên trường modulo;
4. suy diễn toán học để duy trì động $\text{adj } A$ trong độ phức tạp $\mathcal{O}(n^2)$, rồi thu được bao đóng bắc cầu.

Có thể thấy thuật toán này không chỉ giới hạn ở thao tác thêm, xóa một cạnh mỗi lần; nó còn hỗ trợ mỗi thao tác thêm, xóa một số cạnh vào, ra của một đỉnh mà hiệu suất không đổi.

8.6 Tổng kết

Bài viết đã đề xuất các thuật toán sau cho bài toán bao đóng bắc cầu động:

- dùng thuật toán Floyd–Warshall, sắp xếp tô-pô và thuật toán thành phần liên thông mạnh để giải bài toán bao đóng bắc cầu tĩnh;
- dùng thuật toán đường đi ngắn nhất, mở rộng thuật toán Floyd–Warshall và thuật toán gộp cây có hướng để giải bài toán bao đóng bắc cầu động bộ phận;
- dùng thuật toán cây đoạn thời gian và một thuật toán dựa trên đại số tuyến tính để giải bài toán bao đóng bắc cầu động hoàn toàn.

Vì hiện nay trong thi tin học, các bài toán liên quan đến bao đóng bắc cầu động còn rất ít, nên lớp bài này vẫn còn nhiều không gian để khai thác. Hy vọng bài viết này có thể đóng vai trò gợi mở, để trong tương lai thi tin học có ngày càng nhiều nghiên cứu và ứng dụng theo hướng này.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn thầy Chen Heli và thầy Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và chỉ dạy tôi trong nhiều năm.

Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã chỉ dẫn.

Cảm ơn các anh khóa trên Zhang Hengjie và Ren Zhizhou của Đại học Thanh Hoa đã cho tôi ý tưởng và gợi mở trong quá trình viết bài.

Cảm ơn các bạn Sun Yican, Hong Huadun, Feng Zhe, Ye Jianing, Ren Xuandi và các bạn khác của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ tôi.

Cảm ơn tất cả thầy cô và bạn bè từng giúp đỡ tôi.

Tài liệu tham khảo

- [1] Wikipedia, Schwartz–Zippel lemma.

- [2] Wikipedia, Gaussian elimination.
- [3] Liu Rujia, Huang Liang. *Nghệ thuật thuật toán và thi tin học*.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*.
- [5] Sun Yaofeng. *Báo cáo lời giải bài khó của G nhỏ*, bài tập lần thứ hai của đội tuyển tập huấn năm 2017.
- [6] Xu Yinzhan. *Cây đoạn trong một lớp bài toán phân trị*, luận văn đội tuyển tập huấn năm 2014.
- [7] Camil Demetrescu, Giuseppe F. Italiano. “Fully Dynamic Transitive Closure: Breaking Through the $O(n^2)$ Barrier.” FOCS 2000: 381–389.
- [8] M. Nivat. “Amortized Efficiency of a Path Retrieval Data Structure.” TCS 1986: 273–281.
- [9] Piotr Sankowski. “Dynamic Transitive Closure via Dynamic Matrix Inverse.” FOCS 2004: 509–517.

9 Báo cáo ra đề $A + B$ Problem

Wang Leping, Trường THPT trực thuộc Đại học Sư phạm An Huy

Tóm tắt

Bài này là một đề trong trận thi đối kháng đầu tiên của đội tuyển tập huấn năm 2016. Bài viết chủ yếu giới thiệu lời giải của bài và bối cảnh ra đề.

9.1 Đề bài

9.1.1 Mô tả bài toán

Tên bài chỉ để dụ bạn bấm vào thôi~~~

Gần đây JOHNNKRAM đang nghiên cứu lý thuyết đồ thị. Hôm nay cậu gặp một bài như sau: cho một đồ thị có hướng, đảm bảo không có cạnh trùng và không có tự khuyên. Sau khi có các thành phần liên thông mạnh của đồ thị này thành đỉnh, hỏi có bao nhiêu đỉnh có bậc vào bằng 0.

JOHNNKRAM dùng thuật toán Tarjan và dễ dàng giải bài này. Nhưng cậu phát hiện rất nhiều người viết mã ngắn hơn mình nhiều, bèn xin chương trình xem thử, kết quả phát hiện chương trình ấy chỉ in ra 1 ==. Chẳng lẽ dữ liệu ngẫu nhiên thật sự yếu như vậy sao? Thế là cậu sinh ngẫu nhiên rất nhiều đồ thị có hướng, rồi phát hiện đáp án thật sự toàn là 1 ==. Sau đó cậu thay đổi cách sinh ngẫu nhiên, chỉ sinh các đồ thị có kích thước của mỗi thành phần liên thông mạnh đều thuộc một tập S ; khi ấy đáp án lập tức lớn lên.

Bây giờ JOHNNKRAM đã chặn được những người đó, nhưng cậu muốn chứng minh cường độ dữ liệu, nên đặt ra một câu hỏi: với mọi đồ thị có hướng gán nhãn gồm i đỉnh ($1 \leq i \leq n$) mà kích thước của mỗi thành phần liên thông mạnh đều thuộc tập S , kỳ vọng đáp án của bài toán trước là bao nhiêu? Cậu phát hiện mình không chứng minh được, nên đến nhờ bạn chỉ giáo.

9.1.2 Định dạng vào

Dòng đầu vào chứa một số nguyên dương n , biểu thị số đỉnh lớn nhất của đồ thị có hướng được sinh.

Tiếp theo là n dòng; dòng thứ i chứa một số nguyên s_i . Nếu $s_i = 1$ thì i thuộc tập S , ngược lại i không thuộc tập S .

9.1.3 Định dạng ra

In ra tổng cộng n dòng, mỗi dòng chứa một số nguyên. Số nguyên ở dòng thứ i biểu thị kết quả modulo 998244353 của kỳ vọng đáp án bài toán trước, xét trên mọi đồ thị có hướng gán nhãn gồm i đỉnh ($1 \leq i \leq n$) mà kích thước của mỗi thành phần liên thông mạnh đều thuộc tập S . Nếu không có đồ thị hợp lệ thì in 0. Giả sử giá trị kỳ vọng là a/b , trong đó a và b là hai số nguyên dương nguyên tố cùng nhau; nếu bạn in số nguyên x thì cần đảm bảo

$$bx \equiv a \pmod{998244353} \quad \text{và} \quad 0 \leq x < 998244353.$$

9.1.4 Ví dụ vào

3
1
0
0

9.1.5 Ví dụ ra

1
332748119
519087065

9.1.6 Giải thích ví dụ

Trong các đồ thị có hướng 1 đỉnh, có 1 đồ thị hợp lệ có đáp án bằng 1, nên đáp án là 1, sau khi lấy modulo 998244353 vẫn là 1.

Trong các đồ thị có hướng 2 đỉnh, có 2 đồ thị hợp lệ có đáp án bằng 1, và 1 đồ thị hợp lệ có đáp án bằng 2, nên đáp án là $4/3$, sau khi lấy modulo 998244353 là 332748119.

Trong các đồ thị có hướng 3 đỉnh, có 15 đồ thị hợp lệ có đáp án bằng 1, 9 đồ thị hợp lệ có đáp án bằng 2, và 1 đồ thị hợp lệ có đáp án bằng 3, nên đáp án là $36/25$, sau khi lấy modulo 998244353 là 519087065.

9.1.7 Ràng buộc và quy ước

Với toàn bộ dữ liệu kiểm thử, $1 \leq n \leq 100000$, $0 \leq s_i \leq 1$.

Số test	$n \leq$	Quy ước khác
1	4	Không có
2	1000	$\forall 1 \leq i \leq \lfloor n/2 \rfloor, s_i = 0$
3	1000	$\forall 1 \leq i \leq n, s_i = [i == 1]$
4	1000	$\forall 1 \leq i \leq n, s_i = [i == 1]$
5	1000	$\forall 1 \leq i \leq n, s_i = [i == 1]$
6	1000	$\forall 1 \leq i \leq \lfloor n/3 \rfloor, s_i = 0$
7	1000	$\forall 1 \leq i \leq \lfloor n/3 \rfloor, s_i = 0$
8	1000	$\forall 1 \leq i \leq \lfloor n/3 \rfloor, s_i = 0$
9	1000	$\exists x, \forall 1 \leq i \leq n, s_i = [i == x]$
10	1000	$\exists x, \forall 1 \leq i \leq n, s_i = [i == x]$
11	1000	$\exists x, \forall 1 \leq i \leq n, s_i = [i == x]$
12	1000	Không có
13	1000	Không có
14	1000	Không có
15	100000	Không có
16	100000	Không có
17	100000	Không có
18	100000	Không có
19	100000	Không có
20	100000	Không có

9.2 Giới thiệu thuật toán

Trong phần sau, mọi nơi nhắc tới đồ thị đều chỉ đồ thị có gán nhãn.

9.2.1 Thuật toán một

Ta vét cạn từng đồ thị có hướng, kiểm tra nó có thỏa điều kiện hay không và tính đáp án. Số cạnh của một đồ thị có hướng là cấp $\mathcal{O}(n^2)$, còn số đồ thị có hướng n đỉnh là $2^{n(n-1)}$, nên độ phức tạp thời gian của thuật toán một là

$$\mathcal{O}(n^2 \cdot 2^{n(n-1)}).$$

9.2.2 Thuật toán hai

Với test 2, số đỉnh của mỗi thành phần liên thông mạnh lớn hơn $\lfloor n/2 \rfloor$. Biến đổi điều kiện một chút sẽ thấy đồ thị hợp lệ nhất định là liên thông mạnh, còn đáp án của đồ thị liên thông mạnh nhất định là 1, nên chỉ cần in nguyên mẫu s . Độ phức tạp thời gian của thuật toán hai là $\mathcal{O}(n)$.

9.2.3 Thuật toán ba

Với các test 3 ~ 5, kích thước thành phần liên thông mạnh chỉ có thể là 1, tức đồ thị có hướng hợp lệ nhất định là DAG. Gọi g_i là số DAG có i đỉnh, và $f_{i,j}$ là số DAG có i đỉnh và có j đỉnh bậc vào bằng 0. Hiển nhiên

$$g_i = \sum_{j=0}^i f_{i,j}.$$

Với một tập đỉnh S , nếu với mọi $i \in S$, đỉnh i có bậc vào bằng 0 trong DAG G , ta gọi S là một gốc của G . Định nghĩa một DAG có gốc là một cặp có thứ tự (G, S) , trong đó tập đỉnh S là một gốc của DAG G . Để thấy một DAG G_x có j đỉnh bậc vào bằng 0 tương ứng với 2^j DAG có gốc, tức có 2^j DAG có gốc

thỏa $G = G_x$. Mặt khác, với một DAG có gốc, nếu xóa mọi đỉnh trong S và mọi cạnh có đầu mút trong S , phần còn lại vẫn là một DAG. Do đó ta có đẳng thức

$$\sum_{i=0}^n f_{i,n} \cdot 2^i = \sum_{i=0}^n g_{n-i} \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Ở đây $2^{i(n-i)} \binom{n}{i}$ biểu thị việc chọn mỗi cạnh có thể bị xóa có tồn tại hay không, và chọn mỗi đỉnh có nằm trong S hay không. Tách 2^i ở vế trái bằng định lý nhị thức rồi biến đổi đôi chút, ta được

$$\sum_{i=0}^n \sum_{j=i}^n f_{j,n} \binom{j}{i} = \sum_{i=0}^n g_{n-i} \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Chú ý rằng lúc này ý nghĩa của i ở hai vế là như nhau, đều biểu thị kích thước của S . Vì vậy nhân hai vế với $(-1)^i$, ta có

$$\sum_{i=0}^n (-1)^i \sum_{j=i}^n f_{j,n} \binom{j}{i} = \sum_{i=0}^n g_{n-i} \cdot (-1)^i \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Biến đổi vế trái lần nữa:

$$\sum_{i=0}^n f_{i,n} \sum_{j=0}^i (-1)^j \binom{i}{j} = \sum_{i=0}^n g_{n-i} \cdot (-1)^i \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Theo định lý nhị thức,

$$\sum_{j=0}^i (-1)^j \binom{i}{j} = [i == 0],$$

mà $f_{0,n} = [n == 0]$, nên vế trái trở thành $[n == 0]$.

Chuyển g_n sang vế trái, chuyển $[n == 0]$ sang vế phải, rồi nhân hai vế với -1 , ta được

$$g_n = [n == 0] - \sum_{i=1}^n g_{n-i} \cdot (-1)^i \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Như vậy ta đã có một công thức truy hồi có thể tính n giá trị đầu của g_i trong thời gian $\mathcal{O}(n^2)$. Tiếp theo xét cách tính tổng đáp án của mọi DAG.

Xét đóng góp của một đỉnh x vào tổng đáp án. Khi bậc vào của x bằng 0, đóng góp vào tổng đáp án là 1; ngược lại là 0. Vì vậy ta chỉ cần xét có bao nhiêu DAG i đỉnh thỏa đỉnh x có bậc vào bằng 0. Rõ ràng x không có cạnh đi vào. Nếu xóa đỉnh x và mọi cạnh đi ra từ x , phần còn lại là một DAG $i - 1$ đỉnh. Đỉnh x có thể có nhiều nhất $i - 1$ cạnh đi ra, và việc mỗi cạnh trong $i - 1$ cạnh này có tồn tại hay không đều hợp lệ. Vì vậy đóng góp của một đỉnh x vào đáp án là $g_{i-1} \cdot 2^{i-1}$. Đóng góp của mọi đỉnh vào tổng đáp án là như nhau, nên tổng đáp án của mọi DAG i đỉnh là

$$i \cdot g_{i-1} \cdot 2^{i-1}.$$

Tính trực tiếp từ g_{i-1} là được. Độ phức tạp thời gian của thuật toán ba là $\mathcal{O}(n^2)$.

9.2.4 Thuật toán bốn

Với các test 6 ~ 8, số đỉnh của mỗi thành phần liên thông mạnh lớn hơn $\lceil n/3 \rceil$, nói cách khác có nhiều nhất 2 thành phần liên thông mạnh. Khi giữa hai thành phần liên thông mạnh không có cạnh thì đáp án bằng 2, ngược lại đáp án bằng 1. Vì vậy chỉ cần tính số đồ thị có hướng hợp lệ và số đồ thị hợp lệ mà giữa hai thành phần liên thông mạnh không có cạnh là có thể tính tổng đáp án. Do đó điểm mấu chốt hiện nay là tính số đồ thị liên thông mạnh có i đỉnh.

Sau khi co các thành phần liên thông mạnh của một đồ thị có hướng thành đỉnh, phần còn lại là một DAG, nên ta có thể áp dụng công thức truy hồi đã suy ra trước đó. Nhưng cần chú ý rằng bây giờ thứ ta đã biết là số “DAG”, và mỗi đỉnh thật ra tương ứng với một thành phần liên thông mạnh, nên còn phải thêm hệ số.

Định nghĩa trọng số của một tập S là $(-1)^{|S|}$. Gọi d_i là số đồ thị liên thông mạnh có i đỉnh; e_i là tổng trọng số của mọi tập mà mỗi phần tử là một đồ thị liên thông mạnh, và tổng số đỉnh của các phần tử trong tập bằng i ; h_i là số đồ thị có hướng i đỉnh. Suy luận theo ý tưởng trước đó, ta có thể thu được đẳng thức sau, lược bỏ quá trình:

$$\sum_{i=0}^n h_{n-i} \cdot e_i \cdot 2^{i(n-i)} \cdot \binom{n}{i} = [n == 0].$$

Nếu không có ràng buộc khác thì $h_i = 2^{i(i-1)}$. Vì vậy ta có thể tính mọi e_i trong độ phức tạp thời gian $\mathcal{O}(n^2)$. Tiếp theo xét cách tính d_i từ e_i .

Vì đồ thị có gắn nhãn, chỉ cần liệt kê số đỉnh của đồ thị liên thông mạnh chứa đỉnh 1 là có thể thu được công thức truy hồi của e_i theo d_i :

$$e_n = \sum_{i=1}^n -d_i \cdot e_{n-i} \cdot \binom{n-1}{i-1}.$$

Chuyển vế, ta được công thức truy hồi của d_i theo e_i :

$$d_n = -e_n + \sum_{i=1}^{n-1} -d_i \cdot e_{n-i} \cdot \binom{n-1}{i-1}.$$

Sau khi tính mọi d_i , phần còn lại rất đơn giản: chỉ cần liệt kê số thành phần liên thông mạnh và số đỉnh của từng thành phần liên thông mạnh, rồi nhân với các hệ số tương ứng. Độ phức tạp thời gian của thuật toán bốn là $\mathcal{O}(n^2)$.

9.2.5 Thuật toán năm

Với các test 9 ~ 11, kích thước của mọi thành phần liên thông mạnh đều là cùng một số, gọi số này là x . Trước hết dùng công thức truy hồi trong thuật toán bốn để tính mọi d_i , sau đó đặt mọi d_i với $i \neq x$ bằng 0, thu được một dãy d_i mới, ký hiệu là d'_i . Dùng d'_i để tính e_i mới, ký hiệu là e'_i . Lại dùng e'_i để tính h_i mới, ký hiệu là h'_i . Khi đó h'_i chính là số đồ thị có hướng hợp lệ có i đỉnh.

Về tổng đáp án của mọi đồ thị có hướng hợp lệ i đỉnh, vẫn tương tự thuật toán ba: xét đóng góp của mỗi đồ thị liên thông mạnh vào đáp án. Có thể tính ra tổng đáp án khi có i đỉnh bằng

$$d'_x \cdot h'_{i-x} \cdot 2^{x(i-x)} \cdot \binom{i}{x}.$$

Tính trực tiếp là được. Độ phức tạp thời gian của thuật toán năm là $\mathcal{O}(n^2)$.

9.2.6 Thuật toán sáu

Xét cách tính trong trường hợp tổng quát: số đồ thị có hướng hợp lệ i đỉnh và tổng đáp án của mọi đồ thị có hướng hợp lệ i đỉnh.

Trước hết là số đồ thị có hướng hợp lệ. Chú ý rằng cách tính số đồ thị có hướng hợp lệ trong thuật toán năm rất dễ mở rộng sang trường hợp ràng buộc kích thước thành phần liên thông mạnh thuộc một tập nào đó: chỉ cần đặt $d'_i = d_i \cdot s_i$, rồi áp dụng cách trong thuật toán năm để tính h'_i là được.

Tiếp theo là tổng đáp án của mọi đồ thị có hướng hợp lệ. Gọi ans_i là tổng đáp án của mọi đồ thị có hướng hợp lệ i đỉnh. Bây giờ kích thước thành phần liên thông mạnh đã từ một giá trị chuyển thành nhiều

giá trị, nhưng đóng góp của các đồ thị liên thông mạnh có số đỉnh khác nhau vào đáp án là độc lập; ta chỉ cần liệt kê số đỉnh của đồ thị liên thông mạnh và tính riêng từng đóng góp. Vì vậy thu được công thức truy hồi:

$$\text{ans}_n = \sum_{i=1}^n d'_i \cdot h'_{n-i} \cdot 2^{i(n-i)} \cdot \binom{n}{i}.$$

Do đó có thể dựa vào d'_i và h'_i để tính mọi ans_i trong thời gian $\mathcal{O}(n^2)$. Độ phức tạp thời gian của thuật toán sáu là $\mathcal{O}(n^2)$.

9.2.7 Thuật toán bảy

Xét dùng hàm sinh để xử lý các công thức truy hồi.

Đặt

$$\begin{aligned} D(x) &= \sum_{i \geq 0} \frac{d_i x^i}{i!}, & D_1(x) &= \sum_{i \geq 0} \frac{d'_i x^i}{i!}, & E(x) &= \sum_{i \geq 0} \frac{e_i x^i}{i!}, \\ E_1(x) &= \sum_{i \geq 0} \frac{e'_i x^i}{i!}, & H(x) &= \sum_{i \geq 0} \frac{h_i x^i}{i!}, & H_1(x) &= \sum_{i \geq 0} \frac{h'_i x^i}{i!}, \\ A(x) &= \sum_{i \geq 0} \frac{\text{ans}_i x^i}{i!}. \end{aligned}$$

Trước hết xét quan hệ giữa $D(x)$ và $E(x)$, giữa $D_1(x)$ và $E_1(x)$:

$$e_n = \sum_{i=1}^n -d_i \cdot e_{n-i} \cdot \binom{n-1}{i-1}.$$

Chuyển về và chuyển thành dạng hàm sinh, ta được

$$E'(x) = -D'(x)E(x).$$

Chuyển về rồi tích phân hai vế, ta được

$$\ln E(x) = -D(x) \left(\int \frac{F'(x)}{F(x)} dx = \ln F(x) \right).$$

Lấy Exp hai vế, ta được

$$E(x) = e^{-D(x)}.$$

Quan hệ giữa $D_1(x)$ và $E_1(x)$ cũng tương tự.

Tiếp theo xét quan hệ giữa $E(x)$ và $H(x)$, giữa $E_1(x)$ và $H_1(x)$, và giữa $A(x)$ với $D_1(x), H_1(x)$. Chú ý trong công thức truy hồi xuất hiện hệ số $2^{i(n-i)}$, mà hệ số này không thể tách một cách tiện lợi. Ta cần dùng một loại hàm sinh mới. Định nghĩa toán tử một ngôi Δ . Giả sử

$$F(x) = \sum_{i \geq 0} \frac{f_i x^i}{i!},$$

định nghĩa

$$\Delta F(x) = \sum_{i \geq 0} \frac{f_i x^i}{i! 2^{\binom{i}{2}}}.$$

Ta gọi $\Delta F(x)$ là hàm sinh tổ hợp của dãy f_i .

Dùng đẳng thức

$$i(n-i) = \binom{n}{2} - \binom{i}{2} - \binom{n-i}{2},$$

ta có thể tách

$$2^{i(n-i)} = \frac{2^{\binom{n}{2}}}{2^{\binom{i}{2}} \cdot 2^{\binom{n-i}{2}}}.$$

Như vậy ta có thể tính ra các quan hệ giữa những hàm sinh tổ hợp này:

$$\Delta E(x) \Delta H(x) = 1,$$

$$\Delta E_1(x) \Delta H_1(x) = 1,$$

$$\Delta A(x) = \Delta H_1(x) \Delta D_1(x).$$

Bây giờ ta có thể viết toàn bộ quy trình của thuật toán:

$$\Delta E(x) = (\Delta H(x))^{-1},$$

$$D(x) = -\ln E(x),$$

$$d'_i = d_i \cdot s_i,$$

$$E_1(x) = e^{-D_1(x)},$$

$$\Delta H_1(x) = (\Delta E_1(x))^{-1},$$

$$\Delta A(x) = \Delta H_1(x) \Delta D_1(x).$$

Tính kỳ vọng chỉ cần lấy hệ số trong $A(x)$ chia cho hệ số trong $H_1(x)$. Chú ý rằng các thao tác cần thực hiện nhưng không thể cài đặt trong thời gian $\mathcal{O}(n)$ gồm nhân đa thức, nghịch đảo đa thức, Ln đa thức và Exp đa thức; độ phức tạp thời gian phụ thuộc vào độ phức tạp của bốn thao tác này.

Phép nhân đa thức có thể dùng biến đổi số học nhanh để tính trong thời gian $\mathcal{O}(n \log n)$.

Với phép nghịch đảo đa thức, ta dùng phương pháp nhân đôi. Giả sử đa thức gốc là $P(x)$, và nghịch đảo của $P(x)$ theo modulo x^{2^t} là $F_t(x)$. Xét cách tính $F_{t+1}(x)$ từ $F_t(x)$.

$$P(x)F_t(x) \equiv 1 \pmod{x^{2^t}},$$

$$P(x)F_t(x) - 1 \equiv 0 \pmod{x^{2^t}},$$

$$(P(x)F_t(x) - 1)^2 \equiv 0 \pmod{x^{2^{t+1}}},$$

$$1 \equiv 2F_t(x)P(x) - P(x)^2F_t(x)^2 \pmod{x^{2^{t+1}}},$$

$$F_{t+1}(x) \equiv 2F_t(x) - P(x)F_t(x)^2 \pmod{x^{2^{t+1}}}.$$

Vì vậy chỉ cần dùng phép nhân đa thức để tính. Độ phức tạp thời gian thỏa

$$T(n) = T(n/2) + \mathcal{O}(n \log n),$$

nên $T(n) = \mathcal{O}(n \log n)$.

Ln đa thức dùng trực tiếp công thức

$$\ln F(x) = \int \frac{F'(x)}{F(x)} dx$$

để tính. Đạo hàm và tích phân đa thức đều có độ phức tạp thời gian $\mathcal{O}(n)$, nên độ phức tạp thời gian của Ln đa thức là $\mathcal{O}(n \log n)$.

Exp đa thức được giải bằng phương pháp Newton. Giả sử phương trình là $G(F(x)) = 0$, và $F_t(x) \equiv F(x) \pmod{x^{2^t}}$. Công thức Newton là

$$F_{t+1}(x) \equiv F_t(x) - \frac{G(F_t(x))}{G'(F_t(x))} \pmod{x^{2^{t+1}}}.$$

Trong Exp đa thức, $G(F(x)) = e^{P(x)} - F(x)$, phương trình này không tiện tính, nên đổi thành $G(F(x)) = \ln F(x) - P(x)$. Thay vào công thức Newton được

$$F_{t+1}(x) \equiv F_t(x) - (\ln F_t(x) - P(x)) \cdot F_t(x) \pmod{x^{2^{t+1}}}.$$

Dùng Ln đa thức và phép nhân đa thức là có thể tính. Độ phức tạp thời gian thỏa

$$T(n) = T(n/2) + \mathcal{O}(n \log n),$$

nên $T(n) = \mathcal{O}(n \log n)$.

Hiện nay bốn thao tác trên đều có độ phức tạp thời gian $\mathcal{O}(n \log n)$, nên độ phức tạp thời gian của thuật toán bây là $\mathcal{O}(n \log n)$.

9.3 Cách sinh dữ liệu

Chú ý rằng cường độ dữ liệu không liên hệ trực tiếp với giá trị của s_i , nên ngoài các test có yêu cầu đặc biệt, các test còn lại đều được sinh ngẫu nhiên.

9.4 Tình hình điểm số

Trong các thí sinh đội tuyển tập huấn, có 1 người đạt 100 điểm, 3 người đạt 70 điểm, 1 người đạt 35 điểm, 1 người đạt 25 điểm, 2 người đạt 20 điểm, 1 người đạt 15 điểm, 1 người đạt 5 điểm, và 2 người đạt 0 điểm. Nhìn chung, bài này có độ khó khá lớn và độ phân hóa cao.

9.5 Bối cảnh ra đề

Trong vài năm gần đây, các bài đếm xuất hiện rất thường xuyên trong thi tin học, và bài đếm về đồ thị cũng không ít. Tuy nhiên, tuyệt đại đa số các bài đếm về đồ thị là về đồ thị vô hướng, còn bài đếm về đồ thị có hướng lại cực kỳ hiếm. Nguyên nhân nằm ở chỗ so với đồ thị vô hướng, đồ thị có hướng có thêm biến “hướng của cạnh”, và ảnh hưởng của biến này tới đáp án lại rất khó tính.

Trong quá trình học tin học, tôi từng nghiên cứu một số nội dung về hàm sinh. Một ngày nọ, sau khi thấy một bài đếm về đồ thị có hướng, tôi đột nhiên nghĩ: liệu có thể dùng hàm sinh để giải bài toán đếm đồ thị có hướng hay không? Sau đó, tôi tra cứu một số tài liệu và thực hiện một số suy diễn toán học, cuối cùng suy ra thành công công thức truy hồi cho số DAG và số đồ thị liên thông mạnh, đồng thời dùng hàm sinh tối ưu độ phức tạp thời gian xuống $\mathcal{O}(n \log n)$. Rồi tôi lại biến đổi các công thức truy hồi đôi chút, chuyển bài toán đếm thành kỳ vọng, và cuối cùng ra bài này.

9.6 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng trao đổi và học tập.

Cảm ơn thầy Ye Guoping đã giúp đỡ tôi trong học tập, đời sống và nhiều phương diện khác.

Cảm ơn các anh khóa trên Peng Yuxiang, Jin Ce và những người khác về các nghiên cứu hàm sinh.

Cảm ơn bạn Luo Zhezheng đã duyệt bản thảo cho bài viết này.

Cảm ơn bạn Mao Xiao đã giúp đỡ.

Tài liệu tham khảo

- [1] Ronald L. Graham; Donald E. Knuth; Oren Patashnik (1994). “Chapter 7: Generating Functions”. *Concrete Mathematics. A Foundation for Computer Science* (second ed.). Addison–Wesley. pp. 320–380.

- [2] R. W. Robinson. “Counting labeled acyclic digraphs”, pp. 239–273 in F. Harary, editor, *New Directions in the Theory of Graphs*. Academic Press, NY, 1973.
- [3] Huantian Cao. “AutoGF: An Automated System to Calculate Coefficients of Generating Functions.”
- [4] Jin Ce. *Các phép toán hàm sinh và bài toán đếm tổ hợp*, luận văn ứng viên đội tuyển quốc gia Olympic Tin học Trung Quốc năm 2015.
- [5] Peng Yuxiang. *Introduction to Polynomials*, giáo trình bài giảng Trại Đông Olympic Tin học Thanh thiếu niên Toàn quốc năm 2016.

10 Bước đầu tìm hiểu thuật toán chia khối kích thước phi chuẩn

Xu Mingkuan, Trường Trung học Thực nghiệm trực thuộc Đại học Sư phạm Bắc Kinh

Tóm tắt

Chia khối là một thuật toán thực dụng, đơn giản và dễ cài đặt. Nó không chỉ có thể giải nhiều bài toán thao tác trên cấu trúc dữ liệu tuyến tính, mà còn có thể tối ưu một số thuật toán khác. Bài viết này giới thiệu cách xác định kích thước khối tối ưu, dùng một số ví dụ để giới thiệu ứng dụng trực tiếp của chia khối kích thước phi chuẩn, đồng thời giới thiệu vài ứng dụng dùng chia khối kích thước phi chuẩn để tối ưu thuật toán khác.

10.1 Dẫn nhập

Chia khối là một thuật toán quan trọng, thường dùng để xử lý các thao tác trên đoạn của dãy tuyến tính; ưu điểm của nó là thực dụng và dễ cài đặt. Dù cây đoạn có thể xử lý một số tình huống tương đối đơn giản trong độ phức tạp $\mathcal{O}(\log n)$, khi gặp tình huống phức tạp hơn ta có thể phải đào sâu tính chất của thông tin cần duy trì. Điều đó có thể dẫn tới độ phức tạp cao hơn, thậm chí khiến bài toán không giải được. Lúc này ưu thế của chia khối thể hiện rõ: không cần phân tích bài toán quá phức tạp, cũng không cần cài đặt phức tạp. Vì vậy chia khối có tính thực dụng cao.

Chia khối không chỉ giải trực tiếp nhiều bài toán thao tác trên cấu trúc dữ liệu tuyến tính, mà còn có thể tối ưu một số thuật toán khác. Bằng cách chọn kích thước khối phi chuẩn phù hợp, ta có thể tối ưu các bài toán LCA (tổ tiên chung gần nhất), RMQ (truy vấn giá trị nhỏ nhất đoạn), LA (tổ tiên mức k) về độ phức tạp thời gian tối ưu theo lý thuyết.

Trong luận văn đội tuyển tập huấn năm 2013, Luo Jianqiao [1] từng nghiên cứu ứng dụng tư tưởng chia khối trong một lớp bài toán xử lý dữ liệu, Wang Ziyu [2] cũng từng nghiên cứu một số ứng dụng của chia khối; trong luận văn đội tuyển tập huấn năm 2015, Zou Xiaoyao [3] từng nghiên cứu ứng dụng của chia khối trong một lớp bài toán trực tuyến.

Bài viết này dựa trên các kết quả nghiên cứu [1,2,3] và nghiên cứu thêm về thuật toán chia khối có kích thước phi chuẩn.

Cấu trúc bài viết như sau. Mục 2 giới thiệu ngắn gọn thuật toán chia khối. Mục 3 giới thiệu một phương pháp ngắn gọn để xác định kích thước khối tối ưu, đồng thời giới thiệu ứng dụng trực tiếp của chia khối kích thước phi chuẩn trong một số ví dụ. Mục 4 giới thiệu một ứng dụng khác thường dùng chia khối để tối ưu độ phức tạp không gian, và một kỹ thuật dùng chia khối để tối ưu độ phức tạp thời gian của một số thuật toán: Method of Four Russians.

Bài viết nhiều lần nhắc tới thời gian chạy chương trình. Môi trường chạy là hệ điều hành 64-bit Ubuntu 16.04 LTS, Intel Core i3-3240 CPU @ 3.40GHz, bộ nhớ 16GB, GCC/G++ phiên bản 5.4.0, bật tối ưu -O2. Mọi thời gian chạy đều lấy trung vị sau nhiều lần chạy để giảm sai số hết mức có thể.

10.2 Giới thiệu thuật toán chia khối

Chia khối là một thuật toán rất thường gặp trong tin học, thường dùng để xử lý các bài toán như thao tác đoạn trên dãy.

Nếu xử lý thô bạo thao tác đoạn trên một dãy dài n , độ phức tạp là $\mathcal{O}(n)$. Ta có thể chia dãy thành các khối; khi thực hiện thao tác đoạn, phần gồm các khối nguyên vẹn trong đoạn được xử lý cùng nhau, còn phần không nằm trong khối nguyên vẹn được xử lý thô bạo.

Chẳng hạn, nếu cần thực hiện hai loại thao tác: “cộng một số trên đoạn” và “hỏi giá trị tại một điểm”, ta có thể chia khối dãy và duy trì một nhãn cộng cho mỗi khối (ghi lại toàn bộ phần tử trong khối đã được cộng bao nhiêu). Khi sửa đổi, mỗi khối nguyên vẹn được sửa nhãn trực tiếp trong $\mathcal{O}(1)$, còn phần không thuộc khối nguyên vẹn thì sửa thô bạo từng phần tử. Khi hỏi, chỉ cần trả về giá trị phần tử cộng với nhãn cộng của khối chứa nó.

Đặt kích thước khối là S , khi đó có tất cả $\lceil n/S \rceil$ khối. Nếu thao tác trên một phần tử và thao tác trên một khối nguyên vẹn đều có độ phức tạp $\mathcal{O}(1)$, thì một thao tác đoạn có độ phức tạp

$$\mathcal{O}\left(\frac{n}{S} + S\right),$$

đạt cực tiểu $\mathcal{O}(\sqrt{n})$ khi $S = \sqrt{n}$.

Ghi chú hình 4 trong bản gốc: đồ thị của $\frac{n}{S} + S$, S và $\frac{n}{S}$ theo S , với $n = 10^5$, minh họa điểm cân bằng gần $S = \sqrt{n}$.

Vì sao $\mathcal{O}\left(\frac{n}{S} + S\right)$ đạt cực tiểu khi $S = \sqrt{n}$? Ta có thể chứng minh bằng bất đẳng thức trung bình cộng – trung bình nhân:

$$\frac{\sum_{i=1}^n x_i}{n} \geq \sqrt[n]{\prod_{i=1}^n x_i},$$

với mọi $n \geq 2$ số thực không âm $x_1, x_2, \dots, x_n$; dấu bằng xảy ra khi và chỉ khi $x_1 = x_2 = \dots = x_n$.

Lấy $n = 2$, $x_1 = \frac{n}{S}$, $x_2 = S$, ta được

$$\frac{n}{S} + S \geq 2\sqrt{n},$$

dấu bằng xảy ra khi và chỉ khi $\frac{n}{S} = S$, tức $S = \sqrt{n}$.

10.3 Xác định kích thước khối

Với nhiều bài toán chia khối, kích thước khối $\sqrt{n}$ là tối ưu; nhưng cũng có không ít trường hợp $\sqrt{n}$ không tối ưu. Trước hết xét một bài.

Ví dụ 1. Nhập môn chia khối 2 by hzwer. ⁴³ Cho một dãy dài n , có n thao tác. Thao tác gồm hai loại: cộng một số trên đoạn, hoặc hỏi số phần tử nhỏ hơn một giá trị x trong đoạn.

Ta chia dãy thành $\sqrt{n}$ khối, mỗi khối kích thước $\sqrt{n}$. Trước hết xét trường hợp chỉ có thao tác hỏi: phần khối không nguyên vẹn thì liệt kê thống kê; để tìm số phần tử nhỏ hơn một giá trị trong mỗi khối nguyên vẹn, ta có thể sắp xếp trước từng khối (tổng độ phức tạp $\mathcal{O}(n \log n)$), rồi dùng tìm kiếm nhị phân trong khối. Mỗi truy vấn nhị phân trong không quá $\sqrt{n}$ khối và thô bạo không quá $2\sqrt{n}$ phần tử, nên tổng độ phức tạp là

$$\mathcal{O}(n \log n + n\sqrt{n} \log n) = \mathcal{O}(n\sqrt{n} \log n).$$

⁴³Đề và lời giải nguồn: Huang Zhewei [4].

Vậy cộng trên đoạn thì sao? Ta đặt một nhãn cộng cho mỗi khối; mỗi thao tác đánh dấu trực tiếp $\mathcal{O}(1)$ cho mỗi khối nguyên vẹn, còn các khối không nguyên vẹn có ít phần tử nên sửa thô bạo giá trị phần tử. Chú ý sau khi sửa khối không nguyên vẹn, thứ tự phần tử trong khối có thể bị phá, nên hai khối không nguyên vẹn ở đầu và cuối cần được sắp xếp lại.

Khi có nhãn cộng, truy vấn ngoài khối vẫn thô bạo; trong khối thì hỏi số phần tử nhỏ hơn x – nhãn cộng, tức dùng x – nhãn cộng làm giá trị nhị phân.

Cách đặt kích thước $\sqrt{n}$ ở trên đúng là dễ nghĩ và tự nhiên dẫn tới độ phức tạp $\mathcal{O}(n\sqrt{n} \log n)$, nhưng không dễ tối ưu. Ta đặt kích thước khối là S và phân tích lại độ phức tạp.

Với thao tác hỏi, mỗi lần nhị phân trong không quá $\lceil n/S \rceil$ khối và thô bạo không quá $2S$ phần tử, độ phức tạp là

$$\mathcal{O}\left(\frac{n}{S} \log S + S\right).$$

Với thao tác sửa, mỗi lần đánh dấu không quá $\lceil n/S \rceil$ khối và sắp xếp hai khối, độ phức tạp là

$$\mathcal{O}\left(\frac{n}{S} + S \log S\right).$$

Tiền xử lý chỉ cần $\mathcal{O}(n \log n)$. Vì vậy tổng độ phức tạp là

$$\mathcal{O}\left(n \log S \left(\frac{n}{S} + S\right)\right).$$

Biểu thức này hiển nhiên tối ưu khi $S = \sqrt{n}$, tổng độ phức tạp vẫn là $\mathcal{O}(n\sqrt{n} \log n)$; không có tối ưu gì. Vấn đề nằm ở đâu?

Thực ra mỗi lần sửa phần khối không nguyên vẹn không cần $\mathcal{O}(S \log S)$ để sắp xếp: ban đầu mảng đã có thứ tự, sau khi cộng một số vào một phần trong khối, có thể tách mảng cũ thành phần đã sửa và phần chưa sửa; mỗi phần đều là một mảng có thứ tự, nên có thể trộn trong $\mathcal{O}(S)$. Do đó tổng độ phức tạp là

$$\mathcal{O}\left(n \left(\frac{n}{S} \log S + S\right)\right),$$

lấy $S = \sqrt{n \log n}$ thì đạt $\mathcal{O}(n\sqrt{n \log n})$.

10.3.1 Độ phức tạp của kích thước khối

Trong bài trên, cuối cùng ta muốn

$$\mathcal{O}\left(\frac{n \log n}{S} + S\right)$$

đạt cực tiểu. Dùng bất đẳng thức trung bình cộng – trung bình nhân cũng suy ra

$$\frac{n \log n}{S} + S \geq 2\sqrt{n \log n},$$

dấu bằng xảy ra khi và chỉ khi $S = \sqrt{n \log n}$.

Vậy có phải với mọi bài toán chia khối, ta đều có thể đặt kích thước khối là S , rồi cuối cùng dùng bất đẳng thức trung bình cộng – trung bình nhân để suy ra kích thước khối tối ưu và độ phức tạp thấp nhất?

Một số thuật toán chia khối (như Mo có sửa đổi) có độ phức tạp mỗi thao tác là

$$\mathcal{O}\left(\frac{n^2}{S^2} + S\right).$$

Nếu dùng trực tiếp bất đẳng thức trung bình cộng – trung bình nhân, ta chỉ được

$$\frac{n^2}{S^2} + S \geq 2\sqrt{\frac{n^2}{S}},$$

vẫn chưa khử được biến S . Nhưng ta có thể tách S thành $S/2 + S/2$ rồi dùng bất đẳng thức trung bình cộng – trung bình nhân:

$$\frac{n^2}{S^2} + \frac{S}{2} + \frac{S}{2} \geq 3\sqrt[3]{\frac{n^2}{4}} = \mathcal{O}(n^{2/3}),$$

dấu bằng xảy ra khi $S = \sqrt[3]{2n^2}$. Hình 5 trong bản gốc cho đồ thị của $\frac{n^2}{S^2} + S$ theo S với $n = 50000$.

Trong tình huống này vẫn có thể dùng bất đẳng thức trung bình cộng – trung bình nhân để suy ra kích thước khối tối ưu và độ phức tạp thấp nhất, nhưng độ khó suy diễn tăng đáng kể: ta phải nghĩ cách dùng bất đẳng thức để khử S .

Vì trong phần lớn trường hợp, hàm độ phức tạp thời gian của thuật toán chia khối đạt cực tiểu khi đạo hàm theo kích thước khối S bằng 0, ta cũng có thể giải phương trình đạo hàm bằng 0 để thu được kích thước khối tối ưu và độ phức tạp thấp nhất. Chẳng hạn, đạo hàm của $\frac{n^2}{S^2} + S$ là $-2\frac{n^2}{S^3} + 1$, giải

$$-2\frac{n^2}{S^3} + 1 = 0$$

được $S = \sqrt[3]{2n^2}$. Nhưng lấy đạo hàm cũng không hẳn đơn giản.

Dưới đây là một cách gọn để xác định kích thước khối tối ưu. Trong đa số trường hợp, ta có thể suy ra độ phức tạp thời gian dạng $\mathcal{O}(A + B)$, trong đó A giảm khi S tăng, B tăng khi S tăng, chẳng hạn $A = \frac{n \log n}{S}$, $B = S$. Khi đó giải phương trình $A = B$ là được kích thước khối S tối ưu về độ phức tạp; hơn nữa giá trị của một vế trong phương trình $A = B$ chính là độ phức tạp tối ưu.

Xét ví dụ trên:

$$\frac{n^2}{S^2} = S$$

cho $S = n^{2/3}$, và độ phức tạp thời gian tối ưu là $S = n^{2/3}$. Cách này gọn hơn nhiều.

Chứng minh phương pháp trên như sau. Vì $A + B \leq 2 \max(A, B)$ và $\max(A, B) \leq A + B$, nên $\mathcal{O}(A + B) = \mathcal{O}(\max(A, B))$; ta chỉ cần làm $\max(A, B)$ nhỏ nhất. Lại vì A giảm khi S tăng, B tăng khi S tăng, nên nếu tại $S = x$ có $A = B = y$, thì khi $S < x$ ta có $A > y$, khi $S > x$ ta có $B > y$. Do đó $\max(A, B)$ nhỏ nhất khi $A = B$.

Nếu bài toán phức tạp hơn và cuối cùng chỉ suy ra được độ phức tạp dạng $\mathcal{O}(A + B + C)$, trong đó A giảm theo S , còn B, C tăng theo S (hoặc ngược lại: A tăng, B, C giảm), thì tương tự, có thể lần lượt giải $A = B$, $A = C$, rồi thay vào biểu thức ban đầu để so sánh kích thước khối nào tốt hơn.

Thêm vài ví dụ:

$$\mathcal{O}\left(\frac{nm \log n}{S} + nS\right) \Rightarrow S = \sqrt{m \log n}, \quad \mathcal{O}\left(n\sqrt{m \log n}\right);$$

$$\mathcal{O}\left(\frac{n^3}{S^2 w} + mS\right) \Rightarrow S = \sqrt[3]{\frac{n^3}{mw}}, \quad \mathcal{O}\left(\frac{nm^{2/3}}{w^{1/3}}\right);$$

$$\mathcal{O}\left(\frac{n^2}{S^2} + \frac{n}{S} + S\right) \Rightarrow S = n^{2/3}, \quad \mathcal{O}(n^{2/3}).$$

10.3.2 Hằng số của kích thước khối

Sau khi xác định bậc độ phức tạp của kích thước khối, ta tiếp tục xác định hằng số. Trên thực tế, trong đa số tình huống, không xét hằng số đã rất gần với kích thước khối tối ưu: tính ra $\mathcal{O}(\sqrt{n})$ thì lấy $\sqrt{n}$, tính ra $\mathcal{O}(\sqrt{n \log n})$ thì lấy $\sqrt{n \log n}$. Nguyên nhân là hàm thời gian chạy (số phép tính) theo kích thước khối thường khá “phẳng” gần giá trị tối ưu (đạo hàm có trị tuyệt đối nhỏ), nên hằng số của kích thước khối hơi lệch không ảnh hưởng nhiều.

Nhưng cũng có ngoại lệ. Chẳng hạn bài *Nhập môn chia khối 2* by *hzwer* ở trên cuối cùng suy ra bậc của kích thước khối là $\mathcal{O}(\sqrt{n \log n})$, nhưng kích thước khối tối ưu thực tế có hằng số rất nhỏ, đến mức nếu chỉ nhìn kích thước khối thực tế thì dễ tưởng bậc là $\mathcal{O}(\sqrt{n})$. Nguyên nhân là:

- Độ phức tạp mỗi thao tác là $\mathcal{O}(\frac{n}{S} \log S + S)$. Khi phân tích ta đã đổi $\log S$ thành $\log n$; do $S \approx \sqrt{n}$, nên $\log S \approx \frac{1}{2} \log n$, sinh ra hằng số $1/2$.
- Phần $\mathcal{O}(\frac{n}{S} \log S)$ là tìm kiếm nhị phân có hiệu suất cao, và chỉ thao tác hồi mới có độ phức tạp này; phần $\frac{n}{S}$ của thao tác sửa có thể bỏ qua so với nó. Trong khi đó phần $\mathcal{O}(S)$ là trộn có hiệu suất thấp hơn. Để cân bằng hiệu suất hai phần, cần giảm S .

Ghi chú hình 6 trong bản gốc: so sánh điểm tối ưu của $\frac{n}{S} \log n + S$ và $\frac{n}{S} \log S + 2S$, với $n = 50000$. Đồ thị thứ hai có điểm tối ưu dịch về kích thước khối nhỏ hơn.

Dưới đây là một tối ưu hằng số áp dụng được cho nhiều bài toán chia khối. Khi thao tác đoạn và xử lý thô bạo phần không nằm trong khối nguyên vẹn, nếu phần này có nhiều phần tử, vượt quá một nửa độ dài khối, ta có thể đổi sang thao tác trên cả khối nguyên vẹn, rồi thực hiện “thao tác ngược” trên phần không cần thao tác trong khối ấy. Ví dụ cộng đoạn thì đổi thành trừ đoạn; tính tổng đoạn thì đổi thành lấy số đối của tổng. Các thao tác không có tính “trừ được” như gán đoạn không dùng được tối ưu này.

Chẳng hạn, chia dãy $[1..100]$ thành 10 khối $[1..10], [11..20], \dots, [91..100]$, nay cần cộng x vào đoạn $[2..56]$. Cách thông thường đánh dấu $+x$ cho các khối $[11..20], \dots, [41..50]$, rồi cộng x vào các phần tử 2, 3, 4, 5, 6, 7, 8, 9, 10, 51, 52, 53, 54, 55, 56. Sau tối ưu, ta có thể đánh dấu $+x$ cho các khối $[1..10], \dots, [51..60]$, rồi trừ x ở các phần tử 1, 57, 58, 59, 60.

Thêm tối ưu này, hằng số của phần thô bạo giảm một nửa, nên kích thước khối nên tăng thích hợp để cân bằng hiệu suất hai phần.

Ví dụ 2. Thử thách (subtask 2). ⁴⁴ Cho hai xâu tam phân dài n . Có q truy vấn, hỏi sau khi lấy một đoạn con từ mỗi xâu thì có bao nhiêu vị trí tương ứng bằng nhau. Trong đề gốc hỏi số vị trí tương ứng hơn kém 1 theo modulo 3, có thể chuyển thành bằng nhau.

Giới hạn: $n = q = 300000$. Giới hạn thời gian: 3s. Giới hạn bộ nhớ: 2GB.

Trước hết xem thuật toán chuẩn làm thế nào. Ta dùng xâu nhị phân để biểu diễn xâu tam phân đầu vào:

$$0 \rightarrow 100, \quad 1 \rightarrow 010, \quad 2 \rightarrow 001.$$

Khi đó chỉ cần thống kê số bit 1 trong kết quả and theo bit của hai xâu con.

Dùng trực tiếp `std::bitset` thì thời gian mỗi phép tính là $\Theta(n/w)$, chạy vài chục giây. Nếu tự viết một `bitset`, ta có thể chỉ tính phần cần hỏi, dùng bảng theo đoạn để thống kê số bit 1. Dù độ phức tạp vẫn là $\mathcal{O}(nq/w)$, cách này nhanh hơn nhiều, thời gian chạy là 3.898s. Thêm tối ưu đọc vào và mở vòng lặp, thời gian giảm xuống 3.689s.

Khi giảng bài, Wang Yisong [5] nhắc tới một cách truyền thống chia khối FFT $\mathcal{O}(n(\log n)^{1.5})$, nhưng không chạy qua. Bây giờ ta tối ưu thuật toán này.

Nhắc lại bài toán sau khi chuyển hóa: cho hai xâu nhị phân cùng độ dài không quá $9 \cdot 10^5$, có không quá $3 \cdot 10^5$ truy vấn. Mỗi truy vấn hỏi số bit 1 trong kết quả and theo bit giữa một xâu con của xâu thứ nhất và một xâu con của xâu thứ hai.

Chú ý rằng số bit 1 trong and của hai xâu 01 chính là tích vô hướng của chúng; nếu đảo một xâu bằng `std::reverse`, giá trị đó là một hạng trong tích chập.

⁴⁴Nguồn bài: WC 2017.

Ta chia mỗi xâu thành num khối, mỗi khối kích thước S , trong đó $\text{num} = \lceil 3n/S \rceil$. Với mỗi khối của xâu thứ nhất và mỗi khối của xâu thứ hai, đảo khối của xâu thứ hai rồi làm một tích chập (có thể dùng FFT trong $\mathcal{O}(S \log S)$). Khi cài đặt cụ thể, có thể lưu kết quả FFT của 2 num khối, tiết kiệm $2 \text{num}^2 - 2 \text{num}$ lần FFT, nhưng vẫn cần num^2 lần IFFT. Ta thu được một mảng dài $2S$; khi cài đặt, dùng `short` thay vì `int` có thể tiết kiệm không gian và hơi giảm hằng số thời gian. Phần tử thứ i ($1 \leq i \leq S$) của mảng này là tích vô hướng giữa tiền tố dài i của khối xâu thứ nhất và hậu tố dài i của khối xâu thứ hai trước khi đảo. Phần tử thứ $2S - i$ là tích vô hướng giữa hậu tố dài i của khối xâu thứ nhất và tiền tố dài i của khối xâu thứ hai trước khi đảo. Vì vậy ta có thể tiền xử lý trong thời gian

$$\mathcal{O}(\text{num}^2 S \log S) = \mathcal{O}\left(\frac{n^2 \log S}{S}\right)$$

và không gian

$$\mathcal{O}(\text{num}^2 S) = \mathcal{O}\left(\frac{n^2}{S}\right)$$

để biết tích vô hướng giữa tiền tố/hậu tố tùy ý của một khối tùy ý ở xâu thứ nhất với hậu tố/tiền tố tùy ý của một khối tùy ý ở xâu thứ hai.

Khi truy vấn, xét xâu con của xâu thứ nhất được chia khối ra sao. Phần đầu và cuối không thuộc khối nguyên vẹn thì tính thô bạo. Với mỗi khối nguyên vẹn, nếu nó tương ứng với một khối nguyên vẹn của xâu thứ hai, lấy trực tiếp kết quả tiền xử lý. Nếu không, nó nhất định tương ứng với hậu tố của một khối và tiền tố của khối kế tiếp trong xâu thứ hai. Giả sử độ dài hậu tố là x , lấy tích vô hướng giữa tiền tố dài x của khối nguyên vẹn ở xâu thứ nhất với hậu tố dài x của khối này ở xâu thứ hai, cộng với tích vô hướng giữa hậu tố dài $S - x$ của khối nguyên vẹn ở xâu thứ nhất với tiền tố dài $S - x$ của khối kế tiếp ở xâu thứ hai. Do đó một truy vấn được trả lời trong

$$\mathcal{O}(\text{num} + S) = \mathcal{O}\left(\frac{n}{S} + S\right).$$

Tổng độ phức tạp là

$$\mathcal{O}\left(\frac{n^2 \log S}{S} + q\left(\frac{n}{S} + S\right)\right).$$

Lấy $S = \sqrt{n/\log n}$ được $\mathcal{O}(n(\log n)^{1.5} + q\sqrt{n \log n})$. Nhưng lấy $S = \sqrt{n \log n}$ thì được độ phức tạp tốt hơn:

$$\mathcal{O}\left((n + q)\sqrt{n \log n}\right).$$

Nhờ xác định kích thước khối, ta đã chia độ phức tạp cho một log. Hiệu quả thực tế như bảng 1.

Kích thước khối	Thời gian chạy
$256\sqrt{n/\log n}$	130.347s
$512\sqrt{2n/\log n}$	73.173s
$1024\sqrt{n}$	41.272s
$2048\sqrt{2n}$	21.539s
$4096\sqrt{n \log n}$	14.555s
$8192\sqrt{2n \log n}$	14.599s
16384	21.271s

Dù hiệu quả tối ưu độ phức tạp rất rõ, thời gian 14.555s / 14.599s (tương ứng lấy $S = 4096\sqrt{n \log n}$ và $S = 8192\sqrt{2n \log n}$; các cặp thời gian sau cũng hiểu tương tự) vẫn kém xa thuật toán chuẩn. Ta cần vài tối ưu hằng số.

1. Giảm số lần FFT/IFFT. Vì đầu vào đều là số thực (xâu 01), dùng kỹ thuật Mao Xiao nhắc trong luận văn đội tuyển tập huấn năm 2016 [6] có thể gộp mỗi hai lần FFT/IFFT thành một lần. Sau tối ưu, thời gian giảm xuống 11.362s / 12.866s.
2. Tối ưu đọc vào. Có thể dùng fread. Sau tối ưu, thời gian giảm xuống 11.110s / 12.151s.
3. Giảm số lần thô bạo. Khi truy vấn, nếu phần đầu và cuối không thuộc khối nguyên vẹn có độ dài vượt quá nửa khối, ta có thể đổi kết quả thô bạo phần này thành trừ đi kết quả thô bạo phần không nằm trong khối ấy, đồng thời tính thêm một khối nguyên vẹn. Như vậy hằng số của phần này chia đôi. Sau tối ưu, thời gian giảm xuống 8.783s / 6.912s.
4. Giảm quy mô FFT/IFFT. Chú ý giá trị lớn nhất của kết quả tích chập chỉ là $S/3$, lãng phí độ chính xác của kiểu double. Ta có thể dùng một double để lưu mỗi hai hạng liên kề, dùng một số lớn x để phân biệt. Ở đây x cần lớn hơn giá trị lớn nhất của tích chập; lấy $x = S/2$ sẽ tiện thao tác bit vì S là lũy thừa của 2.

Ví dụ, thay vì tính tích chập của $\{a_0, a_1, a_2, a_3\}$ và $\{b_0, b_1, b_2, b_3\}$, ta tính tích chập của $\{a_0 + a_1x, a_2 + a_3x\}$ và $\{b_0 + b_1x, b_2 + b_3x\}$. Kết quả thu được chứa các hạng x^2 ; chuyển mọi hạng x^2 sang hạng hằng của biểu thức kế tiếp là khôi phục được tích chập ban đầu.

Sau cách này, giá trị lớn nhất của kết quả tích chập trở thành

$$\frac{S}{6} \cdot (x+1)^2 = \frac{S^3 + 4S^2 + 4S}{24},$$

với $S = 131072$ cũng chỉ khoảng $9.38 \cdot 10^{13}$, nằm trong phạm vi kiểu double chịu được. Khi chuyển từ double về số nguyên, cần dùng kiểu như long long. Sau tối ưu, thời gian giảm xuống 5.854s / 5.260s.

Một tối ưu khác là bỏ cách trên và đổi double thành float, nhưng thời gian chỉ giảm xuống 8.077s / 6.524s, kém xa tối ưu trên.

5. Trở về bài gốc. Khi chuyển xâu tam phân thành xâu nhị phân, ta đã làm quy mô dữ liệu tăng gấp 3, khiến phần thô bạo chạy rất chậm. Ta có thể không chuyển bài toán, mà làm trực tiếp ba lần tiền xử lý FFT trên xâu tam phân gốc: với các vị trí tương ứng cùng là 0, cùng là 1, cùng là 2, mỗi lần coi ký tự mục tiêu là 1 và các ký tự khác là 0. Như vậy còn có thể cộng kết quả của ba lần tiền xử lý lại để tiết kiệm bộ nhớ.

Khi truy vấn, phần khối nguyên vẹn làm như trước, phần thô bạo đổi thành mô phỏng trực tiếp. Tổng độ phức tạp không đổi, hằng số thời gian của phần thô bạo chia 3, hằng số không gian cũng chia 3. Sau tối ưu, thời gian giảm xuống 1.646s / 1.302s.

Đến đây ta đã đạt 1.302s, nhanh hơn nhiều so với thuật toán chuẩn kể trên (3.689s). Kết hợp ưu điểm của thuật toán chuẩn, ta còn có thể tối ưu độ phức tạp thêm một bước. Nhìn lại phần thô bạo của thuật toán này, ngoài quy mô truy vấn nhỏ hơn thì không khác bài gốc. Vậy tại sao không dùng bitset? Tối ưu này không thể dùng đồng thời với tối ưu hằng số cuối cùng ở trên.

Sau khi dùng bitset, độ phức tạp một truy vấn giảm xuống

$$\mathcal{O}\left(\text{num} + \frac{S}{w}\right) = \mathcal{O}\left(\frac{n}{S} + \frac{S}{w}\right).$$

Tổng độ phức tạp là

$$\mathcal{O}\left(\frac{n^2 \log S}{S} + q\left(\frac{n}{S} + \frac{S}{w}\right)\right).$$

Lấy $S = \sqrt{nw \log n}$ được

$$\mathcal{O}\left((n+q)\sqrt{\frac{n \log n}{w}}\right).$$

Sau tối ưu, thời gian giảm xuống 0.979s / 0.774s. Hai số này lần lượt ứng với $S = 32768\sqrt{nw \log n}$ và $S = 65536\sqrt{2nw \log n}$. Thậm chí bỏ hết các tối ưu hằng số phía trước, cách này vẫn chạy 2.488s / 2.269s.

Ngoài ra, có thể mở vòng lặp ở bước dùng bitset tự viết để and hai xâu con, tra bảng số bit 1 rồi cộng lại. Cuối cùng thời gian giảm xuống 0.964s / 0.749s.

10.4 Ứng dụng của chia khối kích thước phi chuẩn

10.4.1 Tối ưu độ phức tạp không gian của sàng Eratosthenes

Sàng Eratosthenes [7] là một phương pháp đơn giản để tìm số nguyên tố: gạch bỏ mọi bội của các số nguyên tố không vượt quá $\sqrt{n}$, sẽ thu được mọi số nguyên tố không vượt quá n .

Độ phức tạp thời gian của nó là

$$\mathcal{O}\left(n \sum_{\substack{p \leq \sqrt{n} \\ p \text{ nguyên tố}}} \frac{1}{p}\right) = \mathcal{O}(n \log \log n)$$

theo định lý Mertens thứ hai [8], và độ phức tạp không gian là $\mathcal{O}(n)$ bit.

Ta cũng có thể trong lúc sàng số nguyên tố, phân tích thừa số nguyên tố của mỗi số từ 1 đến n ; độ phức tạp thời gian vẫn là

$$\mathcal{O}\left(n \sum_{\substack{p \leq n \\ p \text{ nguyên tố}}} \sum_{i=1}^{\infty} \frac{1}{p^i}\right) = \mathcal{O}\left(n \sum_{\substack{p \leq n \\ p \text{ nguyên tố}}} \frac{1}{p-1}\right) = \mathcal{O}(n \log \log n),$$

độ phức tạp không gian là $\mathcal{O}(n)$.

Thuật toán này chia khối được. Với một đoạn bất kỳ $(L, R]$, nếu đã tiền xử lý mọi số nguyên tố không vượt quá $\sqrt{R}$, ta có thể phân tích thừa số nguyên tố của mọi số trong $(L, R]$ trong thời gian

$$\frac{R}{\log R} + (R - L) \sum_{\substack{p \leq \sqrt{R} \\ p \text{ nguyên tố}}} \sum_{i=1}^{\infty} \frac{1}{p^i}.$$

Vì vậy, nếu chia $(0, n]$ thành các khối kích thước S , độ phức tạp thời gian là

$$\begin{aligned} & \sum_{i=1}^{\lfloor n/S \rfloor} \left(\frac{\sqrt{iS}}{\log(iS)} + S \sum_{\substack{p \leq \sqrt{iS} \\ p \text{ nguyên tố}}} \sum_{j=1}^{\infty} \frac{1}{p^j} \right) \\ &= \mathcal{O}\left(\sum_{i=1}^{\lfloor n/S \rfloor} \left(\frac{\sqrt{iS}}{\log(iS)} + S \log \log(iS) \right)\right) \\ &= \mathcal{O}\left(\frac{n^{1.5}}{S \log n} + n \log \log n\right). \end{aligned}$$

Độ phức tạp không gian là

$$\mathcal{O}\left(\frac{\sqrt{n}}{\log n} + S\right),$$

trong đó $\frac{\sqrt{n}}{\log n}$ là để lưu các số nguyên tố không vượt quá $\sqrt{n}$. Khi tiền xử lý, có thể dùng $\mathcal{O}(n^{1/4})$ thời gian và $\mathcal{O}(1)$ không gian cho mỗi số để kiểm tra thô bạo nó có nguyên tố hay không, nên không cần không gian $\sqrt{n}$.

Do đó lấy $S = \frac{\sqrt{n}}{\log n}$ thì vẫn giữ được độ phức tạp thời gian $\mathcal{O}(n \log \log n)$, đồng thời đạt độ phức tạp không gian $\mathcal{O}(\sqrt{n}/\log n)$. Khi bộ nhớ cho phép, có thể dùng $\mathcal{O}(\sqrt{n})$ không gian để tiền xử lý số nguyên tố, đồng thời lấy $S = \sqrt{n}$, khi đó hằng số thời gian nhỏ hơn. Sàng Eratosthenes chia khối không chỉ tiết kiệm bộ nhớ, mà còn tránh mở mảng lớn, có thể đặt cả mảng vào cache cấp một hoặc cấp hai, nên hiệu suất thực tế cao hơn; xem ví dụ tiếp theo.

Ví dụ 3. Công thức của Tiêu Z. ⁴⁵ Đặt

$$S(n) = \sum_{i=1}^n i \cdot \sigma_1(i^2),$$

trong đó $\sigma_1(x)$ là tổng các ước của x . Nhập một số nguyên dương n , xuất $S(n) \bmod (10^9 + 7)$.

Giới hạn: $n \leq 10^9$. Giới hạn bộ nhớ: 1GB.

Bài này có lời giải $\mathcal{O}(n^{3/4}/\log n)$, nhưng độ khó suy diễn và cài đặt đều cao, lại không liên quan nhiều tới bài viết này nên không nói thêm.

Dưới đây giới thiệu cách chia đoạn lập bảng. Chú ý với mỗi số x , chỉ cần phân tích x thành thừa số nguyên tố là có thể tính tiện $x \cdot \sigma_1(x^2) \bmod (10^9 + 7)$. Cụ thể, đặt

$$x = \prod_{i=1}^k p_i^{\alpha_i},$$

trong đó $p_1, p_2, \dots, p_k$ là các số nguyên tố đôi một khác nhau, $\alpha_1, \alpha_2, \dots, \alpha_k \in \mathbb{N}^*$. Khi đó

$$x \cdot \sigma_1(x^2) = x \prod_{i=1}^k \sum_{j=0}^{2\alpha_i} p_i^j = x \prod_{i=1}^k (p_i^{2\alpha_i+1} - 1)(p_i - 1)^{-1}.$$

Vì vậy có thể chọn một độ dài đoạn len, chẳng hạn 10^7 , dùng sàng Eratosthenes chia khối ở trên để tiền xử lý cục bộ trong thời gian $\mathcal{O}(n \log \log n)$ mọi

$$S(i \cdot \text{len}) \bmod (10^9 + 7) \quad (i \in \mathbb{N}, i \cdot \text{len} \leq n),$$

rồi thông qua bài này trong thời gian $\mathcal{O}(\text{len} \log \log n)$ và độ dài mã $\mathcal{O}((n/\text{len}) \log \text{answer})$.

Kích thước khối	Thời gian chạy
$3052\sqrt{n}/\log n$	36.882
16384	31.330
$31623\sqrt{n}$	29.840
65536	31.613
10^7	56.506
10^8	57.812
10^9 (không chia khối)	59.674

Bảng trên cho thấy chia khối cũng tối ưu thời gian của sàng Eratosthenes rất rõ. Sàng Euler (tức “sàng tuyến tính”, thời gian và không gian đều $\mathcal{O}(n)$) chạy dữ liệu $n = 10^9$ chỉ cần 19.167s, nhưng bất lợi không gian rất lớn (cần ít nhất 6.5GB bộ nhớ), và không thể dùng để lập bảng theo đoạn.

Trên một máy khác dùng Intel Core i5-6500 CPU @ 3.20 GHz và Windows 10, sàng Eratosthenes không chia khối chạy 67.35s, sàng Euler chạy 21.85s, còn sàng Eratosthenes chia khối (kích thước khối vẫn là $31623\sqrt{n}$) có thể đặt cả mảng vào cache cấp một nên chỉ chạy 24.81s; hiệu quả tối ưu càng rõ.

⁴⁵Nguồn bài: Wang Jinghan, Trường Trung học Thực nghiệm trực thuộc Đại học Sư phạm Bắc Kinh.

10.4.2 Method of Four Russians

Method of Four Russians [9] là một kỹ thuật tăng tốc một số thuật toán. Tư tưởng chính của nó là chia khối, tiền xử lý thô bạo bên trong khối, áp dụng thuật toán gốc giữa các khối nguyên vẹn, qua đó giảm quy mô dữ liệu của thuật toán gốc và tối ưu độ phức tạp thời gian. Khác với các kiểu chia khối trước đó, vì tiền xử lý bên trong khối của Method of Four Russians rất tốn thời gian, kích thước khối thường ở cấp $\mathcal{O}(\log n)$.

Mao Xiao trong luận văn đội tuyển tập huấn năm 2016 đã giới thiệu kỹ thuật dùng Method of Four Russians để giảm số phép nhân trong lũy thừa nhanh (tức độ dài chuỗi cộng) từ $2 \log_2 n + C$ (C là hằng số) xuống

$$\log_2 n + \mathcal{O}\left(\frac{\log n}{\log \log n}\right).$$

Nếu chỉ xét số phép nhân ở phần không nhân đôi, đây là ví dụ chia độ phức tạp thời gian cho $\log \log n$; nhưng do lũy thừa nhanh là $\Omega(\log n)$, kỹ thuật này chỉ có thể xem là tối ưu hằng số.

Tiếp theo giới thiệu vài ứng dụng của Method of Four Russians trong tối ưu độ phức tạp thời gian.

Tối ưu nhân ma trận 01. Trong tiểu mục này, nhân ma trận 01 được định nghĩa là: mọi phép toán của phép nhân ma trận thông thường đều thực hiện theo modulo 2.

Giả sử A là ma trận 01 kích thước $m \times n$, B là ma trận 01 kích thước $n \times p$. Định nghĩa tích $C = AB$ là ma trận 01 kích thước $m \times p$, trong đó

$$c_{ij} = \left(\sum_{k=1}^n a_{ik} b_{kj} \right) \bmod 2.$$

Với các định nghĩa khác của nhân ma trận 01, chẳng hạn đảm bảo đầu vào là ma trận 01 nhưng phép nhân là phép nhân ma trận thông thường, một phần cách tối ưu trong tiểu mục này cũng áp dụng được.

Chia khối một chiều. Ta liệt kê từng c_{ij} và xét cách tính nhanh $\sum_{k=1}^n a_{ik} b_{kj}$. Chia khối biểu thức tổng này, đặt kích thước khối là S . Khi đó hai xâu 01 độ dài S có tất cả 4^S trường hợp. Ta có thể tốn $\mathcal{O}(S)$ thời gian tính tổng cho mỗi trường hợp, đồng thời trong $\mathcal{O}(mn + np)$ tiền xử lý mỗi hàng của A và mỗi cột của B ở mỗi khối độ dài S thuộc trường hợp nào. Như vậy tổng độ phức tạp là

$$\mathcal{O}\left(mn + np + mp + \frac{mnp}{S} + 4^S \cdot S\right).$$

Lấy $S = \log_4(mnp)$ được

$$\mathcal{O}\left(mn + np + mp + \frac{mnp}{\log(mnp)}\right).$$

Khi $m = n = p$, độ phức tạp là $\mathcal{O}(n^3 / \log n)$.

Dùng bitset. Thực ra thuật toán trên không có nhiều giá trị thực dụng trong thi tin học, vì nhân ma trận 01 có thể dùng bitset: biểu thức $\sum_{k=1}^n a_{ik} b_{kj}$ có thể thu được bằng cách and hai bitset rồi đếm số bit 1. Khi đó độ phức tạp là

$$\mathcal{O}\left(mn + np + mp + \frac{mnp}{w}\right).$$

Khi $m = n = p$, độ phức tạp là $\mathcal{O}(n^3 / w)$. Với quy mô dữ liệu mà phép nhân ma trận có thể chấp nhận, w lớn hơn $\log n$ rất nhiều.

Chia khối hai chiều. Ta có thể chia các ma trận A, B, C thành các ma trận con $S \times S$. Để tiện, giả sử n, m, p đều là bội của S ; nếu không, có thể thêm 0 ở bên phải hoặc bên dưới các ma trận để làm tròn. Theo phép nhân ma trận phân khối, nếu A_{ij}, B_{ij}, C_{ij} là các khối con, thì

$$C_{ij} = \left(\sum_{k=1}^{n/S} A_{ik} B_{kj} \right) \bmod 2,$$

trong đó lấy modulo 2 cho một ma trận nghĩa là lấy modulo 2 từng phần tử.

Hai ma trận $S \times S$ có tất cả $4S^2$ trường hợp. Ta có thể tốn $\mathcal{O}(S^3)$ thời gian tiền xử lý tích cho mỗi trường hợp, đồng thời trong $\mathcal{O}(mn + np)$ tiền xử lý mỗi khối của A, B thuộc trường hợp nào. Như vậy có thể tính một khối của C trong $\mathcal{O}(nS)$, tổng độ phức tạp là

$$\mathcal{O} \left(mn + np + mp + \frac{mnp}{S} + 4S^2 \cdot S^3 \right).$$

Lấy $S = \sqrt{\log_4(mnp)}$ được

$$\mathcal{O} \left(mn + np + mp + \frac{mnp}{\sqrt{\log(mnp)}} \right).$$

Khi $m = n = p$, độ phức tạp là $\mathcal{O}(n^3 / \sqrt{\log n})$.

Dù độ phức tạp này còn kém chia khối một chiều, nó có thể kết hợp với bitset: khi tiền xử lý tích của hai ma trận $S \times S$, dùng bitset đạt $\mathcal{O}(S^3/w)$; mỗi lần tính một khối của C cũng dùng bitset đạt $\mathcal{O}(nS/w)$. Tổng độ phức tạp là

$$\mathcal{O} \left(mn + np + mp + \frac{mnp}{Sw} + 4S^2 \cdot \frac{S^3}{w} \right),$$

lấy $S = \sqrt{\log_4(mnp)}$ được

$$\mathcal{O} \left(mn + np + mp + \frac{mnp}{w\sqrt{\log(mnp)}} \right).$$

Khi $m = n = p$, độ phức tạp là $\mathcal{O}(n^3 / (w\sqrt{\log n}))$.

Tối ưu LCA và RMQ. LCA [11] (Lowest Common Ancestor), tức tổ tiên chung gần nhất, là bài toán: trong một cây có gốc, tìm tổ tiên chung gần nhất của hai nút. RMQ [12] (Range Minimum Query), tức truy vấn giá trị nhỏ nhất đoạn, là bài toán: trong một dãy số, truy vấn giá trị nhỏ nhất của một đoạn (cũng có thể truy vấn vị trí của giá trị nhỏ nhất).

Hai bài toán này đều kinh điển và có nhiều thuật toán. Dưới đây dùng $\mathcal{O}(f(n)) - \mathcal{O}(g(n))$ để mô tả độ phức tạp: với quy mô dữ liệu n , thuật toán tốn $\mathcal{O}(f(n))$ thời gian tiền xử lý, sau đó mỗi truy vấn tốn $\mathcal{O}(g(n))$. Nếu có m truy vấn, tổng độ phức tạp là $\mathcal{O}(f(n) + m \cdot g(n))$.

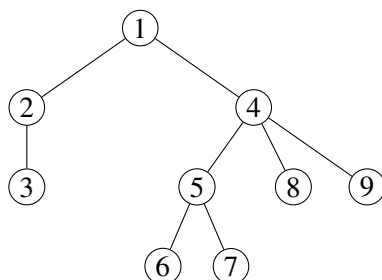
LCA có các thuật toán như vét cạn $\mathcal{O}(n) - \mathcal{O}(n)$ hoặc $\mathcal{O}(n^2) - \mathcal{O}(1)$, nhân đôi $\mathcal{O}(n \log n) - \mathcal{O}(\log n)$, phân rã nặng nhẹ $\mathcal{O}(n) - \mathcal{O}(\log n)$, phân rã chuỗi dài $\mathcal{O}(n) - \mathcal{O}(\sqrt{n})$, và Tarjan offline $\mathcal{O}((n+m)\alpha(n))$.⁴⁶

RMQ có các thuật toán như vét cạn $\mathcal{O}(n) - \mathcal{O}(n)$ hoặc $\mathcal{O}(n^3) - \mathcal{O}(1)$, DP đơn giản $\mathcal{O}(n^2) - \mathcal{O}(1)$, chia khối $\mathcal{O}(n) - \mathcal{O}(\sqrt{n})$, bảng ST $\mathcal{O}(n \log n) - \mathcal{O}(1)$, cây đoạn $\mathcal{O}(n) - \mathcal{O}(\log n)$; Wang Ziyu cũng giới thiệu thuật toán $\mathcal{O}(n \log \log n) - \mathcal{O}(1)$ và $\mathcal{O}(n \log^* n) - \mathcal{O}(1)$ trong luận văn đội tuyển tập huấn năm 2013 [2].

Tiểu mục này tiếp theo giới thiệu thuật toán Method of Four Russians $\mathcal{O}(n) - \mathcal{O}(1)$ cho hai bài toán trên. Đây là độ phức tạp thời gian tối ưu, vì đọc đầu vào đã cần $\Omega(n)$.

⁴⁶ m là số truy vấn, α là hàm Ackermann ngược.

Chuyển LCA thành ± 1 RMQ. Trước hết, ta duyệt Euler toàn bộ cây: bắt đầu DFS từ gốc, mỗi lần đi qua một cạnh (dù đi xuống hay đi lên) thì ghi lại nút hiện tại. Cuối cùng thu được một dãy nút và dãy độ sâu tương ứng.



Ví dụ hình 8: dãy nút là 1, 2, 3, 2, 1, 4, 5, 6, 5, 7, 5, 4, 8, 4, 9, 4, 1; dãy độ sâu là 0, 1, 2, 1, 0, 1, 2, 3, 2, 3, 2, 1, 2, 1, 2, 1, 0.

Hai dãy này có vài tính chất:

- Vì có $n - 1$ cạnh, độ dài dãy là $2n - 1$.
- Mỗi nút xuất hiện trong dãy nút đúng số lần bằng số con của nó cộng 1.
- Trong dãy độ sâu, hiệu của hai số kề nhau luôn là ± 1 .
- LCA của hai nút bất kỳ chính là nút có độ sâu nhỏ nhất giữa hai lần xuất hiện của chúng trong duyệt Euler. Gọi $\text{place}(u)$ là một vị trí xuất hiện bất kỳ của nút u trong dãy nút. Với hai nút u, v , giả sử $\text{place}(u) \leq \text{place}(v)$; nếu vị trí nhỏ nhất của dãy độ sâu trên đoạn $[\text{place}(u), \text{place}(v)]$ là w , thì LCA của u và v là nút ở vị trí w trong dãy nút.

Cái gọi là ± 1 RMQ là RMQ trên dãy mà hiệu hai số kề nhau luôn là ± 1 . Như vậy ta đã chuyển LCA thành ± 1 RMQ trong $\mathcal{O}(n)$.

Thuật toán ST cho RMQ. Thuật toán ST (Sparse Table) tiền xử lý giá trị nhỏ nhất của mọi đoạn có độ dài là lũy thừa của 2, để mỗi lần truy vấn giá trị nhỏ nhất của đoạn bất kỳ trong $\mathcal{O}(1)$.

Có thể dùng DP sau để tiền xử lý trong $\mathcal{O}(n \log n)$. Định nghĩa $f[i][j]$ là giá trị nhỏ nhất của đoạn $[j, j + 2^i)$. Giá trị đầu: $f[0][j] = a[j]$, với a là dãy ban đầu; khi $j > n$ thì $f[i][j] = \infty$. Chuyển:

$$f[i][j] = \min(f[i-1][j], f[i-1][j + 2^{i-1}]).$$

Truy vấn: giá trị nhỏ nhất trên đoạn $[l, r)$ bằng

$$\min(f[\lceil \log_2(r-l) \rceil][l], f[\lceil \log_2(r-l) \rceil][r - 2^{\lceil \log_2(r-l) \rceil}]).$$

Ở đây $\lceil \log_2 x \rceil$ có thể tiền xử lý trong $\mathcal{O}(n)$. Nếu cần truy vấn vị trí nhỏ nhất, chỉ cần sửa nhẹ phần DP và truy vấn: mảng tiền xử lý lưu vị trí của giá trị nhỏ nhất, khi lấy min thì so sánh theo giá trị ở vị trí tương ứng.

Method of Four Russians. Ta chia khối dãy của ± 1 RMQ, đặt kích thước khối là S .

Có thể tính giá trị nhỏ nhất của mỗi khối (và vị trí của nó) trong $\mathcal{O}(n)$. Trên dãy gồm $\lceil n/S \rceil$ giá trị nhỏ nhất này, áp dụng thuật toán ST; thời gian tiền xử lý là $\mathcal{O}(\frac{n}{S} \log \frac{n}{S})$, và mỗi lần truy vấn phần khối nguyên vẹn trong $\mathcal{O}(1)$.

Vậy phần đoạn truy vấn không thuộc khối nguyên vẹn thì sao? Lúc này mới dùng tới tính chất “hiệu hai số kề nhau đều là ± 1 ”: nếu lấy hiệu của mỗi hai số kề nhau trong S số của một khối, ta được $S - 1$ số,

nên có không quá 2^{S-1} loại. Ta có thể với mọi dãy ± 1 trong 2^{S-1} loại này, lấy tổng tiền tố để khôi phục thành một dãy S số (chú ý không biết số đầu tiên là bao nhiêu, có thể tạm chọn 0), rồi tiền xử lý vị trí giá trị nhỏ nhất của mọi $\frac{S(S+1)}{2}$ đoạn. Dễ thấy dù số đầu tiên là bao nhiêu, vị trí giá trị nhỏ nhất của các đoạn này đều như nhau. Cuối cùng, tiền xử lý mỗi khối trong $[n/S]$ khối thuộc loại nào, là có thể truy vấn vị trí giá trị nhỏ nhất của đoạn bất kỳ trong một khối bất kỳ trong $\mathcal{O}(1)$.

Tóm lại, ta đạt mỗi truy vấn $\mathcal{O}(1)$, còn thời gian tiền xử lý là

$$\mathcal{O}\left(\frac{n}{S} \log \frac{n}{S} + 2^S \cdot S^2\right).$$

Lấy $S = \log_2 n$ là đạt $\mathcal{O}(n)$.

So sánh thời gian chạy LCA. Trong bảng 3 và bảng 4, dạng “ $a + b = c$ ” nghĩa là tiền xử lý tốn a giây, n truy vấn tốn tổng cộng b giây, còn nhập cây n đỉnh và trả lời n truy vấn tốn tổng cộng c giây. Tách thời gian tiền xử lý và truy vấn như vậy giúp suy ra: khi cây có n đỉnh và số truy vấn là m , tổng thời gian xấp xỉ $a + \frac{m}{n}b$.

Cây được biểu diễn bằng mảng cha, $\text{father}[i]$ là cha của đỉnh i (gốc là đỉnh 0, $i \in [1, n)$, $\text{father}[i] \in [0, i)$). Với bốn dạng cây, “ngẫu nhiên” nghĩa là mọi $\text{father}[i]$ được sinh đều trên $[0, i)$; “chuỗi” nghĩa là cả cây là một đường ($\text{father}[i] = i - 1$); “cây nhị phân hoàn chỉnh” nghĩa là $\text{father}[i] = \lfloor \frac{i-1}{2} \rfloor$; “hỗn hợp” nghĩa là $n/3$ đỉnh tạo thành một chuỗi, $n/3$ đỉnh tạo thành một cây nhị phân hoàn chỉnh, phần còn lại khoảng $n/3$ đỉnh sinh cha ngẫu nhiên. Mọi truy vấn đều sinh ngẫu nhiên.

Trong thuật toán Tarjan offline, có thể viết DSU chỉ nén đường đi, không hợp theo hạng, hằng số nhỏ nhưng trung bình mỗi thao tác $\mathcal{O}(\log n)$; cũng có thể viết DSU nén đường đi + hợp theo hạng, trung bình mỗi thao tác $\mathcal{O}(\alpha(n))$. Trong bảng, dòng Tarjan thứ nhất là thời gian của DSU chỉ nén đường đi, dòng thứ hai là DSU nén đường đi + hợp theo hạng. “Thuật toán ST” chỉ việc chuyển LCA thành RMQ rồi dùng trực tiếp ST $\mathcal{O}(n \log n) - \mathcal{O}(1)$.

Dạng cây	Ngẫu nhiên	Chuỗi	Cây nhị phân	Hỗn hợp
Four Russians	0.221 + 0.354 = 0.575	0.105 + 0.331 = 0.436	0.056 + 0.339 = 0.395	0.172 + 0.372 = 0.544
ST	0.444 + 0.127 = 0.571	0.190 + 0.161 = 0.351	0.177 + 0.130 = 0.307	0.290 + 0.140 = 0.430
Nặng nhẹ	0.091 + 0.632 = 0.723	0.014 + 0.062 = 0.076	0.008 + 0.945 = 0.953	0.045 + 0.606 = 0.651
Nhân đôi	0.043 + 0.388 = 0.431	0.042 + 0.865 = 0.907	0.009 + 0.390 = 0.399	0.081 + 0.947 = 1.028
Tarjan	0.595/0.754	0.707/0.450	0.424/0.450	0.617/0.556
Vết cạn	0.010 + 0.546 = 0.556	–	0.002 + 0.477 = 0.479	–

Bảng 3: so sánh thời gian chạy LCA, $n = 2 \cdot 10^6$.

Dạng cây	Ngẫu nhiên	Chuỗi	Cây nhị phân	Hỗn hợp
Four Russians	1.189 + 1.956 = 3.145	0.534 + 1.912 = 2.446	0.291 + 1.825 = 2.116	0.860 + 1.935 = 2.795
ST	2.611 + 0.685 = 3.296	1.087 + 0.889 = 1.976	1.012 + 0.688 = 1.700	1.767 + 0.779 = 2.546
Nặng nhẹ	0.522 + 4.415 = 4.937	0.145 + 0.333 = 0.478	0.071 + 6.918 = 6.989	0.336 + 4.391 = 4.727
Nhân đôi	0.308 + 2.089 = 2.397	0.244 + 5.154 = 5.398	0.056 + 2.090 = 2.146	0.579 + 6.180 = 6.759
Tarjan	4.287/5.380	4.961/2.898	2.997/3.071	3.890/3.759
Vết cạn	0.081 + 4.406 = 4.487	–	0.010 + 3.756 = 3.766	–

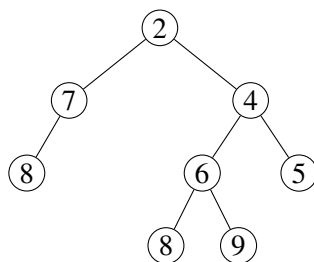
Bảng 4: so sánh thời gian chạy LCA, $n = 10^7$.

Do những năm gần đây trong OI, chấm theo nhóm test ngày càng thịnh hành, ta quan tâm hơn tới hiệu suất trong trường hợp xấu nhất. Trong các thuật toán trên, với $n = 10^7$, Method of Four Russians đúng là nhanh nhất trong trường hợp xấu nhất (trong bốn dạng cây kể trên), nhưng ưu thế so với ST rất nhỏ: dù mỗi truy vấn đều là $\mathcal{O}(1)$, Method of Four Russians phải lấy min của tối đa 4 số, còn ST chỉ lấy min của tối đa 2 số. Khi số truy vấn nhỏ hơn tương đối so với kích thước cây, ưu thế của Method of Four Russians rõ hơn. Chẳng hạn, khi cây có 10^7 đỉnh và $5 \cdot 10^6$ truy vấn, trong trường hợp xấu nhất (cây “ngẫu nhiên”) Method of Four Russians chạy 2.167s, trong khi ST trong trường hợp xấu nhất (cây “ngẫu

nhiên”) chạy 2.953s, phân rã nặng nhẹ trong trường hợp xấu nhất (cây nhị phân hoàn chỉnh) chạy 3.530s, nhân đôi trong trường hợp xấu nhất (cây hỗn hợp) chạy 3.669s, Tarjan offline trong trường hợp xấu nhất (cây ngẫu nhiên) cũng chạy 3.034s (chỉ nén đường đi) / 4.101s (nén đường đi + hợp theo hạng).

Chuyển RMQ thành LCA. RMQ có thể chuyển thành LCA bằng cách dựng cây Descartes (Cartesian tree). Cây Descartes là một cây nhị phân, mỗi nút có hai giá trị: *key* và *value*; *key* thỏa tính chất cây tìm kiếm nhị phân, còn *value* thỏa tính chất heap (trong bài này là heap nhỏ). Ở đây *key* của mỗi nút là vị trí tương ứng trong dãy ban đầu, nên không biểu diễn tường minh; *value* là số tương ứng trong dãy ban đầu.

Trước hết tìm giá trị nhỏ nhất trong dãy (có thể giả sử dãy không có phần tử trùng; nếu có thì dùng chỉ số làm khóa phụ để so sánh, qua đó khử trùng), đặt nó làm gốc. Số này chia dãy ban đầu thành hai nửa; ta lần lượt dựng cây Descartes cho hai dãy con trái và phải, rồi nối gốc của chúng làm con trái và con phải của gốc tương ứng với giá trị nhỏ nhất ban đầu. Như vậy dựng được toàn bộ cây Descartes.

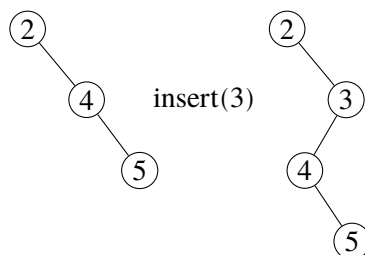


Hình 9: ví dụ cây Descartes của dãy 8, 7, 2, 8, 6, 9, 4, 5.

Để chứng minh với mọi đoạn $[l, r]$, giá trị nhỏ nhất của đoạn này chính là LCA của hai nút tương ứng với l và r trên cây Descartes.

Nhưng dựng đệ quy như vậy quá chậm; có thể dựng tuyến tính không? Câu trả lời là có.

Ta dùng một ngăn xếp đơn điệu để duy trì chuỗi phải của cây Descartes, lần lượt thêm nút từ trái sang phải: bật khỏi đỉnh ngăn xếp những nút lớn hơn nút mới; nối nút mới vào cây con phải của đỉnh ngăn xếp hiện tại (nếu không tồn tại thì đặt nút hiện tại làm gốc); nối nút cuối cùng bị bật ra (nếu có) vào cây con trái của nút mới; cuối cùng đẩy nút mới vào ngăn xếp.



Hình 10: trong bước dựng tuyến tính, ngăn xếp từ 2, 4, 5 chuyển thành 2, 3, và 4 trở thành con trái của 3.

Như vậy ta đã chuyển RMQ thành LCA trong $\mathcal{O}(n)$, rồi áp dụng thuật toán Method of Four Russians ở trên để đạt RMQ $\mathcal{O}(n) - \mathcal{O}(1)$.

Tối ưu LA (tổ tiên mức k). LA [13] (Level Ancestor), tức tổ tiên mức k , là bài toán: trong một cây có gốc, truy vấn tổ tiên thứ k của một nút v ; hoặc truy vấn tổ tiên của v có độ sâu d . Ghi độ sâu của gốc là 0, độ sâu của nút khác là độ sâu của cha cộng 1. Vì có thể tiền xử lý độ sâu của mỗi nút, hai bài toán này tương đương.

Bài toán này có các thuật toán như vét cạn $\mathcal{O}(n) - \mathcal{O}(n)$ hoặc $\mathcal{O}(n^2) - \mathcal{O}(1)$, nhân đôi $\mathcal{O}(n \log n) - \mathcal{O}(\log n)$, phân rã nặng nhẹ $\mathcal{O}(n) - \mathcal{O}(\log n)$, phân rã chuỗi dài $\mathcal{O}(n) - \mathcal{O}(\sqrt{n})$. Tiếp theo giới thiệu thuật toán Method of Four Russians đạt độ phức tạp thời gian tối ưu $\mathcal{O}(n) - \mathcal{O}(1)$.

Thuật toán nhân đôi. Tốn $\mathcal{O}(n \log n)$ thời gian để tiền xử lý tổ tiên cách $1, 2, 4, 8, \dots, 2^i, \dots, 2^\ell$ tầng của mỗi nút v , trong đó $\ell = \lfloor \log_2 \text{depth}(v) \rfloor$. Khi truy vấn tổ tiên thứ k , phân tích nhị phân k , rồi lần lượt nhảy lên.

Ví dụ, muốn hỏi tổ tiên thứ 11 của v , thì bắt đầu từ v , lần lượt nhảy lên 8, 2, 1 tầng. Hiển nhiên mỗi truy vấn tốn $\mathcal{O}(\log n)$.

Phân rã chuỗi dài. Phân rã chuỗi dài nghĩa là mỗi lần chọn một chuỗi dài nhất đi từ gốc xuống (nếu có nhiều chuỗi dài nhất thì chọn tùy ý), xóa nó đi, rồi tiếp tục phân rã chuỗi dài trên mỗi cây sinh ra.

Ta có thể ghi lại dãy nút của mỗi chuỗi được phân rã (theo thứ tự độ sâu tăng dần), đồng thời ghi mỗi nút nằm ở vị trí thứ mấy trong dãy nút của chuỗi chứa nó; ký hiệu nút v ở vị trí $\text{place}(v)$.

Khi truy vấn tổ tiên thứ k của nút v , nếu $\text{place}(v) > k$ thì lấy nút thứ $\text{place}(v) - k$ trong dãy nút của chuỗi chứa v . Ngược lại, truy vấn đệ quy tổ tiên thứ $k - \text{place}(v)$ của cha nút đầu chuỗi chứa v .

Trong quá trình đệ quy trên, mỗi lần đệ quy thêm một tầng, độ dài chuỗi chứa nút đang xét tăng ít nhất 1, nên tối đa đệ quy $\mathcal{O}(\sqrt{n})$ tầng (vì $\sum_{i=1}^{2\lceil \sqrt{n} \rceil} \lceil \sqrt{n} \rceil > n$). Vì vậy mỗi truy vấn có độ phức tạp $\mathcal{O}(\sqrt{n})$.

Ghi chú hình 11 trong bản gốc: một cây 23 nút dùng để minh họa phân rã chuỗi dài; ví dụ truy vấn $LA(17, 5)$ đi qua $LA(17, 5) = LA(15, 4) = LA(12, 3) = LA(1, 0) = 1$.

Thuật toán phân rã thang. Trên cơ sở phân rã chuỗi dài, kéo dài ngược mỗi chuỗi (dãy nút của chuỗi) về phía gốc một đoạn bằng chính độ dài chuỗi (nếu gặp gốc thì dừng). Chuỗi kéo dài này gọi là “thang”, đồng thời giữ nguyên chuỗi (thang) mà mỗi nút thuộc về, dù lúc này một nút có thể bị nhiều thang phủ. Ghi nút v là nút thứ $\text{place}(v)$ trong dãy nút của thang chứa nó.

Khi truy vấn tổ tiên thứ k của v , nếu $\text{place}(v) > k$ thì lấy nút thứ $\text{place}(v) - k$ trong dãy nút của thang chứa v . Ngược lại, truy vấn tổ tiên thứ $k - \text{place}(v) + 1$ của nút đầu thang chứa v .

Trong quá trình đệ quy này, mỗi lần đệ quy thêm một tầng, chiều cao của nút truy vấn (ghi lá có chiều cao 1, nút khác có chiều cao bằng chiều cao lớn nhất của con cộng 1) ít nhất nhân 2, nên tối đa đệ quy $\mathcal{O}(\log n)$ tầng. Vì vậy mỗi truy vấn tốn $\mathcal{O}(\log n)$.

Chẳng hạn trong hình 11 của bản gốc:

$$LA(17, 5) = LA(15, 4) = LA(11, 2) = 1.$$

Chuỗi dài (17) tương ứng thang (15, 17), chuỗi dài (15, 16) tương ứng thang (11, 12, 15, 16), chuỗi dài (10, 11, 12, 13, 14) tương ứng thang (1, 10, 11, 12, 13, 14).

Thuật toán $\mathcal{O}(n \log n) - \mathcal{O}(1)$. Kết hợp thuật toán nhân đôi và thuật toán phân rã thang, ta có được một thuật toán $\mathcal{O}(n \log n) - \mathcal{O}(1)$.

Trước hết lần lượt tốn $\mathcal{O}(n \log n)$ và $\mathcal{O}(n)$ để làm phần tiền xử lý của hai thuật toán. Khi truy vấn tổ tiên thứ k của nút v , ghi chiều cao của v là $\text{height}(v)$. Ta dùng nhân đôi để nhảy bước đầu tiên, đặt khoảng cách nhảy là len , với $\text{len} \geq \lfloor k/2 \rfloor$, và nhảy tới nút u . Khi đó chiều cao của u ít nhất là $\text{height}(v) + \text{len}$. Tiếp theo dùng phân rã thang để truy vấn tổ tiên thứ $k - \text{len}$ của u . Vì

$$\text{height}(u) \geq \text{height}(v) + \text{len} \geq k/2 \geq k - \text{len},$$

tổ tiên thứ $k - \text{len}$ của u nhất định nằm trên thang chứa u , nên có thể tính đáp án trong $\mathcal{O}(1)$.

Thuật toán $\mathcal{O}(n + L \log n) - \mathcal{O}(1)$. Ở đây L là số lá của cây.

Ta có thể trong $\mathcal{O}(n)$ tiền xử lý một lá bất kỳ trong cây con của mỗi nút và khoảng cách từ nút này tới lá ấy. Như vậy có thể trong $\mathcal{O}(1)$ chuyển bài toán hỏi tổ tiên thứ k của một nút bất kỳ thành bài toán hỏi tổ tiên thứ k' của một lá bất kỳ.

Trong thuật toán trên, ta chỉ cần dùng bước nhảy đầu tiên của nhân đôi, nên chỉ cần tiền xử lý tổ tiên cách $1, 2, 4, 8, \dots, 2^i, \dots, 2^\ell$ tầng của mỗi lá v , trong đó $\ell = \lfloor \log_2 \text{depth}(v) \rfloor$. Việc này có thể hoàn thành trong $\mathcal{O}(n + L \log n)$: DFS toàn cây, đồng thời dùng `std::vector` hoặc mảng (thay vì `std::stack`) để duy trì ngăn xếp DFS. Khi duyệt tới một lá, ngăn xếp DFS lần lượt chứa mọi nút trên đường từ lá này tới gốc, nên ta có thể trong $\mathcal{O}(\log n)$ tính tổ tiên cách $1, 2, 4, 8, \dots, 2^i, \dots, 2^\ell$ tầng của lá này.

Khi truy vấn, sau bước nhảy đầu tiên vẫn dùng thuật toán phân rã thang; mỗi truy vấn vẫn tốn $\mathcal{O}(1)$.

Đáng tiếc là trong trường hợp xấu nhất $L = \mathcal{O}(n)$, nên thuật toán này xấu nhất vẫn cần $\mathcal{O}(n \log n)$ thời gian tiền xử lý.

Method of Four Russians cho LA. Để giảm số lá, ta có thể chia khối phần gần lá của cây. Đặt kích thước khối là S , xóa mọi nút có kích thước cây con trong cây gốc không vượt quá S . Khi đó cây mới có không quá n/S lá, vì mỗi lá của cây mới tương ứng với một cây con trong cây gốc có kích thước lớn hơn S . Có thể áp dụng thuật toán trên, tiền xử lý mọi tổ tiên ở tầng lũy thừa của 2 cho mỗi lá của cây mới và phân rã thang của cây mới trong

$$\mathcal{O}\left(n + \frac{n}{S} \log n\right),$$

nhờ đó trả lời truy vấn trên các nút thuộc cây mới trong $\mathcal{O}(1)$.

Với truy vấn ở các nút đã bị xóa, ta có thể trong $\mathcal{O}(n)$ tiền xử lý khoảng cách từ mỗi nút bị xóa tới nút gần nhất không bị xóa (tức lá tương ứng trong cây mới). Nếu khi hỏi tổ tiên thứ k , k không nhỏ hơn khoảng cách này, ta có thể trong $\mathcal{O}(1)$ chuyển truy vấn gốc thành truy vấn trên lá tương ứng trong cây mới, rồi trả lời trong $\mathcal{O}(1)$.

Chỉ còn một tình huống chưa giải quyết: nút hỏi và nút đáp án đều đã bị xóa. Quan sát mọi nút bị xóa, chúng tạo thành một rừng mà mỗi cây có kích thước không quá S .

Xét một cây có gốc kích thước không quá S . Ta có thể duyệt Euler nó: bắt đầu DFS từ gốc, mỗi lần đi qua cạnh hướng xuống thì ghi một dấu ngoặc trái, mỗi lần đi qua cạnh hướng lên thì ghi một dấu ngoặc phải, thu được một dãy ngoặc dài không quá $2S$. Số dãy ngoặc hợp lệ dài không quá $2S$ là tổng S số Catalan đầu, hiển nhiên là $\mathcal{O}(4^S)$; cụ thể gần cấp $4^S/S^{3/2}$, nhưng không cần độ phức tạp chính xác như vậy. Ta có thể với mỗi dãy ngoặc dài không quá $2S$, khôi phục một cây có gốc kích thước không quá S theo thứ tự DFS, rồi dùng thuật toán vết cạn trong $\mathcal{O}(S^2)$ để tính mọi tổ tiên của mọi nút trong cây; tổng thời gian là $\mathcal{O}(4^S \cdot S^2)$.

Ta có thể trong $\mathcal{O}(n)$ tiền xử lý dãy ngoặc tương ứng với mỗi cây trong rừng này, cũng như song ánh giữa nhãn của mỗi nút và vị trí của nó trong thứ tự DFS của cây. Như vậy, với mỗi truy vấn mà nút hỏi và đáp án đều đã bị xóa, ta tra bảng theo dãy ngoặc của cây kích thước không quá S chứa nút hỏi để biết tổ tiên thứ k của nó ở vị trí nào trong thứ tự DFS, rồi đổi về nhãn nút tương ứng trong cây này.

Tóm lại, ta đạt mỗi truy vấn $\mathcal{O}(1)$, còn thời gian tiền xử lý là

$$\mathcal{O}\left(n + \frac{n}{S} \log n + 4^S \cdot S^2\right).$$

Lấy $S = \log_4 n$ là đạt $\mathcal{O}(n)$.

10.4.3 Tiểu kết

Điều kiện để áp dụng Method of Four Russians là: có thể chọn một kích thước khối S sao cho sau khi chia khối bài toán, số loại khối khác nhau có thể xuất hiện nhỏ hơn rất nhiều so với quy mô bài toán gốc.

Với phép nhân ma trận thông thường, chẳng hạn ma trận kích thước 1000×1000 và trị tuyệt đối của mỗi phần tử không vượt quá 10^6 , dù chia thành các ma trận 1×1 , số loại khối khác nhau vẫn rất lớn, bằng bình phương số giá trị có thể của một phần tử ma trận; như vậy không thể áp dụng Method of Four Russians.

Nếu số loại khối khác nhau có thể xuất hiện là $\mathcal{O}(c_1^{S_c})$, trong đó c, c_1 là hằng số và $c_1 > 1$, thì thông thường có thể lấy kích thước khối

$$S = \sqrt[c_3]{c_3 \log_{c_1} n},$$

trong đó c_3 là một hằng số nhỏ, thường không vượt quá 1. Khi đó trong tiền xử lý không vượt quá độ phức tạp của thuật toán gốc, đồng thời chia độ phức tạp theo quy mô dữ liệu của thuật toán gốc cho $\sqrt[c_3]{\log_{c_1} n}$.

10.5 Tổng kết

Thuật toán chia khối tách bài toán gốc thành hai bài toán con: khối nguyên vẹn và khối không nguyên vẹn; lần lượt giải chúng, rồi bằng cách chọn kích thước khối như $S = \sqrt{n}$ để cân bằng hiệu suất hai phần, đạt một độ phức tạp thời gian khá tốt. Khi độ phức tạp của hai bài toán con khác nhau, thuật toán chia khối kích thước phi chuẩn xuất hiện: kích thước khối tối ưu không chỉ có thể là $\mathcal{O}(\sqrt{n})$, mà còn có thể là $\mathcal{O}(\sqrt{nw \log n})$ hoặc $\mathcal{O}(n^{2/3})$, thậm chí có thể là $\mathcal{O}(\log n)$ hoặc $\mathcal{O}(\sqrt{\log n})$.

Bài viết đã bước đầu tìm hiểu thuật toán chia khối kích thước phi chuẩn, giới thiệu một số cách tối ưu thuật toán chia khối, đồng thời giới thiệu ứng dụng của chia khối kích thước phi chuẩn trong tối ưu một số thuật toán khác. Trong đó, ba bài toán LCA, RMQ, LA có thể được tối ưu tới độ phức tạp thời gian tối ưu theo lý thuyết. Dùng chia khối kích thước phi chuẩn không chỉ có thể đạt độ phức tạp thời gian tốt hơn chia khối kích thước thông thường $\sqrt{n}$ cho nhiều bài toán thao tác đoạn, mà còn có thể tối ưu một số thuật toán khác. Hy vọng các ví dụ này có thể gợi mở cho độc giả.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn thầy Hu Weidong đã quan tâm và giúp đỡ tôi trong nhiều năm qua.

Cảm ơn các thầy cô và bạn học khác từng giúp đỡ và gợi mở cho tác giả.

Cảm ơn cha mẹ tôi đã chăm sóc và quan tâm chu đáo tới tôi.

Tài liệu tham khảo

- [1] Luo Jianqiao. *Bàn về ứng dụng của tư tưởng chia khối trong một lớp bài toán xử lý dữ liệu*, luận văn đội tuyển tập huấn quốc gia năm 2013.
- [2] Wang Ziyu. *Ứng dụng của phương pháp chia khối*, luận văn đội tuyển tập huấn quốc gia năm 2013.
- [3] Zou Xiaoyao. *Bàn về chia khối trong một lớp bài toán trực tuyến*, luận văn đội tuyển tập huấn quốc gia năm 2015.
- [4] Huang Zhewei. “[Chia khối] Nhập môn chia khối dãy số 1–9 by hzwer”, <http://hzwer.com/8053.html>.
- [5] Wang Yisong. Slide bình giảng bài *Thử thách*, Trại Đông Toàn quốc năm 2017.
- [6] Mao Xiao. *Tìm hiểu lại biến đổi Fourier nhanh*, luận văn đội tuyển tập huấn quốc gia năm 2016.
- [7] Wikipedia tiếng Anh về sàng Eratosthenes.

- [8] [Wikipedia tiếng Anh về định lý Mertens.](#)
- [9] [Wikipedia tiếng Anh về Method of Four Russians.](#)
- [10] Prof. Erik Demaine. “Lecture 15 – April 12, 2012”, 6.851: Advanced Data Structures (Spring 2012), <https://courses.csail.mit.edu/6.851/spring12/scribe/lec15.pdf>.
- [11] [Wikipedia tiếng Anh về LCA.](#)
- [12] [Wikipedia tiếng Anh về RMQ.](#)
- [13] [Wikipedia tiếng Anh về LA.](#)

11 Cây palindrome và ứng dụng

Weng Wentao, Trường Trung học Kỹ niệm Trung Sơn, thành phố Trung Sơn

Tóm tắt

Bài viết giới thiệu cây palindrome, một cấu trúc dữ liệu khá mới để xử lý các xâu con palindrome. Trước hết bài viết trình bày cấu trúc và cách xây dựng cơ bản của cây palindrome, sau đó đưa ra một số mở rộng trên cây palindrome, và cuối cùng dùng một số bài tập để minh họa năng lực giải quyết bài toán của cây palindrome.

11.1 Dẫn nhập

Palindrome là một loại xâu rất đặc biệt, có nhiều tính chất đẹp. Những năm gần đây, các bài toán về palindrome trong thi thuật toán khá phổ biến, nhưng do các thuật toán liên quan đến palindrome còn nghèo nàn, lời giải của các bài toán thường tương đối đơn điệu. Cây palindrome là một cấu trúc dữ liệu mới, được Mikhail Rubinchik công bố năm 2015; trong nước hiện chưa có nghiên cứu thật sâu về nó. Vì vậy bài viết này sẽ tập trung giới thiệu cây palindrome, đồng thời cho thấy tính linh hoạt mạnh và năng lực giải quyết bài toán của nó, hy vọng đem lại cho độc giả một vài gợi mở về tư duy.

Cấu trúc của bài viết như sau.

Mục 2 đưa ra một số ký hiệu và định nghĩa dùng trong bài.

Mục 3 giới thiệu cấu trúc cơ bản và cách xây dựng cây palindrome.

Mục 4 liệt kê một số mở rộng của cây palindrome, cụ thể là cách chèn và xóa ở hai đầu của cây palindrome, cách xây cây palindrome trên TRIE, và cách làm cây palindrome khả tồn. Mục này kèm một số ví dụ và bài toán để độc giả dễ hiểu và mở rộng.

Mục 5 nêu một số bài toán palindrome điển hình trong thi thuật toán, và cho thấy cách dùng công cụ mạnh là cây palindrome để giải chúng.

11.2 Một số ký hiệu và định nghĩa

Ký hiệu 1. Σ biểu thị kích thước bảng chữ cái.

Ký hiệu 2. $|s|$ biểu thị độ dài của xâu s . s_i biểu thị ký tự thứ i của s . $s[l..r]$, $1 \leq l \leq r \leq |s|$, biểu thị xâu con gồm các ký tự từ vị trí l đến vị trí r của s .

Ký hiệu 3. Ký hiệu $\text{pre}[s][i]$ là $s[1..i]$, gọi là tiền tố i của s . Ký hiệu $\text{suf}[s][i]$ là $s[i..|s|]$, gọi là hậu tố i của s .

Ký hiệu 4. Với xâu t và ký tự c , ký hiệu tc là xâu thu được bằng cách nối ký tự c vào sau t . Ký hiệu ct là xâu thu được bằng cách nối ký tự c vào trước t .

Định nghĩa 2.1. Với hai xâu s, t , định nghĩa s và t khác nhau khi và chỉ khi thỏa ít nhất một trong các điều kiện sau:

- $|s| \neq |t|$.
- $|s| = |t|$, và tồn tại một vị trí j , $1 \leq j \leq |s|$, sao cho $s_j \neq t_j$.

Định nghĩa 2.2. Với xâu $s = s_1s_2 \cdots s_n$, định nghĩa xâu đảo của nó là $s_ns_{n-1} \cdots s_1$.

Định nghĩa 2.3. Một xâu t là xâu con của s khi và chỉ khi tồn tại $1 \leq l \leq r \leq |s|$ sao cho $t = s[l..r]$. Chú ý xâu rỗng không phải là xâu con của s . Với hai xâu con $s[a..b]$, $s[l..r]$ của s , chúng khác nhau khi và chỉ khi các xâu mà chúng biểu thị khác nhau.

Định nghĩa 2.4. Một xâu s là palindrome khi và chỉ khi xâu đảo của s bằng chính s .

Định nghĩa 2.5. Một xâu t là xâu con palindrome của s khi và chỉ khi t là xâu con của s và t là palindrome.

Định nghĩa 2.6. Một xâu t là hậu tố palindrome của s khi và chỉ khi $t \neq s$, t là một hậu tố của s , và t là palindrome. Hậu tố palindrome dài nhất của s được định nghĩa là xâu có độ dài lớn nhất, khác s , trong các hậu tố palindrome. Chú ý một xâu có thể không có hậu tố palindrome; khi đó ta định nghĩa tập hậu tố palindrome của nó chỉ chứa một xâu rỗng. Tương tự, có thể định nghĩa tiền tố palindrome và tiền tố palindrome dài nhất của một xâu.

11.3 Cấu trúc và cách xây dựng cây palindrome

11.3.1 Cấu trúc

Cây palindrome của một xâu s là một rừng gồm hai cây. Gọi hai gốc của chúng lần lượt là *even* và *odd*. Mỗi nút trên cây ứng với một xâu; trừ hai gốc, các nút tương ứng một-một với các xâu con palindrome của s . Mỗi cạnh trên cây có một ký tự tương ứng, và mọi cạnh đi ra từ cùng một nút có ký tự đôi một khác nhau. Với một nút i , gọi fa_i là cha của nó, thì xâu s_i ứng với i được tạo bằng cách thêm vào hai đầu của xâu s_{fa_i} ứng với fa_i ký tự c trên cạnh nối i với fa_i , tức $s_i = cs_{fa_i}c$. Đặc biệt, gốc *even* ứng với xâu rỗng; gốc *odd* ứng với một xâu có độ dài -1 và không thật sự tồn tại; các con của *odd* ứng với ký tự trên cạnh đi ra. Trong bài viết này, ta sẽ trực tiếp dùng một nút trên cây để chỉ xâu tương ứng của nó.

Ngoài ra, với một nút i trên cây palindrome, định nghĩa con trở thất bại $fail_i$ trở tới nút ứng với hậu tố palindrome dài nhất của i . Đặc biệt, định nghĩa $fail_{even}$ và $fail_{odd}$ đều là *odd*.

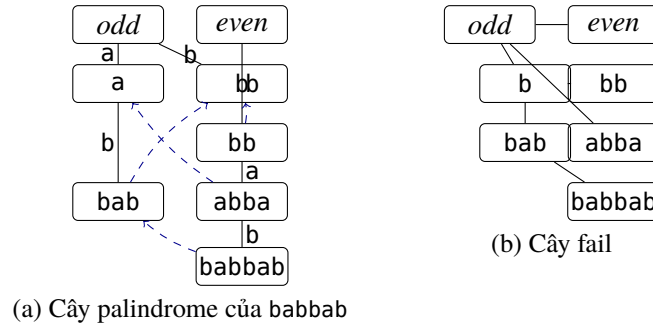
Cụ thể, với mỗi nút trên cây palindrome, ghi lại ba loại giá trị sau:

- len , độ dài xâu ứng với nút đó; riêng *even* và *odd* có len lần lượt là 0 và -1 .
- $go[c]$, nút được nối bởi cạnh đi ra có ký tự c ; nếu không tồn tại thì để rỗng.
- $fail$, nút mà con trở thất bại của nút này trở tới.

Từ các con trở fail, có thể định nghĩa cây fail của một chuỗi s là cây tạo bởi các cạnh fail. Với một nút i trên cây fail, định nghĩa chuỗi fail của nó là một tập, bằng chuỗi fail của fail _{i} hợp với i . Đặc biệt, định nghĩa chuỗi fail của *odd* chỉ gồm chính nó.

Ví dụ 1. Cây palindrome và cây fail tương ứng với chuỗi babbab.

Trong hình, nét đứt màu xanh biểu thị chuyển fail, nét liền biểu thị chuyển go, chữ trên nét liền là ký tự chuyển, và chuỗi ghi trên nút là chuỗi con palindrome tương ứng trong s .



11.3.2 Số nút và số chuyển

Có thể thấy rằng nếu một chuỗi s có n chuỗi con palindrome khác nhau, thì cây palindrome tương ứng của s có đúng $n + 2$ nút; vấn đề trở thành phân tích số chuỗi con palindrome khác nhau của s .

Định lý 3.1. Với một chuỗi s , số chuỗi con palindrome khác nhau nhiều nhất là $|s|$.

Chứng minh. Dùng quy nạp toán học.

- Khi $|s| = 1$, chỉ có một chuỗi con $s[1..1]$, và nó là palindrome, nên kết luận đúng.
- Khi $|s| > 1$, đặt $s = s'c$, trong đó c là ký tự cuối của s , và giả sử kết luận đúng với s' . Xét các chuỗi con palindrome kết thúc ở ký tự cuối c ; giả sử các đầu trái của chúng theo thứ tự từ trái sang phải là $l_1, l_2, \dots, l_k$. Vì $s[l_1..|s|]$ là palindrome, nên với mọi vị trí $l_1 \leq p \leq |s|$, ta có $s[p..|s|] = s[l_1..l_1 + |s| - p]$. Do đó, với chuỗi con palindrome $s[l_i..|s|]$, ta đều có $s[l_i..|s|] = s[l_1..l_1 + |s| - l_i]$. Khi $i \neq 1$, luôn có $l_1 + |s| - l_i < |s|$, nên $s[l_i..|s|]$ đã xuất hiện trong $s[1..|s| - 1]$. Vì vậy mỗi lần nhiều nhất chỉ sinh thêm một chuỗi palindrome khác nhau, tức $s[l_1..|s|]$. Kết luận vẫn đúng với s .

Theo quy nạp toán học, Định lý 3.1 đúng. ■

Vì vậy số trạng thái của cây palindrome là cấp $\mathcal{O}(|s|)$. Xét các chuyển: số chuyển của một nút là $\mathcal{O}(\Sigma)$, nhưng với một trạng thái, nút có thể chuyển tới nó thật ra là duy nhất, nên tổng số chuyển là $\mathcal{O}(|s|)$. Mỗi nút chỉ có một con trở thất bại, do đó tổng số nút và tổng số chuyển của cây palindrome đều là $\mathcal{O}(|s|)$.

11.3.3 Cách xây dựng

Ta dùng phương pháp tăng dần để xây cây palindrome của chuỗi s . Giả sử đã xây xong cây palindrome của s , tiếp theo cần thêm một ký tự mới c vào cuối s , và duy trì cây palindrome của sc .

Định lý 3.2. Trong các chuỗi con palindrome kết thúc ở ký tự mới thêm c , số chuỗi chưa từng xuất hiện trong s nhiều nhất là 1, và chuỗi đó nhất định là hậu tố palindrome dài nhất của sc .

Chứng minh. Suy trực tiếp từ quá trình chứng minh Định lý 3.1. ■

Mỗi lần chỉ cần tìm hậu tố palindrome dài nhất của sc . Giả sử hậu tố palindrome dài nhất của sc là $s[i..|s|]c$. Theo định nghĩa palindrome, hoặc $i > |s|$, tức hậu tố palindrome dài nhất chỉ gồm ký tự c , hoặc $s[i + 1..|s|]$ cũng là một palindrome. Nói cách khác, $s[i + 1..|s|]$ phải là một hậu tố palindrome của s , hoặc là xâu có độ dài 0 hoặc -1 (cho thuận tiện, chính là *even* và *odd* trên cây palindrome). Bây giờ cần tìm trên chuỗi fail của nút ứng với hậu tố palindrome dài nhất của s một nút t có độ dài lớn nhất thỏa $s[|s| - \text{len}_t] = c$; khi đó hậu tố palindrome dài nhất của sc là ctc .

Khi chèn, nếu trên cây palindrome chưa có nút biểu thị ctc , cần tạo một nút mới để biểu thị ctc ; nếu đã có thì dùng trực tiếp nút đó. Nếu tạo nút mới, còn phải tìm nút mà con trở fail của nút mới trở tới, tương đương tìm lại trên chuỗi fail của fail[t] một nút v dài nhất thỏa $s[|s| - \text{len}_v] = c$. Cần chú ý: nếu $\text{len}_t = -1$, thì fail của ctc phải trở tới nút độ dài 0; ngược lại, hậu tố palindrome dài nhất của ctc chắc chắn nằm trên cây palindrome, nên chỉ cần trở tới nút tương ứng.

Ví dụ 2. Chẳng hạn $s = \text{babbab}$, ký tự mới thêm là a . Hậu tố palindrome dài nhất của s là babbab , chuỗi fail của nó là $\{\text{babbab}, \text{bab}, \text{b}, \text{even}, \text{odd}\}$. Xâu con palindrome hợp lệ dài nhất là b , nên hậu tố palindrome dài nhất của sa là aba . Tiếp tục tìm hậu tố palindrome dài nhất của aba , “xâu con palindrome” hợp lệ là *odd*, nên hậu tố palindrome dài nhất của aba là nút ứng với chuyển a của *odd*, tức a .

Cây của babbab Sau khi thêm a : thêm nút aba , với $\text{fail}(\text{aba}) = a$.

Trong cài đặt cụ thể, chỉ cần bắt đầu từ hậu tố palindrome dài nhất trên cây palindrome, đi lên gốc theo con trở fail cho tới khi hợp lệ. Gọi cách chèn này là thuật toán chèn cơ bản.

Định lý 3.3. Cách cài đặt này có tổng độ phức tạp thời gian $\mathcal{O}(|s| \log \Sigma)$, độ phức tạp không gian $\mathcal{O}(|s|)$.

Chứng minh. Xét tổng số lần nhảy lên theo chuỗi fail. Chia đóng góp thành hai phần: đóng góp khi tìm hậu tố palindrome dài nhất của xâu mới, và đóng góp khi tìm hậu tố palindrome dài nhất của nút mới. Hai phần này hiển nhiên cùng bậc, nên chỉ xét phần trước. Đặt hàm thế năng $\Phi(s)$ bằng độ dài hậu tố palindrome dài nhất của s . Khi thêm ký tự mới c vào sau s , trước hết có $\Phi(sc) \leq \Phi(s) + 1$. Mỗi khi đi lên một bước theo chuỗi fail, $\Phi(sc)$ giảm 1, và luôn có $\Phi(sc) \geq 0$. Vì vậy thế năng tổng cộng chỉ tăng $\mathcal{O}(|s|)$, và cũng chỉ giảm $\mathcal{O}(|s|)$; do đó độ phức tạp của các lần nhảy trên chuỗi fail là $\mathcal{O}(|s|)$.

Mỗi lần thêm ký tự mới, cần kiểm tra nút t đã có cạnh chuyển c hay chưa; nếu có thì không cần thêm nút mới, nếu không thì thêm. Cần xây một cây cân bằng (hoặc dùng trực tiếp map của C++) cho mỗi nút để ghi các chuyển của nó; độ phức tạp mỗi lần truy vấn là $\mathcal{O}(\log \Sigma)$, tổng độ phức tạp thời gian là $\mathcal{O}(|s| \log \Sigma)$, độ phức tạp không gian là $\mathcal{O}(|s|)$. Trong thực tế, Σ có thể nhỏ, có thể mở thẳng một mảng cho mỗi nút để ghi chuyển; nếu không tính chi phí cấp phát mảng, độ phức tạp thời gian có thể giảm xuống $\mathcal{O}(|s|)$, nhưng độ phức tạp không gian là $\mathcal{O}(|s| \Sigma)$. ■

11.3.4 Ứng dụng cơ bản

Ví dụ 3. Bài toán kinh điển. Cho một xâu s chỉ gồm chữ cái thường. Với mỗi tiền tố $s[1..i]$ của s , hãy tính số xâu con palindrome khác nhau và số xâu con palindrome khác vị trí.

$$|s| \leq 10^5.$$

Vì cây palindrome được xây tăng dần, sau khi thêm mỗi ký tự ta có thể nhận được cấu trúc cây palindrome tương ứng. Số xâu con palindrome khác nhau bằng số nút hiện có trên cây palindrome trừ 2

(hai gốc). Số xâu con palindrome khác vị trí tương đương số trước đó cộng với số palindrome kết thúc ở vị trí hiện tại; con số này bằng số hậu tố palindrome của xâu hiện tại, tức bằng độ dài chuỗi fail của nút đó (chú ý không tính *odd* và *even*). Trên cây palindrome, duy trì thêm số nút trên chuỗi fail của mỗi nút. Cách làm này có độ phức tạp thời gian $\mathcal{O}(|s| \log \Sigma)$, độ phức tạp không gian $\mathcal{O}(|s|)$.

Ví dụ 4. Palindrome. ⁴⁷ Xét một xâu s chỉ gồm chữ thường. Định nghĩa giá trị xuất hiện của một xâu con t của s là số lần t xuất hiện trong s nhân với độ dài của t . Hãy tìm giá trị xuất hiện lớn nhất trong tất cả các xâu con palindrome của s .

$$|s| \leq 300000.$$

Trước hết xây cây palindrome của s . Với một nút t trên cây palindrome, định nghĩa right_t là tập các đầu nút phải của những lần xuất hiện của t trong s . Khi đó giá trị xuất hiện của t là $\text{len}_t \times |\text{right}_t|$, chỉ cần tính $|\text{right}_t|$ cho mỗi nút. Với mỗi tiền tố $\text{pre}[s][i]$ của s , ta tìm được hậu tố palindrome dài nhất của nó, gọi là u , rồi thêm i vào right_u . Sau khi xử lý mọi tiền tố, right của mỗi nút chính là hợp các right của các con nó trên cây fail. Có thể chứng minh rằng trên cây fail, các tập right của các con của cùng một nút đôi một rời nhau, nên kích thước right của nút đó bằng tổng kích thước right của các con. Cách làm này cũng có độ phức tạp thời gian $\mathcal{O}(|s| \log \Sigma)$, độ phức tạp không gian $\mathcal{O}(|s|)$.

11.4 Mở rộng cây palindrome

11.4.1 Chèn ở đầu

Trong cách xây dựng cây palindrome trước đó, ta đã thực hiện thao tác thêm một ký tự vào cuối xâu và duy trì cây palindrome mới. Bây giờ có thể thử thêm một ký tự vào đầu xâu và duy trì cây palindrome mới.

Tương tự chèn ở cuối, khi chèn ở đầu, chẳng hạn xâu hiện tại là s và cần tìm cây palindrome của cs , các palindrome mới sinh bắt đầu bằng c nhiều nhất chỉ có 1, và chính là tiền tố palindrome dài nhất của cs . Từ quá trình trước đó, ta biết chỉ cần tìm một tiền tố palindrome dài nhất t của s thỏa $s[|t| + 1] = c$, khi đó tiền tố palindrome dài nhất mới là ctc . Để tìm t , ta có thể duy trì thêm trên mỗi nút cây palindrome một con trỏ fail', biểu thị nút ứng với tiền tố palindrome dài nhất của nút này; mỗi lần thêm ký tự mới, chỉ cần đi theo chuỗi fail' của tiền tố palindrome dài nhất của s để tìm t hợp lệ dài nhất.

Chú ý rằng mọi nút trên cây palindrome đều là palindrome (trừ các nút gốc). Với một palindrome t , xâu đảo của t bằng chính t ; tức với mỗi hậu tố palindrome $t[i..|t|]$ của t , xâu đảo của nó trùng với $t[1..|t| - i + 1]$, mà bản thân nó lại là palindrome. Do đó xâu đảo của $t[i..|t|]$ bằng $t[i..|t|]$, tức $t[i..|t|] = t[1..|t| - i + 1]$. Vì vậy tập xâu tiền tố palindrome và tập xâu hậu tố palindrome của t thật ra là như nhau. Nói cách khác, con trỏ fail' của một nút trên cây palindrome chính là con trỏ fail của nó.

Vì vậy chỉ cần duy trì tiền tố palindrome dài nhất và hậu tố palindrome dài nhất của xâu s , ta có thể hỗ trợ chèn ở cả đầu và cuối xâu. Cần chú ý rằng khi chèn ở đầu (hoặc cuối), thao tác này có thể ảnh hưởng tới hậu tố (hoặc tiền tố) palindrome dài nhất; duy trì thêm là được. Phân tích độ phức tạp tương tự trước đó, thu được tổng độ phức tạp thời gian $\mathcal{O}(|s| \log \Sigma)$, độ phức tạp không gian $\mathcal{O}(|s|)$.

Ví dụ 5. Chẳng hạn $s = \text{babbab}$, ký tự cần thêm vào đầu là b . Tiền tố palindrome dài nhất của s là babbab , chuỗi fail của nó là $\{\text{babbab}, \text{bab}, \text{b}, \text{even}, \text{odd}\}$. Xâu con palindrome hợp lệ dài nhất là bab , nên tiền tố palindrome dài nhất của bs là bbabb . Tiếp tục tìm hậu tố palindrome dài nhất của bbabb , “xâu

⁴⁷APIO2014, “Palindromes”.

con palindrome” hợp lệ là *even*, nên hậu tố palindrome dài nhất của *bbabb* là nút ứng với chuyển *b* của *even*, tức *bb*.

Sau khi thêm *b* vào đầu *babbab*, cây palindrome có thêm nút *bbabb*; các nút cũ như *bb*, *abba*, *babbab* vẫn được dùng lại.

Ví dụ 6. Victor and String. ⁴⁸ Ban đầu có một xâu rỗng *s*. Tiếp theo thực hiện *n* thao tác, thuộc bốn loại sau:

- Thêm một ký tự cho trước vào đầu *s*.
- Thêm một ký tự cho trước vào cuối *s*.
- Hỏi hiện tại *s* có bao nhiêu palindrome khác nhau.
- Hỏi hiện tại *s* có bao nhiêu palindrome khác vị trí.

$$n \leq 10^5.$$

Nếu không có chèn ở đầu, bài này hoàn toàn giống Ví dụ 3: chỉ cần duy trì thêm độ sâu của mỗi nút trên cây fail, và khi thêm ký tự ở cuối thì cập nhật số palindrome khác vị trí. Bây giờ cần hỗ trợ chèn ở đầu. Vấn đề vẫn tương tự: số palindrome khác nhau trong *s* vẫn là số nút trên cây palindrome, nhưng tập *right* của mỗi nút có vẻ không dễ duy trì. Chú ý rằng mỗi lần chèn ở đầu, chỉ cần cập nhật kích thước tập *right*, nên chỉ cần cộng 1 vào kích thước *right* của tiền tố palindrome dài nhất sau khi thêm ký tự mới. Số palindrome khác vị trí tương đương cộng thêm độ dài chuỗi fail của nút ứng với tiền tố palindrome dài nhất.

11.4.2 Xóa ký tự cuối

Xét một vấn đề thú vị hơn: cho một xâu *s* và cây palindrome tương ứng, liệu có thể tìm cây palindrome của xâu mới sau khi xóa ký tự cuối của *s* hay không?

Một ý tưởng rất đơn giản là: với mỗi tiền tố của *s*, ghi lại hậu tố palindrome dài nhất của nó; mỗi nút trên cây palindrome ghi số lần nút này là hậu tố palindrome dài nhất của một tiền tố. Khi xóa ký tự cuối của *s*, thao tác này tương đương xóa hậu tố palindrome dài nhất của nó (vì các hậu tố palindrome khác vẫn còn xuất hiện trong *s*); ta chỉ cần giảm số lần ở nút tương ứng trên cây palindrome đi 1, và nếu số lần của một nút về 0 thì xóa nút đó khỏi cây palindrome. Đáng tiếc là độ phức tạp của cách làm này không đúng khi có thao tác chèn. Xét ví dụ sau.

Ví dụ 7. Ký hiệu *push(c)* là thêm ký tự *c* vào cuối, và *pop()* là xóa ký tự cuối. Xét dãy thao tác:

$$\underbrace{\text{push}(a), \dots, \text{push}(a)}_n \quad \underbrace{\text{push}(b), \text{pop}(), \dots, \text{push}(b), \text{pop}()}_n.$$

Có thể thấy với mỗi *push(b)*, ta đều nhảy lên *n* lần trên chuỗi fail. Mỗi lần xóa *b*, phần thể năng đã bị tiêu hao này lại được khôi phục, nên tổng độ phức tạp là $\mathcal{O}(n^2)$.

Lỗi hổng của thuật toán này là: độ phức tạp của phần chèn trong cây palindrome được suy ra bằng phân tích thể năng. Nếu có thể xóa một ký tự, ta có thể liên tục lặp lại thao tác tiêu hao nhiều thể năng, khiến độ phức tạp tổng thể trở thành $\mathcal{O}(n^2)$. Do đó, để cài đặt thao tác xóa, trước hết nên tìm một thuật toán chèn ổn định, có độ phức tạp không phụ thuộc vào phân tích thể năng.

⁴⁸Bestcoder #52 1004, “Victor and String”.

11.4.3 Một thuật toán chèn không dựa trên phân tích thể năng

Điểm nghẽn của thuật toán chèn trước đó là: mỗi lần chèn một ký tự c , ta phải đi lên theo chuỗi fail của hậu tố palindrome dài nhất hiện tại t để tìm nút đầu tiên v sao cho tiền thân của v trong s (tức ký tự đứng ngay trước v) là c . Chú ý rằng ngoài trường hợp $v = t$, tiền thân của v luôn nằm trong t . Nói cách khác, với mỗi t , ta cần tìm một hậu tố palindrome dài nhất của t có tiền thân là c ; hậu tố này chỉ liên quan đến t , không liên quan đến s . Vì vậy, với mỗi nút t trong cây palindrome, duy trì thêm một mảng chuyển thất bại `quick[c]`, lưu hậu tố palindrome dài nhất của t có tiền thân là c . Khi chèn, trước hết chỉ cần kiểm tra tiền thân của hậu tố palindrome dài nhất hiện tại t có phải c hay không; nếu không, nút hợp lệ v chính là `quick[c]` của t .

Tiếp theo xét cách duy trì `quick` của mỗi nút. Với một nút t , `quick` của t hầu như không khác `quick` của fail_t , vì các hậu tố palindrome của t chỉ là các hậu tố palindrome của fail_t cộng thêm chính fail_t . Trước hết sao chép `quick` của fail_t làm `quick` của t . Gọi c là tiền thân của fail_t trong t ; dùng fail_t để cập nhật `quick[c]` của t . Nếu làm thô trực tiếp, độ phức tạp thời gian và không gian của mỗi lần chèn đều là $\mathcal{O}(\Sigma)$.

Có thể làm `quick` của mỗi t khả tồn; một cách là dùng cây đoạn khả tồn để khả tồn hóa mảng. Mỗi lần sao chép chỉ cần sao chép phiên bản, độ phức tạp giảm xuống $\mathcal{O}(1)$. Độ phức tạp thời gian và không gian của một lần chèn là $\mathcal{O}(\log \Sigma)$. Cuối cùng ta thu được một thuật toán chèn đơn lẻ không dựa trên phân tích thể năng, với độ phức tạp thời gian và không gian $\mathcal{O}(\log \Sigma)$.

11.4.4 Chèn và xóa ở hai đầu

Tiếp theo thử đồng thời hỗ trợ chèn, xóa ở hai đầu xâu và duy trì cây palindrome mới.

Trước hết đưa vào một khái niệm. Một xâu con $s[l..r]$ của s được gọi là quan trọng khi và chỉ khi $s[l..r]$ là palindrome, không tồn tại $r' > r$ sao cho $s[l..r']$ là palindrome, và không tồn tại $l' < l$ sao cho $s[l'..r]$ là palindrome. Với mỗi vị trí r , hiển nhiên nhiều nhất chỉ có một $l \leq r$ sao cho $s[l..r]$ là quan trọng; gọi $s[l..r]$ là hậu tố quan trọng của r , và r có thể không có hậu tố quan trọng. Tương tự, có thể định nghĩa tiền tố quan trọng cho l . Tiếp theo xét ảnh hưởng của bốn thao tác:

- Thêm ký tự mới ở cuối.

Gọi r là vị trí cuối ban đầu. Khi đó r chắc chắn có một hậu tố quan trọng, và hậu tố này chính là hậu tố palindrome dài nhất hiện tại của s . Áp dụng trực tiếp thuật toán chèn ở cuối để tìm cây palindrome mới và hậu tố palindrome dài nhất mới. Gọi l là đầu trái của hậu tố palindrome dài nhất mới, thì $s[l..r+1]$ là quan trọng. Chú ý thao tác này có thể khiến tiền tố palindrome dài nhất của $s[l..r+1]$ không còn quan trọng. Duy trì thêm một thùng `cnt[l][r]`, biểu thị $s[l..r]$ bị đánh dấu “không quan trọng” bao nhiêu lần; `cnt[l][r]` có thể cài bằng băm.

- Xóa ký tự cuối.

Gọi l là đầu trái của hậu tố quan trọng của r . Giảm `cnt` tương ứng với tiền tố palindrome dài nhất của $s[l..r]$ đi 1; nếu giảm về 0, nghĩa là nó lại trở thành quan trọng. Còn một vấn đề: ta cần biết nút cây palindrome ứng với $s[l..r]$ có cần bị xóa hay không. Với mỗi nút t trên cây palindrome, định nghĩa hai trọng số `impt` và `childt`: `impt` là số cặp $1 \leq l' \leq r' \leq |s|$ sao cho $s[l'..r'] = t$ và $s[l'..r']$ là quan trọng; `childt` là số con của t trên cây fail. Khi tình trạng quan trọng của một xâu thay đổi, cần đồng thời thay đổi giá trị `imp` của nút tương ứng trên cây palindrome. Nếu `imp` và `child` của một nút đều bằng 0, nút đó không thể còn xuất hiện trong s , nên xóa trực tiếp. Chú ý mỗi lần nhiều nhất chỉ có thể xóa một nút, và khi xóa một nút cần giảm `child` của nút fail của nó đi 1.

- Thêm ký tự mới ở đầu.

Tương tự thêm ở cuối: chỉ cần tìm tiền tố palindrome dài nhất mới, rồi sửa tình trạng quan trọng và cnt của hậu tố palindrome dài nhất của nó.

- Xóa ký tự đầu.

Tương tự xóa ở cuối: chỉ cần xử lý đúng thay đổi của tình trạng quan trọng và thay đổi của cây palindrome.

Ví dụ 8. Xét xâu hiện tại aacbaaa. Các xâu con quan trọng của nó là $s[1..2]$, $s[3..3]$, $s[4..4]$, $s[5..7]$. Nếu xóa ký tự đầu, xâu trở thành acbaaa, các xâu con quan trọng là $s[1..1]$, $s[2..2]$, $s[3..3]$, $s[4..6]$. Tiếp tục xóa ký tự cuối, xâu trở thành acbaa, các xâu con quan trọng là $s[1..1]$, $s[2..2]$, $s[3..3]$, $s[4..5]$. Các giá trị imp trên cây fail thay đổi tương ứng với những xâu quan trọng này.

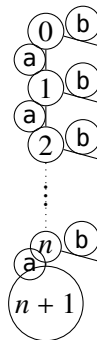
11.4.5 Xây cây palindrome trên TRIE

Cho một cây TRIE có n nút. Một xâu hợp lệ trên TRIE được định nghĩa là xâu gồm các ký tự trên các cạnh đi từ gốc tới một nút bất kỳ. Bây giờ cần tìm hợp của các cây palindrome của mọi xâu hợp lệ trên TRIE.

Tương tự việc một xâu s chỉ có $\mathcal{O}(|s|)$ xâu con palindrome khác nhau, có thể chứng minh rằng với một TRIE có n nút, tập các xâu con palindrome của các xâu hợp lệ cũng chỉ có $\mathcal{O}(n)$ phần tử. Xét xâu ứng với một nút: nó tương đương xâu của cha nối thêm một ký tự, nên nhiều nhất chỉ sinh thêm một xâu con palindrome, và xâu đó nhất định là hậu tố palindrome dài nhất của xâu ứng với nút này. Vì vậy số xâu con palindrome khác nhau là $\mathcal{O}(n)$, nên kích thước cây palindrome cũng là $\mathcal{O}(n)$. Với mỗi nút trên TRIE, ghi lại nút trên cây palindrome ứng với hậu tố palindrome dài nhất của xâu tương ứng; mỗi lần mở rộng tới con của nó, áp dụng trực tiếp thuật toán chèn không dựa trên phân tích thể năng để tìm hậu tố palindrome dài nhất ứng với nút con. Tổng độ phức tạp thời gian và không gian là $\mathcal{O}(n \log \Sigma)$.

Nhưng ở đây có một câu hỏi thú vị: nếu áp dụng thuật toán chèn cơ bản nhất, độ phức tạp sẽ là bao nhiêu?

Định lý 4.1. Với cây TRIE sau, thuật toán chèn cơ bản nhất có độ phức tạp $\mathcal{O}(\text{tổng độ sâu của mọi lá})$.



Chứng minh. Định nghĩa $\Phi(i)$ là độ sâu trên cây fail của hậu tố palindrome dài nhất của nút i trên cây TRIE. Khi đó hiển nhiên $\Phi(i) = i$. Với mỗi nút không đánh số được mở rộng từ nút i , vì ký tự trên cạnh chuyển khác với mọi ký tự trước đó, chắc chắn phải tiêu hao hết $\Phi(i)$. Do đó tổng thể năng bị tiêu hao là

$$\sum_{i=1}^n \Phi(i) = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \mathcal{O}(\text{tổng độ sâu của mọi lá}).$$

■

Vì vậy khi xây cây palindrome cho TRIE, bắt buộc phải dùng thuật toán chèn không dựa trên phân tích thể năng, nếu không độ phức tạp thời gian sẽ suy biến thành $\mathcal{O}(n^2)$.

Ví dụ 9. Tree Palindromes. ⁴⁹ Cho một cây N đỉnh gốc 1, mỗi cạnh trên cây có một chữ thường. Với mỗi đỉnh trên cây, hãy tìm hậu tố palindrome dài nhất của xâu tạo bởi đường đi từ gốc tới đỉnh đó, rồi xuất tổng đáp án của mọi đỉnh.

$$N \leq 100000.$$

Cây trong đề chính là một TRIE. Xây trực tiếp cây palindrome trên TRIE này; trong quá trình xây, có thể tìm hậu tố palindrome dài nhất kết thúc ở mỗi đỉnh. Bài này có bảng chữ cái nhỏ, khi chèn không cần khả tồn hóa chuyển thất bại, chỉ cần mở thẳng một mảng để ghi. Độ phức tạp thời gian $\mathcal{O}(N\Sigma)$.

11.4.6 Cây palindrome khả tồn

Thuật toán chèn cơ bản nhất dựa trên phân tích thể năng; nếu làm nó khả tồn thì độ phức tạp không đúng. Vì vậy xét khả tồn hóa thuật toán chèn không dựa trên phân tích thể năng. Chuyển của một nút trên cây palindrome được lưu bằng một cây cân bằng hoặc mảng độ dài Σ . Nếu dùng cây cân bằng, có thể dùng TREAP khả tồn, khi đó chỉ cần ghi phiên bản tương ứng; nhưng cách này có lượng mã rất lớn. Một cách khác là, giống như dùng cây đoạn khả tồn để lưu chuyển thất bại, cũng dùng cây đoạn khả tồn để lưu mảng độ dài Σ , khi đó cũng chỉ cần ghi thông tin phiên bản.

Với một phiên bản nút trên cây palindrome của phiên bản hiện tại, chỉ cần ghi độ dài của nó, phiên bản nút mà con trở fail trở tới, và phiên bản cây đoạn tương ứng với cạnh chuyển và chuyển thất bại. Trong cách làm này, độ phức tạp không gian của một nút đơn là $\mathcal{O}(1)$. Với cây palindrome của mỗi phiên bản, dùng một cây đoạn khả tồn để lưu; mỗi lá lưu thông tin phiên bản của nút tương ứng trên cây palindrome. Mỗi lần chèn có thể cần tạo một nút mới trên cây palindrome, đồng thời cần lưu hậu tố palindrome dài nhất ứng với phiên bản này. Thuật toán này có độ phức tạp thời gian và không gian cho mỗi lần chèn là $\mathcal{O}(\log(|s| + \Sigma))$.

Rõ ràng có thể kết hợp cây palindrome khả tồn với cây palindrome hỗ trợ chèn và xóa ở hai đầu. Tuy nhiên, do tác giả cũng chưa từng cài đặt phiên bản cây palindrome này, nên không thể ước lượng độ phức tạp cài đặt mã cuối cùng; độc giả quan tâm có thể thử tự cài.

11.5 Ứng dụng

11.5.1 Country

50

Cho một xâu s độ dài n . Cần chọn từ s càng nhiều xâu con càng tốt, sao cho các xâu con được chọn đều là palindrome, và không có xâu nào là xâu con của một xâu khác.

$$n \leq 100000.$$

Trước hết xây cây palindrome của s , khi đó mỗi nút trên cây palindrome đều là một xâu có thể chọn. Bây giờ cần giải quyết điều kiện một xâu không được là xâu con của xâu khác.

Định lý 5.1. Nếu với hai palindrome a và b , a là xâu con của b , thì hoặc $a = b$, hoặc a là xâu con của hậu tố palindrome dài nhất của b , hoặc a là xâu con của palindrome thu được bằng cách bỏ hai ký tự ở hai đầu của b .

⁴⁹ICPC Preparatory Series by Team Akatsuki, “Three Palindromes”.

⁵⁰GDKOI2013, bài *Quy hoạch thành phố của Vương quốc núi lớn*.

Chứng minh. Chỉ cần xét trường hợp a là hậu tố (hoặc tiền tố) của b . Vì a là một palindrome, nếu a là hậu tố (hoặc tiền tố) của b , thì a chắc chắn là một hậu tố (hoặc tiền tố) palindrome của b , do đó chắc chắn là hậu tố (hoặc tiền tố) của hậu tố (hoặc tiền tố) palindrome dài nhất của b . ■

Theo định lý này, ta có thể xây một đồ thị có hướng G . Các nút của G tương ứng với các nút trên cây palindrome. Với một nút i , nối một cạnh có hướng từ i tới cha của i trên cây palindrome, và một cạnh có hướng từ i tới fail _{i} . Khi đó nếu nút j là xâu con của nút i , chắc chắn tồn tại một đường đi từ i tới j . Bài toán chuyển thành: cho một đồ thị có hướng G , chọn càng nhiều nút càng tốt sao cho giữa mọi cặp nút không tồn tại đường đi. Đây là bài toán phản xích dài nhất. Theo định lý Dilworth, phản xích dài nhất của một DAG bằng phủ đường ít nhất của đồ thị đó; chỉ cần tìm phủ đường ít nhất trên G . Độ phức tạp thời gian $\mathcal{O}(\text{Maxflow}(n, n))$.

11.5.2 Palindromeness

51

Định nghĩa chỉ số palindrome của một xâu như sau:

- Nếu một xâu không phải palindrome, chỉ số palindrome của nó là 0.
- Xâu chỉ có một ký tự có chỉ số palindrome bằng 1.
- Với các palindrome khác, định nghĩa đệ quy chỉ số bằng 1 cộng với chỉ số palindrome của xâu con gồm $\lfloor |s|/2 \rfloor$ ký tự đầu.

Cho một xâu s , hãy tính tổng chỉ số palindrome của mọi xâu con của s .

$$|s| \leq 10^5.$$

Xây cây palindrome của s , đồng thời duy trì kích thước tập right của mỗi nút. Phần còn lại là tính chỉ số palindrome của mỗi palindrome. Xét duy trì thêm trên mỗi nút i của cây palindrome một con trỏ half _{i} , trỏ tới hậu tố palindrome dài nhất của i có độ dài không vượt quá một nửa độ dài của i . Nếu độ dài của half _{i} đúng bằng $\lfloor |s|/2 \rfloor$, thì chỉ số palindrome của i bằng chỉ số palindrome của half _{i} cộng 1; nếu không thì bằng 1. Có hai cách duy trì half _{i} :

- Nhân đôi trên cây fail. Các tổ tiên của i trên cây fail ứng với những xâu có độ dài giảm dần, nên có thể nhân đôi để tìm tổ tiên dài nhất có độ dài không vượt quá một nửa độ dài của i . Độ phức tạp thời gian $\mathcal{O}(|s| \log |s|)$.
- Gọi j là cha của i trên cây palindrome. Khi đó half _{i} chắc chắn là con của một hậu tố palindrome nào đó của half _{j} . Đi trực tiếp theo chuỗi fail của half _{j} để tìm nút hợp lệ đầu tiên. Có thể dùng phương pháp giống chứng minh độ phức tạp của thuật toán chèn cơ bản để chứng minh tổng độ phức tạp của thuật toán này là $\mathcal{O}(|s|)$.

11.5.3 Virus synthesis

52

Cho một xâu s độ dài n , chỉ gồm các chữ cái in hoa A, G, T, C. Ban đầu có một xâu rỗng t . Mỗi bước có thể thực hiện một trong các thao tác sau với t :

- Thêm một ký tự vào đầu hoặc cuối t .

⁵¹CodeChef April Lunchtime 2015, “Palindromeness”.

⁵²Central Europe Regional Contest 2014, “Virus synthesis”.

- Nối xâu đảo của t vào đầu hoặc cuối t .

Hỏi cần ít nhất bao nhiêu bước để biến t thành s .

$$n \leq 10^5.$$

Chú ý rằng xâu sau thao tác loại hai chắc chắn là một palindrome. Xét liệt kê một xâu con palindrome $s[i..j]$ của s , tính chi phí nhỏ nhất để dựng $s[i..j]$, rồi dùng giá trị này cộng $i - 1 + n - j$ để cập nhật đáp án. Xây cây palindrome của s ; bây giờ cần tính chi phí nhỏ nhất để dựng mỗi nút. Tính chi phí của mỗi palindrome theo thứ tự từ ngắn tới dài. Với một palindrome t , trước hết xét thao tác loại một: có thể mở rộng từ tiền tố (hoặc hậu tố) palindrome dài nhất của t để được t , hoặc mở rộng từ cha của t ; chi phí của hai phương án đều tính trực tiếp được. Xét việc thu được t bằng thao tác loại hai: trước hết độ dài của t phải chẵn. Đặt $t = bb^r$, tìm tiền tố (hoặc hậu tố) palindrome dài nhất c của b ; khi đó chi phí của t bằng chi phí của c , cộng chi phí mở rộng c thành b , rồi cộng thêm 1. Áp dụng trực tiếp thuật toán trong mục 5.2 để tìm half_t , từ đó thu được c .

Tổng độ phức tạp thời gian là $\mathcal{O}(n)$.

11.5.4 Gen

53

Cho một xâu s độ dài n , chỉ gồm chữ cái thường. Có q truy vấn l, r : hỏi trong $s[l..r]$ có bao nhiêu palindrome khác nhau về bản chất. Bắt buộc online.

$$n, q \leq 10^5.$$

Đặt $L = \sqrt{n}$, cứ mỗi L vị trí đặt một điểm mốc. Với mỗi truy vấn, nếu $r - l \leq L$, có thể duyệt cây palindrome bằng vét cạn trực tiếp; ngược lại đoạn truy vấn chắc chắn đi qua một điểm mốc. Duyệt cây palindrome từ mỗi điểm mốc tới mỗi đầu phải, mỗi lần tương đương chèn ký tự vào đầu cây palindrome. Bằng một số ý tưởng ghi nhớ toàn cục, có thể bỏ chi phí $\mathcal{O}(\Sigma)$ trong $\mathcal{O}((n + q)\sqrt{n})$ lần chèn. Tổng độ phức tạp là $\mathcal{O}((n + q)\sqrt{n} + n\Sigma)$. Chi tiết cài đặt cụ thể hơn có thể tham khảo tài liệu [5].

11.5.5 AlphaDog (nguyên tác)

Với một xâu s , định nghĩa giá trị đặc biệt $F(s)$ là

$$F_s = \sum_{1 \leq x \leq y \leq |s|} \text{LCP}(x, y).$$

LCP là viết tắt của *Longest Common Palindrome*; nói cách khác, $\text{LCP}(x, y)$ là độ dài của xâu t dài nhất thỏa ba điều kiện:

- t là palindrome.
- Tồn tại một $i \leq x$ sao cho $s[i..x] = t$.
- Tồn tại một $j \leq y$ sao cho $s[j..y] = t$.

Ban đầu có một xâu rỗng s . Tiếp theo thực hiện q lần chèn, mỗi lần thêm một ký tự mới vào cuối xâu. Sau mỗi lần thêm, hỏi giá trị F của xâu hiện tại. Bắt buộc online.

⁵³Lần thi chéo thứ tư của ứng viên đội tuyển quốc gia Trung Quốc năm 2017, bài Gen.

$$q \leq 10^5.$$

Trước hết xét cách tính $LCP(x, y)$. Gọi a là hậu tố palindrome dài nhất của $s[1..x]$, b là hậu tố palindrome dài nhất của $s[1..y]$. Khi đó $LCP(x, y)$ bằng độ dài LCA của a, b trên cây fail. Mỗi lần chèn một ký tự mới, cần tính đóng góp mà ký tự này đem lại: tương đương mỗi lần có thể nối thêm một nút mới vào cây fail, rồi truy vấn tổng độ dài LCA của một nút với mọi nút khác. Bài toán này có hai cách làm.

- Trên cây fail, đặt khoảng cách từ nút i tới $fail_i$ là $len_i - len_{fail_i}$. Xét cách tính khoảng cách giữa hai nút:

$$dis(x, y) = deep(x) + deep(y) - 2 \cdot deep(lca(x, y)).$$

Do đó

$$deep(lca(x, y)) = \frac{deep(x) + deep(y) - dis(x, y)}{2}.$$

Bây giờ, với x cho trước, để tính $\sum_y deep(lca(x, y))$, chỉ cần tính $\sum_y dis(x, y)$. Bài toán động thêm nút trên cây và hỏi tổng khoảng cách từ một nút tới mọi nút khác có cách làm kinh điển bằng phân trị điểm động. Ý tưởng đại khái là duy trì cấu trúc phân trị điểm của một cây, khi chèn thì dùng tư tưởng cây thay thế để tái cấu trúc phân trị điểm của cây này; chi tiết cài đặt có thể tham khảo [6]. Tổng độ phức tạp thời gian là $\mathcal{O}(q \log^2 q)$.

- Với một $lca(x, y)$, đóng góp của nó là độ dài của nó, tương đương tổng độ dài các cạnh từ nó tới gốc. Có thể chuyển đóng góp thành tổng đóng góp của từng cạnh. Với một cạnh, giả sử đó là cạnh từ i tới $fail_i$. Cạnh này đóng góp khi và chỉ khi x và y đều nằm trong cây con của i . Với mỗi cạnh, đặt size là số nút trong cây con của nó, và duy trì $size \times len$, trong đó len là độ dài cạnh. Khi thêm một nút x , thao tác này tương đương truy vấn tổng $size \times len$ trên mọi cạnh từ x tới gốc, đồng thời cộng len vào $size \times len$ của mọi cạnh này. Có thể duy trì bằng Link-Cut Tree. Vì vậy tổng độ phức tạp thời gian là $\mathcal{O}(q \log q)$.

11.6 Tổng kết

Cây palindrome là cấu trúc dữ liệu chuyên dùng để xử lý một lớp bài toán về palindrome. Nó tận dụng những tính chất đẹp vốn có của palindrome, kết hợp với cấu trúc cây, nhờ đó có thể dễ dàng giải nhiều bài toán mà các thuật toán thông thường khó với tới. Tuy nhiên bản thân nó cần chi phí không gian khá lớn, độ phức tạp cài đặt mã cũng cao; đồng thời, thuật toán này thiên về xử lý hậu tố palindrome, nên không phù hợp với mọi bài toán palindrome (với một số bài, thuật toán Manacher đã đủ dùng). Cũng như mảng hậu tố và automaton hậu tố có ưu thế riêng, cây palindrome và thuật toán Manacher cũng có trọng tâm khác nhau khi xử lý bài toán palindrome.

Thuật toán cây palindrome vẫn còn nhiều không gian để khai thác: khi chèn hoặc xóa ký tự trong xâu, liệu có thể duy trì cây palindrome tương ứng không? Với một cây xâu hoặc một DAG, liệu có thể xây dựng cây palindrome tương ứng không? Những hướng này vẫn có giá trị nghiên cứu lớn. Hy vọng bài viết này có thể đóng vai trò gợi mở, đem lại cho mọi người nhiều cảm hứng hơn trong các bài toán palindrome.

11.7 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn các huấn luyện viên đội tuyển tập huấn quốc gia Zhang Ruizhe và Yu Linyun đã hướng dẫn và giúp đỡ.

Cảm ơn thầy Song Xinbo và thầy Xiong Chao của Trường Trung học Kỹ niệm Trung Sơn đã quan tâm và chỉ dẫn tôi trong nhiều năm qua.

Cảm ơn đàn anh Yang Le ở Đại học Thanh Hoa đã đọc duyệt bản thảo.

Cảm ơn các bạn Shen Rui của Trường Trung học Thường Châu, Mao Xiao và Ouyang Siqi của Trường Trung học Yali đã giúp đỡ và gợi mở cho tôi.

Cảm ơn các thầy cô và bạn học khác từng giúp đỡ và gợi mở cho tôi.

Cảm ơn cha mẹ tôi đã luôn quan tâm và chăm sóc chu đáo.

Tài liệu tham khảo

- [1] Aditya Shah, “TREEPAL’s Editorial”, [CodeChef Discuss](#), [TREEPAL’s Editorial](#).
- [2] Mikhail Rubinchik, Arseny M. Shur, “EERTREE: An Efficient Data Structure for Processing Palindromes in Strings”, arXiv:1506.04862v2 [cs.DS], 17 Aug 2015.
- [3] Victor Wonder, *Cách xây dựng và ứng dụng của cây palindrome*.
- [4] Xu Yi, *Bàn sơ về bài toán xâu con palindrome*, luận văn đội tuyển tập huấn quốc gia năm 2014.
- [5] Weng Wentao, báo cáo lời giải bài *Gen*.
- [6] WC2014, *Tình yêu hoa tử kinh*, <http://uoj.ac/problem/55>.

Ghi chú về trích xuất

Nội dung bài đầu được trích bằng pdftotext từ trang vật lý 3 đến 13 của PDF, tương ứng trang bài viết 1 đến 11; trang vật lý 14 là bài kế tiếp. Không cần OCR. Hai dòng kết thúc vòng lặp trong mã giả của nguồn in là **end for** = 0; bản dịch giữ nguyên ký hiệu này như một dị biệt của nguồn.

Bài thứ hai được trích bằng pdftotext từ trang vật lý 14 đến 26 của PDF, tương ứng trang bài viết 12 đến 26. Các hình minh họa nhỏ được dựng lại bằng tikz; ký hiệu ma trận con xóa hàng/cột $A^{I,J}$ và các phân số trong chứng minh Định lý 4.1 được kiểm tra lại bằng trang render vì lớp văn bản của PDF làm mất chi tiết này.

Bài thứ ba được trích bằng pdftotext từ trang vật lý 29 đến 33 của PDF, tương ứng trang bài viết 27 đến 31; trang vật lý 34 là bài kế tiếp. Bài này không có hình, chỉ có một bảng giới hạn dữ liệu được dựng lại trong LaTeX. Công thức khai triển $E(x)$ ở thuật toán sáu được giữ theo bản in: về phải dùng x^k trong tổng, dù về mặt ký hiệu đây có vẻ là lỗi nguồn thay vì x^i . Dòng định nghĩa $P(x)$ cũng được giữ theo bản in, nơi không có thừa số lũy thừa của x trong tổng.

Bài thứ tư được trích bằng pdftotext từ trang vật lý 34 đến 51 của PDF, tương ứng trang bài viết 32 đến 49; trang vật lý 52 là bài kế tiếp. Bài này không có hình; hai bảng và hai mã giả được dựng lại bằng LaTeX. Các ký hiệu bị lớp văn bản trích thành $<$, $($ hoặc dấu phẩy đã được đối chiếu với trang render và chuyển về $\notin$, $\subset$, $\neq$ khi rõ nghĩa. Bản in ở điều kiện cực đại của S trên trang 36 ghi $V \cup \{v\}$; bản dịch dùng $S \cup \{v\}$ theo nội dung toán học ngay sau đó. Hai dòng mã giả **end for** = 0 và **return** $I = 0$ được giữ theo bản in.

Bài thứ năm được trích bằng pdftotext từ trang vật lý 52 đến 76 của PDF, tương ứng trang bài viết 50 đến 74; trang vật lý 77 là bài kế tiếp. Các bảng và hình minh họa nhỏ được dựng lại bằng LaTeX/tikz. Các hình được kiểm tra bằng trang render sau khi biên dịch; không dùng ảnh cắt từ PDF. Nguồn không có mẫu mã chương trình cần giữ nguyên.

Bài thứ sáu được trích bằng pdftotext từ trang vật lý 77 đến 91 của PDF, tương ứng trang bài viết 75 đến 89; trang vật lý 92 là bài kế tiếp. Bài này không có hình. Các công thức ma trận trong mục 5.2

được kiểm tra bằng trang render vì lớp văn bản trích ra làm nhiều ngoặc và chỉ số; các ma trận B và B^{-1} được viết lại bằng công thức theo phân tử để giữ đúng nội dung mà gọn hơn bản in. Nguồn không có mẫu mã chương trình cần giữ nguyên.

Bài thứ bảy được trích bằng pdftotext từ trang vật lý 92 đến 100 của PDF, tương ứng trang bài viết 90 đến 98; trang vật lý 101 là bài kế tiếp. Bài này không có hình và không có mẫu mã chương trình. Bảng test và các công thức hàm sinh trên trang 94–96 được kiểm tra bằng trang render vì lớp văn bản trích ra làm nhiều ký hiệu trần, nhị thức và chỉ số lũy thừa; dòng tích phân dẫn tới $E(x) = e^{-D(x)}$ được giữ theo bản in dù biểu thức trong ngoặc có vẻ là ghi chú công thức $\ln F(x)$ chứ không phải một vế toán học hoàn chỉnh.

Bài thứ tám được trích bằng pdftotext từ trang vật lý 101 đến 123 của PDF, tương ứng trang bài viết 99 đến 121; trang vật lý 124 là bài kế tiếp. Bài này không có ảnh raster. Các đồ thị hàm số trong hình 4–7 được thay bằng ghi chú hình ngắn gọn vì việc dựng lại chính xác trục và đường cong không ảnh hưởng nội dung thuật toán; các hình cây nhỏ được dựng lại bằng tikz. Nguồn không có mẫu mã chương trình cần giữ nguyên.

Bài thứ chín được trích bằng pdftotext từ trang vật lý 124 đến 140 của PDF, tương ứng trang bài viết 122 đến 138; trang vật lý 141 là bài kế tiếp. Bài này không có ảnh raster và không có mẫu mã chương trình. Các sơ đồ cây palindrome, cây fail và TRIE được dựng lại hoặc tóm lược bằng LaTeX/tikz; phần văn bản và công thức giữ nguyên các định danh thuật toán như fail, go, quick, half, TRIE, TREAP, push và pop.